

The NetCDF Developer's Handbook

The Authoritative Guide to Writing High-Performance
Programs for Scientific Data Management

Edward Hartnett

Copyright © 2026 by Edward Hartnett

All rights reserved.

No portion of this book may be reproduced in any form without written permission from the publisher or author, except as permitted by U.S. copyright law.

This publication is designed to provide accurate and authoritative information in regard to the subject matter covered. It is sold with the understanding that neither the author nor the publisher is engaged in rendering legal, investment, accounting or other professional services. While the publisher and author have used their best efforts in preparing this book, they make no representations or warranties with respect to the accuracy or completeness of the contents of this book and specifically disclaim any implied warranties of merchantability or fitness for a particular purpose. No warranty may be created or extended by sales representatives or written sales materials. The advice and strategies contained herein may not be suitable for your situation. You should consult with a professional when appropriate. Neither the publisher nor the author shall be liable for any loss of profit or any other commercial damages, including but not limited to special, incidental, consequential, personal, or other damages.

Book Cover: The image shows Earth from geostationary orbit, about 36 000 km from the planet's surface. It was captured on 15 November 2025 by the MTG-Sounder's Infrared Sounder instrument, using the medium-wave infrared channel, which measures humidity in Earth's atmosphere. Blue colours correspond to regions in the atmosphere with higher humidity, while red colours correspond to lower humidity in the atmosphere.

The outlines of landmasses are not visible in this image. The areas of least humidity, shown in dark red, are seen approximately over the Sahara Desert and the Middle East (top of image), while a large area of low humidity also covers part of the South Atlantic Ocean (centre of image). Numerous patches of high humidity are seen in dark blue over the eastern part of the African continent as well as in high and low latitudes.

CREDIT

Data processed by industrial partners Thales and OHB, under the supervision of Eumetsat and ESA. Visual produced by Eumetsat.

1 edition 2026

For Maureen: without her support, none of this would have been possible.

Preface

In the beginning, it is said, we only had zeros. Then ones were invented and we could start getting work done. When God created Fortran and gave us Fortran Formatted I/O, we could start to share data with each other. That is, as long as the person you wanted to share with was using the same hardware floating point format, the same hardware endianness, the same operating system and the same compiler. And lastly, you had to go find the program that wrote the data, and copy the Fortran format statement into your reading program. Then you could share data. This worked until the total number of programmers became greater than 42, and you couldn't remember all of their names.

So this was the problem that NetCDF (and many others) set out to solve. Create a binary file format for scientific data that could be shared across hardware, software, and the great religious chasms known as programming languages. NetCDF-3 followed the Fortran-77 data model of fixed length rectangular arrays of numbers in row-major order. The NetCDF "Classic Format" specification fits on a single one-sided page of paper. When I first implemented a NetCDF-3 reader in Java, it took only a few hundred lines of a notoriously verbose language. Like all great technology, it made itself invisible by just reliably working.

Other scientific data formats from the same time period as NetCDF were targeted for specific uses, such as FITS for astronomical data, GeoTIFF for image data, and meteorological data from the World Meteorological Organization (WMO).

WMO needed a variable length binary format for weather observations from around the world, with severe constraints on communication bandwidth. They also needed an exchange file format for georeferenced 2-D gridded data. These became BUFR and GRIB, respectively. Both rely on metadata tables (that are separately stored and maintained from the data) in order to know what the data means, aka semantics, and in BUFR's case, to be able to even read the data, aka syntax. A correctly managed central repository could have made the dependence on external tables a non-problem, and indeed the formal design requires just that. The communities of practice writing these files, usually national weather services, failed rather miserably on this account, caught between the rock of time critical operational services and the hard place of plodding world bureaucracies. Certain weather services are better than others, and for sure the later versions of GRIB and BUFR offer some improvements. Unfortunately to this day,

to reliably read data in BUFR or GRIB from a source you don't personally have experience with requires tracking down the program that wrote the data, and finding and copying their table into your reading software. Plus ça change ...

Along with NetCDF, HDF also was designed as a general purpose scientific data format. While NetCDF-3 followed the KISS principle, HDF went for the feature rich, kitchen sink approach. The first version, HDF4, was used extensively by NASA's Earth Observing System, who built a specialized software layer on top called HDF-EOS, providing georeferencing, coordinate transformations, and other complex services specialized for NASA data. HDF4 was not able to evolve to keep pace with increasingly complex data requirements and had to be abandoned in favor of a completely redesigned HDF5.

HDF5 has been a major success and is widely used, not only for earth science and remote sensing, but also bioinformatics, physics, financials, aerospace etc. Its APIs are correspondingly more complex and general purpose to a fault.

The strength of NetCDF-3 is its simple format and its use of arbitrary key/value metadata pairs called attributes. Attributes allow the user to document their data in whatever way they like, no permission needed from the NetCDF library or from the WMO committee on What Attributes Are Allowed and What They Mean.

The weakness of NetCDF-3 is its simple format and its use of arbitrary key/value metadata pairs called attributes. Arbitrary means it is up to the writer as to what attributes are put into the file. We say you can use NetCDF to write crap Fortran-like files, just like you can use modern languages to write crap spaghetti-code Fortran-like programs. NetCDF solves the syntactical problems of reading shared data, but not the semantic problem of what it means. WMO's efforts to control the metadata wasn't misguided, just poorly engineered. In the place of central tables, sets of semantic conventions for NetCDF have emerged to specify metadata required for specific scientific communities. The best example of these are the Climate and Forecast (CF) conventions for climate and forecast model data output. These communities are like "vertical markets", with NetCDF as the common syntactic layer.

The simplicity of NetCDF's Fortran-77 data model eventually found its limits in exoscale data sets that beg for more sophisticated data structures and ways to speed up data access. Here's where Ed Hartnett's NetCDF story begins. Ed merged the NetCDF-3 and HDF5 data models and APIs to create NetCDF-4 in a way that keeps the essential simplicity of NetCDF-3 and hides as much of the complexity of the HDF5 APIs as possible. He built the NetCDF-4 library so it could use both the classic NetCDF-3 file format and the HDF5 format as its storage layer, and the user need only build one library to read both.

The NetCDF-4 data model adds hierarchical groups, variable length data, and "user defined" types called Compounds, aka structures or records. The data format inherits the best features of HDF5, such as data chunking and compression. It hides some of the features of HDF5 that create unneeded complexity, such as cyclical groups and multiple names for the same variable.

NetCDF-4 is surprisingly not a strict subset of HDF5. NetCDF has a notion of *coordinate variables* which assign coordinate values to each index in a data array. HDF5 tried to do something more general with *dimension scales*, but missed the boat by not making them correspond one-to-one with array indices. The Netcdf-4 library fixes the problem under the covers, but unless you know the intentions of the data writer, the HDF5 data model alone does not give you coordinate variables. You need the Netcdf-4 data model.

Details aside, NetCDF-4, along with metadata conventions such as CF, is an important contribution to writing earth science data in a way that will be readable in 100 years. Ed continues to make that a reality with this book and many other contributions over the years.

To allow data to be read for an indefinite time into the future is not easy. You can't count on your hardware, OS, or programming language to be around. The software library you are working so assiduously on may not be compilable in the future. The only thing you

can be sure of is the data format itself: the way that the data is laid out in a linear sequence of bytes. The data model is a human-readable semantic description of the data format. The format specification is the syntactical description. These two things are the core documents of any data format. The API is a language specific implementation of them. Your future self must be able to reconstruct your library using only these. That means you can't fix problems just in the software library. If there is an error or ambiguity in the file format, you must fix it there and create a new version and document the change.

Ed Hartnett has the luxury of having a commanding height overview of what needs to be documented and why, so that irreplaceable scientific data is not lost during the dark ages. Here is his version of that documentation. Long may it live and prosper.

- John Caron, April 2026

Table of Contents

Chapter 1 Introduction.....	1
Learning Objectives.....	1
What is NetCDF?.....	1
Why Does NetCDF Exist?.....	2
Who Uses NetCDF?.....	2
NASA Earth Observing System (EOSDIS).....	2
NOAA Weather and Climate.....	2
ECMWF and European Climate Infrastructure.....	3
Global Climate Modeling (CMIP).....	3
NCAR and the University Research Community.....	3
Oceanography.....	3
What All NetCDF Communities Have in Common.....	4
Who is this Book for?.....	4
Example Programs.....	4
Quick Start in C.....	5
Quick Start in Fortran.....	8
Reader Roadmap.....	11
Introduction – Key Takeaways.....	11
Chapter 2 Obtaining NetCDF.....	13
Learning Objectives.....	13
Dependencies.....	13
Choosing Your Installation Method.....	14
Package Managers vs. Building from Source.....	15
A Note on Multiple Methods.....	16
Unix Systems.....	16
Package Management Systems.....	16
Windows.....	17
vcpkg.....	17
MSYS2.....	17
Cygwin.....	17
Windows Subsystem for Linux (WSL).....	18

Conda (cross-platform).....	18
Spack (HPC environments).....	18
Building from Source.....	19
Building from Source with Autotools.....	19
Building from Source with CMake.....	21
Building from Source on Windows.....	22
Building for Parallel I/O with MPI.....	23
The Parallel Software Stack.....	23
Prerequisites.....	23
Building HDF5 with Parallel Support.....	23
Building PnetCDF (Optional).....	24
Building netCDF-Fortran with Parallel Support.....	24
Installing NetCDF-Java.....	25
Prerequisites.....	25
Adding NetCDF-Java to a Maven Project.....	25
Adding NetCDF-Java to a Gradle Project.....	25
Choosing the Right Artifact.....	26
Manual JAR Download.....	26
NetCDF-4 Write Support via JNI.....	26
ToolsUI.....	27
Verifying Your Installation and Troubleshooting.....	27
Checking nc-config and nf-config.....	27
C Smoke Test.....	27
Fortran Smoke Test.....	28
Verifying Parallel Support.....	28
Parallel I/O Smoke Test.....	28
Common Problems.....	29
Obtaining NetCDF – Key Takeaways.....	32
Chapter 3 NetCDF Data Models.....	33
Learning Objectives.....	33
Example Code.....	33
Introduction to Data Models.....	33
The Classic Model.....	34
The File.....	36
Dimensions.....	38

Variables.....	39
Attributes.....	42
Fill Values and Fill Mode.....	44
Reading and Writing Data.....	46
The Enhanced Model.....	48
Why the Enhanced Model Was Needed.....	49
Compatibility Considerations.....	51
Multiple Unlimited Dimensions.....	51
Groups.....	52
New Atomic Types.....	53
User Defined Types and Strings.....	54
NetCDF-4 Attribute Functions.....	60
Reading and Writing Data with the Enhanced Model.....	61
Creating a NetCDF-4 File.....	61
Writing and Reading New Atomic Types.....	62
Variables in Groups.....	64
Appending Along Multiple Unlimited Dimensions.....	65
Writing and Reading Compound Variables.....	65
Writing and Reading VLEN Variables.....	67
Writing and Reading String Variables.....	68
Writing and Reading Enum Variables.....	69
Writing and Reading Opaque Variables.....	70
Summary of I/O Support by Type and Language.....	72
Choosing Your Data Model.....	72
Decision Framework.....	72
Use Case Examples.....	73
Performance Considerations.....	74
NetCDF Data Models – Key Takeaways.....	74
Chapter 4 Binary Formats.....	75
Learning Objectives.....	75
History of NetCDF Binary Format Development.....	75
Opening NetCDF Files of Different Formats.....	76
Choosing a Format.....	76
Format Details.....	77
Classic Format.....	78
CDF-5.....	79

NetCDF-4/HDF5 Format.....	79
Storage Architecture.....	80
Creating and Inspecting NetCDF-4/HDF5 Files.....	81
ncZarr Cloud Format.....	82
Zarr Storage Model.....	82
Using ncZarr.....	83
Zarr and the Enhanced Model.....	83
When Not to Use ncZarr.....	83
Converting Between Formats.....	84
Legacy Formats.....	84
HDF4 SD (Read-Only).....	84
64-bit Offset Format.....	85
Binary Formats – Key Takeaways.....	85
Chapter 5 Command-Line Utilities.....	87
Learning Objectives.....	87
Working with NetCDF Files on the Command Line.....	87
Built-in Utilities: ncdump and ncgen.....	87
ncdump: Inspecting NetCDF Files.....	87
CDL: The Common Data Language.....	90
ncgen: Creating NetCDF Files from CDL.....	91
The ncdump/ncgen Round-Trip Workflow.....	92
The nccopy Utility.....	93
Hands-On: A Complete ncdump/ncgen/nccopy Workflow.....	96
More Utilities: the NetCDF Command Operators (NCO).....	99
Command-Line Utilities – Key Takeaways.....	100
Chapter 6 NetCDF C Examples.....	101
Classic Model Examples.....	101
Simple Example in C.....	101
Coordinate Variables.....	106
Enhanced Model Examples in C.....	112
Different NetCDF Binary Files.....	112
Compression.....	119
User-Defined Types.....	125
Groups and New Atomic Types.....	132
Chapter 7 NetCDF Fortran Examples.....	145

Fortran Classic Model Examples.....	145
Simple Example with Classic Format.....	145
Coordinate Variables.....	150
Fortran Enhanced Model Examples.....	155
Different NetCDF Binary Files.....	155
Simple File with NetCDF/HDF5 Format.....	162
Compression.....	163
User-Defined Types.....	164
Groups and New Atomic Types.....	166
Most Important Fortran API Functions.....	177
Dataset Operations.....	177
Dimension Operations.....	178
Variable Operations.....	178
Variable I/O.....	178
Attribute Operations.....	179
NetCDF-4 Group Operations.....	179
NetCDF-4 User-Defined Types.....	179
NetCDF-4 Chunking and Compression.....	179
NetCDF-4 Parallel I/O.....	180
Error Handling.....	180
Data Types.....	180
Mode Flags.....	180
Storage Options (NetCDF-4).....	181
Special Constants.....	181
Error Codes.....	181
Programming with NetCDF in Fortran – Key Takeaways.....	181
Chapter 8 Programming with NetCDF in Java.....	183
Learning Objectives.....	183
Re-implementation in Java.....	183
Understanding the NetCDF-Java API.....	183
Error Handling.....	184
Logging.....	184
Opening, Creating, and Closing Files.....	185
Opening Files with Options.....	185
Creating Files.....	186

Metadata.....	186
Reading Dimensions.....	186
Reading Variables.....	187
Reading Attributes.....	187
Writing Metadata.....	188
Metadata Best Practices.....	188
Reading and Writing Data.....	189
Reading Entire Variables.....	189
Reading Scalar Variables.....	189
Reading Subsets with Section Specification.....	189
Reading with Explicit Ranges.....	190
Accessing Array Elements.....	190
Type-Specific Access.....	190
Writing Data.....	191
Writing Subsets.....	191
Common Pitfalls.....	192
Performance Considerations.....	193
Advanced Features.....	193
NetcdfDataset for Enhanced Functionality.....	193
NetCDF Markup Language (NcML).....	194
Aggregation with NcML.....	194
Working with Groups (NetCDF-4).....	194
Multiple File Formats.....	195
Maven and Gradle Integration.....	195
Maven Configuration.....	195
Gradle Configuration.....	196
Modular Dependencies.....	196
Comparison with C and Fortran APIs.....	197
Key Differences.....	197
API Correspondence.....	197
Code Example Comparison.....	197
ToolsUI Application.....	198
Best Practices.....	198
Most Important API Classes and Methods.....	199
Core Classes.....	199
Opening and Closing Files.....	199

File Information.....	200
Variable Operations.....	200
Array Operations.....	200
Index Operations.....	201
Dimension Operations.....	201
Attribute Operations.....	201
Group Operations (NetCDF-4).....	201
Coordinate System Support (NetcdfDataset).....	201
Data Types.....	202
Array Section Syntax.....	202
Remote File Access.....	202
Programming with NetCDF in Java – Key Takeaways.....	203
Chapter 9 Attributes and Conventions.....	205
Learning Objectives.....	205
Storing Earth Science Data.....	205
The Climate and Forecast (CF) Conventions for Earth Science Data.....	205
Essential Attributes for Self-Describing Data.....	205
Variable Attributes.....	205
Global Attributes.....	206
Adding Attributes in C.....	206
Adding Attributes in Fortran.....	208
Coordinate Variables and Coordinate Systems.....	209
Defining Coordinate Variables.....	209
Auxiliary Coordinate Variables.....	209
Vertical Coordinates.....	210
Time Encoding and Calendars.....	210
Time Units.....	210
Calendar Systems.....	211
Example: Complete Time Coordinate.....	211
Missing Values and Fill Values.....	211
The _FillValue Attribute.....	211
The missing_value Attribute.....	212
Valid Range.....	212
CF Conventions in Detail.....	212
Standard Names.....	212

Units Conventions.....	213
Cell Methods.....	213
Axis Attributes.....	213
Other Common Conventions.....	213
ACDD (Attribute Convention for Data Discovery).....	213
COARDS (Cooperative Ocean/Atmosphere Research Data Service).....	214
Best Practices for Attributes and Conventions.....	214
Checking CF Compliance.....	214
Summary.....	215
Attributes & Conventions – Key Takeaways.....	215
Chapter 10 NetCDF-4/HDF5 Performance Fundamentals.....	217
Learning Objectives.....	217
Getting the Best from NetCDF.....	217
Define Mode.....	217
Chunk Sizes.....	217
Why Chunking Matters.....	218
Setting Chunk Sizes.....	218
Choosing Optimal Chunk Sizes.....	218
The Chunk Cache.....	218
Compression with Deflate.....	219
Shuffle Filter.....	219
Fill Values.....	220
Endianness.....	221
NetCDF-4/HDF5 Performance Fundamentals – Key Takeaways.....	221
Chapter 11 Parallel I/O with NetCDF.....	223
Learning Objectives.....	223
NetCDF on Supercomputers.....	223
Why Parallel I/O Matters.....	223
Building NetCDF with Parallel I/O Support.....	223
For NetCDF-4/HDF5 Parallel I/O.....	224
For PnetCDF (CDF-5) Parallel I/O.....	224
Parallel I/O with NetCDF-4/HDF5.....	224
Creating a Parallel NetCDF-4 File.....	224
Opening an Existing Parallel File.....	225

Collective vs. Independent I/O.....	225
Writing Data in Parallel.....	225
Example: Complete Parallel Write.....	226
Parallel I/O with PnetCDF (CDF-5).....	227
Using PnetCDF Through NetCDF.....	227
PnetCDF Native API.....	227
Domain Decomposition Strategies.....	228
1D Decomposition (Time).....	228
2D Decomposition (Spatial).....	228
3D Decomposition.....	229
Performance Optimization for Parallel I/O.....	229
Align Chunking with Domain Decomposition.....	229
Use Collective I/O.....	230
Tune MPI-IO Hints.....	230
Avoid Small, Random Writes.....	230
Common Pitfalls and Solutions.....	230
Pitfall 1: Metadata Operations in Parallel.....	230
Pitfall 2: Mixing Collective and Independent I/O.....	231
Pitfall 3: Unbalanced Decomposition.....	231
Fortran Parallel I/O.....	231
Benchmarking Parallel I/O Performance.....	232
Summary.....	232
Parallel I/O – Key Takeaways.....	233
Chapter 12 Advanced Compression Techniques.....	235
Learning Objectives.....	235
Compression in the Modern Age.....	235
NetCDF-4.9.0 - zstd and quantization.....	235
The NetCDF Expansion Pack (NEP) - lz4 and More.....	236
Choosing a Compression Algorithm.....	236
Advanced Compression – Key Takeaways.....	236
Chapter 13 NetCDF Architecture and Extensibility.....	239
Learning Objectives.....	239
Extending NetCDF.....	239
The Dispatch Table.....	239

How It Works.....	239
Format Detection.....	240
Extensibility.....	240
User-Defined Formats.....	241
The V2 API.....	241
Architecture & Extensibility – Key Takeaways.....	241
Chapter 14 Remote Data Access with OPeNDAP.....	243
Learning Objectives.....	243
Getting Remote Data.....	243
What is OPeNDAP?.....	243
Client/Server Architecture.....	243
OPeNDAP URLs.....	244
URL Service Endpoints.....	244
Constraint Expressions.....	245
Using OPeNDAP with NetCDF Programs.....	245
Constraint Expressions in URLs.....	246
Using OPeNDAP with Fortran Programs.....	246
Exploring Remote Datasets.....	247
Using a Web Browser.....	247
Using ncdump.....	247
Using .info Endpoint.....	247
Performance Considerations.....	248
Request Only What You Need.....	248
Minimize the Number of Requests.....	248
Request Multiple Variables Together.....	248
Use Compression-Aware Servers.....	249
DAP2 vs DAP4.....	249
DAP2 (Data Access Protocol 2).....	249
DAP4 (Data Access Protocol 4).....	249
Common Issues and Troubleshooting.....	250
URL Not Recognized.....	250
Network Timeouts.....	250
Authentication.....	250
Constraint Expression Errors.....	250
OPeNDAP in Scientific Workflows.....	250

Climate Data Analysis.....	251
Satellite Data Distribution.....	251
Real-Time Data Access.....	251
Federated Data Systems.....	251
Best Practices.....	251
Summary.....	252
Remote Data Access (OPeNDAP) – Key Takeaways.....	252
Chapter 15 Performance Testing and Benchmarking.....	253
Learning Objectives.....	253
Testing Performance.....	253
Why Benchmark?.....	253
Basic Timing Techniques.....	253
Using the time Command.....	253
Timing in C Code.....	254
Timing in Fortran Code.....	254
Measuring File Size and Compression Ratio.....	255
Benchmarking Chunking Strategies.....	255
Example: Time Series Data.....	255
Guidelines for Chunk Size Testing.....	256
Benchmarking Compression Settings.....	256
Compression Benchmarking Results Interpretation.....	256
Testing Access Patterns.....	257
Pattern 1: Sequential Time Slice Access.....	257
Pattern 2: Random Spatial Subsets.....	257
Pattern 3: Time Series at Points.....	257
Chunk Cache Tuning.....	258
Cache Size Guidelines.....	258
Creating a Benchmark Suite.....	258
Interpreting Benchmark Results.....	260
Write Performance.....	260
Read Performance.....	260
File Size.....	260
Overall Efficiency.....	260
Best Practices for Benchmarking.....	260
Example Benchmark Results.....	261

Automated Testing.....	261
Performance Testing & Benchmarking – Key Takeaways.....	262
Chapter 16 The Past, Present, and Future of NetCDF.....	263
Learning Objectives.....	263
How We Got Here.....	263
The Past.....	263
Ancient Times.....	263
NetCDF-3 Era.....	263
NetCDF-4 Era.....	264
Modern Times.....	264
Key Milestones.....	265
The Present.....	269
The Future.....	271
The Cloud.....	271
Compression.....	271
The NetCDF Expansion Pack.....	271
Parallel I/O.....	272
Data, Data, and More Data.....	272
The Past, Present, and Future of NetCDF – Key Takeaways.....	272

Chapter 1 Introduction

Learning Objectives

- Understand NetCDF's purpose and core capabilities
- Recognize the scale of modern scientific data challenges
- Identify target audience and book structure

What is NetCDF?

NetCDF is a family of scientific data formats and software libraries used to store, share, and document array-oriented data. It is most common in Earth science, climate, oceanography, and remote sensing, but it applies to any field that produces gridded or time-varying measurements.

At its core, netCDF provides a simple data model:

- Dimensions define the axes of data, such as time, latitude, longitude, or height.
- Variables are multi-dimensional arrays that use those dimensions, such as temperature(time, lat, lon).
- Attributes are named metadata attached to a file or a variable, used for units, standard names, coordinate references, and provenance.

This model is the reason netCDF scales well. It keeps data and metadata together, so a file remains understandable even years later, even if the original documentation is gone.

NetCDF also emphasizes portability. A netCDF file written on one machine can be read on another without worrying about byte order or floating-point representation. That mattered when labs routinely exchanged data across incompatible systems, and it still matters when archives must remain readable for decades.

Finally, netCDF is not just a file format. It is an ecosystem:

- C and Fortran libraries provide low-level, high-performance access for model and instrument pipelines.
- NetCDF-Java supports many community conventions and is widely used for analysis and data services.
- Command-line tools like `ncdump` and `ncgen` make it easy to inspect and generate files without writing code.

Together, the format, the data model, and the APIs make netCDF a practical standard for scientific data that must be large, portable, and self-describing.

Why Does NetCDF Exist?

Before common data formats like netCDF, every instrument and model produced data in its own format. Metadata lived in separate documentation, if it existed at all, and when that documentation was lost, the data became useless.

NetCDF solved three problems at once. Its data model was simple: arrays with named dimensions and descriptive attributes. Its files were machine-independent, so a file written on a Cray could be read on a PC without byte-swapping code. And its metadata was self-describing. Units, coordinates, and provenance traveled with the data, not on a floppy disk that someone might lose.

The original classic formats hit a wall at 2 GB. In 2008, NetCDF-4.0 switched to HDF5 as a storage backend, removing all size limits and adding compression, groups, new types, and parallel I/O. NASA, ESA, and NOAA adopted it as a standard.

The data keep growing. Higher instrument resolution, cheaper satellite launches, and faster supercomputers all drive exponential increases. Doubling the resolution of an atmospheric model in every dimension produces 16 times more data.

NetCDF continues to evolve, with better performance and a growing ecosystem of tools, to keep pace. For a detailed history, see Chapter 16 (*The Past, Present, and Future of NetCDF*).

Who Uses NetCDF?

NetCDF is embedded in the infrastructure of Earth science. The following communities represent its largest users.

NASA Earth Observing System (EOSDIS)

NASA's EOSDIS manages over 100 petabytes of Earth science data across twelve Distributed Active Archive Centers, serving millions of data requests per year. NetCDF-4 is a primary distribution format for satellite missions including Aqua, Terra, Suomi NPP, and the Joint Polar Satellite System. When NASA's Earth science community standardized on HDF5 and NetCDF-4 for its next-generation data products, it cemented netCDF's role at the center of global remote sensing.

If you work with satellite Level 2 products or large HDF5-based archives, see Chapter 10 (*NetCDF-4/HDF5 Performance Fundamentals*) and Chapter 14 (*Remote Data Access with OPeNDAP*).

NOAA Weather and Climate

NOAA's operational weather models (including the Global Forecast System (GFS), the High-Resolution Rapid Refresh (HRRR), and the Unified Forecast System (UFS)) all produce gridded output in netCDF. Climate Data Records, reanalysis products, and ocean observations rely on netCDF for long-term archival and distribution through the National Centers for Environmental Information. The GOES-R satellite series, which produces approximately 2 TB of data per day, distributes all Level 2 products in netCDF-4; compression and chunking make this volume practical to store and deliver.

For operational model output pipelines and GOES-R data, see Chapter 7 (*NetCDF Fortran Examples*), Chapter 12 (*NetCDF-4/HDF5 Performance Fundamentals*), and Chapter 13 (*Parallel I/O with NetCDF*).

ECMWF and European Climate Infrastructure

The European Centre for Medium-Range Weather Forecasts produces ERA5, widely considered the most important climate reanalysis dataset, covering the global atmosphere at 31 km resolution from 1940 to the present. ERA5 and dozens of other climate datasets are distributed in netCDF format through the Copernicus Climate Data Store to hundreds of thousands of users worldwide. The European Space Agency's Climate Change Initiative standardized on netCDF with CF conventions for satellite-derived climate records covering sea surface temperature, ice sheets, greenhouse gases, and more.

For working with reanalysis datasets and CF-compliant satellite records, see Chapter 9 (*Attributes and Conventions*) and Chapter 14 (*Remote Data Access with OPeNDAP*).

Global Climate Modeling (CMIP)

The Coupled Model Inter-comparison Project coordinates climate model experiments across dozens of modeling centers worldwide. Its output forms the scientific basis for IPCC assessment reports. All CMIP data is stored and distributed in netCDF following strict CF and CMIP conventions. CMIP6 produced over 20 petabytes of model output from more than 100 distinct climate models, making it one of the largest coordinated uses of netCDF in existence.

For producing or consuming CF/CMIP compliant model output, see Chapter 9 (*Attributes and Conventions*), Chapter 12 (*Advanced Compression Techniques*), and Chapter 11 (*Parallel I/O with NetCDF*).

NCAR and the University Research Community

The National Center for Atmospheric Research hosts Unidata, the program that created and maintains the netCDF library. The Community Earth System Model (CESM), the Weather Research and Forecasting model (WRF), and the Model for Prediction Across Scales (MPAS) all use netCDF for input and output. Thousands of university researchers worldwide read and write netCDF daily in climate modeling, atmospheric chemistry, and hydrology.

For building research workflows around WRF, CESM, or MPAS, see Chapter 6 (*NetCDF C Examples*), Chapter 7 (*NetCDF Fortran Examples*), and Chapter 15 (*Performance Testing and Benchmarking*).

Oceanography

Physical, chemical, and biological oceanography adopted netCDF early and thoroughly. The Argo program distributes data from nearly 4,000 autonomous profiling floats in netCDF. NOAA's Ocean Climate Laboratory, the Copernicus Marine Service, and the Ocean Observatories Initiative all use netCDF as their primary data format. The CF conventions that govern most netCDF metadata originated in the oceanographic and climate modeling communities.

For Argo float data and ocean observing networks, see Chapter 3 (*NetCDF Data Models*), Chapter 9 (*Attributes and Conventions*), and Chapter 5 (*Command-Line Utilities*).

What All NetCDF Communities Have in Common

These communities share a common pattern: they produce enormous volumes of array-oriented data, they need that data to be portable across platforms and decades, and they require self-describing metadata so that files remain usable long after the people who created them have moved on. NetCDF was built for exactly this problem.

Who is this Book for?

This book is for scientists and programmers who work with netCDF data and want to get the job done efficiently. Whether you need to read a single file or build a pipeline that processes terabytes, the goal is the same: spend less time on data plumbing and more time on science.

The programming chapters assume you can read C or Fortran code and are comfortable compiling programs from the command line. Java coverage is included for NetCDF-Java users.

By bringing together the most current information in one place, this book aims to flatten the learning curve. Time spent on netCDF programming is time taken away from real science, and the fastest path through is understanding how the format works and what tools are available.

A good understanding of the netCDF ecosystem, and the tools already available, will save researchers a great deal of time. There is no need to spend weeks writing a utility program in Fortran, when one of the built-in command line tools, or tools from the NCO library, can fill the need without custom code.

Example Programs

Most of the C and Fortran examples in this book come from the examples directory of the NetCDF Expansion Pack (NEP). The NEP project can be found on GitHub.

NEP extends NetCDF-4 with powerful new capabilities for scientific data workflows:

- Ultra-Fast LZ4 Compression: 2-3x faster than DEFLATE with excellent compression ratios - ideal for real-time data processing and HPC workflows
- High-Ratio BZIP2 Compression: Superior compression for archival storage - reduce storage costs while maintaining data integrity
- NASA CDF File Reader: Access Common Data Format files directly through the familiar NetCDF API - no conversion needed
- GeoTIFF File Reader: Access GeoTIFF files, as if they are netCDF, with full CF metadata.
- Drop-In Compatibility: Works with existing NetCDF C and Fortran applications without code changes

NEP can be installed from source with Cmake or autotools. It is also supported by spack.

Quick Start in C

Fundamentally, netCDF is about storing arrays of data (“variables”). Arrays have axes (“dimensions”), and we can attach metadata to files and arrays (“attributes”).

In the following simple example, we create a netCDF file with a variable “data”, two dimensions “X” and “Y”, and some attributes to demonstrate how metadata may be associated with files or arrays. We then close and re-open the file, and read all metadata and data, and checking that the values are all correct.

This example demonstrates how to create, write, and and close, then open, read, and close a netCDF file.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <netcdf.h>

#define FILE_NAME "quickstart.nc"
#define ERRCODE 2
#define ERR(e) {printf("Error: %s\n", nc_strerror(e)); return ERRCODE;}

int main()
{
    int ncid, x_dimid, y_dimid, data_varid;
    int dimids[2];
    int retval;

    int data_out[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int data_in[2][3];

    /* ===== WRITE PHASE ===== */
    printf("Creating NetCDF file: %s\n", FILE_NAME);

    /* Create the NetCDF file (NC_CLOBBER overwrites existing file) */
    if ((retval = nc_create(FILE_NAME, NC_CLOBBER, &ncid)))
        ERR(retval);

    /* Define dimensions: X=2, Y=3 */
    if ((retval = nc_def_dim(ncid, "X", 2, &x_dimid)))
        ERR(retval);
    if ((retval = nc_def_dim(ncid, "Y", 3, &y_dimid)))
        ERR(retval);

    /* Define the variable with dimensions X and Y */
    dimids[0] = x_dimid;
    dimids[1] = y_dimid;
    if ((retval = nc_def_var(ncid, "data", NC_INT, 2, dimids, &data_varid)))
        ERR(retval);

    /* Add global attribute */
    if ((retval = nc_put_att_text(ncid, NC_GLOBAL, "description",
                                strlen("a quickstart example"), "a quickstart example")))
        ERR(retval);

    /* Add variable attribute */
    if ((retval = nc_put_att_text(ncid, data_varid, "units",
                                strlen("m/s"), "m/s")))
        ERR(retval);
```

```

/* End define mode - ready to write data */
if ((retval = nc_enddef(ncid)))
    ERR(retval);

/* Write the data to the file */
if ((retval = nc_put_var_int(ncid, data_varid, &data_out[0][0])))
    ERR(retval);

/* Close the file */
if ((retval = nc_close(ncid)))
    ERR(retval);

printf("*** SUCCESS writing file!\n");

/* ===== READ PHASE ===== */
printf("\nReopening file for validation...\n");

/* Open the file for reading */
if ((retval = nc_open(FILE_NAME, NC_NOWRITE, &ncid)))
    ERR(retval);

/* Verify metadata: check number of dimensions, variables, and attributes */
int ndims_in, nvars_in, ngatts_in;
if ((retval = nc_inq(ncid, &ndims_in, &nvars_in, &ngatts_in, NULL)))
    ERR(retval);

printf("File contains: %d dimensions, %d variables, %d global attributes\n",
       ndims_in, nvars_in, ngatts_in);

if (ndims_in != 2 || nvars_in != 1 || ngatts_in != 1) {
    printf("Error: Unexpected file structure\n");
    return 1;
}

/* Verify dimension sizes */
size_t len_x, len_y;
if ((retval = nc_inq_dimlen(ncid, x_dimid, &len_x)))
    ERR(retval);
if ((retval = nc_inq_dimlen(ncid, y_dimid, &len_y)))
    ERR(retval);

if (len_x != 2 || len_y != 3) {
    printf("Error: Expected dimensions X=2, Y=3, found X=%zu, Y=%zu\n", len_x, len_y);
    return 1;
}
printf("Verified: X=%zu, Y=%zu\n", len_x, len_y);

/* Verify global attribute */
char desc_in[100];
size_t desc_len;
if ((retval = nc_inq_attlen(ncid, NC_GLOBAL, "description", &desc_len)))
    ERR(retval);
if ((retval = nc_get_att_text(ncid, NC_GLOBAL, "description", desc_in)))
    ERR(retval);
desc_in[desc_len] = '\0';
if (strcmp(desc_in, "a quickstart example") != 0) {
    printf("Error: expected description 'a quickstart example', got '%s'\n", desc_in);
    return 1;
}
}

```



```

printf("Verified: global attribute 'description' = '%s'\n", desc_in);

/* Verify variable attribute */
char units_in[100];
size_t units_len;
if ((retval = nc_inq_attlen(ncid, data_varid, "units", &units_len)))
    ERR(retval);
if ((retval = nc_get_att_text(ncid, data_varid, "units", units_in)))
    ERR(retval);
units_in[units_len] = '\0';
if (strcmp(units_in, "m/s") != 0) {
    printf("Error: expected units 'm/s', got '%s'\n", units_in);
    return 1;
}
printf("Verified: variable attribute 'units' = '%s'\n", units_in);

/* Read the data back */
if ((retval = nc_get_var_int(ncid, data_varid, &data_in[0][0])))
    ERR(retval);

/* Verify data correctness */
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        if (data_in[i][j] != data_out[i][j]) {
            printf("Error: data[%d][%d] = %d, expected %d\n",
                    i, j, data_in[i][j], data_out[i][j]);
            return 1;
        }
    }
}

printf("Verified: all 6 data values correct (1, 2, 3, 4, 5, 6)\n");

/* Close the file */
if ((retval = nc_close(ncid)))
    ERR(retval);

printf("\n*** SUCCESS: All validation checks passed!\n");
return 0;
}

```

After creating this file, we can use the built-in command line utility `ncdump` to see what it contains. The `ncdump` command line utility comes with `netcdf-c` and is installed in the `bin` directory of the `netcdf-c` install (you may have to include this `bin` directory in your `PATH`).

The `ncdump` utility will be fully explained in Chapter 5 (*Command Line Utilities*). Briefly, it dumps the contents of a netCDF file to `stdout` as ASCII. If the file contains much data, this dump is far too large for use, but the `-h` option causes `ncdump` to print only the metadata of the file. The `ncdump -h` output for the file produced by the quickstart program is:

```

netcdf quickstart {
dimensions:
    X = 2 ;
    Y = 3 ;
variables:
    int data(X, Y) ;
        data :units = "m/s" ;

// global attributes:

```

```

        :description = "a quickstart example" ;
data:

  data =
    1, 2, 3,
    4, 5, 6 ;
}

```

Quick Start in Fortran

The same quick start program in Fortran looks like this:

```

program f_quickstart
  use netcdf
  implicit none

  character(len=*), parameter :: FILE_NAME = "f_quickstart.nc"
  integer, parameter :: XDIM = 2, YDIM = 3

  integer :: ncid, x_dimid, y_dimid, data_varid
  integer :: dimids(2)
  integer :: retval

  integer :: data_out(XDIM, YDIM)
  integer :: data_in(XDIM, YDIM)

  integer :: i, j
  integer :: ndims_in, nvars_in, ngatts_in
  integer :: len_x, len_y
  character(len=100) :: desc_in, units_in

  ! ===== WRITE PHASE =====
  print *, "Creating NetCDF file: ", FILE_NAME

  ! Create the NetCDF file (nf90_clobber overwrites existing file)
  retval = nf90_create(FILE_NAME, nf90_clobber, ncid)
  call check(retval, "creating file")

  ! Define dimensions: X=2, Y=3
  retval = nf90_def_dim(ncid, "X", XDIM, x_dimid)
  call check(retval, "defining X dimension")

  retval = nf90_def_dim(ncid, "Y", YDIM, y_dimid)
  call check(retval, "defining Y dimension")

  ! Define the variable with dimensions X and Y
  ! Note: Fortran column-major order means first dimension varies fastest
  dimids(1) = x_dimid
  dimids(2) = y_dimid
  retval = nf90_def_var(ncid, "data", nf90_int, dimids, data_varid)
  call check(retval, "defining variable")

  ! Add global attribute
  retval = nf90_put_att(ncid, nf90_global, "description", "a quickstart example")
  call check(retval, "adding global attribute")

  ! Add variable attribute
  retval = nf90_put_att(ncid, data_varid, "units", "m/s")
  call check(retval, "adding variable attribute")

```

```

! End define mode - ready to write data
retval = nf90_enddef(ncid)
call check(retval, "ending define mode")

! Generate data: sequential integers (1, 2, 3, 4, 5, 6)
! Fortran column-major: fill by column to match C row-major layout
do j = 1, YDIM
  do i = 1, XDIM
    data_out(i, j) = (j-1) * XDIM + i
  end do
end do

! Write the data to the file
retval = nf90_put_var(ncid, data_varid, data_out)
call check(retval, "writing data")

! Close the file
retval = nf90_close(ncid)
call check(retval, "closing file")

print *, "**** SUCCESS writing file!"

! ===== READ PHASE =====
print *, ""
print *, "Reopening file for validation..."

! Open the file for reading
retval = nf90_open(FILE_NAME, nf90_nowrite, ncid)
call check(retval, "reopening file")

! Verify metadata: check number of dimensions, variables, and attributes
retval = nf90_inquire(ncid, ndims_in, nvars_in, ngatts_in)
call check(retval, "inquiring file")

print *, "File contains:", ndims_in, "dimensions,", nvars_in, "variables,", &
      ngatts_in, "global attributes"

if (ndims_in /= 2 .or. nvars_in /= 1 .or. ngatts_in /= 1) then
  print *, "Error: Unexpected file structure"
  stop 1
end if

! Verify dimension sizes
retval = nf90_inquire_dimension(ncid, x_dimid, len=len_x)
call check(retval, "inquiring X dimension")

retval = nf90_inquire_dimension(ncid, y_dimid, len=len_y)
call check(retval, "inquiring Y dimension")

if (len_x /= XDIM .or. len_y /= YDIM) then
  print *, "Error: Expected dimensions X=", XDIM, ", Y=", YDIM, &
    ", found X=", len_x, ", Y=", len_y
  stop 1
end if
print *, "Verified: X=", len_x, ", Y=", len_y

! Verify global attribute
retval = nf90_get_att(ncid, nf90_global, "description", desc_in)
call check(retval, "reading global attribute")
if (trim(desc_in) /= "a quickstart example") then

```

```

    print *, "Error: expected description 'a quickstart example', got '", trim(desc_in), ""
    stop 1
end if
print *, "Verified: global attribute 'description' = '", trim(desc_in), ""

! Verify variable attribute
retval = nf90_get_att(ncid, data_varid, "units", units_in)
call check(retval, "reading variable attribute")
if (trim(units_in) /= "m/s") then
    print *, "Error: expected units 'm/s', got '", trim(units_in), ""
    stop 1
end if
print *, "Verified: variable attribute 'units' = '", trim(units_in), ""

! Read the data back
retval = nf90_get_var(ncid, data_varid, data_in)
call check(retval, "reading data")

! Verify data correctness
do j = 1, YDIM
    do i = 1, XDIM
        if (data_in(i, j) /= data_out(i, j)) then
            print *, "Error: data(", i, ",", j, ") = ", data_in(i, j), &
                ", expected ", data_out(i, j)
            stop 1
        end if
    end do
end do

print *, "Verified: all 6 data values correct (1, 2, 3, 4, 5, 6)"

! Close the file
retval = nf90_close(ncid)
call check(retval, "closing file after reading")

print *, ""
print *, "**** SUCCESS: All validation checks passed!"
contains
!> @brief Error handling subroutine
!! @param status NetCDF return status code
!! @param context Descriptive message about the operation
subroutine check(status, context)
    integer, intent(in) :: status
    character(len=*), intent(in) :: context

    if (status /= nf90_noerr) then
        print *, "Error ", trim(context), ": ", trim(nf90_strerror(status))
        stop 2
    end if
end subroutine check
end program f_quickstart

end program f_quickstart

The ncdump output from the netCDF file created here shows:
netcdf f_quickstart {
dimensions:
    X = 2 ;

```

```

        Y = 3 ;
variables:
    int data(X, Y) ;
        data :units = "m/s" ;

// global attributes:
        :description = "a quickstart example" ;
data:

    data =
        1, 2, 3,
        4, 5, 6 ;
}

```

Reader Roadmap

If you want to install netCDF: Chapter 2

If you want to understand data models and formats: Chapters 3-4

If you want to use command-line tools: Chapter 5

If you want to write programs in C: Chapter 6

If you want to write programs in Fortran: Chapter 7

If you want to write programs in Java: Chapter 8

If you want to learn about attributes and conventions: Chapter 9

If you want netCDF-4/HDF5 performance (chunking and compression): Chapter 10

If you want parallel I/O for HPC: Chapter 11

If you want advanced compression techniques: Chapter 12

If you want internals and extensibility: Chapter 13

If you want remote data access with OPeNDAP: Chapter 14

If you want performance testing and benchmarking: Chapter 15

If you want to learn about the past and future of NetCDF: Chapter 16

Introduction – Key Takeaways

- **Self-describing and portable:** NetCDF files carry their metadata with them — dimensions, units, coordinate systems, and provenance — so data remains usable across machines and decades without external documentation.
- **Built for scale:** NetCDF-4 removed the classic format's size limits and added compression, chunking, and parallel I/O, enabling it to handle TB-scale daily output from modern instruments like GOES-R.
- **Ubiquitous in Earth science:** NASA, NOAA, ECMWF, ESA, and the global climate modeling community (CMIP) all standardized on netCDF, collectively managing hundreds of petabytes.
- **Three ways in:** You can work with netCDF from the command line (ncdump, ncgen), through C/ Fortran APIs, or through Java — this book covers all three paths.

- **Quick start pattern:** Create → define dimensions → define variables → add attributes → write data → close. The C and Fortran examples in this chapter demonstrate this workflow in under 50 lines each.
- **Read the chapter you need:** The Reader Roadmap on the previous page lets you jump directly to installation (Ch. 2), programming (Ch. 6–8), performance tuning (Ch. 10–11), or remote access (Ch. 14).

Chapter 2 Obtaining NetCDF

Learning Objectives

- Choose an installation method (package manager, Conda/Spack, or source build) based on your operating system, access level, and feature requirements
- Install the netCDF C and Fortran libraries on Linux, macOS, or Windows
- Identify the separate packages for the C, Fortran, and C++ netCDF interfaces and their dependency relationships
- Build HDF5 and netCDF from source using Autotools or CMake, selecting configure options for netCDF-4, parallel I/O, and other features
- Build a parallel-capable netCDF stack with MPI, parallel HDF5, and optionally PnetCDF
- Add NetCDF-Java to a Maven or Gradle project and choose the correct artifact for your needs
- Verify your installation using nc-config, nf-config, and smoke-test programs in C and Fortran
- Diagnose common build and link errors such as missing headers, unresolved symbols, and library-version mismatches

Dependencies

The netCDF C library lies at the heart of the netCDF ecosystem. Even the Java implementation uses the C library to provide write capability for netCDF-4 files.

The netcdf-c library, in turn, optionally depends on other libraries to provide advanced features. When built with no dependencies the netcdf-c library is capable of reading and writing all the classic formats, but not HDF5 or remote data.

Dependency	Capability
HDF5	Required to read/write netCDF/HDF5 files, use the enhanced model.
zlib	Required for compression in netCDF/HDF5 files, or with ncZarr.
Libcurl	Required for OpeNDAP remote access.
HDF4	Allows read-only access to HDF4 SD files.
BLOSC	Required for ncZarr.
Pnetcdf	Allows parallel I/O with classic formats. (Requires MPI)
LZ4	Required for ncZarr, optionally used by netCDF/HDF5 files.
AWS SDK	Required for ncZarr if using S3 storage.
MPI	Required for parallel I/O.

NetCDF C/Fortran Dependencies

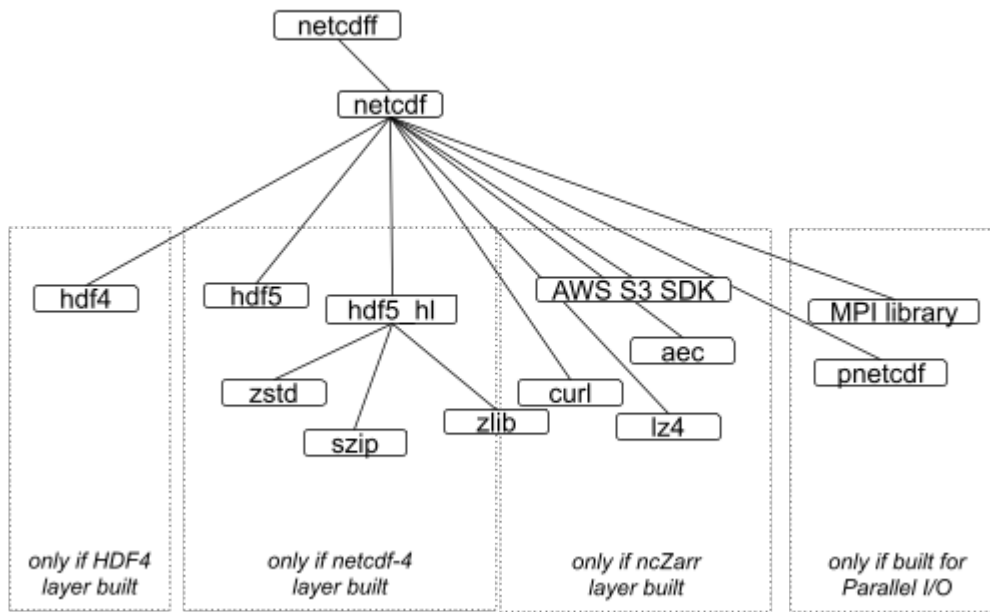


Figure 1: The netCDF ecosystem is centered on the netcdf-c library. The netcdf-c library can be built with many optional dependencies; without the related dependency, the netcdf-c feature will not be built. For example, a netcdf-c library built without the HDF5 dependency will not be able to read or write netCDF-4 files, but will still be able to read and write classic format netCDF files.

Some common ways to build the netcdf-c library include:

- build everything – have all dependencies installed before building netcdf-c.
- Build without remote access – build with HDF5, zlib, (optionally) HDF4. The library can read and write all local netCDF classic format or netCDF/HDF5 format files, but cannot access files remotely.
- Build for parallel I/O – build with MPI compilers and include HDF5, pnetCDF. The library can do parallel I/O on netCDF classic format or netCDF/HDF5 files, but can't do remote access or ncZarr.

Choosing Your Installation Method

NetCDF can be installed in many ways. The right choice depends on your operating system, whether you have administrator access, and which features you need. Use the table below to find the method that fits your situation, then skip to the corresponding section in this chapter.

Situation	Recommended Method	Section
Linux desktop or server with root access	System package manager (apt, dnf)	Unix Package Management Systems
macOS developer	Homebrew	Unix Package Management Systems
Windows with Visual Studio	vcpkg	Windows
Windows, prefer GCC and command-line tools	MSYS2	Windows
Windows, comfortable with Linux tools	WSL	Windows
Shared system or HPC cluster, no root access	Conda or Spack	Conda / Spack
Need the latest unreleased features	Build from source	Building from Source
Need custom compiler flags or optimization	Build from source	Building from Source
Need parallel I/O with MPI	Build from source with MPI	Building for Parallel I/O
Java project	Maven or Gradle	Installing NetCDF-Java

Package Managers vs. Building from Source

Package managers are the right choice for most users. They resolve the entire dependency chain (zlib, HDF5, netCDF-C, netCDF-Fortran) automatically, and they make upgrades straightforward. The trade-off is that you get whatever version and configuration the package maintainer chose.

Build from source when you need:

- **A specific version** — for example, a release that fixes a bug you hit, or a development snapshot from GitHub.
- **Custom configure options** — such as disabling netCDF-4 to eliminate the HDF5 dependency, or enabling PnetCDF for parallel classic I/O.
- **A particular compiler** — for example, building everything with Intel compilers to match the rest of your software stack.
- **Parallel I/O** — HDF5 must be built with `--enable-parallel` and an MPI compiler. Most package managers ship a serial HDF5, so parallel builds almost always require building from source.

If none of these apply, start with a package manager. You can always rebuild from source later if your needs change.

A Note on Multiple Methods

Do not mix installation methods on the same system. If you install netCDF from Homebrew and then build a second copy from source, your compiler may find headers from one installation and libraries from the other. This produces confusing linker errors. Pick one method and use it consistently. If you need to switch, remove the old installation first.

Unix Systems

Package Management Systems

The easiest way to install the netCDF C and Fortran libraries is with a package management system. NetCDF is available through most standard package managers on Unix-like systems.

Common Package Names

NetCDF is distributed as separate packages for different language interfaces:

Package Type	Purpose	Common Package Names
C library (runtime)	Core netCDF library	libnetcdf, netcdf
C library (development)	Headers and build files for C	libnetcdf-dev, netcdf-devel
Fortran library (runtime)	Fortran interface	libnetcdf, netcdf-fortran
Fortran library (development)	Headers and build files for Fortran	libnetcdf-dev, netcdf-fortran-devel
C++ library (modern)	Modern C++ interface	libnetcdf-cxx4, netcdf-cxx4
C++ library (legacy)	Legacy C++ interface	libnetcdf-cxx-legacy

Installation Examples

Ubuntu/Debian (apt):

```
# Search for available packages
apt search libnetcdf
# Install C library
sudo apt install libnetcdf-dev
# Install Fortran library
sudo apt install libnetcdf-fortran-dev
```

RHEL/Fedora/CentOS (dnf/yum):

```
# Search for available packages
dnf search netcdf
# Install C library
sudo dnf install netcdf-devel
# Install Fortran library
sudo dnf install netcdf-fortran-devel
```

macOS (Homebrew):

```
# Search for available packages
brew search netcdf
# Install netCDF (includes C library)
brew install netcdf
# Install Fortran library
brew install netcdf-fortran
```

Note that the C library (libnetcdf) and Fortran libraries (libnetcdf-f) are separate packages. The Fortran library depends on the C library. The legacy C++ library (libnetcdf-cxx-legacy) and the modern C++ library (libnetcdf-c++4) are also available as separate packages on most systems.

Windows

NetCDF can be installed on Windows through several methods. Choose the one that best fits your development environment.

vcpkg

vcpkg is a C/C++ package manager that integrates with Visual Studio and CMake. It is the most straightforward way to get netCDF on Windows with a native compiler:

```
vcpkg install netcdf-c vcpkg install netcdf-cxx4
```

To use the installed libraries with CMake, pass the vcpkg toolchain file:

```
cmake .. -DCMAKE_TOOLCHAIN_FILE=C:/vcpkg/scripts/buildsystems/vcpkg.cmake
```

vcpkg resolves dependencies automatically, so HDF5 and zlib are installed for you.

MSYS2

MSYS2 provides a Unix-like environment on Windows with the MinGW-w64 toolchain. It uses pacman, the same package manager as Arch Linux:

```
pacman -S mingw-w64-x86_64-netcdf pacman -S mingw-w64-x86_64-netcdf-fortran
```

After installation, use the MinGW64 shell to compile programs. The nc-config and nf-config utilities work the same as on Linux:

```
nc-config --cflags --libs
```

MSYS2 is a good choice if you prefer GCC and a command-line workflow on Windows.

Cygwin

Cygwin provides a POSIX-compatible environment on Windows, including a full set of GNU tools. The Cygwin installer includes netCDF packages:

From the Cygwin setup program, select these packages: libnetcdf-devel libnetcdf-fortran-devel

You can also install packages from the Cygwin terminal using the command-line installer:

```
apt-cyg install libnetcdf-devel libnetcdf-fortran-devel
```

Programs compiled under Cygwin link against the Cygwin POSIX layer (cygwin1.dll) rather than producing native Windows binaries. This means Cygwin executables must be run from a Cygwin shell. If you need native Windows binaries, use MSYS2 or vcpkg instead.

Both Autotools and CMake work under Cygwin, so the Unix build-from-source instructions in this chapter apply directly. The nc-config and nf-config utilities are available after installation:

```
nc-config --version nf-config --version
```

Windows Subsystem for Linux (WSL)

WSL runs a full Linux distribution inside Windows. All of the Linux instructions in this chapter apply directly under WSL. Install Ubuntu from the Microsoft Store, then use apt as described in the Ubuntu/Debian section above:

```
sudo apt install libnetcdf-dev libnetcdf-fortran
```

WSL is the simplest option if you are already comfortable with Linux tools and do not need a native Windows binary.

Conda (cross-platform)

Conda is widely used in scientific computing and works on Linux, macOS, and Windows. It does not require root access, making it a good choice for shared systems and HPC environments. The conda-forge channel provides up-to-date netCDF packages:

```
# Install C library
conda install -c conda-forge libnetcdf

# Install Fortran library
conda install -c conda-forge netcdf-fortran

# Install both at once
conda install -c conda-forge libnetcdf netcdf-fortran
```

Conda automatically resolves dependencies, so HDF5 and zlib are installed for you. To verify the installation:

```
nc-config --version
nf-config --version
```

If you use conda environments, activate the environment before compiling programs, and use nc-config --cflags --libs to get the correct compiler and linker flags for that environment.

Spack (HPC environments)

Spack is a package manager designed for supercomputers and HPC clusters where users cannot install system packages. It builds libraries from source with fine-grained control over compilers, MPI implementations, and feature flags:

```
# Install C library with default options
spack install netcdf-c

# Install Fortran library
spack install netcdf-fortran

# Install with specific options
spack install netcdf-c +parallel-netcdf +mpi ^hdf5+mpi
After installation, load the packages into your environment:

spack load netcdf-c
```

```
spack load netcdf-fortran
```

Spack handles the entire dependency chain (zlib, HDF5, netCDF-C, netCDF-Fortran) and can build multiple versions side by side with different configurations. This is particularly useful when you need netCDF built with a specific MPI implementation or compiler to match the rest of your software stack.

Building from Source

If you want to get the very latest netCDF code, or want to use special compiler optimizations, build the code from source. The NetCDF team maintains both Autotools and Cmake build systems. Use whichever is most convenient to you: they are equivalent and support the same features.

According to the NetCDF team, the autotools build is going to be dropped eventually, leaving only the Cmake build to maintain.

Building from Source with Autotools

Building netCDF from source provides the most flexibility and control over which features are enabled. However, netCDF has dependencies that must be satisfied before building.

Note: If you installed netCDF using a package manager (as described in the previous section), the package manager automatically handles all dependencies including HDF5. You do not need to manually install HDF5. The instructions below are only for users building netCDF from source.

Building HDF5 (Required for netCDF-4 Support When Building from Source)

If you are building netCDF from source and want to use netCDF-4/HDF5 format files (which provide compression, groups, and other advanced features), you must first build and install the HDF5 library. NetCDF-4 support requires HDF5 version 1.8.0 or later, but works best with the latest version.

Option 1: Install HDF5 from Package Manager (Recommended for most users)

The easiest approach is to install HDF5 through your system's package manager:

```
# Ubuntu/Debian
sudo apt install libhdf5-dev
# RHEL/Fedora/CentOS
sudo dnf install hdf5-devel
# macOS
brew install hdf5
```

Option 2: Build HDF5 from Source

If you need a specific HDF5 version or custom configuration, you can build from source. Download HDF5 from the HDF Group website (<https://www.hdfgroup.org/downloads/hdf5/>) and follow their build instructions. HDF5 requires zlib:

```
# Install zlib development files
sudo apt-get install zlib1g-dev
```

After building HDF5, verify the installation by examining the hdf5.settings file in your HDF5 installation directory (typically /usr/local/share/ or similar). This file contains important information about how HDF5 was configured, including which features are enabled and where libraries are located.

For detailed HDF5 build instructions, see the HDF5 documentation at <https://portal.hdfgroup.org/documentation/index.html>.

The netcdf-c Library

The netcdf-c library supports a standard autotools build. Using autoconf, automake, and libtool, the netcdf-c library and tests are packaged into a tarball that can be installed on any Unix system.

Some environment variables may be set before the configure step, in order to help the build. Some of the more common variables are listed below.

Environment Variable	Purpose	Example
CPPFLAGS	C pre-processor flags. List the include directories here.	'-I/usr/local/hdf5-1.10.4/include -I/usr/local/pnetcdf-1.9.0/include'
LDFLAGS	Loader flags. List the lib directories here.	'-L/usr/local/hdf5-1.10.4/lib -L/usr/local/pnetcdf-1.9.0/lib'
CFLAGS	C compiler flags. Control optimization, debugging symbols, and warnings.	'-Wall -g'

To install:

```
./configure  
make check  
make install
```

The configure step may involve many options which control how the netCDF C library is built. There are many options; some of the more commonly-used are listed below.

Option	Effect	Notes
--prefix=/dir	Install netcdf-c library and include files under directory /dir.	/dir will be created if needed; the default is /usr/local. Under /dir make install in create directories bin, include, lib, and share.
--help	Display all available configure options.	
--disable-netcdf-4	Causes netCDF to build without HDF5.	Only classic formats are available if the library is built without netcdf-4. The DAP2 interface permits use of OPeNDAP with classic-only files.
--enable-benchmarks	Build and run benchmarking programs as part of tests.	Will cause tests to run more slowly.
--enable-hdf4	Build with HDF4 SD read-only access.	HDF4 must be built with --disable-netcdf, or it will define a netCDF API that will conflict.
--enable-parallel-tests	Run parallel I/O tests with MPI	Requires MPI build, which occurs

	using mpiexec.	when HDF5 is built with --enable-parallel and an MPI-enabled C compiler (ex. mpicc) is used.
--enable-pnetcdf	Use the parallel-netcdf library to provide parallel I/O for classic format files.	Requires MPI build, and finding parallel-netcdf header and library files.
--enable-logging	Turn on internal netcdf-c logging. User code turns on the logging with nc_set_log_level().	Causing I/O to the console will impact performance. Logging should never be left on in production systems.

The netcdf-fortran Libraries

There are two netCDF Fortran APIs. The original Fortran API, now known as the F77 API, closely echoes the C API. Every function in the C library has a close corresponding function in the F77 API.

There is also a F90 Fortran API. The F90 API makes use of features like function overloading and optional parameters. It can yield a more compact set of code than the F77 API.

In the Fortran APIs, dimensions are listed in opposite order from the C API. This matches the difference between the way C and Fortran store arrays of data. C uses a row-major ordering, and Fortran uses a column-major ordering.

NetCDF handles this by the simple expedient of reversing the order of the dimension IDs when defining a dimension from Fortran, and likewise reversing the orders of the start, count, and index arrays, where needed.

Building from Source with CMake

The netCDF-C library also supports CMake as a build system. CMake is a cross-platform build tool that generates native build files for your system. Building with CMake requires version 3.15 or later.

To build netCDF-C with CMake, first create a build directory, then run cmake to configure, build, test, and install:

```
mkdir build
cd build
cmake ..
make
ctest
make install
```

CMake accepts options on the command line to control which features are enabled. Some commonly used options are listed below.

Option	Effect	Notes
-DCMAKE_INSTALL_PREFIX=/dir	Install netCDF under directory /dir.	Default is /usr/local.

-DENABLE_NETCDF_4=ON	Enable netCDF-4/HDF5 support.	ON by default if HDF5 is found.
-DENABLE_DAP=ON	Enable OPeNDAP support.	ON by default if libcurl is found.
-DENABLE_PARALLEL4=ON	Enable parallel I/O for netCDF-4.	Requires HDF5 built with parallel support and an MPI compiler.
-DENABLE_PNETCDF=ON	Enable parallel I/O for classic formats using PnetCDF.	Requires PnetCDF library.
-DENABLE_TESTS=ON	Build and run the test suite.	ON by default.
-DHDF5_DIR=/path	Point CMake to a specific HDF5 installation.	Use when HDF5 is installed in a non-standard location.
-DCMAKE_C_FLAGS="-Wall -g"	Set C compiler flags.	Controls optimization, debugging symbols, and warnings.

If HDF5 or other dependencies are installed in non-standard locations, you can tell CMake where to find them:

```
cmake .. -DHDF5_DIR=/usr/local/hdf5-1.14.0 -DCMAKE_INSTALL_PREFIX=/usr/local/netcdf
```

To build without netCDF-4 support:

```
cmake .. -DENABLE_NETCDF_4=OFF
```

The netcdf-fortran library can also be built with CMake. After installing netcdf-c, point the Fortran build at the netcdf-c installation:

```
mkdir build
cd build
cmake .. -DCMAKE_PREFIX_PATH=/usr/local/netcdf
make
ctest
make install
```

Building from Source on Windows

The netCDF-C library supports CMake on Windows. Autotools is not available natively on Windows, so CMake is the only option for a from-source build.

The CMake build steps are the same as on Unix, with one difference: specify the build configuration explicitly:

```
mkdir build cd build cmake .. -DCMAKE_INSTALL_PREFIX=C:/netcdf cmake --build . --config Release ctest --build-config Release cmake --install . --config Release
```

If HDF5 is installed in a non-standard location, point CMake to it:

```
cmake .. -DHDF5_DIR=C:/HDF5/cmake -DCMAKE_INSTALL_PREFIX=C:/netcdf
```

Note that the netCDF-Fortran library requires a Fortran compiler on Windows. Intel Fortran (ifx) integrates with Visual Studio; gfortran is available through MSYS2.

Building for Parallel I/O with MPI

NetCDF supports parallel I/O through two mechanisms: parallel access to netCDF-4/HDF5 files, and parallel access to classic format files through PnetCDF. Both require MPI. Building a parallel-capable netCDF stack involves more steps than a serial build, because HDF5 itself must be built with parallel support.

The Parallel Software Stack

A parallel netCDF installation requires building the dependency chain with MPI support at each layer.

Prerequisites

You need an MPI implementation installed before building anything else. Most systems provide one through the package manager:

```
# Ubuntu/Debian
sudo apt install libopenmpi-dev openmpi-bin
# RHEL/Fedora/CentOS
sudo dnf install openmpi-devel
module load mpi/openmpi-x86_64
# macOS
brew install open-mpi
```

Verify MPI is working:

```
mpicc --version
mpiexec -n 2 echo "MPI works"
```

Building HDF5 with Parallel Support

HDF5 must be built with `--enable-parallel` to support parallel netCDF-4 I/O. This is a different build from the serial HDF5 described earlier in this chapter. You cannot use a serial HDF5 for parallel netCDF; the library must be built with MPI from the start.

Using Autotools:

```
CC=mpicc ./configure --enable-parallel --prefix=/usr/local/hdf5-parallel
make
make check
make install
```

Using CMake:

```
mkdir build && cd build
CC=mpicc cmake .. -DHDF5_ENABLE_PARALLEL=ON \
                  -DCMAKE_INSTALL_PREFIX=/usr/local/hdf5-parallel
make
ctest
make install
```

The key points are:

- Set `CC=mpicc` (or your MPI C compiler wrapper) so HDF5 links against MPI.
- Use `--enable-parallel` (Autotools) or `-DHDF5_ENABLE_PARALLEL=ON` (CMake).
- Install to a separate prefix from any serial HDF5 to avoid conflicts.

- Verify the build by checking hdf5.settings in the installation's share directory. Look for Parallel HDF5: yes.

Building PnetCDF (Optional)

PnetCDF provides parallel I/O for classic format files (CDF-1, CDF-2, CDF-5). If you only need parallel I/O for netCDF-4 files, you can skip this step.

```
CC=mpicc ./configure --prefix=/usr/local/pnetcdf
make
make check
make install
```

Building netCDF-C with Parallel Support

With parallel HDF5 installed, build netCDF-C using the MPI compiler and pointing to the parallel HDF5:

Using Autotools:

```
CC=mpicc CPPFLAGS="-I/usr/local/hdf5-parallel/include" \
LDLAGS="-L/usr/local/hdf5-parallel/lib" \
./configure --prefix=/usr/local/netcdf-parallel \
--enable-parallel-tests
make
make check
make install
```

To also enable PnetCDF:

```
CC=mpicc CPPFLAGS="-I/usr/local/hdf5-parallel/include -I/usr/local/pnetcdf/include" \
LDLAGS="-L/usr/local/hdf5-parallel/lib -L/usr/local/pnetcdf/lib" \
./configure --prefix=/usr/local/netcdf-parallel \
--enable-parallel-tests \
--enable-pnetcdf
make
make check
make install
```

Using CMake:

```
mkdir build && cd build
CC=mpicc cmake .. -DHDF5_DIR=/usr/local/hdf5-parallel \
-DENABLE_PARALLEL4=ON \
-DCMAKE_INSTALL_PREFIX=/usr/local/netcdf-parallel
make
ctest
make install
```

The `--enable-parallel-tests` flag (Autotools) tells the test suite to run parallel tests using `mpiexec`. Without it, the library is still built with parallel support, but the parallel tests are skipped.

Building netCDF-Fortran with Parallel Support

Build netCDF-Fortran against the parallel netCDF-C installation. Use both the MPI C and Fortran compilers:

```
CC=mpicc FC=mpif90 \
CPPFLAGS="-I/usr/local/netcdf-parallel/include" \
```

```
LDFLAGS="-L/usr/local/netcdf-parallel/lib" \  
./configure --prefix=/usr/local/netcdf-parallel  
make  
make check  
make install
```

Installing NetCDF-Java

NetCDF-Java is a pure Java implementation of the netCDF data model. Unlike the C and Fortran libraries, which are installed system-wide, netCDF-Java is distributed as JAR files and managed through Java build tools. It does not depend on the netCDF-C library for reading files.

Prerequisites

You need a Java Development Kit (JDK) version 8 or later. Check your Java version:

```
java -version
```

If Java is not installed, download it from <https://adoptium.net/> or install it through your package manager:

Ubuntu/Debian:

```
sudo apt install default-jdk
```

RHEL/Fedora:

```
sudo dnf install java-17-openjdk-devel
```

macOS:

```
brew install openjdk
```

You also need a build tool. Maven and Gradle are the two standard choices for Java projects. Most of the examples below use Maven.

Adding NetCDF-Java to a Maven Project

NetCDF-Java is hosted in the Unidata Maven repository, not in Maven Central. Add the repository and the dependency to your pom.xml.

The cdm-core artifact provides the core data model and netCDF-3 support. The logback-classic dependency is required because netCDF-Java uses SLF4J for logging and will print warnings at startup if no logging implementation is present.

Run `mvn clean compile` to download the dependencies and verify the configuration.

Adding NetCDF-Java to a Gradle Project

In your build.gradle:

```
repositories { mavenCentral() maven { url  
"https://artifacts.unidata.ucar.edu/repository/unidata-releases/" } }  
dependencies { implementation 'edu.ucar:cdm-core:5.5.3' implementation  
'ch.qos.logback:logback-classic:1.2.11' }
```

Run `./gradlew build` to download dependencies and compile.

Choosing the Right Artifact

NetCDF-Java is split into modules. Use the smallest set that covers your needs:

- `cdm-core` provides the core data model and netCDF-3 read/write support. Every project needs this.
- `netcdf4` adds the ability to read netCDF-4 and HDF5 files. Most scientific users need this.
- `opendap` adds OPeNDAP remote data access (DAP2 and DAP4 protocols).
- `grib` adds GRIB-1 and GRIB-2 format support, used in weather and climate modeling.
- `bufr` adds BUFR format support, used in meteorological observations.
- `netcdfAll` bundles all of the above into a single artifact. It is the largest but requires no decisions about which modules to include.

For most netCDF work, `cdm-core` plus `netcdf4` is sufficient. If you also access remote data through OPeNDAP, add `opendap`.

If you are unsure which artifacts you need, start with `netcdfAll`. It includes everything and is the simplest way to get started. You can switch to individual artifacts later to reduce your application size.

Manual JAR Download

For simple experiments without a build tool, download the all-in-one JAR directly:

```
https://artifacts.unidata.ucar.edu/repository/unidata-releases/edu/ucar/netcdfAll/5.5.3/netcdfAll-5.5.3.jar
```

Then compile and run with the JAR on the classpath:

```
javac -cp netcdfAll-5.5.3.jar MyProgram.java java -cp netcdfAll-5.5.3.jar:. MyProgram
```

On Windows, use a semicolon instead of a colon:

```
java -cp netcdfAll-5.5.3.jar;. MyProgram
```

Manual JAR management is fine for learning but not recommended for production projects. Use Maven or Gradle for proper dependency management.

NetCDF-4 Write Support via JNI

NetCDF-Java can read netCDF-4/HDF5 files in pure Java. However, writing netCDF-4 files requires the `netcdf-c` library installed on the system, accessed through JNI (Java Native Interface).

If you only need to read netCDF-4 files, no additional installation is needed. If you need to write netCDF-4 files, install the `netcdf-c` library as described earlier in this chapter, then set the Java library path when running your program:

Linux:

```
java -Djava.library.path=/usr/local/lib -jar myapp.jar
```

macOS:

```
java -Djava.library.path=/usr/local/lib -jar myapp.jar
```

Windows:

```
java -Djava.library.path=C:\netcdf\bin -jar myapp.jar
```

Writing netCDF-3 files works in pure Java without any native library.

ToolsUI

ToolsUI is a graphical application bundled with netCDF-Java for browsing and debugging netCDF files. Download the standalone JAR:

```
https://downloads.unidata.ucar.edu/netcdf-java/5.5.3/toolsUI-5.5.3.jar
```

Run it with:

```
java -Xmx1g -jar toolsUI-5.5.3.jar
```

The `-Xmx1g` flag allocates 1 GB of heap memory. Increase it if you work with large files. ToolsUI can open netCDF, HDF5, GRIB, and OPeNDAP datasets, display metadata and data values, and test coordinate system recognition.

Verifying Your Installation and Troubleshooting

After installing netCDF — whether from a package manager or a source build — verify that the libraries are correctly installed and that your compiler can find them.

Checking nc-config and nf-config

The `nc-config` utility is installed with the netCDF-C library. It reports the installed version, enabled features, and the compiler and linker flags needed to build programs against it. The Fortran equivalent is `nf-config`, installed with the netCDF-Fortran library.

To see a complete summary of your installation:

```
nc-config --all
nf-config --all
```

This prints the version, installation prefix, compiler used, and which features are enabled. Check that the features you need are listed. If `nc-config` is not found, the netCDF bin directory may not be in your `PATH`.

You can query individual settings:

```
nc-config --version      # Library version
nc-config --prefix       # Installation prefix
nc-config --has-nc4      # NetCDF-4/HDF5 support?
nc-config --has-dap      # OPeNDAP remote access?
nc-config --has-parallel # Parallel I/O?
nc-config --has-parallel4 # Parallel netCDF-4?
nc-config --has-pnetcdf  # PnetCDF for classic parallel?
```

If `nc-config --has-nc4` prints no, the library was built without HDF5 support and netCDF-4 features are unavailable. You will need to rebuild from source with HDF5, or install a package that includes netCDF-4 support.

C Smoke Test

Compile and run a minimal program that prints the netCDF library version:

```
#include <stdio.h>
#include <netcdf.h>
int main()
```

```
{
    printf("netCDF version: %s\n", nc_inq_libvers());
    return 0;
}
```

Compile and run it using nc-config to get the correct flags:

```
cc -o nc_version nc_version.c $(nc-config --cflags --libs)
./nc_version
```

If the program compiles, links, and prints a version string, your C installation is working.

Fortran Smoke Test

Compile and run a minimal Fortran program that prints the netCDF-Fortran library version:

```
program nf_version
    use netcdf
    implicit none
    print *, "netCDF-Fortran version: ", nf90_inq_libvers()
end program nf_version
```

Compile and run it using nf-config:

```
gfortran -o nf_version nf_version.f90 $(nf-config --fflags --flibs)
./nf_version
```

If the program compiles, links, and prints a version string, your Fortran installation is working.

Verifying Parallel Support

After installation, confirm that parallel I/O is enabled:

```
nc-config --has-parallel
nc-config --has-parallel4
nc-config --has-pnetcdf
```

All should print yes for a fully parallel build. If --has-parallel4 prints no, netCDF was not linked against a parallel HDF5. You must rebuild HDF5 with --enable-parallel and then rebuild netCDF.

Parallel I/O Smoke Test

If you built netCDF with parallel support, verify it with a minimal MPI program:

```
#include <stdio.h>
#include <mpi.h>
#include <netcdf.h>
#include <netcdf_par.h>
int main(int argc, char **argv)
{
    int ncid, rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (nc_create_par("parallel_test.nc", NC_NETCDF4|NC_CLOBBER,
                    MPI_COMM_WORLD, MPI_INFO_NULL, &ncid))
        return 1;
    nc_close(ncid);
    if (rank == 0)
        printf("Parallel netCDF is working.\n");
}
```

```

    MPI_Finalize();
    return 0;
}

```

Compile and run:

```

mpicc -o par_test par_test.c $(nc-config --cflags --libs)
mpiexec -n 4 ./par_test

```

If this prints "Parallel netCDF is working," your parallel installation is correct. If it fails, verify that `nc-config --has-parallel4` prints yes and that HDF5 was built with `--enable-parallel`.

Using `nc-config` in Makefiles

The most practical use of `nc-config` is to get the correct compiler and linker flags without hard-coding paths that differ between systems:

```

CFLAGS = $(shell nc-config --cflags)
LDFLAGS = $(shell nc-config --libs)
my_program: my_program.c
$(CC) $(CFLAGS) -o $@ $< $(LDFLAGS)
For Fortran:
FFLAGS = $(shell nf-config --fflags)
FLIBS = $(shell nf-config --flibs)
my_program: my_program.f90
$(FC) $(FFLAGS) -o $@ $< $(FLIBS)

```

The `--cflags` flag outputs the include directory (e.g., `-I/usr/local/include`). The `--libs` flag outputs the library directory and all required libraries (e.g., `-L/usr/local/lib -lnetcdf -lhdf5_hl -lhdf5 -lz`). For Fortran, `--fflags` and `--flibs` serve the same purpose, and `--flibs` includes both `-lnetcdff` and `-lnetcdf` along with their transitive dependencies.

Common Problems

The table below lists the errors you are most likely to encounter. Find your error message in the left column, then read across for the cause and fix. Detailed explanations follow the table.

Error Message	Cause	Fix
netcdf.h: No such file or directory	C development headers not installed, or include path not set	Install <code>libnetcdf-dev</code> (Debian/Ubuntu) or <code>netcdf-devel</code> (RHEL/Fedora), or compile with <code>\$(nc-config --cflags)</code>
Cannot open module file 'netcdf.mod'	Fortran development package not installed, or module path not set	Install <code>libnetcdff-dev</code> or <code>netcdf-fortran-devel</code> , or compile with <code>\$(nf-config --fflags)</code>
cannot find -lnetcdf	Library directory not in linker search path	Add <code>-L/path/to/lib</code> or compile with <code>\$(nc-config --libs)</code>
cannot find -lnetcdff	Fortran library not installed, or wrong library name used	Install the Fortran package; note the double-f in <code>-lnetcdff</code>

error while loading shared libraries: libnetcdf.so	Shared library found at compile time but not at runtime	Set LD_LIBRARY_PATH (Linux) or DYLD_LIBRARY_PATH (macOS), or run ldconfig
undefined reference to 'nc_create'	Linking against wrong library, or library path missing	Verify -lnetcdf appears after your .o files; add -L if needed
HDF5 header version does not match library version	Headers from one HDF5 installation, library from another	Remove the conflicting install; use a single installation method
nc-config: command not found	netCDF bin/ directory not in PATH	export PATH=/usr/local/bin:\$PATH or use the full path to nc-config
nc-config --has-nc4 prints no	netCDF built without HDF5	Rebuild from source with HDF5, or install a package that includes netCDF-4
NC_ENOPAR from nc_create_par	netCDF or HDF5 built without parallel support	Rebuild HDF5 with --enable-parallel (CC=mpicc), then rebuild netCDF
UnsatisfiedLinkError (Java, writing netCDF-4)	JNI cannot find the netCDF-C shared library	Set -Djava.library.path=/usr/local/lib when running your Java program
Could not find artifact edu.ucar:cdm-core	Unidata Maven repository not configured	Add the Unidata repository to your pom.xml or build.gradle

Header not found: netcdf.h: No such file or directory

The netCDF include directory is not in the compiler search path. Fix it by using nc-config or adding the path manually:

```
cc -o my_program my_program.c $(nc-config --cflags --libs)
```

Or explicitly:

```
cc -I/usr/local/include -o my_program my_program.c -L/usr/local/lib -lnetcdf
```

Fortran module not found: Fatal Error: Cannot open module file 'netcdf.mod'

The netCDF-Fortran module directory is not in the compiler search path. Use nf-config or add the path manually:

```
gfortran -o my_program my_program.f90 $(nf-config --fflags --flibs)
```

Or explicitly:

```
gfortran -I/usr/local/include -o my_program my_program.f90 -L/usr/local/lib -lnetcdff -lnetcdf
```

Note the double f in -lnetcdff. This is the Fortran library, not the C library (-lnetcdf). Mixing these up is a common source of linker errors.

Library not found: cannot find -lnetcdf

The netCDF library directory is not in the linker search path. Add it with -L:


```
cc -o my_program my_program.c -L/usr/local/lib -lnetcdf
```

Or set the environment variable before compiling:

```
export LDFLAGS="-L/usr/local/lib"
```

Runtime error: error while loading shared libraries: libnetcdf.so

The system can find the library at compile time but not at runtime. Set the library path:

Linux:

```
export LD_LIBRARY_PATH=/usr/local/lib:$LD_LIBRARY_PATH
./my_program
```

macOS:

```
export DYLD_LIBRARY_PATH=/usr/local/lib:$DYLD_LIBRARY_PATH
./my_program
```

For a permanent fix on Linux, add the library directory to `/etc/ld.so.conf.d/` and run `ldconfig`, or install to a standard system prefix like `/usr/local`.

NetCDF-4 features not available

If `nc-config --has-nc4` prints no, the library was built without HDF5 support. NetCDF-4 features (compression, groups, user-defined types) are unavailable. Either:

- Install a package that includes netCDF-4 support (most distribution packages do), or
- Rebuild netCDF from source with HDF5 installed and netCDF-4 enabled.

Java: UnsatisfiedLinkError when writing netCDF-4 files

The JNI bridge cannot find the netCDF-C shared library. Verify that `nc-config --version` works, then set the Java library path to the directory containing the shared library. See instructions above.

Reading netCDF-4 files works in pure Java without any native library. Only writing netCDF-4 files requires the C library via JNI.

Java: Could not find artifact edu.ucar:cdm-core

The Unidata Maven repository is missing from your `pom.xml`. Add the repository block as described in the Installing NetCDF-Java section.

Java: SLF4J: Failed to load class org.slf4j.impl.StaticLoggerBinder

Add the logback-classic dependency to your project. This is a warning, not a fatal error, but it should be fixed to suppress the message.

Java: OutOfMemoryError with large files

Increase the heap size with `-Xmx` (for example, `-Xmx4g` for 4 GB), or read data in smaller sections instead of loading entire variables at once.

Obtaining NetCDF – Key Takeaways

- Pick one method and stick with it. Mixing package-manager installs with source builds on the same system causes header/library mismatches and confusing linker errors.
- Package managers handle dependencies for you. On Linux use apt or dnf; on macOS use Homebrew; on Windows use vcpkg or MSYS2. Conda and Spack are good choices on shared or HPC systems where you lack root access.
- The C and Fortran libraries are separate packages. Install libnetcdf-dev and libnetcdf-f-dev (note the double-f) if you need both. The Fortran library depends on the C library.
- Build from source when you need a specific version, custom compiler flags, or parallel I/O. The dependency chain is: zlib, then HDF5, then netCDF-C, then netCDF-Fortran. Build them in that order.
- Both Autotools and CMake are supported. They are equivalent in capability. The netCDF team plans to drop Autotools eventually, leaving only CMake.
- Parallel I/O requires MPI at every layer. Build HDF5 with --enable-parallel (or -DHDF5_ENABLE_PARALLEL=ON) using CC=mpicc, then build netCDF-C against that parallel HDF5. A serial HDF5 cannot be used for parallel netCDF.
- NetCDF-Java is pure Java for reads; writing netCDF-4 requires the C library via JNI. Add cdm-core from the Unidata Maven repository. Use netcdfAll if you are unsure which modules you need.
- Verify with nc-config --all and a smoke test. Compile a one-line C or Fortran program that calls nc_inq_libvers() or nf90_inq_libvers(). If it links and runs, your installation is correct.
- When something goes wrong, check three things: (1) Are the development headers installed? (2) Is the library directory in the linker search path? (3) Do the header and library versions match.

Chapter 3 NetCDF Data Models

Learning Objectives

- Differentiate classic vs enhanced data models
- Implement dimensions, variables, and attributes
- Understand coordinate variable conventions
- Create and use groups for hierarchical data organization
- Work with multiple unlimited dimensions in netCDF-4/HDF5 files
- Utilize user-defined types: compound, VLEN, enum, and opaque
- Choose between data models for specific use cases

Example Code

The example C and Fortran code provided in this chapter are from the examples section of the NetCDF Expansion Pack. Full copies of the example programs used in this chapter can be found in chapters 6 and 7, and online at the NetCDF Expansion Pack GitHub site. As part of the expansion pack, they can be built and run.

Introduction to Data Models

This chapter explores the fundamental data models that form the foundation of netCDF. Understanding these models is essential for anyone working with scientific data, as they determine how you structure, organize, and access your data files.

We begin with the classic model, which has served the scientific computing community for decades. This model provides a simple yet powerful framework based on three core components: dimensions that define data axes, variables that store multidimensional arrays, and attributes that provide descriptive metadata. The classic model's elegance lies in its simplicity and widespread compatibility.

Next we examine the enhanced model, introduced with netCDF-4, which addresses the growing complexity of modern scientific datasets. The enhanced model builds upon the classic foundation by adding hierarchical groups for organization, multiple unlimited dimensions for flexible data growth, and user-defined types for complex data structures. These features enable you to work with larger, more complex datasets while maintaining backward compatibility.

Throughout this chapter, you will learn to differentiate between classic and enhanced models, implement dimensions and variables effectively, utilize coordinate variables for meaningful data representation, and leverage user-defined types when appropriate. Each concept includes practical examples in both C and Fortran, demonstrating how these data models work in real-world applications.

The skills you develop here provide the groundwork for all subsequent chapters, where we will apply these data model concepts to solve specific scientific data challenges.

The Classic Model

The netCDF classic data model was introduced in netCDF-2, and has been supported by every release since. This is how netCDF sees data.

All example code from this chapter is from the examples section of the NetCDF Expansion Pack.

NetCDF Classic Data Model

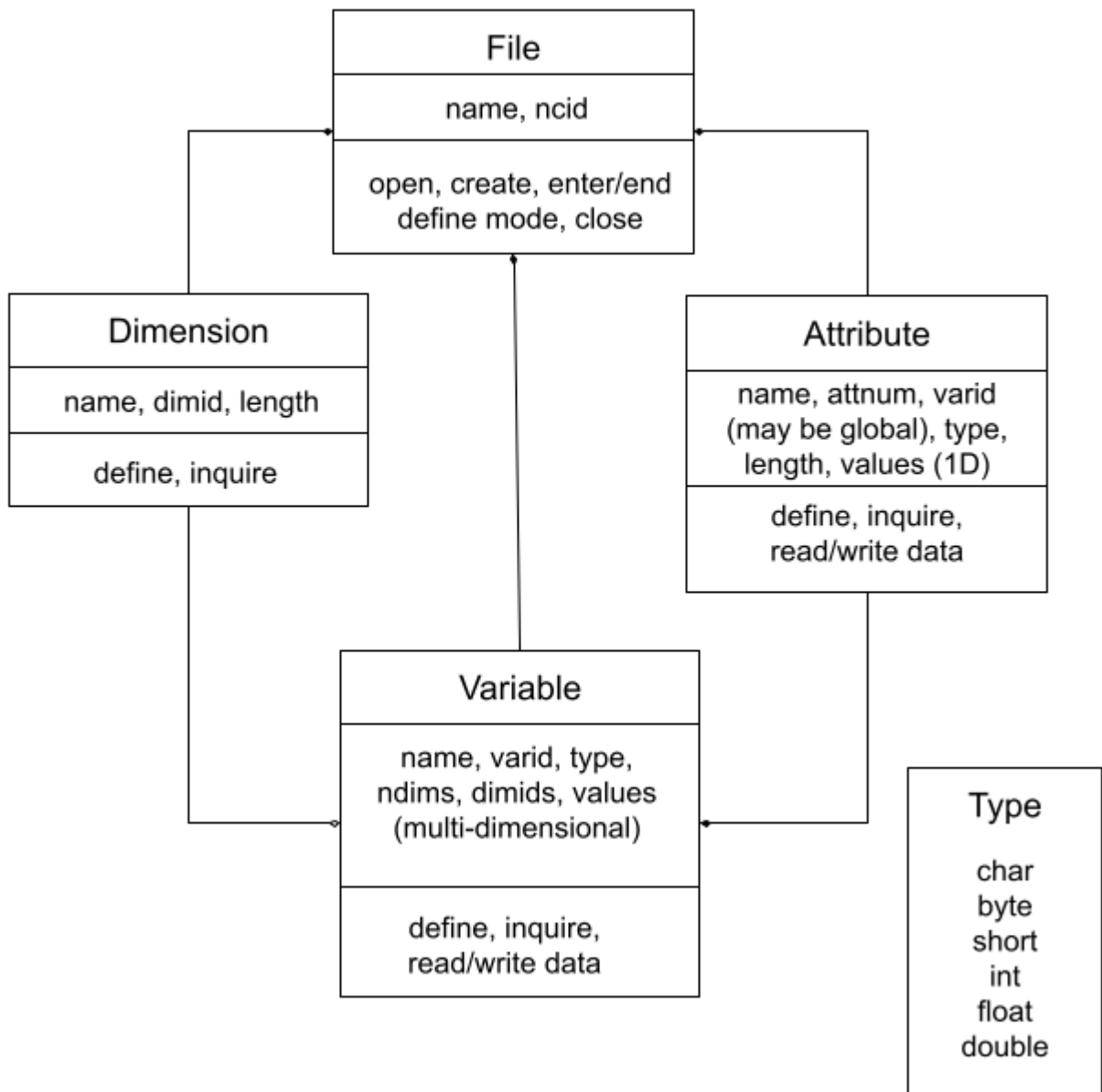


Figure 2: The netCDF classic data model. A file contains dimensions and variables, and may have global attributes. Each variable is associated with zero or more dimensions and may have its own attributes. The datatype of each variable and attribute must be one of the six classic types: `NC_BYTE`, `NC_CHAR`, `NC_SHORT`, `NC_INT`, `NC_FLOAT`, or `NC_DOUBLE`.

The File

The netCDF file contains:

- Dimensions which define the axes of the arrays of data.
- Variables are associated with zero or more dimensions, and are where the data are stored.
- Attributes are small arrays, frequently text arrays, with metadata for a variable or the file.

Creating or Opening a NetCDF File

Create new files with the `nc_create()` function in C:

```
/* Create the NetCDF file (NC_CLOBBER overwrites existing file) */
if ((retval = nc_create(FILE_NAME, NC_CLOBBER, &ncid)))
    ERR(retval);
```

In Fortran we use `nf90_create()`:

```
! Create the NetCDF file (NF90_CLOBBER overwrites existing file)
retval = nf90_create(FILE_NAME, NF90_CLOBBER, ncid)
if (retval /= nf90_noerr) call handle_err(retval)
```

Open existing files in C with the `nc_open()` function:

```
/* Open the file for reading */
if ((retval = nc_open(FILE_NAME, NC_NOWRITE, &ncid)))
    ERR(retval);
```

In Fortran we open existing files with `nf90_open()`.

```
! Open the file for reading
retval = nf90_open(FILE_NAME, NF90_NOWRITE, ncid)
if (retval /= nf90_noerr) call handle_err(retval)
```

Whenever opening or creating a netCDF file, an integer file ID will be returned (the `ncid`). This ID is needed for all subsequent function calls which ally to that file.

Inquiring about an Open NetCDF File

We can learn how many dimensions, variables, and global attributes are in the file with the `nc_inq()` function in C:

```
/* Verify metadata: check number of dimensions, variables, attributes, unlimited dim */
int ndims_in, nvars_in, ngatts_in, unlimdimid_in;
if ((retval = nc_inq(ncid, &ndims_in, &nvars_in, &ngatts_in, &unlimdimid_in)))
    ERR(retval);
```

In Fortran we use the `nf90_inquire()` function:

```
! Verify metadata: check number of dimensions and variables
retval = nf90_inquire(ncid, ndims_in, nvars_in, ngatts_in, unlimdimid_in)
if (retval /= nf90_noerr) call handle_err(retval)
```

Closing a NetCDF File

Always close netCDF files with the `nc_close()` function in C.

```
/* Close the file */
if ((retval = nc_close(ncid)))
    ERR(retval);
```

Or in Fortran,:

```
! Close the file
retval = nf90_close(ncid)
if (retval /= nf90_noerr) call handle_err(retval)
```

Define Mode and Data Mode

A netCDF file operates in one of two modes: define mode and data mode. Understanding these modes is essential with classic formats because the netCDF library enforces strict rules about which operations are permitted in each mode. Calling a function in the wrong mode produces an error. (These rules are greatly relaxed with netCDF/HDF5 files).

When you create a new file with `nc_create()`, the file starts in define mode. In define mode you can define dimensions, define variables, and add attributes. You cannot write data to variables while in define mode.

Once you have finished defining the structure of the file, call `nc_enddef()` to leave define mode and enter data mode. In data mode you can read and write variable data. You cannot define new dimensions or variables while in data mode.

In C:

```
/* End define mode, enter data mode */
if ((retval = nc_enddef(ncid)))
    ERR(retval);
```

In Fortran:

```
! End define mode, enter data mode
retval = nf90_enddef(ncid)
if (retval /= nf90_noerr) call handle_err(retval)
```

If you attempt to write data while still in define mode, the library returns the `NC_EINDEFINE` error. If you attempt to define a new dimension or variable while in data mode, the library returns `NC_ENOTINDEFINE`.

You can switch back to define mode on an existing file by calling `nc_redef()` in C or `nf90_redef()` in Fortran. This is useful when you need to add a new variable or attribute to a file that already contains data.

In C:

```
/* Re-enter define mode */
if ((retval = nc_redef(ncid)))
    ERR(retval);
```

In Fortran:

```
! Re-enter define mode
retval = nf90_redef(ncid)
if (retval /= nf90_noerr) call handle_err(retval)
```

When you open an existing file with `nc_open()`, the file starts in data mode.

Every example program in this chapter follows the same pattern: create the file (enters define mode), define dimensions and variables, call `enddef` (enters data mode), write data, close the file. You can see this sequence in the `simple_2D.c` and `f_simple_2D.f90` programs in the NetCDF Expansion Pack.

NetCDF-4/HDF5 files relax the define mode requirement. When a file is created with the `NC_NETCDF4` flag, the library allows you to interleave define and data operations without explicit `enddef` and `redef`

calls. The library handles the mode transitions internally. However, calling `nc_enddef()` on a netCDF-4/HDF5 file is not an error. It simply has no effect. Programs may call `nc_enddef()` regardless of format to maintain a consistent coding pattern that works with all netCDF file types.

For classic format files (`NC_CLASSIC_MODEL`, `NC_64BIT_OFFSET`, `NC_64BIT_DATA`), the define mode and data mode distinction is strictly enforced. The reason is that classic format files store all metadata in a contiguous header at the beginning of the file. When you call `nc_enddef()`, the library computes the final header size, writes the header, and positions the file for data writing. Switching back to define mode with `nc_redef()` may require the library to move data within the file to make room for an expanded header, which can be expensive for large files.

Dimensions

Dimensions have a name and a length. The length may be fixed, or unlimited.

An unlimited dimension can grow as data are added to the file. In the classic model, only one dimension may be unlimited. This dimension is sometimes called the "record dimension" because each write along this dimension adds a new record to the file.

In C create dimensions with the `nc_def_dim()` function.

```
/* Define dimensions */
if ((retval = nc_def_dim(ncid, "x", NX, &x_dimid)))
    ERR(retval);
if ((retval = nc_def_dim(ncid, "y", NY, &y_dimid)))
    ERR(retval);
```

In Fortran, use the `nf90_def_dim()` function.

```
! Define dimensions
retval = nf90_def_dim(ncid, "x", NX, x_dimid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_def_dim(ncid, "y", NY, y_dimid)
if (retval /= nf90_noerr) call handle_err(retval)
```

Notice that the `ncid` is required, this must refer to an open file. When the dimension is created, an integer ID is returned, the `dimid`. This ID is used later to specify the dimensions associated with variables.

Once a dimension is defined, the `nc_inq_dim()` function ("inq" for inquire) will return information about it:

```
/* Verify dimensions using nc_inq_dim() */
char dim_name[NC_MAX_NAME + 1];
size_t len_x, len_y;
if ((retval = nc_inq_dim(ncid, x_dimid, dim_name, &len_x)))
    ERR(retval);
```

In Fortran, use the `nf90_inquire_dimension()` function:

```
! Verify dimensions using nf90_inquire_dimension()
retval = nf90_inquire_dimension(ncid, x_dimid, name=dim_name, len=len_x)
if (retval /= nf90_noerr) call handle_err(retval)
```

Dimensions are used to construct variables. When a variable is defined, the number and sizes of the dimensions used will determine the size of the data.

Unlimited Dimensions

In the classic netCDF model, dimensions typically have a fixed size that is determined when the file is created. However, a special type of dimension called the unlimited dimension can grow dynamically as you add data to the file. This feature is particularly valuable for time series data where you may not know the final length of the time axis when you start collecting data.

The unlimited dimension serves as the foundation for incremental data writing. When you create a file with an unlimited dimension, you establish the framework for adding new records over time without needing to redefine the entire file structure. Each time you write data along the unlimited dimension, netCDF automatically extends the file to accommodate the new data. This approach is efficient for applications like weather models, satellite data streams, or sensor networks where data arrives continuously.

Only one unlimited dimension may exist in a classic model file. This limitation stems from the underlying storage format of classic netCDF files, which organizes data in records that grow sequentially. The unlimited dimension is therefore sometimes called the record dimension because each write operation along this dimension adds a new record to the file. Common practice uses the unlimited dimension for time, making it natural to append new time steps as they become available.

Variables that use the unlimited dimension must follow specific ordering rules. In C programs, the unlimited dimension must be the first dimension in the variable definition, while in Fortran programs it must be the last dimension. This requirement ensures that data is stored in a contiguous layout that supports efficient record-wise access. When you define variables with the unlimited dimension in the wrong position, netCDF returns an error indicating the dimension is in the wrong index (unless using the enhanced model – see below).

The practical impact of unlimited dimensions becomes clear when working with large datasets. Instead of allocating space for the maximum possible number of time steps upfront, you can start with a small file and let it grow as needed. This approach saves storage space and reduces initial file creation time.

Variables

Variables are the place where data are stored. Each variable represents an N-dimensional array. The maximum number of dimensions available is 1024 in the C library, but most physical systems are modeled with 4 dimensions or less.

Variables have a type, and there are six available types in the classic model.

Type	C Type (most systems)	Size (bytes)
NC_BYTE	unsigned char	1
NC_CHAR	signed char	1
NC_SHORT	short int	2
NC_INT	int	4
NC_FLOAT	float	4
NC_DOUBLE	double	8

The C types given are for most systems (those that have a 4-byte int). Technically, C does not specify the sizes of types, so it is possible for a compiler to have a different size for an int. In practice, this is only seen on 8-bit embedded systems; the programming of netCDF on such systems is outside the scope of this work.

Unlike C, the sizes of netCDF types (in bytes) are true on every system. NetCDF specifies that an NC_INT is a 32-bit integer. The sizes of every netCDF type are as specified in the table above. This is necessary to ensure that netCDF files are inter-operable between different systems.

The type of the variable, along with its name and dimensions, are provided to the nc_def_var() functions:

```
/* Define the variable (dimension order: y, x for C row-major) */
dimids[0] = y_dimid;
dimids[1] = x_dimid;
if ((retval = nc_def_var(ncid, "data", NC_INT, NDIMS, dimids, &varid)))
    ERR(retval);
```

In Fortran, use nf90_def_var():

```
! Define the variable (dimension order: x, y for Fortran column-major)
dimids(1) = x_dimid
dimids(2) = y_dimid
retval = nf90_def_var(ncid, "data", NF90_INT, dimids, varid)
if (retval /= nf90_noerr) call handle_err(retval)
```

To write to a var, use one of the nc_put_var functions, which will be explained in chapter 6 for C, chapter 7 for Fortran. Here's an example in C:

```
/* Write the data to the file */
if ((retval = nc_put_var_int(ncid, varid, &data_out[0][0])))
    ERR(retval);
```

And in Fortran:

```
! Write the data to the file
retval = nf90_put_var(ncid, varid, data_out)
if (retval /= nf90_noerr) call handle_err(retval)
```

Coordinate Variables

A coordinate variable is a one-dimensional variable with the same name as its dimension. The netCDF library does not enforce this convention, but it is so fundamental that most tools and libraries depend on it. A coordinate variable stores the physical values that correspond to each index along a dimension. Without one, index 5 along the latitude dimension is just a number. With one, index 5 maps to 42.5 degrees north.

Tools like Panoply, ncview, and CF-compliant libraries detect coordinate variables automatically to label axes, create plots, and enable geographic subsetting. The CF Conventions require coordinate variables for metadata compliance.

To create a coordinate variable, define the dimension, then define a variable with the same name using only that dimension. Include units and long_name attributes to make the data self-describing.

In this code from the NetCDF Expansion Pack example program, coord_vars.c, we create latitude and longitude dimensions ("lat" and "lon"), then define coordinate variables with the same names.

```
/* Define dimensions */
if ((retval = nc_def_dim(ncid, "lat", NLAT, &lat_dimid)))
    ERR(retval);
```

```

if ((retval = nc_def_dim(ncid, "lon", NLONG, &lon_dimid)))
    ERR(retval);

/* Define coordinate variables (same name as dimension) */
if ((retval = nc_def_var(ncid, "lat", NC_FLOAT, 1, &lat_dimid, &lat_varid)))
    ERR(retval);
if ((retval = nc_def_var(ncid, "lon", NC_FLOAT, 1, &lon_dimid, &lon_varid)))
    ERR(retval);

```

Later in the code, we write the values of the coordinate variables:

```

/* Write coordinate variables */
if ((retval = nc_put_var_float(ncid, lat_varid, lat)))
    ERR(retval);
if ((retval = nc_put_var_float(ncid, lon_varid, lon)))
    ERR(retval);

```

In Fortran, we define and write coordinate variables the same way, using the `f_coord_vars.f90` example from the NetCDF Expansion Pack:

```

! Define dimensions
retval = nf90_def_dim(ncid, "lat", NLAT, lat_dimid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_def_dim(ncid, "lon", NLONG, lon_dimid)
if (retval /= nf90_noerr) call handle_err(retval)

! Define coordinate variables (same name as dimension)
retval = nf90_def_var(ncid, "lat", NF90_FLOAT, lat_dimid, lat_varid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_def_var(ncid, "lon", NF90_FLOAT, lon_dimid, lon_varid)
if (retval /= nf90_noerr) call handle_err(retval)

```

And later, writing the coordinate data:

```

! Write coordinate variables
retval = nf90_put_var(ncid, lat_varid, lat)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_put_var(ncid, lon_varid, lon)
if (retval /= nf90_noerr) call handle_err(retval)

```

(The full example for coordinate variables can be seen for C in chapter 6, and for Fortran in chapter 7.)

The coordinate values written are the latitude and longitude for each data point. Use floating-point types for geographic coordinates: latitude ranges from -90 to +90 degrees, longitude from -180 to +180 (or 0 to +360). Time coordinates typically use double precision to represent hours or days since a reference date. Vertical coordinates represent pressure levels or depth, and require careful attention to units through the units attribute.

Some datasets use two-dimensional coordinate variables for non-regular grids, where latitude and longitude vary along multiple dimensions. This occurs in satellite data and model output where each grid point has its own geographic position.

Best practices for coordinate variables:

- Create them for all spatial and temporal dimensions
- Include units and long_name attributes
- Use appropriate numeric types for precision
- Follow CF Conventions for standard coordinate names

- Write the coordinate data after defining the variable

Attributes

Attributes store ancillary data (often text strings) which describe the data arrays. Attributes can be associated with the file as a whole (a.k.a. “global attributes”), or with a specific variable (“variable attributes”).

Attributes are always 1-dimensional arrays. The attribute includes a name, type, and length (which may be zero).

Attributes are read into memory only when needed. When the user inquires about any attribute data in a file, all attributes in that file are read (all the attributes in the group, if the enhanced model is being used – see below). This allows files with thousands of attributes to be opened very quickly. We use `nc_put_att_text()` to write text attributes in C, and `nf90_put_att()` in Fortran.

Global Attributes

Global attributes are used to store metadata about the entire file. Instead of a varid, use `NC_GLOBAL` (in C):

```
/* Add a global attribute */
if ((retval = nc_put_att_text(ncid, NC_GLOBAL, "title",
                             strlen("Simple 2D Example"), "Simple 2D Example")))
    ERR(retval);
```

or `NF90_GLOBAL` in Fortran:

```
! Add a global attribute
retval = nf90_put_att(ncid, NF90_GLOBAL, "title", "Simple 2D Example")
if (retval /= nf90_noerr) call handle_err(retval)
```

Variable Attributes

Variable attributes are used to store metadata about a particular variable. For example, a “units” attribute can store the scientific units of the data in a variable. When creating a variable attribute, the variable ID is required:

```
/* Add a variable attribute */
if ((retval = nc_put_att_text(ncid, varid, "units",
                             strlen("m/s"), "m/s")))
    ERR(retval);
```

For Fortran:

```
! Add a variable attribute
retval = nf90_put_att(ncid, varid, "units", "m/s")
if (retval /= nf90_noerr) call handle_err(retval)
```

Writing Attributes

In C, the write function name encodes the attribute type. In Fortran, `nf90_put_att` is overloaded and accepts any scalar or array value directly.

NetCDF type	C write function	C read function
NC_CHAR	nc_put_att_text	nc_get_att_text
NC_BYTE	nc_put_att_schar	nc_get_att_schar
NC_SHORT	nc_put_att_short	nc_get_att_short
NC_INT	nc_put_att_int	nc_get_att_int
NC_FLOAT	nc_put_att_float	nc_get_att_float
NC_DOUBLE	nc_put_att_double	nc_get_att_double

In Fortran, all types use `nf90_put_att(ncid, varid, name, value)` and `nf90_get_att(ncid, varid, name, value)`. The compiler resolves the correct implementation from the type of value.

The most common attribute type is text. `coord_vars.c` attaches CF convention metadata to coordinate variables:

```
if ((retval = nc_put_att_text(ncid, lat_varid, "units", 13, "degrees_north")))
    ERR(retval);
if ((retval = nc_put_att_float(ncid, temp_varid, "_FillValue", NC_FLOAT, 1, &fill_value)))
    ERR(retval);
```

The Fortran equivalent passes the value directly without a length or type argument:

```
retval = nf90_put_att(ncid, lat_varid, "units", "degrees_north")
retval = nf90_put_att(ncid, temp_varid, "_FillValue", fill_value)
```

Reading Attributes

Before reading a text attribute in C, call `nc_inq_attlen()` to get the length so you can allocate the right buffer. Numeric attributes have a known size from their type. (This code is from the NetCDF Expansion Pack example `coord_vars.c`.)

```
size_t att_len;
char att_text[NC_MAX_NAME + 1];
if ((retval = nc_inq_attlen(ncid, lat_varid, "units", &att_len)))
    ERR(retval);
if ((retval = nc_get_att_text(ncid, lat_varid, "units", att_text)))
    ERR(retval);
att_text[att_len] = '\0';
```

In Fortran, `nf90_get_att` writes directly into a character variable of sufficient length;

`nf90_inquire_attribute` provides the length if you need it.

```
retval = nf90_inquire_attribute(ncid, lat_varid, "units", len=att_len)
retval = nf90_get_att(ncid, lat_varid, "units", att_text)
```

The NetCDF Expansion Pack program `dump_classic_metadata.c` shows the general pattern for reading an attribute of unknown type: call `nc_inq_att()` to get the type and length, then dispatch to the appropriate typed getter.

Fill Values and Fill Mode

A fill value is a sentinel that NetCDF returns for any element that was never written. It lets readers distinguish "no data here" from a legitimate zero or negative number. Without a fill value, unwritten storage contains whatever bytes happened to be on disk—garbage.

Default Fill Values

Every NetCDF type has a built-in default fill value:

Type	Default fill value
NC_BYTE	-127
NC_SHORT	-32767
NC_INT	-2147483647
NC_FLOAT	9.9692099683868690e+36
NC_DOUBLE	9.9692099683868690e+36
NC_CHAR	\0

Use the default when any out-of-range sentinel is acceptable. Override it when you have an established convention or when the default could collide with valid data. Always set a fill value for variables that may be partially written—sparse grids, time series with gaps, or any array where some elements may never be written. Never store a legitimate data value that equals the fill value; CF-aware tools treat it as missing and will silently exclude it from computations.

Setting a Custom Fill Value

Call `nc_def_var_fill()` (C) or `nf90_def_var_fill()` (Fortran) during define mode, before `nc_enddef()`. The second argument is the *no-fill flag*: pass `NC_FILL` (0) to enable fill mode, `NC_NOFILL` (1) to disable it. The fill value must match the variable's type exactly; passing a float fill value for an integer variable will truncate it silently. (This example code is from the NetCDF Expansion Pack program `simple_2D.c`.)

```
int fill_value = FILL_VALUE; /* FILL_VALUE = -9999 */
if ((retval = nc_def_var_fill(ncid, varid, NC_FILL, &fill_value)))
    ERR(retval);
```

In Fortran, we use the F90 version of the function (example from NetCDF Expansion Pack program `f_simple_2D.f90`.)

```
retval = nf90_def_var_fill(ncid, varid, 0, FILL_VALUE)
if (retval /= nf90_noerr) call handle_err(retval)
```

The example program `simple_2D.c` writes a 6×12 integer array but leaves the last row unwritten. Those 6 elements return -9999 on any subsequent read.

What Fill Mode Actually Does

When fill mode is NC_FILL, the library pre-fills every element with the fill value before any application write. This costs one extra write pass per variable at nc_enddef() time. The payoff is predictable behavior for partial writes.

When fill mode is NC_NOFILL, the library skips that pre-fill pass. Use NC_NOFILL when you know you will write every element—it can cut I/O time roughly in half for large variables with many small writes. Note that NC_NOFILL also suppresses the _FillValue attribute entirely, so readers have no sentinel to check against.

Querying the Fill Value

To check the fill value on an existing file, use nc_inq_var_fill() (C) or nf90_inq_var_fill() (Fortran). Both return the no-fill flag and the fill value itself.

```
int no_fill, fill_value_in;
if ((retval = nc_inq_var_fill(ncid, varid, &no_fill, &fill_value_in)))
    ERR(retval);
```

In Fortran:

```
retval = nf90_inq_var_fill(ncid, varid, no_fill, fill_value_in)
if (retval /= nf90_noerr) call handle_err(retval)
```

The _FillValue Attribute

nc_def_var_fill() works by writing a _FillValue attribute internally — the two are the same mechanism, not alternatives. The attribute is visible in ncdump output and to any reader (CDL tools, Python xarray, etc.) regardless of which API wrote it.

The practical difference is the **no-fill flag**. Calling nc_def_var_fill() with NC_NOFILL both suppresses the pre-fill write pass *and* removes the _FillValue attribute. Writing the attribute directly with nc_put_att_* cannot do that.

CF Conventions require the fill value to appear as _FillValue. The NetCDF Expansion Pack program coord_vars.c writes it directly as an attribute, which is the typical CF pattern:

```
float fill_value = -999.0;
if ((retval = nc_put_att_float(ncid, temp_varid, "_FillValue", NC_FLOAT, 1, &fill_value)))
    ERR(retval);
```

In Fortran:

```
fill_value = -999.0
retval = nf90_put_att(ncid, temp_varid, "_FillValue", fill_value)
if (retval /= nf90_noerr) call handle_err(retval)
```

Writing _FillValue directly is equivalent to calling nc_def_var_fill() with NC_FILL; the library implementation deletes any existing _FillValue attribute and writes a new one with the given value. Use nc_def_var_fill() when you also need to control the no-fill flag; use nc_put_att_* when the CF attribute pattern is more readable in context. Either way, the resulting file is identical.

Reading and Writing Data

Variables are defined during define mode but data is written and read only in data mode. The netCDF C library provides a family of typed functions for both operations; the Fortran library uses generic overloaded functions instead. This section covers the full read/write cycle: writing an entire variable at once, writing a subset with start/count arrays, reading data back, and handling the variable ID lookup you need when opening an existing file.

Writing an Entire Variable

The simplest write operation transfers all elements of a variable in one call. In C, use `nc_put_var_<type>()` where `<type>` matches the variable's netCDF type:

```
/* Write entire 2D integer array */
if ((retval = nc_put_var_int(ncid, varid, &data_out[0][0]))
    ERR(retval);
```

In Fortran, `nf90_put_var()` is overloaded and accepts any array type directly:

```
! Write entire 2D integer array
retval = nf90_put_var(ncid, varid, data_out)
if (retval /= nf90_noerr) call handle_err(retval)
```

The full set of typed write functions in C mirrors the attribute functions:

NetCDF type	C write function	C read function
NC_CHAR	nc_put_var_text	nc_get_var_text
NC_BYTE	nc_put_var_schar	nc_get_var_schar
NC_SHORT	nc_put_var_short	nc_get_var_short
NC_INT	nc_put_var_int	nc_get_var_int
NC_FLOAT	nc_put_var_float	nc_get_var_float
NC_DOUBLE	nc_put_var_double	nc_get_var_double

In Fortran, all types use `nf90_put_var(ncid, varid, values)` and `nf90_get_var(ncid, varid, values)`. The compiler selects the correct implementation from the type of values.

Writing a Subset with Start and Count

Many applications need to write only part of a variable—a single time step, a spatial tile, or a block of records. The `nc_put_vara_<type>()` functions in C accept a start array and a count array that specify the hyperslab to write. `start` gives the zero-based index of the first element along each dimension; `count` gives how many elements to write along each dimension.

This example from `simple_2D.c` in the NetCDF Expansion Pack writes the first NY-1 rows of a two-dimensional integer variable, leaving the last row unwritten:

```
size_t start[NDIMS] = {0, 0};
size_t count[NDIMS] = {NY - 1, NX};
if ((retval = nc_put_vara_int(ncid, varid, start, count, &data_out[0][0])))
    ERR(retval);
```

The Fortran equivalent passes `start` and `count` as keyword arguments to `nf90_put_var()`. In Fortran the indices are 1-based and the dimension order is reversed relative to C:

```
start_idx = (/ 1, 1 /)
count_idx = (/ NX, NY - 1 /)
retval = nf90_put_var(ncid, varid, data_out(:, 1:NY-1), start=start_idx, count=count_idx)
if (retval /= nf90_noerr) call handle_err(retval)
```

The same `start/count` mechanism works for appending to an unlimited dimension. The `unlimited_dim.c` example in the NetCDF Expansion Pack writes three initial time steps and then reopens the file to append two more:

```
/* Write initial timesteps (0, 1, 2) */
size_t start[3] = {0, 0, 0};
size_t count[3] = {INITIAL_TIMESTEPS, NLAT, NLON};
if ((retval = nc_put_vara_float(ncid, temp_varid, start, count, &temp_data[0][0][0])))
    ERR(retval);
/* Later: append timesteps 3 and 4 */
start[0] = INITIAL_TIMESTEPS;
count[0] = APPEND_TIMESTEPS;
if ((retval = nc_put_vara_float(ncid, temp_varid, start, count,
                                &temp_data[INITIAL_TIMESTEPS][0][0])))
    ERR(retval);
```

Reading an Entire Variable

To read all elements of a variable in one call, use `nc_get_var_<type>()` in C:

```
/* Read entire 2D integer array */
if ((retval = nc_get_var_int(ncid, varid, &data_in[0][0])))
    ERR(retval);
```

In Fortran, use `nf90_get_var()`:

```
! Read entire 2D integer array
retval = nf90_get_var(ncid, varid, data_in)
if (retval /= nf90_noerr) call handle_err(retval)
```

The buffer you pass must be large enough to hold the entire variable. For large variables this can be impractical; use `nc_get_vara_<type>()` with `start` and `count` to read one hyperslab at a time.

Looking Up Variable IDs in an Existing File

When you write a file and immediately reopen it, you know the variable IDs from the define phase. When you open a file written by someone else—or open your own file in a later program run—you must look up the IDs by name. Use `nc_inq_varid()` in C:

```
/* Look up variable ID by name */
if ((retval = nc_inq_varid(ncid, "temperature", &temp_varid)))
    ERR(retval);
```

In Fortran, use `nf90_inq_varid()`:

```
! Look up variable ID by name
retval = nf90_inq_varid(ncid, "temperature", temp_varid)
if (retval /= nf90_noerr) call handle_err(retval)
```

Once you have the variable ID you can query its type, number of dimensions, and dimension IDs with `nc_inq_var()` in C or `nf90_inquire_variable()` in Fortran, as shown in the Inquiring about an Open NetCDF File section above.

A Complete Read/Write Cycle

The `simple_2D.c` and `f_simple_2D.f90` programs in the NetCDF Expansion Pack demonstrate the complete cycle. The write phase creates the file, defines dimensions and a variable with a custom fill value, leaves define mode, writes a partial array, and closes the file. The read phase reopens the file, verifies metadata with `nc_inq()` and `nc_inq_dim()`, reads the data with `nc_get_var_int()`, and confirms that written elements contain the expected sequential integers while unwritten elements contain the fill value.

The key steps in order are:

- `nc_create()` or `nc_open()` — obtain the `ncid`
- `nc_def_dim()`, `nc_def_var()`, `nc_put_att_*`(), `nc_def_var_fill()` — define mode only
- `nc_enddef()` — enter data mode (classic formats)
- `nc_put_var_*`() or `nc_put_vara_*`() — write data
- `nc_get_var_*`() or `nc_get_vara_*`() — read data
- `nc_close()` — flush buffers and release the file

(The full examples for reading and writing data are covered in detail for C in chapter 6 and for Fortran in chapter 7.)

The Enhanced Model

The classic model works well for many datasets, but its constraints become limiting as data grow more complex. A flat namespace makes it difficult to organize multi-instrument or multi-model output. A single unlimited dimension restricts how data can be extended over time. The six atomic types do not support 64-bit integers or unsigned integers, and leave no room for structured or variable-length data. The enhanced model removes these restrictions by adding hierarchical groups, multiple unlimited dimensions, and user-defined types, all available in HDF5.

Starting with netCDF-4.0, the netCDF C library can read and write HDF5 files using the netCDF API. These files are known as netCDF-4/HDF5 files. They are fully functional HDF5 files, and also fully functional netCDF files.

We call the expanded data model the “enhanced model.”

Why the Enhanced Model Was Needed

Scientific data requirements evolved beyond what the classic model could provide. Climate models now generate terabytes of output with hundreds of variables. Satellite missions produce multi-instrument datasets that need hierarchical organization. Modern sensors collect 64-bit integer data and require structured data types for complex observations.

The flat namespace of the classic model forces all variables into a single level. A climate model with temperature, pressure, humidity, and wind data for multiple pressure levels and forecast times results in dozens of variables with long, cumbersome names. Organizing these into logical groups would make the data more manageable and self-documenting.

The single unlimited dimension limitation affects time series analysis. Imagine a dataset with both time and ensemble dimensions. You want to add new timesteps and new ensemble members, but the classic model forces you to choose which dimension can grow. This constraint requires workarounds like creating multiple files or pre-allocating maximum sizes.

The limited type system restricts modern data representation. Many instruments now output 64-bit integer counts or unsigned integer values. Structured observations like weather reports contain multiple related fields that would benefit from compound types. Variable-length data such as text observations or ragged arrays have no natural representation in the classic model.

HDF5 Foundation

The enhanced model builds on HDF5, a powerful hierarchical data format designed for scientific computing. HDF5 provides the underlying storage mechanism that makes enhanced features possible. Chunked storage allows efficient access to subsets of large datasets. Compression reduces storage requirements while maintaining fast access. Hierarchical groups enable natural data organization.

When you create a netCDF-4/HDF5 file, you get the benefits of both worlds. The file follows netCDF conventions and can be read by any netCDF-aware application. At the same time, HDF5 tools can access the same file, providing additional flexibility for data management and analysis.

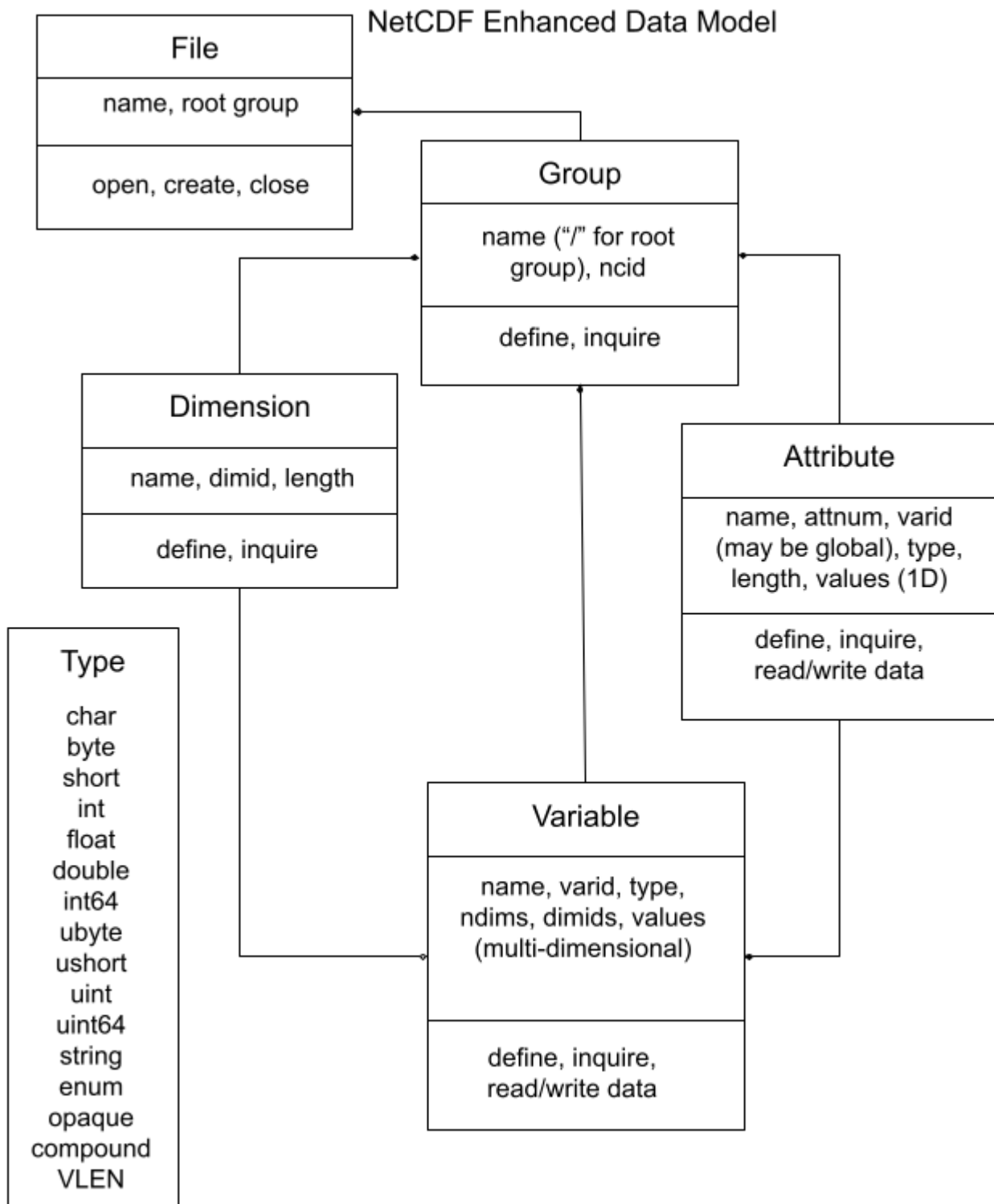


Figure 3: The netCDF enhanced (NetCDF-4) data model. A file contains a hierarchy of groups. Each group can contain dimensions, variables, attributes, and user-defined types. Variables and attributes may use any of the 12 atomic data types or a user-defined type. User-defined types include compound, variable-length, opaque, and enum types.

Compatibility Considerations

In the enhanced model, the dimensions, variables, and global attributes of the classic model are found in each group. That is, each group functions as a classic model file. Each group can have dimensions, global attributes, and variables. By default, there is one group, the root group.

Use of the enhanced model is only available with netCDF-4/HDF5 files. The enhanced model cannot be used with netCDF classic binary formats.

In limited cases it is desirable to produce netCDF-4/HDF5 files which do not permit the enhanced model to be used. These netCDF-4/HDF5 files can then be readily converted to other netCDF binary formats, and the code to read/write such files may have been written before the enhanced model was developed. For these cases, netcdf-c includes the NC_CLASSIC_MODEL mode flag in nc_create(). Files created with NC_CLASSIC_MODEL can only contain elements from the classic model.

Multiple Unlimited Dimensions

Unlike the classic model, the enhanced model allows any number of unlimited dimensions. Variables can use multiple unlimited dimensions in any position. This flexibility supports complex data growth patterns like adding both new timesteps and new ensemble members.

A typical use case is to use the unlimited dimension as the time dimension in weather models. Each timestep of the model is run, and all variables are written for one time. Then the model processes another timestep and writes another set of variable data for the new timestep. (In this case the unlimited dimension is sometimes called the “record” dimension, and the timestep data for all variables are a “record” of data.)

Due to the way the classic binary format stores data, only one unlimited dimension can be permitted. As each record is added, the netCDF classic format file is increased in size to hold the new record, which is appended to the end of the file.

```
#define NC_EUNLIMIT (-54) /**< NC_UNLIMITED size already in use */
```

This error is returned when the user attempts to define a second dimension in a classic binary format file, or a netCDF-4/HDF5 file with NC_CLASSIC_MODEL specified.

An additional restriction is that for any variable using the unlimited dimension, that dimension must be the slowest-varying dimension. In C this means the unlimited dimension must come first in the list of dimensions for the variable. In Fortran, the unlimited dimension must be last.

```
#define NC_EUNLIMPOS (-47) /**< NC_UNLIMITED in the wrong index */
```

The NC_EUNLIMPOS error is returned if the user attempts to define a variable with the unlimited dimension not the slowest varying dimension, if the file is a classic format file, or a netCDF-4/HDF5 file with NC_CLASSIC_MODEL defined.

With HDF5, multiple unlimited dimensions are permitted. Since data are written in chunks, there is no restriction on the dimensions. Chunks are added to the file in any order, and contain information which allows HDF5 to properly locate the chunk of data within the global data space.

For netCDF-4/HDF5 files, any number of unlimited dimensions may be defined. Any variable may use one or more unlimited dimensions, and those dimensions can appear anywhere in the variable’s list of dimensions.

```

/* Define two unlimited dimensions */
if ((retval = nc_def_dim(ncid, "station", NC_UNLIMITED, &station_dimid)))
    ERR(retval);
if ((retval = nc_def_dim(ncid, "time", NC_UNLIMITED, &time_dimid)))
    ERR(retval);

/* Define variable using both unlimited dimensions */
dimids[0] = station_dimid;
dimids[1] = time_dimid;
if ((retval = nc_def_var(ncid, "temperature", NC_FLOAT, NDIMS, dimids, &varid)))
    ERR(retval);

```

In Fortran:

```

retval = nf90_def_dim(ncid, "station", NF90_UNLIMITED, station_dimid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_def_dim(ncid, "time", NF90_UNLIMITED, time_dimid)
if (retval /= nf90_noerr) call handle_err(retval)

dimids(1) = time_dimid
dimids(2) = station_dimid
retval = nf90_def_var(ncid, "temperature", NF90_FLOAT, dimids, varid)
if (retval /= nf90_noerr) call handle_err(retval)

```

If NC_CLASSIC_MODEL is used when the file is created (or NF90_CLASSIC_MODEL in Fortran), then the classic model behavior is enforced; only one unlimited dimension will be permitted in the file, and variables that use it must define it as their slowest varying dimension.

Groups

Groups provide a tree-like structure for organizing data. Each group acts like a mini-netCDF file with its own dimensions, variables, and attributes. You can create groups for instruments, experiments, or any logical division of your data. This structure makes large datasets more navigable and self-documenting.

Each group acts as a file in the classic model. That is, each group may contain variables, dimensions, and global attributes. When defining a new group, a new ncid is returned which applies to that group. The ncid returned from nc_open() and nc_create() point to the unnamed root group of the file.

In the groups.c example in the NetCDF Expansion Pack, we create a file:

```

if ((retval = nc_create(FILE_NAME, NC_CLOBBER|NC_NETCDF4, &ncid)))
    ERR(retval);

```

The ncid returned as a parameter in this function points to the root group. To create subgroups:

```

/* Create SubGroup1 */
printf("Creating SubGroup1\n");
if ((retval = nc_def_grp(ncid, "SubGroup1", &grp1_id)))
    ERR(retval);

/* Create SubGroup2 */
printf("Creating SubGroup2\n");
if ((retval = nc_def_grp(ncid, "SubGroup2", &grp2_id)))
    ERR(retval);

```

We can then create a nested group under subgroup 2:

```

if ((retval = nc_def_grp(grp2_id, "NestedGroup", &nested_id)))
    ERR(retval);

```

When creating a variable in a group, use the group ID as the ncid in the nc_def_var() call:

```

/* SubGroup1: NC_USHORT variable (2D: x, y) */

```

```
printf(" SubGroup1: ushort_var (NC_USHORT, 2D: x, y)\n");
if ((retval = nc_def_var(grp1_id, "ushort_var", NC_USHORT, NDIMS_2D,
                        dimids_2d, &ushort_varid)))
    ERR(retval);
```

In Fortran:

```
! Create SubGroup1
print *, "Creating SubGroup1"
retval = nf90_def_grp(ncid, "SubGroup1", grp1_id)
if (retval /= nf90_noerr) call handle_err(retval)

! Create SubGroup2
print *, "Creating SubGroup2"
retval = nf90_def_grp(ncid, "SubGroup2", grp2_id)
if (retval /= nf90_noerr) call handle_err(retval)
```

We can also create nested subgroups:

```
! Create NestedGroup under SubGroup2
print *, "Creating NestedGroup under SubGroup2"
retval = nf90_def_grp(grp2_id, "NestedGroup", nested_id)
if (retval /= nf90_noerr) call handle_err(retval)
```

Visibility of Dimensions

Dimensions can be used in any variable in their group, or any subgroup. That is, dimensions in the root group are visible throughout the entire file, but dimensions defined in a group may only be used in that group, or subgroups of that group.

New Atomic Types

HDF5 supports additional types, including unsigned integer types, 64-bit integer types, and a string type. These types are exposed in the netCDF-4 API.

Type	C Type (most systems)	Size (bytes)
NC_UBYTE	unsigned char	1
NC_USHORT	unsigned short	2
NC_UINT	unsigned int	4
NC_INT64	long long	8
NC_UINT64	unsigned long long	8
NC_STRING	N/A	variable

64-bit Integer Type

NetCDF classic format supports 32-bit integers as the largest integer type. Starting with version 4.0, netCDF added support for signed and unsigned 64-bit integers in netCDF-4/HDF5 files.

Starting in version 4.5.0, the CDF-5 binary format was added, which is an extension of the netCDF classic binary format. CDF-5 also allows 64-bit signed and unsigned integer types. Define these like any other variable, with one of the new types:

```
if ((retval = nc_def_var(nested_id, "int64_var", NC_INT64, NDIMS_2D, dimids_2d, &int64_varid)))
    ERR(retval);
```

Or in Fortran:

```
retval = nf90_def_var(nested_id, "int64_var", NF90_INT64, dimids_2d, int64_varid)
if (retval /= nf90_noerr) call handle_err(retval)
```

Unsigned Integer Types

NetCDF-4 adds unsigned integer types when using netCDF-4/HDF5 files.

We can define an unsigned short in C like this:

```
if ((retval = nc_def_var(grp1_id, "ushort_var", NC_USHORT, NDIMS_2D, dimids_2d, &ushort_varid)))
    ERR(retval);
```

In Fortran:

```
retval = nf90_def_var(grp1_id, "ushort_var", NF90_USHORT, dimids_2d, ushort_varid)
if (retval /= nf90_noerr) call handle_err(retval)
```

User Defined Types and Strings

User-defined types are a feature in HDF5 which allows users to define their own types, and then use these types as if they were pre-defined atomic types. To use a user-defined type, the type must be defined (while the file is in define mode). This returns a type ID which may be used when defining a variable or attribute.

The Compound Type

The compound type is similar to a C struct. It can contain an arbitrary set of other types (including user-defined types). When matched to a C struct in the code, this can provide dramatically faster access to a collection of variables.

Consider the case of the model restart file. This is a file that is written to hold the values of many model variables at each timestep. As each timestep of the model is run, the restart file is updated with all their values. In this way the model may be restarted at any timestep.

But writing the variables in this way is slow. If there are 100 restart variables, then there must be 100 fseeks to find the appropriate places to write data, and data are written in 100 operations. However, if the restart variables are stored in a struct, and the struct is matched to a compound type (which contains the 100 variables), then there will only be one fseek, and the data may be written in one operation.

In the `user_types.c` example from the NetCDF expansion pack, we define a struct to hold some WeatherObs:

```
/* Compound type: Weather observation */
typedef struct {
    double time;
    float temperature;
    float pressure;
    float humidity;
} WeatherObs;
```

We can then define a corresponding compound data type:

```
/* 1. Define Compound Type */
printf("\n--- Compound Type (weather observation) ---\n");
nc_type compound_typeid;
if ((retval = nc_def_compound(ncid, sizeof(WeatherObs), "weather_obs_t",
                             &compound_typeid)))
```



```

    ERR(retval);
    if ((retval = nc_insert_compound(ncid, compound_typeid, "time",
                                    offsetof(WeatherObs, time), NC_DOUBLE)))
        ERR(retval);
    if ((retval = nc_insert_compound(ncid, compound_typeid, "temperature",
                                    offsetof(WeatherObs, temperature),
                                    NC_FLOAT)))
        ERR(retval);
    if ((retval = nc_insert_compound(ncid, compound_typeid, "pressure",
                                    offsetof(WeatherObs, pressure),
                                    NC_FLOAT)))
        ERR(retval);
    if ((retval = nc_insert_compound(ncid, compound_typeid, "humidity",
                                    offsetof(WeatherObs, humidity),
                                    NC_FLOAT)))
        ERR(retval);
    printf("Defined compound type with 4 fields\n");

```

Using the new typeid, we can define a variable:

```

    if ((retval = nc_def_var(ncid, "observations", compound_typeid, 1,
                            &obs_dimid, &compound_varid)))
        ERR(retval);

```

Later, we define some sample data and write it to the variable:

```

/* Write compound data */
WeatherObs obs_data[NOBS];
for (int i = 0; i < NOBS; i++) {
    obs_data[i].time = 1000.0 + i * 3600.0;
    obs_data[i].temperature = 20.0 + i * 2.0;
    obs_data[i].pressure = 1013.0 + i * 0.5;
    obs_data[i].humidity = 60.0 - i * 5.0;
}
if ((retval = nc_put_var(ncid, compound_varid, obs_data)))
    ERR(retval);
printf("Wrote %d compound observations\n", NOBS);

```

The Compound Type in Fortran

The NetCDF Fortran interface does not support reading or writing variables of compound type. While the API includes functions to define compound types and query their metadata, there is no practical way to move compound data between a Fortran program and a NetCDF file. The reason comes down to a fundamental mismatch between how Fortran and C represent structured data in memory.

In C, a compound type maps naturally to a struct, and the NetCDF C library reads and writes compound data by copying raw bytes between the file and a buffer that matches the struct layout. The programmer controls the byte offsets of each field when defining the compound type, and these offsets must match the memory layout of the C struct. This works because C struct layout is predictable and the programmer can use `offsetof()` to get exact field positions.

Fortran derived types (TYPE) do not offer the same guarantees. The Fortran standard does not specify how a compiler lays out the fields of a derived type in memory. Different compilers, and even different optimization levels on the same compiler, can insert different amounts of padding between fields or reorder them. There is no Fortran equivalent of `offsetof()` in the language standard. The `SEQUENCE` attribute forces declaration order but still does not guarantee specific padding or alignment. This means there is no portable way to construct a Fortran derived type whose memory layout matches the byte offsets required by a NetCDF compound type definition.

A determined programmer could work around this using ISO_C_BINDING to define interoperable types with BIND(C), which would give C-compatible layout. But this pushes the Fortran code into territory where the programmer is essentially writing C semantics in Fortran syntax, and it requires careful manual management of offsets and sizes. The NetCDF Fortran API would also need new overloads of `nf90_put_var` and `nf90_get_var` that accept raw byte buffers or C pointers, which do not exist today. The single compound type test in the `netcdf-fortran` library, written in 2009, only exercises type definition and inquiry. It never attempts to write or read compound data, which reflects the state of the implementation.

For Fortran users who need to work with compound data, the practical alternatives are to decompose the compound into separate scalar variables, or to call the NetCDF C API directly through ISO_C_BINDING. Neither is elegant, but both are reliable and portable across compilers.

The VLEN Type

The variable length type allows the user to define a type which will include ragged arrays. That is, each element of a variable will be an array, and these arrays are of variable length. Special functions are required to help the user deal with VLEN types, and they may be of limited use to Fortran users.

The VLEN may be of any type, including user-defined types.

The VLEN type, like the String, Opaque, and Enum types, is supported in netCDF-4 in order to give full access to existing HDF5 files, which may use these types. NetCDF users may wish to avoid creating files with VLEN types, in order to ensure maximum portability and usefulness of the data. Using fixed length arrays, and setting the length to the maximum size needed, users can store the same data, without VLENS. If compression is turned on, the excess values will be compressed out of the file, and the space used will be similar to that used by VLENS.

Fortran does not deal well with VLENS. Data in VLENS may be hard to read from Fortran programs.

In this example from `user_types.c` in the NetCDF Expansion Pack, we define a VLEN of NC_INT:

```
nc_type vlen_typeid;
if ((retval = nc_def_vlen(ncid, "obs_per_day_t", NC_INT, &vlen_typeid)))
    ERR(retval);
```

We can then define a variable of this type:

```
if ((retval = nc_def_var(ncid, "obs_per_day", vlen_typeid, 1, &day_dimid,
                        &vlen_varid)))
    ERR(retval);
```

We construct some variable length arrays, and write them to this variable:

```
/* Write vlen data */
nc_vlen_t vlen_data[NDAYS];
int day1_obs[] = {10, 15, 20};
int day2_obs[] = {12, 18, 22, 25};
int day3_obs[] = {8, 14};

vlen_data[0].len = 3;
vlen_data[0].p = day1_obs;
vlen_data[1].len = 4;
vlen_data[1].p = day2_obs;
vlen_data[2].len = 2;
vlen_data[2].p = day3_obs;

if ((retval = nc_put_var(ncid, vlen_varid, vlen_data)))
```

```
ERR(retval);
```

VLENs and Fortran

Variable-length types present a particularly difficult challenge for the Fortran interface. In C, each element of a vlen variable is represented by an `nc_vlen_t` struct containing a length field and a pointer to a heap-allocated array. Each element can have a different number of values, and the caller is responsible for freeing the memory after reading by calling `nc_free_vlen`. This design is natural in C, where programmers routinely work with raw pointers and manual memory management.

Fortran has no native equivalent of this structure. Fortran allocatable arrays and pointers carry hidden descriptor metadata that includes rank, bounds, and stride information. This metadata is compiler-specific and does not match the simple two-field layout of `nc_vlen_t`. You cannot pass a Fortran allocatable array where the C library expects a length-and-pointer pair.

The memory management model is also foreign to Fortran. When the C library reads vlen data, it allocates memory internally and hands back pointers that the caller must explicitly free. Fortran programs expect the runtime to manage the lifetime of allocatable memory. Mixing C heap allocations with Fortran deallocation is undefined behavior. The Fortran API does include `nf_free_vlen` and `nf_free_vlens` functions to address this, but using them correctly requires the programmer to understand the C memory model underneath, which defeats much of the purpose of having a Fortran interface.

There is also a structural mismatch. Fortran arrays are rectangular. A two-dimensional Fortran array has the same number of columns in every row. A vlen variable is inherently ragged, with each element holding a different number of values. There is no standard Fortran data structure that represents this naturally. A programmer could build an array of derived types, each containing an allocatable component, but mapping that to and from the flat `nc_vlen_t` buffers expected by the C library requires manual packing and unpacking through `ISO_C_BINDING`.

The `netcdf-fortran` library includes the type definition and inquiry functions for vlen, and they work correctly because they just pass integers and strings down to the C layer. But no one has built the Fortran-side machinery to actually move vlen data in and out of variables, and there are no tests in the `netcdf-fortran` repository that attempt it. Fortran users who need per-element variable-length storage are better served by using a fixed-size dimension large enough to hold the longest element and padding shorter ones with fill values, or by calling the NetCDF C API directly through `ISO_C_BINDING`.

The String Type

The string type is implemented using the `VLEN` type, in which the type is restricted to character types. The string type may be used to store arrays of strings in an efficient manner.

NetCDF provides the string type to increase compatibility between netCDF and HDF5. Similar results can be achieved without the string type by defining a maximum string length, and then using arrays of `NC_CHAR` to hold the strings. With compression turned on, this will also result in the efficient storage of arrays of strings, without the complexity of using VLENs.

Strings and Fortran

The NetCDF-4 data model introduced a string type that is distinct from the classic `char` type used in earlier versions of NetCDF. In C, the string type stores an array of pointers, where each element points to

a separately allocated, null-terminated character string of arbitrary length. This is a natural fit for C programs, which routinely work with arrays of `char*` and manual memory management. The NetCDF C library handles the allocation internally on read and expects the caller to free the memory afterward by calling `nc_free_string`.

The Fortran interface does not support the NetCDF-4 string type for variable data. The `NF90_STRING` constant is defined, and string attributes can be written using `nf_put_att_string`, but there are no overloads of `nf90_put_var` or `nf90_get_var` that handle string-typed variables. The reason is the same pointer-and-heap problem that affects `vlen` types. Each element of a string variable is a separate heap allocation of variable size, represented in memory as a raw C pointer. Fortran has no equivalent of an array of `char` pointers. A Fortran `CHARACTER` array has fixed-length elements stored contiguously in memory with no pointers involved. Mapping between the two representations would require `ISO_C_BINDING` to manipulate `C_PTR` values and manually copy bytes between C strings and Fortran character buffers, including handling null termination that Fortran does not use. The `netcdf-fortran` test suite contains no tests that create a variable of type `NF90_STRING` and write or read data to it.

Fortran programs that need to store text in NetCDF files should use the classic `NF90_CHAR` type with an extra dimension whose length equals the maximum string length. This maps directly to Fortran's fixed-length `CHARACTER` type. A program can declare a `character(len=80)` array, and `nf90_put_var` and `nf90_get_var` handle the data transfer without difficulty. Shorter strings are padded with spaces or null characters to fill the fixed length. This approach is portable across all Fortran compilers, well tested, and has been the standard practice in the Fortran NetCDF community since the original NetCDF-3 interface. The NetCDF-4 string type should be considered a feature available only through the C, Java, and Python interfaces.

The Enum Type

The user-defined enumeration type allows the user to define enumerated values as a data type, as with an `enum` statement in C.

When the user defines an enum type, a base integer type is specified, and then the valid values must each be specified. Only valid values will be writable as the value of an enum variable.

In the NetCDF Expansion Pack example `user_types.c`, we define a C enum type to represent cloud cover:

```
/* Enum type: Cloud cover categories */
typedef enum {
    CLEAR = 0,
    PARTLY_CLOUDY = 1,
    CLOUDY = 2,
    OVERCAST = 3
} CloudCover;
```

We can then define an enum type for these data:

```
/* 3. Define Enumeration Type */
printf("\n--- Enumeration Type (cloud cover) ---\n");
nc_type enum_typeid;
if ((retval = nc_def_enum(ncid, NC_INT, "cloud_cover_t", &enum_typeid)))
    ERR(retval);
CloudCover clear = CLEAR;
CloudCover partly = PARTLY_CLOUDY;
CloudCover cloudy = CLOUDY;
CloudCover overcast = OVERCAST;
```

```

if ((retval = nc_insert_enum(ncid, enum_typeid, "CLEAR", &clear)))
    ERR(retval);
if ((retval = nc_insert_enum(ncid, enum_typeid, "PARTLY_CLOUDY",
                             &partly)))
    ERR(retval);
if ((retval = nc_insert_enum(ncid, enum_typeid, "CLOUDY", &cloudy)))
    ERR(retval);
if ((retval = nc_insert_enum(ncid, enum_typeid, "OVERCAST", &overcast)))
    ERR(retval);
printf("Defined enum type with 4 categories\n");

```

With the type defined, we can define a variable of that type:

```

if ((retval = nc_def_var(ncid, "cloud_cover", enum_typeid, 1, &obs_dimid,
                        &enum_varid)))

```

Now we can write an array of enum:

```

/* Write enum data */
CloudCover cloud_data[NOBS] = {CLEAR, PARTLY_CLOUDY, CLOUDY, PARTLY_CLOUDY, OVERCAST};
if ((retval = nc_put_var(ncid, enum_varid, cloud_data)))
    ERR(retval);

```

An enum type can also be defined and used in Fortran:

```

! Define enum type: cloud_cover_t based on NF90_INT
retval = nf90_def_enum(ncid, NF90_INT, "cloud_cover_t", enum_typeid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_insert_enum(ncid, enum_typeid, "CLEAR", CLEAR)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_insert_enum(ncid, enum_typeid, "PARTLY_CLOUDY", PARTLY_CLOUDY)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_insert_enum(ncid, enum_typeid, "CLOUDY", CLOUDY)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_insert_enum(ncid, enum_typeid, "OVERCAST", OVERCAST)
if (retval /= nf90_noerr) call handle_err(retval)
print *, "Defined enum type cloud_cover_t with 4 members"

```

When writing data, the `nf_put_var()` function from the V2 F77 API. It allows generic, untyped I/O.

```

! Write enum data using nf_put_var (generic untyped I/O)
retval = nf_put_var(ncid, enum_varid, cloud_out)
if (retval /= nf90_noerr) call handle_err(retval)
print *, "Wrote ", NOBS, " cloud cover values"

```

The Opaque Type

The opaque type provides a “bag of bytes” of a fixed size, and stores objects of that size. No mechanism is provided in the netCDF or HDF5 library to look inside these objects, hence the name.

The opaque type is provided for maximal HDF5 compatibility. Most netCDF users will use arrays of a known type, rather than the opaque type.

The example program `user_types.c` in the NetCDF Expansion Pack demonstrates how to use the opaque type. First we define the type, specifying its size and a name for the type:

```

nc_type opaque_typeid;
if ((retval = nc_def_opaque(ncid, CALIB_SIZE, "calibration_t",
                           &opaque_typeid)))
    ERR(retval);

```

With the type defined, we can define a variable:

```

if ((retval = nc_def_var(ncid, "calibration", opaque_typeid, 0, NULL,
                        &opaque_varid)))

```

```
ERR(retval);
```

And then we can write some data:

```
/* Write opaque data */
unsigned char calib_data[CALIB_SIZE];
for (int i = 0; i < CALIB_SIZE; i++) {
    calib_data[i] = (unsigned char)(i * 17);
}
if ((retval = nc_put_var(ncid, opaque_varid, calib_data)))
    ERR(retval);
printf("Wrote %d bytes of opaque calibration data\n", CALIB_SIZE);
```

We can also define and write opaque data with Fortran:

```
! Define opaque type: calibration_t with CALIB_SIZE bytes
retval = nf90_def_opaque(ncid, CALIB_SIZE, "calibration_t", opaque_typeid)
if (retval /= nf90_noerr) call handle_err(retval)
```

We can write data with the `nf_put_var1()` function:

```
! Write opaque data using nf_put_var1 (generic untyped I/O)
index(1) = 1
retval = nf_put_var1(ncid, opaque_varid, index, calib_out)
if (retval /= nf90_noerr) call handle_err(retval)
```

NetCDF-4 Attribute Functions

NetCDF-4 (HDF5-backed files) extends the classic type set with six additional atomic types. The C API adds a typed function for each one. Fortran continues to use the same overloaded `nf90_put_att` and `nf90_get_att`; the compiler selects the correct implementation from the type of the value argument.

New Types in NetCDF-4

The six new types and their C write/read functions are:

NetCDF-4 type	C type	C write function	C read function
NC_UBYTE	unsigned char	nc_put_att_ubyte	nc_get_att_ubyte
NC_USHORT	unsigned short	nc_put_att_ushort	nc_get_att_ushort
NC_UINT	unsigned int	nc_put_att_uint	nc_get_att_uint
NC_INT64	long long	nc_put_att_longlong	nc_get_att_longlong
NC_UINT64	unsigned long long	nc_put_att_ulonglong	nc_get_att_ulonglong
NC_STRING	char **	nc_put_att_string	nc_get_att_string

All six are available only in NetCDF-4 format files. Attempting to use them on a classic-format file returns `NC_ENOTNC4`.

Strings

`NC_STRING` stores variable-length character strings, unlike `NC_CHAR` which stores fixed-length character arrays. Attributes of type `NC_STRING` are arrays of pointers; the library allocates the string

memory on read and you must free it with `nc_free_string()`. `dump_nc4_metadata.c` shows the pattern (example code from NetCDF Expansion Pack program `dump_nc4_metadata.c`.)

```
char **val = malloc(len * sizeof(char *));
if ((retval = nc_get_att_string(ncid, varid, att_name, val)))
    ERR(retval);
for (i = 0; i < len; i++)
    printf("%s\\\"%s\\\" ", i ? ", " : "", val[i] ? val[i] : "(null)");
nc_free_string(len, val);
free(val);
```

In Fortran, `NC_STRING` variables and attributes are not handled portably by the Fortran 90 interface. The Fortran `dump_nc4_metadata` example notes this explicitly and reports string-typed attributes by type name only, without reading their values.

Reading Attributes of Unknown Type

When reading a file whose attribute types are not known in advance, use `nc_inq_att()` (C) or `nf90_inquire_attribute()` (Fortran) to retrieve the type and length, then dispatch to the appropriate typed getter. The NetCDF Expansion Pack program `dump_nc4_metadata.c` does this for every attribute it encounters, covering all classic and NetCDF-4 types including `NC_STRING`.

Reading and Writing Data with the Enhanced Model

The enhanced model extends the read/write API in several directions. New atomic types require typed functions in C. Variables inside groups need the group's `ncid` rather than the file's root `ncid`. User-defined types introduce their own write and read patterns. Strings require memory management that differs from all other types. This section covers each area in turn, following the pattern established in the Classic Model section above.

Creating a NetCDF-4 File

All enhanced-model features require the `NC_NETCDF4` flag at file creation. Without it, the library creates a classic-format file and rejects any enhanced-model operation.

In C:

```
if ((retval = nc_create(FILE_NAME, NC_CLOBBER|NC_NETCDF4, &ncid)))

    ERR(retval);
```

In Fortran:

```
retval = nf90_create(FILE_NAME, NF90_CLOBBER + NF90_NETCDF4, ncid)
```

```
if (retval != nf90_noerr) call handle_err(retval)
```

To enforce classic-model behavior in a netCDF-4/HDF5 file, add [NC_CLASSIC_MODEL](#) (C) or [NF90_CLASSIC_MODEL](#) (Fortran) alongside [NC_NETCDF4](#). This prevents enhanced features from being used while still producing an HDF5 file.

Writing and Reading New Atomic Types

The five additional integer types available in netCDF-4 each have their own typed C functions. These follow the same naming pattern as the classic types but use the type name as a suffix.

NetCDF-4 type	C write function	C read function
NC_UBYTE	nc_put_var_uchar	nc_get_var_uchar
NC_USHORT	nc_put_var_ushort	nc_get_var_ushort
NC_UINT	nc_put_var_uint	nc_get_var_uint
NC_INT64	nc_put_var_longlong	nc_get_var_longlong
NC_UINT64	nc_put_var_ulonglong	nc_get_var_ulonglong

This code from [groups.c](#) in the NetCDF Expansion Pack writes one variable of each type, each into a different group:

```
if ((retval = nc_put_var_uchar(ncid, ubyte_varid, &ubyte_data[0][0])))
```

```
    ERR(retval);
```

```
if ((retval = nc_put_var_ushort(grp1_id, ushort_varid, &ushort_data[0][0])))
```



```
ERR(retval);
```

```
if ((retval = nc_put_var_uint(grp2_id, uint_varid, &uint_data[0][0])))
```

```
ERR(retval);
```

```
if ((retval = nc_put_var_longlong(nested_id, int64_varid, &int64_data[0][0])))
```

```
ERR(retval);
```

```
if ((retval = nc_put_var_ulonglong(nested_id, uint64_varid, &uint64_data[0][0][0])))
```

```
ERR(retval);
```

Reading them back uses the parallel `nc_get_var_*` functions:

```
if ((retval = nc_get_var_uchar(ncid, ubyte_varid, &ubyte_in[0][0])))
```

```
ERR(retval);
```

```
if ((retval = nc_get_var_ushort(grp1_id, ushort_varid, &ushort_in[0][0])))
```

```
ERR(retval);
```

```
if ((retval = nc_get_var_longlong(nested_id, int64_varid, &int64_in[0][0])))
```

```
ERR(retval);
```

```
if ((retval = nc_get_var_ulonglong(nested_id, uint64_varid, &uint64_in[0][0][0])))
```

```
ERR(retval);
```

Notice that the `ncid` passed to each call is the group ID where that variable lives, not the root file ID. Variables defined in a group are only accessible through that group's `ncid`.

In Fortran, `nf90_put_var` and `nf90_get_var` are overloaded and handle all five new types automatically. The correct implementation is selected from the declared type of the array argument, just as with classic types. No new function names are needed.

Variables in Groups

When reading a file with groups, use `nc_inq_grp_ncid()` to navigate to a group by name. The returned group ID then serves as the `ncid` for all variable operations in that group. This code from `groups.c` navigates the group hierarchy after reopening the file:

```
if ((retval = nc_inq_grp_ncid(ncid, "SubGroup1", &grp1_id)))  
  
    ERR(retval);  
  
if ((retval = nc_inq_grp_ncid(ncid, "SubGroup2", &grp2_id)))  
  
    ERR(retval);  
  
if ((retval = nc_inq_grp_ncid(grp2_id, "NestedGroup", &nested_id)))  
  
    ERR(retval);
```

Once you have the group ID, `nc_inq_varid()` and the typed get/put functions work exactly as they do in the root group. Variable IDs are scoped to their group; the same integer ID value in two different groups refers to two different variables.

In Fortran, `nf90_inq_grp_ncid()` returns the group ID:

```
retval = nf90_inq_grp_ncid(ncid, "SubGroup1", grp1_id)  
  
if (retval /= nf90_noerr) call handle_err(retval)
```

Appending Along Multiple Unlimited Dimensions

In the classic model only one unlimited dimension is permitted. In netCDF-4/HDF5 files any number of unlimited dimensions may be defined, and they may appear in any position in a variable's dimension list. The `multi_unlimited.c` example in the NetCDF Expansion Pack demonstrates a variable indexed by both an unlimited `station` dimension and an unlimited `time` dimension:

```
if ((retval = nc_def_dim(ncid, "station", NC_UNLIMITED, &station_dimid)))

    ERR(retval);

if ((retval = nc_def_dim(ncid, "time", NC_UNLIMITED, &time_dimid)))

    ERR(retval);

dimids[0] = station_dimid;

dimids[1] = time_dimid;

if ((retval = nc_def_var(ncid, "temperature", NC_FLOAT, NDIMS, dimids, &varid)))

    ERR(retval);
```

Appending along either dimension uses `nc_put_vara_float()` with the appropriate `start` and `count` arrays, exactly as with a single unlimited dimension. Because HDF5 stores data in chunks, there is no restriction on which dimension is unlimited or where unlimited dimensions appear.

Writing and Reading Compound Variables

Compound type variables are written and read with the generic `nc_put_var()` and `nc_get_var()` functions in C. These functions transfer raw bytes between the file and a C struct whose layout matches the compound type definition. The `user_types.c` example from the NetCDF Expansion Pack writes an array of `WeatherObs` structs:

```
WeatherObs obs_data[NOBS];
```

```

for (int i = 0; i < NOBS; i++) {

    obs_data[i].time = 1000.0 + i * 3600.0;

    obs_data[i].temperature = 20.0 + i * 2.0;

    obs_data[i].pressure = 1013.0 + i * 0.5;

    obs_data[i].humidity = 60.0 - i * 5.0;

}

if ((retval = nc_put_var(ncid, compound_varid, obs_data)))

    ERR(retval);

```

Reading uses `nc_get_var()` into a matching struct array:

```

WeatherObs obs_read[NOBS];

if ((retval = nc_get_var(ncid, compound_varid, obs_read)))

    ERR(retval);

```

The struct layout must match the byte offsets registered when the type was defined with `nc_insert_compound()`. Use `offsetof()` when defining the type and declare the struct fields in the same order to ensure the layouts agree.

Compound variable I/O is not available from Fortran. The Fortran interface has no mechanism to match derived type memory layouts to the byte offsets required by a compound type definition. Fortran users who need to store grouped fields should decompose them into separate scalar variables, or call the NetCDF C API directly through `ISO_C_BINDING`. See the discussion under The Compound Type above for a full explanation.

Writing and Reading VLEN Variables

Variable-length type variables are written and read with `nc_put_var()` and `nc_get_var()` using arrays of `nc_vlen_t`. Each element of the array has a `len` field for the number of values and a `p` pointer to the data for that element. After reading, call `nc_free_vlen()` to release the library-allocated memory. This code from `user_types.c` demonstrates both:

```
/* Write */

nc_vlen_t vlen_data[NDAYS];

int day1_obs[] = {10, 15, 20};

int day2_obs[] = {12, 18, 22, 25};

int day3_obs[] = {8, 14};

vlen_data[0].len = 3; vlen_data[0].p = day1_obs;

vlen_data[1].len = 4; vlen_data[1].p = day2_obs;

vlen_data[2].len = 2; vlen_data[2].p = day3_obs;

if ((retval = nc_put_var(ncid, vlen_varid, vlen_data)))

    ERR(retval);

/* Read */

nc_vlen_t vlen_read[NDAYS];

if ((retval = nc_get_var(ncid, vlen_varid, vlen_read)))

    ERR(retval);
```

```
/* ... use vlen_read[d].len and (int *)vlen_read[d].p ... */
```

```
if ((retval = nc_free_vlen(vlen_read)))
```

```
    ERR(retval);
```

VLEN variable I/O is not available from Fortran for the same reasons as compound types. The `nc_vlen_t` struct depends on C pointer semantics and heap management that have no portable Fortran equivalent. See the discussion under The VLEN Type above.

Writing and Reading String Variables

String variables in C use `nc_put_var_string()` and `nc_get_var_string()`. The write call accepts an array of `const char *` pointers. The read call fills an array of `char *` pointers, each pointing to library-allocated memory; call `nc_free_string()` after use. This code from `user_types.c` writes station names and then reads them back:

```
/* Write */
```

```
const char *station_names[NSTATIONS] = {
```

```
    "Boulder, CO", "Cape Canaveral, FL",
```

```
    "Wallops Island, VA", "White Sands, NM"
```

```
};
```

```
if ((retval = nc_put_var_string(ncid, string_varid, station_names)))
```

```
    ERR(retval);
```

```
/* Read */
```

```

char *station_read[NSTATIONS];

if ((retval = nc_get_var_string(ncid, string_varid, station_read)))

    ERR(retval);

/* ... use station_read[i] ... */

if ((retval = nc_free_string(NSTATIONS, station_read)))

    ERR(retval);

```

String variable I/O is not supported in the Fortran interface. Fortran programs that need to store text should use `NF90_CHAR` variables with an extra character-length dimension instead. See the discussion under Strings and Fortran above.

Writing and Reading Enum Variables

Enum variables store integer data, so the write and read operations transfer integer arrays. In C, use `nc_put_var()` and `nc_get_var()` with a buffer whose C type matches the enum's base integer type. In the `user_types.c` example the enum is based on `NC_INT` and the underlying C type is `CloudCover` (an `int`-sized enum):

```

CloudCover cloud_data[NOBS] = {CLEAR, PARTLY_CLOUDY, CLOUDY, PARTLY_CLOUDY,
OVERCAST};

if ((retval = nc_put_var(ncid, enum_varid, cloud_data)))

    ERR(retval);

CloudCover cloud_read[NOBS];

if ((retval = nc_get_var(ncid, enum_varid, cloud_read)))

```

```
ERR(retval);
```

In Fortran, enum variable I/O uses the F77 generic functions `nf_put_var` and `nf_get_var`, which transfer raw bytes without type checking. The variable must be defined with `nf_def_var` rather than `nf90_def_var`, because `nf90_def_var` validates the type argument against built-in types and rejects user-defined type IDs. This code from the `f_user_types.f90` example in the NetCDF Expansion Pack shows the pattern:

```
! Define using F77 API (nf_def_var accepts user-defined type IDs)
```

```
retval = nf_def_var(ncid, "cloud_cover", enum_typeid, 1, dimids, enum_varid)
```

```
if (retval /= nf90_noerr) call handle_err(retval)
```

```
! Write enum data as integer array
```

```
retval = nf_put_var(ncid, enum_varid, cloud_out)
```

```
if (retval /= nf90_noerr) call handle_err(retval)
```

```
! Read enum data back
```

```
retval = nf_get_var(ncid, enum_varid, cloud_in)
```

```
if (retval /= nf90_noerr) call handle_err(retval)
```

The integer values written must be members of the enum type; the library enforces this on write.

Writing and Reading Opaque Variables

Opaque variables store fixed-size binary blobs. In C, `nc_put_var()` and `nc_get_var()` treat the buffer as raw bytes; no type conversion is performed. In `user_types.c` an array of `unsigned char` fills a scalar opaque variable:


```

unsigned char calib_data[CALIB_SIZE];

for (int i = 0; i < CALIB_SIZE; i++)

    calib_data[i] = (unsigned char)(i * 17);

if ((retval = nc_put_var(ncid, opaque_varid, calib_data)))

    ERR(retval);

unsigned char calib_read[CALIB_SIZE];

if ((retval = nc_get_var(ncid, opaque_varid, calib_read)))

    ERR(retval);

```

In Fortran, a fixed-length `character` buffer of the correct size serves as the container. As with enum variables, `nf_def_var` is needed to define the variable, and `nf_put_var1` / `nf_get_var1` handle single-element I/O. From `f_user_types.f90`:

```

character(len=CALIB_SIZE) :: calib_out, calib_in

! Write opaque data

index(1) = 1

retval = nf_put_var1(ncid, opaque_varid, index, calib_out)

if (retval /= nf90_noerr) call handle_err(retval)

! Read opaque data

retval = nf_get_var1(ncid, opaque_varid, index, calib_in)

```

if (retval != nf90_noerr) call handle_err(retval)

Summary of I/O Support by Type and Language

Type	C write/read	Fortran write/read
New atomics	nc_put_var_/nc_get_var_	nf90_put_var / nf90_get_var (overloaded)
Compound	nc_put_var / nc_get_var	Not supported
VLEN	nc_put_var / nc_get_var + nc_free_vlen	Not supported
String	nc_put_var_string / nc_get_var_string + nc_free_string	Not supported
Enum	nc_put_var / nc_get_var	nf_put_var / nf_get_var (F77 API)
Opaque	nc_put_var / nc_get_var	nf_put_var1 / nf_get_var1 (F77 API)

(The full enhanced-model examples for C are covered in chapter 6 and for Fortran in chapter 7.)

Choosing Your Data Model

Selecting the appropriate NetCDF data model is a critical decision that impacts file compatibility, performance, and future flexibility. This section provides a practical framework for making this choice based on your specific requirements.

Decision Framework

The first step in choosing your data model is to assess your compatibility requirements. The classic model is required when you need to share data with legacy systems or older NetCDF libraries, work with tools that only support NetCDF-3 format, need maximum portability across different scientific software, or are targeting embedded systems with limited HDF5 support. The enhanced model becomes viable when all users have NetCDF-4.0 or later, HDF5 libraries are available in your environment, and cross-platform compatibility includes modern systems.

Next, evaluate your data complexity. Simple datasets with single instrument or model output, flat variable organization, one time dimension for growth, and standard atomic data types only work well with the classic model. Complex datasets involving multiple instruments or experiments, hierarchical data organization needs, multiple growth dimensions like time plus ensemble, or requirements for 64-bit integers, unsigned types, strings, or structured data benefit from the enhanced model.

Finally, consider your performance requirements. The classic model offers advantages for faster access to simple datasets, lower memory overhead, simpler file structure, and better performance on very small files. The enhanced model provides superior compression options, efficient chunked access for large datasets, parallel I/O capabilities, and better subset performance for complex queries.

Use Case Examples

Domain	Data Characteristics	Recommended Model	Rationale
Simple Climate Station	Single location, time series, temperature/precipitation	Classic	One unlimited dimension (time), flat structure, maximum compatibility
Weather Model Restart	Model state variables, structured data, performance critical	Enhanced	Compound types for structured data, compression, performance needs
Multi-Instrument Satellite	Multiple sensors, hierarchical organization, various data types	Enhanced	Groups for instrument organization, multiple unlimited dims, new types
Ocean Buoy Network	Many stations, irregular data, text observations	Classic	Simple structure, wide compatibility, text as fixed-length char arrays
Climate Model Ensemble	Multiple model runs, time + ensemble growth, large datasets	Enhanced	Multiple unlimited dimensions, groups for organization, compression
Atmospheric Chemistry	Many species, vertical levels, time series, 64-bit counts	Enhanced	Need for 64-bit integers, groups for species organization, compression
Reanalysis Dataset	Global grid, multiple variables, long time series, large files	Enhanced	Compression essential, chunking for spatial subsets, groups for variables
Educational Dataset	Simple examples, broad audience, learning tools	Classic	Universal compatibility, simple structure for learning
Real-time Sensor	Continuous data growth,	Classic	Proven reliability, simple

Network	multiple stations, reliability critical		structure for robust operations
---------	---	--	---------------------------------

Performance Considerations

File size thresholds provide practical guidance for model selection. Files smaller than 10 megabytes often perform faster with the classic model due to simpler structure. Files between 10 megabytes and 1 gigabyte can work well with either model, so the choice should be based on feature requirements. Files larger than 1 gigabyte typically benefit from the enhanced model due to compression and chunking advantages.

Access patterns also influence the optimal choice. Sequential time series access works excellently with the classic model. Spatial subsetting operations are superior with the enhanced model when proper chunking is used. Complex queries benefit from the organizational features of enhanced model groups and types. Frequent small writes may have lower overhead with the classic model.

Enhanced model compression can reduce file sizes by fifty to ninety percent for many scientific datasets, but this adds CPU overhead during both writes and reads. When performance is critical, you should benchmark with your actual data patterns to understand the trade-offs.

NetCDF Data Models – Key Takeaways

- Classic model: dimensions + variables + attributes, one unlimited dimension
- Enhanced model adds groups, multiple unlimited dimensions, new atomic types, and user-defined types
- Multiple unlimited dimensions require netCDF-4/HDF5 format
- Coordinate variable convention: a 1D variable with the same name as its dimension stores the physical coordinate values
- Groups provide hierarchical organization; each group functions as a classic model file
- New atomic types include unsigned integers, 64-bit integers, and strings
- User-defined types (compound, VLEN, enum, opaque) enable complex data structures; Fortran support varies by type
- NC_CLASSIC_MODEL flag creates netCDF-4/HDF5 files restricted to classic model features
- Choose classic for compatibility and simplicity; choose enhanced for complex, large, or hierarchical datasets

Chapter 4 Binary Formats

Learning Objectives

- Choose the right binary format (Classic, CDF-5, NetCDF-4/HDF5, ncZarr) for a given set of requirements
- Create netCDF files in each format using the appropriate creation flags in C and Fortran
- Detect the binary format of an existing netCDF file using `nc_inq_format()` and `ncdump`
- Explain the tradeoffs between contiguous and chunked storage in HDF5
- Convert files between formats using `nccopy`
- Identify when ncZarr is and is not appropriate for a workflow

History of NetCDF Binary Format Development

NetCDF was first released in 1989 with a single binary data format, now known as netCDF classic format. This remains the default format for netCDF files.

At the time, the 2 GB file size limitation of the classic format was not a practical concern. Operating systems and storage hardware imposed the same constraints. A file larger than 2 GB was simply not possible. As manufacturers and operating systems moved to 64-bit architectures through the 1990s and 2000s, those limits fell away, and netCDF had to evolve.

The timeline of format introductions:

- 2004, netCDF 3.6.0: The 64-bit offset format raised the file size ceiling beyond 2 GB, though individual variables remained limited to about 4 GB.
- 2008, netCDF 4.0: The netCDF-4/HDF5 format used HDF5 as a storage layer, adding compression, groups, user-defined types, and multiple unlimited dimensions.
- 2016, netCDF 4.4.0: The CDF-5 format extended the classic binary layout with 64-bit integers throughout, removing all size limitations while preserving the simplicity of the classic data model. CDF-5 originated in the PnetCDF project for high-performance parallel I/O.
- 2021, netCDF 4.8.0: The ncZarr format added support for Zarr, a cloud-native storage format designed for efficient HTTP access to data in object stores like Amazon S3, Google Cloud Storage, and Azure Blob Storage.

Each new format addressed limitations of its predecessors while the older formats continued to be fully supported. The choice of format depends on file size requirements, feature needs, and the target computing environment. The sections that follow describe each format in detail.

Opening NetCDF Files of Different Formats

When opening a netCDF file, you do not need to specify or even know the binary format. The library detects the format automatically. The same `nc_open()` call works for classic, 64-bit offset, CDF-5, NetCDF-4/HDF5, and ncZarr files.

Format only needs to be specified when creating a file.

From the command line, learn the format of a file with `ncdump`'s `-k` option:

```
$ ncdump -k format_classic.nc
classic
$ ncdump -k format_netcdf4.nc
netCDF-4
```

The format can also be determined programmatically. After opening a file, you can query which format it uses with `nc_inq_format()`. This returns one of five format constants:

In C:

```
int ncid, format;

if ((retval = nc_open("mydata.nc", NC_NOWRITE, &ncid)))
    ERR(retval);

if ((retval = nc_inq_format(ncid, &format)))
    ERR(retval);

if (format == NC_FORMAT_CLASSIC)
    printf("Classic (CDF-1)\n");
else if (format == NC_FORMAT_64BIT_OFFSET)
    printf("64-bit Offset (CDF-2)\n");
else if (format == NC_FORMAT_64BIT_DATA)
    printf("CDF-5\n");
else if (format == NC_FORMAT_NETCDF4)
    printf("NetCDF-4/HDF5\n");
else if (format == NC_FORMAT_NETCDF4_CLASSIC)
    printf("NetCDF-4/HDF5 Classic Model\n");
```

In Fortran:

```
integer :: ncid, format_type
retval = nf90_open("mydata.nc", NF90_NOWRITE, ncid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_inquire(ncid, formatNum=format_type)
if (retval /= nf90_noerr) call handle_err(retval)
```

This is useful when writing tools that need to handle files of any format, or when verifying that a file was created in the expected format. The `format_variants` example program in this book demonstrates this technique.

Choosing a Format

Before diving into the details of each format, here is a quick guide. Start from the top and follow the first condition that matches your situation.

Do you need to access data in cloud object storage (S3, Google Cloud, Azure)?

Use ncZarr. It is the only netCDF format designed for HTTP-based access to data stored in object stores.

Do you need compression, groups, user-defined types, or multiple unlimited dimensions?

Use NetCDF-4/HDF5. It is the most feature-rich format and the recommended default for new applications.

Do you need parallel I/O without depending on the HDF5 library?

Use CDF-5 with PnetCDF. It provides MPI-based parallel I/O using the classic data model, without requiring HDF5.

Do you need 64-bit integer data types or variables larger than 4 GB, but not HDF5 features?

Use CDF-5. It extends the classic binary layout with 64-bit integers throughout, removing all size limitations.

Are you writing small files (under 2 GB) that must be readable by the oldest netCDF software?

Use Classic format. Every version of the netCDF library ever released can read it.

Are you maintaining code that already uses the 64-bit offset format?

Continue using it for compatibility with existing workflows, but consider migrating to CDF-5 when practical. CDF-5 removes the 4 GB per-variable limitation while keeping the same classic data model.

Not sure?

Use NetCDF-4/HDF5. It handles the widest range of use cases, supports compression that can dramatically reduce file sizes, and is the format most actively developed and tested.

Format Details

The sections that follow describe each format in detail. The table below summarizes key differences:

Feature	Classic	64-bit Offset	CDF-5	NetCDF-4/HDF5	NetCDF-4/HDF5 Classic Model	ncZarr
Max File Size	2 GB	Unlimited	8 EB	8 EB	8 EB	8 EB
Max Variable Size	2 GB	4 GB*	8 EB	8 EB	8 EB	8 EB
Max Dim Size	2 ³²	2 ³²	2 ⁶³	2 ⁶³	2 ⁶³	2 ⁶³
Unlimited Dimensions	1	1	1	Unlimited	1	Unlimited
Compression	No	No	No	Yes	No	Yes
Parallel I/O	No	No	Yes†	Yes	No	Yes‡
Groups	No	No	No	Yes	No	Yes

User-defined Types	No	No	No	Yes	No	Yes
64-bit Integers	No	No	Yes	Yes	No	Yes
Cloud Optimized	No	No	No	No	No	Yes

**Fixed variable limit and record size limit ~4GB*

‡Requires PnetCDF

‡Requires Zarr-compatible storage

Understanding which format to use can help ensure optimal performance and compatibility for your application.

For most new applications, netCDF-4/HDF5 is the recommended format. It provides the most features and flexibility while maintaining excellent performance. NetCDF-4/HDF5 supports compression, which can dramatically reduce file sizes. It offers hierarchical groups for organizing complex datasets, user-defined types for specialized data structures, and multiple unlimited dimensions for greater flexibility. The format handles extremely large files efficiently and supports all the features of the classic data model plus the enhanced model extensions.

For high-performance computing with parallel I/O: both netCDF-4/HDF5 (with parallel HDF5) and CDF-5 (with PnetCDF) provide excellent performance for large-scale scientific computing. HDF5's chunked storage excels for random access patterns and when compression is needed, while CDF-5's contiguous storage is optimal for sequential access patterns. Both support efficient parallel I/O through their respective libraries. The choice should be based on feature requirements and access patterns.

Classic Format

The classic netCDF binary format is the original netCDF binary format. It remains the default format for netCDF data, and provides great performance, especially for files that are not very large, and therefore don't require compression.

The classic format starts to hit limitations at the 2 GB range. This relates to the maximum byte offset that can be stored in a 32-bit integer.

Since the classic format is the default, there is no flag to select it. The NC_CLOBBER flag can be used to overwrite existing files of the same name. In C:

```
/* CDF-1: Original classic format (2GB file/variable limit).
 * No format flag needed – NC_CLOBBER alone creates a classic CDF-1 file. */
printf("Creating classic (CDF-1) format file: format_classic.nc\n");
if ((retval = nc_create("format_classic.nc", NC_CLOBBER, &ncid))
    ERR(retval);
```

In Fortran:

```
! CDF-1: Original classic format (2GB file/variable limit).
! No format flag needed - NF90_CLOBBER alone creates a classic CDF-1 file.
print *, "Creating classic (CDF-1) format file: f_format_classic.nc"
retval = nf90_create("f_format_classic.nc", NF90_CLOBBER, ncid)
```


CDF-5

CDF-5 (also called PnetCDF format) was introduced in netCDF 4.4.0. It extends the classic binary format by removing size limitations while keeping the simplicity and performance of the classic data model. It was developed by the PnetCDF project at Argonne National Laboratory and Northwestern University for high-performance parallel I/O.

CDF-5 uses 64-bit integers throughout the file structure. Variables can be up to 2^{63} bytes (effectively unlimited). Dimensions can also reach 2^{63} , far exceeding earlier limits. The format adds NC_INT64 and NC_UINT64 data types not available in classic or 64-bit offset formats.

The file structure matches the classic format: metadata first, then data. This keeps it simple to implement, efficient for sequential access, and familiar to anyone who knows the classic format.

CDF-5 is a good choice when:

- Files exceed 64-bit offset format limits but you do not need netCDF-4/HDF5 features
- Parallel I/O is needed (PnetCDF provides MPI-based parallel writes to CDF-5 files)
- Data consists of 64-bit integers
- Classic model simplicity is preferred and data does not compress well

Limitations: no built-in compression, no groups, no user-defined types, and not all software reads CDF-5 (netCDF-C 4.4.0+ required).

To create a CDF-5 file, pass NC_64BIT_DATA (or the equivalent NC_CDF5) to nc_create():

```
/* CDF-5: 64-bit data format (unlimited variable sizes) */
printf("Creating NC_64BIT_DATA format file: format_64bit_data.nc\n");
if ((retval = nc_create("format_64bit_data.nc", NC_64BIT_DATA | NC_CLOBBER, &ncid)))
    ERR(retval);
```

In Fortran:

```
! CDF-5: 64-bit data format (unlimited variable sizes)
print *, "Creating NF90_64BIT_DATA format file: f_format_64bit_data.nc"
retval = nf90_create("f_format_64bit_data.nc", NF90_64BIT_DATA, ncid)
if (retval /= nf90_noerr) call handle_err(retval)
```

The file then works like any other netCDF file through the same API. For parallel I/O, build netCDF with PnetCDF support and use nc_create_par()/nc_open_par() instead of nc_create()/nc_open().

NetCDF-4/HDF5 Format

Starting with version 4.0, the netCDF C library can read and write HDF5 files.

HDF5 (Hierarchical Data Format 5) is a data format and associated libraries originally developed at the Supercomputer Center at the University of Illinois Champaign-Urbana campus. The HDF5 team later formed their own company, the HDF Group, to manage and develop the HDF5 and HDF4 formats. HDF5 was a completely new data format, written using the lessons learned from the HDF4 project.

HDF5 includes many advanced features. In addition to supporting almost all of the features of the classic netCDF library, HDF5 supports hierarchical groups, additional types, user-defined types, and pluggable filters for data. These filters allow for built-in data compression, among other features.

Storage Architecture

HDF5 organizes a file as a rooted tree of groups and datasets. Every HDF5 file begins with a superblock that points to the root group. Groups contain other groups or datasets, forming a hierarchy similar to a filesystem. Metadata such as dimension information, attributes, and dataspace descriptions are stored in B-tree and heap structures that allow efficient lookup even when a file contains thousands of objects. The netCDF-4 library maps its flat namespace of dimensions, variables, and attributes onto this tree, placing everything in the root group unless the program explicitly creates subgroups.

Each dataset in an HDF5 file can use one of two storage layouts. Contiguous layout stores all of a variable's data in a single, unbroken block, which gives the best sequential read performance but does not support compression or extensible dimensions. Chunked layout divides the data into fixed-size rectangular tiles that can be stored anywhere in the file. Chunked storage is required for compression, because HDF5 applies filters to individual chunks rather than to the dataset as a whole. It also allows datasets to grow along any dimension without rewriting existing data. The netCDF-4 library defaults to chunked storage for any variable that has an unlimited dimension or that has compression enabled, and uses contiguous storage otherwise.

Data Types

HDF5 greatly expands the number of data types available in netCDF. Unsigned and 64-bit integer types are now available.

HDF5 also introduces user-defined types. Users define these types and then use them through the netCDF API as if they were native types.

HDF5 Dimension Scales

HDF5 has no concept of shared dimensions. Datasets have shapes, but there is no way to express that two datasets share a dimension rather than just coincidentally having the same size.

HDF5 addresses this with dimension scales. A dimension scale is a special dataset that represents coordinate values along a dimension. Other datasets attach to dimension scales to declare which dimensions they use.

NetCDF uses this mechanism to map its dimension model onto HDF5. For each netCDF dimension, the library creates a dimension scale in the HDF5 file and attaches each variable's dataset to the appropriate scales. Coordinate variables (variables with the same name as a dimension) are stored directly as dimension scale datasets. This mapping is transparent to netCDF users. Dimensions work exactly as they do in classic files. The dimension scales are only visible when examining files with HDF5 tools like h5dump.

One performance issue emerged over time: when files contain hundreds or thousands of objects, reading dimension scale information becomes very slow. Starting with netcdf-c-4.4.0, netCDF stores dimension information as an attribute table instead, which makes file opening much faster.

HDF5 Chunking

One of the most important features of HDF5 is chunked storage. In the classic netCDF formats, variable data are stored contiguously in the file. HDF5 offers an alternative: chunked storage, where data are divided into fixed-size chunks that can be scattered throughout the file.

Chunking enables compression (HDF5 can only compress chunked datasets), allows efficient partial I/O by reading only the chunks containing requested data, and permits datasets to grow along any dimension. The chunk size is set with the `nc_def_var_chunking()` function.

For detailed information on chunking and how to optimize chunk sizes for your application, see Chapter 10: NetCDF-4/HDF5 Performance Fundamentals.

HDF5 Filters

HDF5 filters are operations applied to data as it is written to or read from a file. The most common use is compression, but filters can also perform checksumming, encryption, or custom data transformations. Filters are applied to each chunk independently.

The netCDF C library provides access to several HDF5 filters, including deflate (zlib compression) and shuffle filters. Filters are specified on a per-variable basis, allowing optimization of each variable based on its characteristics.

For detailed information on compression filters, the shuffle filter, and choosing compression settings, see Chapter 10: NetCDF-4/HDF5 Performance Fundamentals and Chapter 12: Advanced Compression Techniques.

Creating and Inspecting NetCDF-4/HDF5 Files

Creating NetCDF/HDF5 Files

To create a file in NetCDF/HDF5 format, use the `NC_NETCDF4` flag in `nc_create()`:

```
/* NetCDF-4: HDF5-based format (groups, compression, user-defined types) */
printf("Creating NC_NETCDF4 format file: format_netcdf4.nc\n");
if ((retval = nc_create("format_netcdf4.nc", NC_NETCDF4 | NC_CLOBBER, &ncid)))
    ERR(retval);
```

Or in Fortran:

```
! NetCDF-4: HDF5-based format (groups, compression, user-defined types)
print *, "Creating NF90_NETCDF4 format file: f_format_netcdf4.nc"
retval = nf90_create("f_format_netcdf4.nc", NF90_NETCDF4, ncid)
if (retval /= nf90_noerr) call handle_err(retval)
```

The Classic Model Flag

Creating netCDF/HDF5 files allows the use of the enhanced model, including groups, new data types, multiple unlimited dimensions, and compression.

However, we can also create a netCDF/HDF5 file which will conform to the rules of the classic data model. This feature was originally added to enable the use of existing netCDF tests with the new netCDF-4 code. However, it has also proved useful for some users.

With the `NC_CLASSIC_MODEL` flag, we can create a netCDF-4/HDF5 file which will not allow groups, multiple unlimited dimensions, etc. It will force the file to follow the rules of the classic model. The resulting data files can be easily converted to any of the netCDF classic formats.

In C:

```
/* NetCDF-4 Classic Model: HDF5 storage with classic data model restrictions */
printf("Creating NC_NETCDF4|NC_CLASSIC_MODEL format file: format_netcdf4_classic.nc\n");
if ((retval = nc_create("format_netcdf4_classic.nc",
                       NC_NETCDF4 | NC_CLASSIC_MODEL | NC_CLOBBER, &ncid)))
    ERR(retval);
```

Or in Fortran:

```
! NetCDF-4 Classic Model: HDF5 storage with classic data model restrictions
print *, "Creating NF90_NETCDF4+NF90_CLASSIC_MODEL format file"
retval = nf90_create("f_format_netcdf4_classic.nc", &
                    IOR(NF90_NETCDF4, NF90_CLASSIC_MODEL), ncid)
if (retval /= nf90_noerr) call handle_err(retval)
```

Using h5dump

HDF5 comes with the `h5dump` utility. Just as `ncdump` shows the contents of a netCDF file, `h5dump` shows the contents of an HDF5 file.

`h5dump` is an invaluable tool for debugging netCDF-4 programming problems. With `h5dump`, programmers can see exactly how HDF5 is expressing the netCDF-4 file.

ncZarr Cloud Format

Starting with version 4.8.0, the netCDF C library added support for the Zarr storage format. Zarr is a cloud-native format designed for efficient access to large scientific datasets stored in object storage systems like Amazon S3, Google Cloud Storage, and Azure Blob Storage.

The ncZarr implementation allows netCDF to read and write Zarr datasets using the familiar netCDF API. This means that existing netCDF code can be adapted to work with cloud storage with minimal changes. The same `nc_open()`, `nc_create()`, and data access functions work with Zarr datasets, just as they do with classic and HDF5 formats.

Zarr Storage Model

Unlike the classic netCDF formats which store data in a single binary file, Zarr stores data in a collection of files organized in a directory structure. Each chunk of a variable's data is stored in a separate file, and metadata is stored in JSON files with names like `.zarray`, `.zgroup`, and `.zattrs`.

This design is particularly well-suited to object storage systems, where data are accessed via HTTP requests. When reading a subset of a large array, only the chunks containing the requested data need to be retrieved from storage. This can dramatically reduce the amount of data transferred, especially when working with very large datasets.

Zarr datasets can be stored in multiple backends: local filesystem directories (for testing or local workflows), cloud object storage (S3, Google Cloud, Azure for production cloud-native applications), or even ZIP archives (for portable distribution). The storage backend is selected automatically based on the URL scheme or path provided.

Using ncZarr

To create a Zarr dataset, use the `NC_ZARR` flag when calling `nc_create()`. The path may be a local directory or a URL pointing to cloud storage:

```
nc_create("s3://mybucket/mydata.zarr#mode=nczarr,s3",
         NC_ZARR|NC_NETCDF4, &ncid);
```

The fragment portion of the URL (after the #) contains parameters that control how ncZarr accesses the storage. For S3 storage, authentication credentials are typically provided through environment variables or AWS configuration files.

Reading Zarr datasets works similarly. The `nc_open()` function will automatically detect the Zarr format and use the appropriate dispatch layer:

```
nc_open("s3://mybucket/mydata.zarr#mode=nczarr,s3", NC_NOWRITE, &ncid);
```

Zarr and the Enhanced Model

Zarr datasets created through ncZarr support the netCDF enhanced model, including groups, user-defined types, and multiple unlimited dimensions. The chunk sizes, compression filters, and other storage parameters are controlled through the same netCDF API functions used for HDF5 files, such as `nc_def_var_chunking()` and `nc_def_var_deflate()`.

One important difference from HDF5 is that Zarr uses a different set of compression codecs. While HDF5 uses filters like `zlib` and `szip`, Zarr supports codecs like `blosc`, `zstd`, and `lz4`. The netCDF library automatically maps between the netCDF compression API and the appropriate Zarr codec.

When Not to Use ncZarr

ncZarr is the right choice for cloud object storage, but it is not the best fit for every situation. Consider using NetCDF-4/HDF5 or a classic format instead when any of the following apply.

Local filesystem workloads: Zarr stores each chunk as a separate file in a directory tree. On a local filesystem this creates thousands of small files, which adds overhead from filesystem metadata operations. For local storage, a single HDF5 file will generally perform better and is simpler to manage, copy, and archive.

Interoperability with non-netCDF Zarr tools: The Python `zarr` library, `xarray`, and other tools in the Python ecosystem can read and write Zarr datasets, but they do not all handle ncZarr metadata conventions the same way. If your primary consumers use these tools directly rather than through the netCDF C library, test interoperability carefully. Metadata attributes that ncZarr adds (such as `_NCZARR_ATTR` and `_NCZARR_SUPERBLOCK`) may require special handling on the reading side.

Stable, long-term archival: The Zarr specification is still evolving. The transition from Zarr V2 to V3 introduces changes to metadata layout and codec handling. ncZarr currently targets Zarr V2. If your data

must remain readable for decades with minimal maintenance, NetCDF-4/HDF5 or CDF-5 are more mature choices with longer track records of backward compatibility.

Simple deployment requirements: Accessing data in cloud object storage requires configuring authentication credentials, endpoint URLs, and region settings. For S3, this typically means setting environment variables or maintaining AWS configuration files. If your application runs in environments where this configuration is difficult to manage, a single-file format like HDF5 avoids that complexity entirely.

Maximum read performance for sequential access: Zarr is optimized for random access to individual chunks over HTTP. For workloads that read entire variables sequentially, the contiguous storage layout of classic formats or HDF5 with contiguous storage will be faster, since the data can be read in a single sequential pass without per-chunk overhead.

Converting Between Formats

The `nccopy` utility, included with the netCDF C distribution, converts files from one format to another. The `-k` flag selects the output format. To convert a classic file to NetCDF-4/HDF5:

```
nccopy -k nc4 input.nc output.nc
```

To convert back to classic format:

```
nccopy -k classic input.nc output.nc
```

The supported format names are classic, 64-bit-offset, cdf5, nc4, and nc4-classic.

You can also add compression during conversion with the `-d` flag, which sets the deflate level from 0 (no compression) to 9 (maximum compression). For example, `nccopy -k nc4 -d 4 input.nc output.nc` converts to NetCDF-4/HDF5 and applies level-4 deflate compression to every variable. The `-s` flag enables the shuffle filter, which often improves compression ratios for integer data.

Not every conversion works. Converting a NetCDF-4/HDF5 file that uses groups, user-defined types, or multiple unlimited dimensions to a classic format will fail, because those features have no representation in the classic data model. `nccopy` will report an error rather than silently dropping data. If you need to move data from the enhanced model to a classic format, first flatten the group hierarchy and simplify the types, then convert. For files that stay within the classic data model, conversion between any of the classic-family formats (classic, 64-bit offset, CDF-5) and NetCDF-4/HDF5 classic model is straightforward.

Legacy Formats

The formats in this section cannot (HDF4) and should not (64-bit offset) be created with netCDF. However, files of these formats exist and can still be used by netCDF.

HDF4 SD (Read-Only)

The netCDF C library supports read-only access to HDF4 SD files. This functionality is available to all of the language libraries that use the C library as their base, including the netCDF Fortran APIs.

The HDF4 format was developed at the Champaign-Urbana campus of the University of Illinois. The team that created HDF4 later went on to create HDF5, which is the basis for the netCDF-4/HDF5 format.

HDF4 included several sub-formats, including image data. One of these sub-formats was the SD file type, for Scientific Data. The HDF4 SD model closely matches the netCDF classic data model.

HDF4 SD files are used to distribute the data from NASA's Aura and Terra satellites.

64-bit Offset Format

The 64-bit offset format was introduced in netCDF version 3.6.0 to address the 2 GB file size limitation of the classic format. However, it has been superseded by the CDF-5 format, which removes all size limitations while maintaining the classic data model.

For new applications, use CDF-5 instead of the 64-bit offset format. CDF-5 provides all the benefits of 64-bit offset format without its limitations (4 GB per-variable limit, limited record counts). The 64-bit offset format remains supported for backward compatibility with existing files and legacy applications.

The 64-bit offset format uses 64-bit integers for file offsets instead of 32-bit integers, allowing files to exceed the 2 GB limitation of the classic format. However, it retains several important limitations:

Individual Variable Size Limit: Each individual variable is still limited to approximately 4 GB in size. This is because the variable size is stored as a 32-bit integer in the file header. For a file with multiple variables, the total file size can exceed 4 GB, but no single variable can be larger than 4 GB.

Record Variable Limitations: For files with record (unlimited dimension) variables, the number of records is limited by the 32-bit record count. Additionally, the size of each record is limited. These constraints can be problematic for long time series data.

No Compression: Like the classic format, the 64-bit offset format does not support compression. All data are stored uncompressed, which can result in very large files for data that would compress well.

No Enhanced Model Features: The 64-bit offset format does not support groups, user-defined types, or any of the enhanced model features introduced with netCDF-4.

Binary Formats – Key Takeaways

- NetCDF supports five binary formats: Classic, 64-bit Offset, CDF-5, NetCDF-4/HDF5, and ncZarr. Each was introduced to address limitations of its predecessors.
- Format detection is automatic on read. The same `nc_open()` call works for all formats. Format only matters at file creation time.
- NetCDF-4/HDF5 is the recommended default for new applications. It supports compression, groups, user-defined types, and multiple unlimited dimensions.
- CDF-5 removes all size limitations of the classic data model without requiring HDF5. It is the preferred format for parallel I/O with PnetCDF.
- ncZarr is the only netCDF format designed for cloud object storage (S3, Google Cloud, Azure). It stores each chunk as a separate object, which suits HTTP access patterns but creates overhead on local filesystems.

- HDF5 chunked storage enables compression and efficient partial reads, but adds indexing overhead compared to contiguous storage. Chunk size selection affects performance.
- Classic format remains useful for small files that must be readable by every version of the netCDF library ever released.
- The 64-bit offset format is a legacy format. Use CDF-5 instead for new work.
- `nccopy -k` converts between formats. Not all conversions are possible: files using enhanced model features (groups, user-defined types) cannot be converted to classic formats.
- HDF5 dimension scales map the netCDF dimension model onto HDF5 transparently. Starting with netCDF 4.4.0, dimension information is stored as attribute tables for faster file opening.

Chapter 5 Command-Line Utilities

Learning Objectives

- Use `ncdump` to inspect the structure, metadata, and data values of any netCDF file
- Read and write CDL, the text representation of netCDF files
- Create netCDF files from CDL descriptions with `ncgen`
- Edit netCDF metadata by round-tripping through CDL text
- Convert files between netCDF formats using `nccopy`
- Compress and shuffle-filter variables to reduce file sizes
- Rechunk netCDF-4/HDF5 files to match specific access patterns
- Recognize when NCO operators can replace custom programs for routine data tasks

Working with NetCDF Files on the Command Line

NetCDF files are binary. You cannot open one in a text editor and read its contents. But you do not always need to write a program to work with them. The netCDF C distribution ships with three command-line utilities for inspecting, creating, and converting netCDF files: `ncdump`, `ncgen`, and `nccopy`. A separate open-source toolkit called NCO adds operators for subsetting, concatenation, arithmetic, and attribute editing.

This chapter covers all four. It starts with `ncdump` and `ncgen`, then covers `nccopy` for format conversion and optimization, and ends with a brief overview of NCO. For programmatic access to netCDF files, see Chapter 6: Programming with NetCDF in C and Chapter 7: Programming with NetCDF in Fortran.

Built-in Utilities: `ncdump` and `ncgen`

The netCDF distribution includes two utilities that every netCDF user should know: `ncdump` and `ncgen`. Together they convert between binary netCDF files and a human-readable text format called CDL (Common Data Language). They require no programming and are available on any system where the netCDF C library is installed.

`ncdump`: Inspecting NetCDF Files

The `ncdump` utility reads a netCDF file and prints its contents as CDL text. It is the single most useful tool for working with netCDF data. Use it to verify that a program created the right file structure, to check data values, and to understand files you receive from other sources.

Viewing File Structure

The `-h` flag prints only the header: dimensions, variables, and attributes, with no data values. This is the flag you will use most often.

```
ncdump -h example.nc
```

Output looks like this:

```
netcdf example {  
dimensions:  
time = UNLIMITED ; // (12 currently)  
lat = 180 ;  
lon = 360 ;  
variables:  
float temperature(time, lat, lon) ;  
temperature:units = "K" ;  
temperature:_FillValue = -999.f ;  
double time(time) ;  
time:units = "days since 2000-01-01" ;  
float lat(lat) ;  
lat:units = "degrees_north" ;  
float lon(lon) ;  
lon:units = "degrees_east" ;  
// global attributes:  
:title = "Monthly surface temperature" ;  
:Conventions = "CF-1.8" ;  
}
```

This tells you everything about the file structure at a glance: three dimensions, four variables, and two global attributes.

Viewing Data Values

To display the data for specific variables, use the `-v` flag followed by a comma-separated list of variable names:

```
ncdump -v lat,lon example.nc
```

This prints the full header plus the data values for the `lat` and `lon` variables. It is useful for checking coordinate variables without dumping a large data variable.

The `-c` flag is a shortcut that displays data for all coordinate variables (variables that share a name with a dimension). For a file with `time`, `lat`, and `lon` dimensions that also has `time`, `lat`, and `lon` variables, `-c` displays the data for all three:

```
ncdump -c example.nc
```

To dump the entire file including all data values, run `ncdump` with no data-selection flags:

```
ncdump example.nc
```

For large files this produces enormous output. Use `-v` to select only the variables you need.

Viewing Special Attributes

NetCDF-4/HDF5 files store chunking and compression settings as special attributes that `ncdump -h` does not show by default. The `-s` flag adds these to the output:

```
ncdump -hs example.nc
```

This reveals attributes like `_ChunkSizes`, `_DeflateLevel`, `_Shuffle`, and `_Endianness`. Use `-hs` when you need to verify compression or chunking settings.

Controlling Numeric Precision

By default `ncdump` prints floating-point values with enough digits to distinguish adjacent representable values. You can override this with the `-p` flag:

```
ncdump -p 3 -v temperature example.nc
```

This prints floating-point values with 3 significant digits, which produces more compact output when full precision is not needed.

Displaying Time as Human-Readable Strings

When a time variable has a units attribute in the form "days since 2000-01-01" or similar, the `-t` flag tells `ncdump` to print time values as date-time strings instead of raw numbers:

```
ncdump -t -v time example.nc
```

Output:

```
time = "2000-01-01", "2000-02-01", "2000-03-01", ... ;
```

This is much easier to read than raw offset values.

Useful Flag Combinations

Command	What It Shows
<code>ncdump -h file.nc</code>	Structure only (dimensions, variables, attributes)
<code>ncdump -hs file.nc</code>	Structure plus chunking and compression settings
<code>ncdump -c file.nc</code>	Structure plus coordinate variable data
<code>ncdump -v temp file.nc</code>	Structure plus data for the temp variable
<code>ncdump -t -v time file.nc</code>	Time variable as human-readable dates
<code>ncdump -k file.nc</code>	Just the format kind (classic, netCDF-4, etc.)

Debugging with `ncdump`

When a program creates a netCDF file, the first step in verifying the output is to run `ncdump -h` on it. Check that:

- All expected dimensions exist and have the correct sizes
- All expected variables exist with the correct types and dimensions
- Attributes contain the correct values
- Unlimited dimensions show the expected record count

If the structure looks correct, use `-v` to spot-check data values. For example, after writing a temperature array, verify a few values:

```
ncdump -v temperature example.nc | head -30
```

If values are all zeros or all fill values, the program likely wrote to the wrong variable or used incorrect start/count arrays. If values are present but wrong, check the data type and byte order.

CDL: The Common Data Language

CDL is the text format that ncdump produces and ncgen consumes. Understanding CDL syntax is essential for using the ncdump/ncgen round-trip workflow and for creating netCDF files from text descriptions.

CDL Structure

A CDL file has four sections, all contained within a top-level block named after the dataset:

```
netcdf example {  
  dimensions:  
    x = 10 ;  
    y = 20 ;  
    time = UNLIMITED ;  
  variables:  
    float temperature(time, y, x) ;  
    temperature:units = "K" ;  
    temperature:_FillValue = -999.f ;  
    int time(time) ;  
    time:units = "hours since 2000-01-01" ;  
  data:  
    time = 0, 1, 2, 3, 4, 5 ;  
}
```

The **dimensions** section declares dimension names and sizes. Use UNLIMITED for a dimension that can grow.

The **variables** section declares variable names, types, and dimensions. Attributes appear indented below the variable they belong to. Global attributes appear with no variable prefix.

The **data** section contains the actual values. Each variable's data is a comma-separated list terminated by a semicolon. The data section is optional. If omitted, variables are filled with their fill value.

CDL Data Types

CDL uses type suffixes to distinguish numeric literals:

CDL Type	Suffix	Example
byte	b	42b
short	s	1000s
int	(none)	42
float	f	3.14f
double	(none for decimals)	3.14
int64	(NetCDF-4 only)	1000000000000
string	(NetCDF-4 only)	"hello"

In the variables section, use the type keywords: byte, short, int, float, double, ubyte, ushort, uint, int64, uint64, string.

CDL for NetCDF-4 Features

NetCDF-4 CDL supports groups, which are declared with the group keyword:

```
netcdf example {
dimensions:
x = 10 ;
group: observations {
    variables:
float temperature(x) ;
temperature:units = "K" ;
    data:
temperature = 300.1, 300.2, 300.3, 300.4, 300.5,
300.6, 300.7, 300.8, 300.9, 301.0 ;
}
group: forecasts {
    variables:
float temperature(x) ;
temperature:units = "K" ;
}
}
```

Groups can contain their own dimensions, variables, attributes, and nested groups.

Fill Values in CDL

The special value `_` (underscore) represents a fill value in CDL data:

```
data:
temperature = 300.1, 300.2, _, _, 300.5 ;
```

This writes fill values at positions 3 and 4. The fill value used depends on the variable's `_FillValue` attribute, or the default fill value for the type if no `_FillValue` is set.

ncgen: Creating NetCDF Files from CDL

The `ncgen` utility reads a CDL file and creates a binary netCDF file. It is the complement of `ncdump`.

Basic Usage

To create a netCDF file from a CDL description:

```
ncgen -o example.nc example.cdl
```

The `-o` flag specifies the output file name. Without `-o`, `ncgen` writes to standard output as a C program (which is rarely what you want).

Choosing the Output Format

The `-k` flag selects the output format:

```
ncgen -k nc3 -o classic.nc example.cdl
ncgen -k nc4 -o netcdf4.nc example.cdl
ncgen -k nc7 -o nc4classic.nc example.cdl
ncgen -k nc5 -o cdf5.nc example.cdl
```

Valid -k values:

Value	Format
nc3 or classic	NetCDF classic
nc6 or 64-bit-offset	64-bit offset
nc4 or netCDF-4	NetCDF-4/HDF5
nc7 or netCDF-4 classic model	NetCDF-4 classic model
nc5 or 64-bit-data	CDF-5

If you omit -k, ncgen creates a classic format file by default. If the CDL uses NetCDF-4 features like groups or user-defined types, ncgen automatically selects NetCDF-4 format.

Generating Code Instead of Files

ncgen can also generate C or Fortran source code that creates the described netCDF file. This is useful for bootstrapping programs:

```
ncgen -lc example.cdl > create_example.c
ncgen -lf example.cdl > create_example.f90
```

The generated code includes all the nc_create, nc_def_dim, nc_def_var, and nc_put_var calls needed to produce the file. You can use this as a starting point and modify it for your needs.

The ncdump/ncgen Round-Trip Workflow

One of the most practical uses of ncdump and ncgen together is editing netCDF files as text. The workflow has three steps:

1. Dump the file to CDL:

```
ncdump example.nc > example.cdl
```

2. Edit the CDL file with any text editor. You can rename variables, change attribute values, add new attributes, modify data values, or restructure dimensions.

3. Regenerate the netCDF file:

```
ncgen -o modified.nc example.cdl
```

This workflow is useful for:

- Fixing metadata errors (wrong units, missing attributes)
- Creating template files that a program can later fill with data
- Making small test files for debugging
- Documenting file structures in a readable format

For large files, dump only the header with `ncdump -h`, edit the metadata, and regenerate a skeleton file without data. Then use a program to write the actual data values.

Limitations of the Round-Trip

The round-trip is not lossless in all cases. Floating-point values may lose precision if `ncdump` does not print enough digits (though the default precision is usually sufficient). Very large files are impractical to dump and regenerate because the CDL text can be many times larger than the binary file. For files larger than a few hundred megabytes, use `nccopy` or a program to make modifications instead.

The nccopy Utility

The `nccopy` utility copies a netCDF file while optionally converting its format, compressing its variables, and changing its chunk layout. It is part of the standard netCDF C distribution and requires no programming.

Basic Usage

At its simplest, `nccopy` copies a file:

```
nccopy input.nc output.nc
```

This produces an identical copy. The utility becomes useful when you add flags to change the output format, compression, or chunking.

Format Conversion

The `-k` flag selects the output format:

```
nccopy -k nc4 classic.nc netcdf4.nc
nccopy -k nc3 netcdf4.nc classic.nc
nccopy -k nc5 classic.nc cdf5.nc
```

Valid `-k` values:

Value	Format
nc3 or classic or 1	NetCDF classic
nc6 or 64-bit-offset or 2	64-bit offset
nc4 or netCDF-4 or 3	NetCDF-4/HDF5
nc7 or netCDF-4 classic model or 4	NetCDF-4 classic model
nc5 or 64-bit-data or 5	CDF-5

Not every conversion succeeds. Converting a NetCDF-4/HDF5 file that uses groups, user-defined types, or multiple unlimited dimensions to a classic format will fail, because those features have no representation in the classic data model. `nccopy` reports an error rather than silently dropping data. Conversions that stay within the classic data model work in any direction.

Compression

The `-d` flag applies zlib deflate compression to every variable in the output file. The argument is a compression level from 0 (no compression) to 9 (maximum compression). Use level 1. Higher levels are much slower and produce little additional compression for typical scientific data.

```
nccopy -k nc4 -d 1 input.nc compressed.nc
```

Compression requires NetCDF-4/HDF5 format. If the input file is classic format, combine `-k nc4` with `-d` to convert and compress in one step.

The `-s` flag enables the shuffle filter, which rearranges bytes within each chunk before compression. Shuffle improves compression ratios for integer data and for floating-point data with slowly varying values. It adds negligible overhead and is worth enabling whenever you compress.

```
nccopy -k nc4 -d 1 -s input.nc compressed.nc
```

The shuffle filter provides little benefit for random or noisy data, already-compressed data, or character strings.

Rechunking

NetCDF-4/HDF5 files store variable data in chunks. The chunk shape determines which access patterns are fast and which are slow. If a file was created with chunk sizes that do not match your access pattern, `nccopy` can rechunk it.

The `-c` flag specifies chunk sizes. The format is `dimension_name/size` for each dimension, separated by commas:

```
nccopy -k nc4 -c "time/1,lat/180,lon/360" input.nc rechunked.nc
```

This sets the chunk shape for every variable that uses the time, lat, and lon dimensions. A time chunk size of 1 with large spatial chunks is good for reading individual time slices. Reversing the proportions (large time, small spatial) is better for extracting time series at a single location.

You can also specify chunks per variable by using the variable name instead of dimension names:

```
nccopy -k nc4 -c "temperature/1,180,360" input.nc rechunked.nc
```

Guidelines for choosing chunk sizes:

- Target total chunk sizes between 100 KB and 1 MB
- Match the chunk shape to your dominant access pattern
- For time-slice access, use a time chunk of 1 and large spatial chunks
- For time-series access at a point, use large time chunks and spatial chunks of 1
- When in doubt, let `nccopy` choose defaults by omitting `-c`

Combining Options

The flags compose naturally. A common operation is to convert a classic file to NetCDF-4/HDF5 with compression, shuffle, and new chunking in a single command:

```
nccopy -k nc4 -d 1 -s -c "time/1,lat/180,lon/360" input.nc optimized.nc
```


Subsetting Variables

The -v flag copies only the named variables (plus any coordinate variables they depend on):

```
nccopy -v temperature,pressure input.nc subset.nc
```

The -V flag copies only the named variables without pulling in coordinate variables automatically.

Other Useful Flags

Flag	Purpose
-u dim	Make the named dimension unlimited in the output
-m bytes	Set the copy buffer size (default is 5 MB; increase for large files)
-h bytes	Set the output file header padding (useful for files you plan to modify later)

Verifying Results

After running nccopy, verify the output:

```
ncdump -h output.nc
```

Check that dimensions, variables, and attributes match expectations. To verify compression and chunking settings:

```
ncdump -hs output.nc
```

To confirm that data values are identical:

```
ncdump -v temperature input.nc > in.cdl  
ncdump -v temperature output.nc > out.cdl  
diff in.cdl out.cdl
```

For compressed files, the data values will be identical even though the binary files differ. Compression and decompression are transparent to all netCDF readers.

To check the effect on file size:

```
ls -lh input.nc output.nc
```

Practical Examples

Convert a classic file to compressed NetCDF-4/HDF5:

```
nccopy -k nc4 -d 1 -s legacy.nc modern.nc
```

Rechunk a file for time-slice access:

```
nccopy -k nc4 -c "time/1,lat/180,lon/360" badly_chunked.nc rechunked.nc
```

Prepare a file for cloud object storage with larger chunks to reduce the number of HTTP requests:

```
nccopy -k nc4 -d 1 -s -c "time/10,lat/180,lon/360" local.nc cloud.nc
```

Extract two variables into a smaller file:

```
nccopy -v temperature,pressure large.nc small.nc
```

Limitations

- nccopy always creates a new file. It cannot modify a file in place.
- Compression and chunking require NetCDF-4/HDF5 format. Classic formats store data contiguously without chunks.

- CDF-5 supports chunking but not compression.
- Very large files may need a larger copy buffer (-m) to avoid excessive I/O operations.
- Some advanced HDF5 features (such as external links or non-netCDF datatypes) may not survive conversion.

Hands-On: A Complete ncdump/ncgen/nccopy Workflow

The previous sections described each utility and its flags. This section walks through a complete workflow so you can see how the tools fit together. All you need is a system with the netCDF C library installed.

Step 1: Write a CDL File

Create a file called `var_var.cdl` with the following contents:

```
netcdf var_var {
dimensions:
time = UNLIMITED ;
lat = 4 ;
lon = 8 ;
variables:
float temperature(time, lat, lon) ;
temperature:units = "K" ;
temperature:long_name = "surface temperature" ;
temperature:_FillValue = -999.f ;
double time(time) ;
time:units = "days since 2025-01-01" ;
time:calendar = "standard" ;
float lat(lat) ;
lat:units = "degrees_north" ;
float lon(lon) ;
lon:units = "degrees_east" ;
// global attributes:
:title = "Example surface temperature" ;
:Conventions = "CF-1.8" ;
data:
time = 0, 1, 2 ;
lat = -60, -20, 20, 60 ;
lon = 0, 45, 90, 135, 180, 225, 270, 315 ;
temperature =
    271.5, 275.3, 280.1, 285.7, 290.2, 288.4, 276.9, 272.0,
    283.6, 289.1, 295.4, 300.8, 302.1, 299.7, 291.3, 284.5,
    298.2, 301.0, 303.5, 305.1, 304.8, 302.9, 299.6, 296.3,
    265.4, 270.2, 278.8, 284.3, 283.1, 277.5, 269.0, 264.7,
    272.1, 276.0, 281.3, 286.4, 291.0, 289.1, 277.5, 272.8,
    284.2, 290.0, 296.1, 301.5, 302.8, 300.3, 292.0, 285.1,
    298.9, 301.7, 304.2, 305.8, 305.5, 303.6, 300.3, 297.0,
    266.0, 271.0, 279.5, 285.0, 283.8, 278.2, 269.7, 265.3,
    270.8, 274.5, 279.3, 284.9, 289.5, 287.7, 276.2, 271.3,
    282.9, 288.4, 294.7, 300.1, 301.4, 299.0, 290.6, 283.8,
    297.5, 300.3, 302.8, 304.4, 304.1, 302.2, 298.9, 295.6,
    264.7, 269.5, 278.1, 283.6, 282.4, 276.8, 268.3, 264.0 ;
```

```
}
```

This describes a small dataset: three time steps of surface temperature on a 4-by-8 latitude-longitude grid. The values are plausible temperatures in Kelvin, varying by latitude.

Step 2: Create a NetCDF File

Use `ncgen` to produce a classic-format netCDF file:

```
ncgen -o var_var.nc var_var.cdl
```

`ncgen` reads the CDL and writes a binary netCDF file. The file is small (about 1 KB), but it has the same structure as a real climate dataset.

Step 3: Inspect the File with `ncdump`

View the header:

```
ncdump -h var_var.nc
```

Output:

```
netcdf var_var {
dimensions:
time = UNLIMITED ; // (3 currently)
lat = 4 ;
lon = 8 ;
variables:
float temperature(time, lat, lon) ;
temperature:units = "K" ;
temperature:long_name = "surface temperature" ;
temperature:_FillValue = -999.f ;
double time(time) ;
time:units = "days since 2025-01-01" ;
time:calendar = "standard" ;
float lat(lat) ;
lat:units = "degrees_north" ;
float lon(lon) ;
lon:units = "degrees_east" ;
// global attributes:
:title = "Example surface temperature" ;
:Conventions = "CF-1.8" ;
}
```

Check the coordinate variables:

```
ncdump -c var_var.nc
```

This prints the header plus the data for time, lat, and lon. Verify that the three time values, four latitudes, and eight longitudes match what you wrote in the CDL.

View the time variable as dates:

```
ncdump -t -v time var_var.nc
```

Output:

```
time = "2025-01-01", "2025-01-02", "2025-01-03" ;
```

Check the format:

```
ncdump -k var_var.nc
```

Output:

```
classic
```

Step 4: Edit Metadata with the Round-Trip

Suppose the title attribute is wrong and you want to fix it. Dump the file to CDL:

```
ncdump var_var.nc > var_var_edit.cdl
```

Open `var_var_edit.cdl` in a text editor and change the title:

```
:title = "Example surface temperature, January 2025" ;
```

Regenerate the file:

```
ncgen -o var_var_fixed.nc var_var_edit.cdl
```

Verify the change:

```
ncdump -h var_var_fixed.nc | grep title
```

Output:

```
:title = "Example surface temperature, January 2025" ;
```

The data values are unchanged. Only the attribute was modified.

Step 5: Convert and Compress with *nccopy*

Convert the classic file to NetCDF-4/HDF5 with compression and shuffle:

```
nccopy -k nc4 -d 1 -s var_var.nc var_var_nc4.nc
```

Verify the format and compression settings:

```
ncdump -hs var_var_nc4.nc
```

The output now includes special attributes showing the storage details:

```
temperature:_DeflateLevel = 1 ;  
temperature:_Shuffle = "true" ;  
temperature:_ChunkSizes = 3, 4, 8 ;
```

Compare file sizes:

```
ls -l var_var.nc var_var_nc4.nc
```

For this small file the compressed version may actually be larger because of HDF5 metadata overhead.

Compression pays off with larger datasets where data values dominate the file size.

Step 6: Rechunk for a Different Access Pattern

The default chunking stores the entire array in one chunk. Rechunk so that each time step is a separate chunk:

```
nccopy -k nc4 -d 1 -s -c "time/1,lat/4,lon/8" var_var.nc var_var_chunked.nc
```

Verify:

```
ncdump -hs var_var_chunked.nc
```

Look for:

```
temperature:_ChunkSizes = 1, 4, 8 ;
```

Each chunk now holds one time slice. Reading a single time step requires reading one chunk instead of the entire variable. For a large dataset this difference determines whether a time-slice read takes milliseconds or minutes.

What You Have Now

After these steps you have four files:

- `var_var.nc` -- classic format, no compression
- `var_var_fixed.nc` -- classic format with corrected metadata
- `var_var_nc4.nc` -- NetCDF-4/HDF5, compressed, default chunking
- `var_var_chunked.nc` -- NetCDF-4/HDF5, compressed, one-time-step chunks

You created all of them without writing a single line of C or Fortran. The rest of this chapter describes each utility and its options in detail. You can use these files to experiment with any of the flags discussed in the sections that follow.

More Utilities: the NetCDF Command Operators (NCO)

The NetCDF Operators (NCO) are a suite of command-line utilities for manipulating netCDF files. NCO was developed by Charlie Zender at the University of California, Irvine, and is widely used in the climate science community.

NCO provides operators for common tasks such as averaging, concatenating, subsetting, and arithmetic operations on netCDF files. The operators are designed to work efficiently with large files and can process files in parallel.

Tool	Name	Description	Example Usage
ncks	NetCDF Kitchen Sink	Swiss Army knife of NCO. Extracts variables, subsets dimensions, changes metadata, and converts between formats.	Extract variable: <code>ncks -v temperature input.nc output.nc</code> Subset dimension: <code>`ncks -d time,0,9 input.nc output.nc`</code> (first 10 time steps)
ncrcat	NetCDF Record Concatenator	Concatenates files along the record (unlimited) dimension.	<code>ncrcat file1.nc file2.nc file3.nc output.nc</code> Combines multiple time steps from different files into one.
ncra	NetCDF Record Averager	Averages files along the record dimension.	<code>ncra file1.nc file2.nc file3.nc output.nc</code> Creates a file containing the average of input files.
ncap2	NetCDF Arithmetic Processor	Performs arbitrary arithmetic operations on variables.	<code>ncap2 -s 'temp_k=temp_c+273.15' input.nc output.nc</code> Creates a new variable by adding 273.15 to <code>temp_c</code> .
ncatted	NetCDF	Modifies attributes	<code>ncatted -a</code>

	Attribute Editor	without rewriting the entire file.	units,temperature,o,c,"degrees_Celsius" input.nc Overwrites the units attribute.
ncdiff	NetCDF Differencer	Computes the difference between two files.	ncdiff file1.nc file2.nc diff.nc
ncbo	NetCDF Binary Operator	Performs binary operations (add, subtract, multiply, divide) between two files.	ncbo --op_typ=add file1.nc file2.nc sum.nc

NCO is developed and maintained by Charlie Zender at the University of California, Irvine. It is a separate project from the netCDF C library and must be installed independently. Most Linux package managers provide it (for example, apt install nco on Debian and Ubuntu), and it can also be built from source.

A full treatment of NCO is beyond the scope of this book, but the operators are worth knowing about because they eliminate the need to write custom programs for many routine data processing tasks. Concatenating monthly files into an annual time series, computing a climatological mean, converting temperature units across an entire dataset, or fixing an incorrect attribute value are all one-line operations with the right NCO command.

The NCO documentation on-line includes a detailed user guide, operator reference pages, and worked examples. Readers who process large collections of netCDF files on a regular basis will find it a valuable addition to the built-in utilities covered in this chapter.

Command-Line Utilities – Key Takeaways

- `ncdump` is the first tool to reach for when working with netCDF files. The `-h` flag shows structure, `-hs` adds chunking and compression details, `-v` selects specific variables, and `-c` shows all coordinate variables.
- `ncgen` creates binary netCDF files from CDL text. The `-k` flag selects the output format, and the `-lc` and `-lf` flags generate C or Fortran source code.
- The `ncdump/ncgen` round-trip lets you edit netCDF metadata and data as plain text, but it is impractical for files larger than a few hundred megabytes.
- `nccopy` converts formats (`-k`), compresses (`-d 1 -s`), and rechunks (`-c`) in a single command. It always creates a new file and cannot modify one in place.
- Use zlib compression level 1. Higher levels are much slower and produce little additional compression for typical scientific data.
- Match chunk shapes to your dominant access pattern. Use small time chunks and large spatial chunks for time-slice reads. Reverse the proportions for time-series extraction at a point.
- NCO provides operators for concatenation (`ncrcat`), subsetting (`ncks`), arithmetic (`ncap2`), and attribute editing (`ncatted`) that handle many routine tasks without writing code.

Chapter 6 NetCDF C Examples

The examples in this chapter are from the NetCDF Expansion Pack. The examples can be built and run as part of that package. Each example is a complete, self-contained C program that creates a netCDF file, writes data, then reopens the file to verify that all metadata and data were stored correctly. The programs increase in complexity, starting with basic 2D array I/O, then adding coordinate variables and CF conventions, then demonstrating compression filters, user-defined types, and hierarchical groups. All examples use the same error-handling macro introduced earlier in this chapter.

Classic Model Examples

Simple Example in C

This is a basic example demonstrating 2D array creation and reading in NetCDF.

This example shows the fundamental workflow for working with NetCDF files:

- Creating a new NetCDF file
- Defining dimensions and variables
- Adding global and variable attributes
- Writing data to variables
- Closing and reopening the file
- Querying file structure with `nc_inq()`, `nc_inq_dim()`, and `nc_inq_var()`
- Reading and verifying attributes and data

The program creates a 2D integer array (6x12) with sequential values (0, 1, 2, ..., 71), writes it to a NetCDF-4 file with a global attribute ("title") and a variable attribute ("units"), then reopens the file to verify metadata, attributes, and data correctness. This demonstrates the complete read-write cycle that forms the foundation of NetCDF programming.

Learning Objectives

- Understand basic NetCDF file structure (dimensions, variables, attributes, data)
- Learn dimension and variable definition workflow
- Add global and variable attributes
- Master data writing and reading operations
- Query file metadata with `nc_inq()`, `nc_inq_dim()`, and `nc_inq_var()`
- Implement error handling patterns with `nc_strerror()`
- Verify metadata, attribute, and data integrity

Key Concepts

- **Dimensions:** Named axes that define array shapes (x=6, y=12)
- **Variables:** Named data arrays with defined dimensions and types
- **Attributes:** Metadata attached to variables or the file (global)
- **NetCDF-4 Format:** HDF5-based format with enhanced features
- **Define Mode:** Metadata definition phase before data writing
- **Data Mode:** Phase where actual data is written/read

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <netcdf.h>

#define FILE_NAME "simple_2D.nc"
#define NDIMS 2
#define NX 6
#define NY 12
#define ERRCODE 2
#define ERR(e) {printf("Error: %s\n", nc_strerror(e)); exit(ERRCODE);}

int main()
{
    int ncid, varid;
    int x_dimid, y_dimid;
    int dimids[NDIMS];
    int retval;

    int data_out[NY][NX];
    int data_in[NY][NX];

    /* ===== WRITE PHASE ===== */
    printf("Creating NetCDF file: %s\n", FILE_NAME);

    /* Initialize data with sequential integers (0, 1, 2, 3, ...) */
    for (int i = 0; i < NY; i++)
        for (int j = 0; j < NX; j++)
            data_out[i][j] = i * NX + j;

    /* Create the NetCDF file (NC_CLOBBER overwrites existing file) */
    if ((retval = nc_create(FILE_NAME, NC_CLOBBER|NC_NETCDF4, &ncid)))
        ERR(retval);

    /* Define dimensions */
    if ((retval = nc_def_dim(ncid, "x", NX, &x_dimid)))
        ERR(retval);
    if ((retval = nc_def_dim(ncid, "y", NY, &y_dimid)))
        ERR(retval);

    /* Define the variable (dimension order: y, x for C row-major) */
    dimids[0] = y_dimid;
    dimids[1] = x_dimid;
    if ((retval = nc_def_var(ncid, "data", NC_INT, NDIMS, dimids, &varid)))
        ERR(retval);
}
```



```

/* Add a global attribute */
if ((retval = nc_put_att_text(ncid, NC_GLOBAL, "title",
                             strlen("Simple 2D Example"), "Simple 2D Example")))
    ERR(retval);

/* Add a variable attribute */
if ((retval = nc_put_att_text(ncid, varid, "units",
                             strlen("m/s"), "m/s")))
    ERR(retval);

/* End define mode */
if ((retval = nc_enddef(ncid)))
    ERR(retval);

/* Write the data to the file */
if ((retval = nc_put_var_int(ncid, varid, &data_out[0][0])))
    ERR(retval);

/* Close the file */
if ((retval = nc_close(ncid)))
    ERR(retval);

printf("*** SUCCESS writing file!\n");

/* ===== READ PHASE ===== */
printf("\nReopening file for validation...\n");

/* Open the file for reading */
if ((retval = nc_open(FILE_NAME, NC_NOWRITE, &ncid)))
    ERR(retval);

/* Verify metadata: check number of dimensions, variables, attributes, unlimited dim */
int ndims_in, nvars_in, ngatts_in, unlimdimid_in;
if ((retval = nc_inq(ncid, &ndims_in, &nvars_in, &ngatts_in, &unlimdimid_in)))
    ERR(retval);

if (ndims_in != NDIMS) {
    printf("Error: Expected %d dimensions, found %d\n", NDIMS, ndims_in);
    exit(ERRCODE);
}
printf("Verified: %d dimensions\n", ndims_in);

if (nvars_in != 1) {
    printf("Error: Expected 1 variable, found %d\n", nvars_in);
    exit(ERRCODE);
}
printf("Verified: %d variable\n", nvars_in);

if (ngatts_in != 1) {
    printf("Error: Expected 1 global attribute, found %d\n", ngatts_in);
    exit(ERRCODE);
}
printf("Verified: %d global attribute\n", ngatts_in);

if (unlimdimid_in != -1) {
    printf("Error: Expected no unlimited dimension, found dimid %d\n", unlimdimid_in);
    exit(ERRCODE);
}
printf("Verified: no unlimited dimension\n");

```

```

/* Verify dimensions using nc_inq_dim() */
char dim_name[NC_MAX_NAME + 1];
size_t len_x, len_y;
if ((retval = nc_inq_dim(ncid, x_dimid, dim_name, &len_x)))
    ERR(retval);

if (strcmp(dim_name, "x") != 0) {
    printf("Error: Expected dimension name 'x', found '%s'\n", dim_name);
    exit(ERRCODE);
}
if (len_x != NX) {
    printf("Error: Expected x dimension = %d, found %zu\n", NX, len_x);
    exit(ERRCODE);
}
printf("Verified: dimension '%s' = %zu\n", dim_name, len_x);

if ((retval = nc_inq_dim(ncid, y_dimid, dim_name, &len_y)))
    ERR(retval);

if (strcmp(dim_name, "y") != 0) {
    printf("Error: Expected dimension name 'y', found '%s'\n", dim_name);
    exit(ERRCODE);
}
if (len_y != NY) {
    printf("Error: Expected y dimension = %d, found %zu\n", NY, len_y);
    exit(ERRCODE);
}
printf("Verified: dimension '%s' = %zu\n", dim_name, len_y);

/* Verify variable using nc_inq_var() */
char var_name[NC_MAX_NAME + 1];
nc_type var_type;
int var_ndims;
int var_dimids[NDIMS];
if ((retval = nc_inq_var(ncid, varid, var_name, &var_type, &var_ndims, var_dimids, NULL)))
    ERR(retval);

if (strcmp(var_name, "data") != 0) {
    printf("Error: Expected variable name 'data', found '%s'\n", var_name);
    exit(ERRCODE);
}
if (var_type != NC_INT) {
    printf("Error: Expected variable type NC_INT, found %d\n", var_type);
    exit(ERRCODE);
}
if (var_ndims != NDIMS) {
    printf("Error: Expected %d dimensions, found %d\n", NDIMS, var_ndims);
    exit(ERRCODE);
}
if (var_dimids[0] != y_dimid || var_dimids[1] != x_dimid) {
    printf("Error: Unexpected dimension IDs for variable\n");
    exit(ERRCODE);
}
printf("Verified: variable '%s' type NC_INT, %d dims\n", var_name, var_ndims);

/* Verify global attribute */
char title_in[100];
size_t title_len;
if ((retval = nc_inq_attlen(ncid, NC_GLOBAL, "title", &title_len)))
    ERR(retval);

```

```

if ((retval = nc_get_att_text(ncid, NC_GLOBAL, "title", title_in)))
    ERR(retval);
title_in[title_len] = '\0';
if (strcmp(title_in, "Simple 2D Example") != 0) {
    printf("Error: Expected title 'Simple 2D Example', found '%s'\n", title_in);
    exit(ERRCODE);
}
printf("Verified: global attribute 'title' = '%s'\n", title_in);

/* Verify variable attribute */
char units_in[100];
size_t units_len;
if ((retval = nc_inq_attlen(ncid, varid, "units", &units_len)))
    ERR(retval);
if ((retval = nc_get_att_text(ncid, varid, "units", units_in)))
    ERR(retval);
units_in[units_len] = '\0';
if (strcmp(units_in, "m/s") != 0) {
    printf("Error: Expected units 'm/s', found '%s'\n", units_in);
    exit(ERRCODE);
}
printf("Verified: variable attribute 'units' = '%s'\n", units_in);

/* Read the data back */
if ((retval = nc_get_var_int(ncid, varid, &data_in[0][0])))
    ERR(retval);

/* Verify data correctness */
int errors = 0;
for (int i = 0; i < NY; i++) {
    for (int j = 0; j < NX; j++) {
        int expected = i * NX + j;
        if (data_in[i][j] != expected) {
            printf("Error: data[%d][%d] = %d, expected %d\n",
                i, j, data_in[i][j], expected);
            errors++;
        }
    }
}

if (errors > 0) {
    printf("**** FAILED: %d data validation errors\n", errors);
    exit(ERRCODE);
}

printf("Verified: all %d data values correct (0, 1, 2, ..., %d)\n",
    NX * NY, NX * NY - 1);

/* Close the file */
if ((retval = nc_close(ncid)))
    ERR(retval);

printf("\n*** SUCCESS: All validation checks passed!\n");
return 0;
}

```

This results in the following netCDF file:

```

netcdf simple_2D {
dimensions:
    x = 6 ;

```

```

        y = 12 ;
variables:
    int data(y, x) ;
        data:units = "m/s" ;

// global attributes:
        :title = "Simple 2D Example" ;
data:

data =
    0, 1, 2, 3, 4, 5,
    6, 7, 8, 9, 10, 11,
    12, 13, 14, 15, 16, 17,
    18, 19, 20, 21, 22, 23,
    24, 25, 26, 27, 28, 29,
    30, 31, 32, 33, 34, 35,
    36, 37, 38, 39, 40, 41,
    42, 43, 44, 45, 46, 47,
    48, 49, 50, 51, 52, 53,
    54, 55, 56, 57, 58, 59,
    60, 61, 62, 63, 64, 65,
    66, 67, 68, 69, 70, 71 ;
}

```

Coordinate Variables

This example introduces the concept of coordinate variables - special 1D variables that share the same name as their dimension and provide values along that axis. Coordinate variables are essential for geospatial data, defining latitude, longitude, time, or other dimensional coordinates.

The program creates a 2D temperature field (4x5 grid) with latitude and longitude coordinate variables following Climate and Forecast (CF) conventions. CF conventions are the standard metadata conventions for climate and forecast data.

This example can be found in the NetCDF Expansion Pack.

Learning Objectives

- Understand coordinate variables and their relationship to dimensions
- Learn CF convention metadata attributes (units, standard_name, long_name, axis)
- Master attribute definition and retrieval (nc_put_att, nc_get_att)
- Work with multi-dimensional geospatial data
- Implement proper metadata for scientific data interoperability

Key Concepts

- Coordinate Variable: 1D variable with same name as its dimension (e.g., lat(lat))
- CF Conventions: Standardized metadata for climate/forecast data
- Attributes: Metadata attached to variables (units, standard_name, etc.)
- _FillValue: Special attribute indicating missing/undefined data values

- Geospatial Grid: Regular lat/lon grid for spatial data

CF Convention Attributes Used

- ``units``: Physical units (degrees_north, degrees_east, K)
- ``standard_name``: CF standard name vocabulary (latitude, longitude, air_temperature)
- ``long_name``: Human-readable descriptive name
- ``axis``: Coordinate axis identifier (X, Y, Z, T)
- ``_FillValue``: Value representing missing data

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <netcdf.h>

#define FILE_NAME "coord_vars.nc"
#define NLAT 4
#define NLON 5
#define ERRCODE 2
#define ERR(e) {printf("Error: %s\n", nc_strerror(e)); exit(ERRCODE);}

int main()
{
    int ncid, lat_varid, lon_varid, temp_varid;
    int lat_dimid, lon_dimid;
    int dimids[2];
    int retval;

    float lat[NLAT] = {-45.0, -15.0, 15.0, 45.0};
    float lon[NLON] = {-120.0, -60.0, 0.0, 60.0, 120.0};
    float temperature[NLAT][NLON];

    float lat_in[NLAT];
    float lon_in[NLON];
    float temperature_in[NLAT][NLON];

    /* ===== WRITE PHASE ===== */
    printf("Creating NetCDF file: %s\n", FILE_NAME);

    /* Initialize temperature data (synthetic: varies with lat and lon) */
    for (int i = 0; i < NLAT; i++)
        for (int j = 0; j < NLON; j++)
            temperature[i][j] = 273.15 + i * 5.0 + j * 2.0;

    /* Create the NetCDF file */
    if ((retval = nc_create(FILE_NAME, NC_CLOBBER|NC_NETCDF4, &ncid)))
        ERR(retval);

    /* Define dimensions */
    if ((retval = nc_def_dim(ncid, "lat", NLAT, &lat_dimid)))
        ERR(retval);
    if ((retval = nc_def_dim(ncid, "lon", NLON, &lon_dimid)))
        ERR(retval);

    /* Define coordinate variables (same name as dimension) */
    if ((retval = nc_def_var(ncid, "lat", NC_FLOAT, 1, &lat_dimid, &lat_varid)))
```

```

    ERR(retval);
if ((retval = nc_def_var(ncid, "lon", NC_FLOAT, 1, &lon_dimid, &lon_varid)))
    ERR(retval);

/* Add CF convention attributes to latitude */
if ((retval = nc_put_att_text(ncid, lat_varid, "units", 13, "degrees_north")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, lat_varid, "standard_name", 8, "latitude")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, lat_varid, "long_name", 8, "Latitude")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, lat_varid, "axis", 1, "Y")))
    ERR(retval);

/* Add CF convention attributes to longitude */
if ((retval = nc_put_att_text(ncid, lon_varid, "units", 12, "degrees_east")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, lon_varid, "standard_name", 9, "longitude")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, lon_varid, "long_name", 9, "Longitude")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, lon_varid, "axis", 1, "X")))
    ERR(retval);

/* Define temperature variable */
dimids[0] = lat_dimid;
dimids[1] = lon_dimid;
if ((retval = nc_def_var(ncid, "temperature", NC_FLOAT, 2, dimids, &temp_varid)))
    ERR(retval);

/* Add CF convention attributes to temperature */
if ((retval = nc_put_att_text(ncid, temp_varid, "units", 1, "K")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, temp_varid, "standard_name", 15, "air_temperature")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, temp_varid, "long_name", 15, "Air Temperature")))
    ERR(retval);

float fill_value = -999.0;
if ((retval = nc_put_att_float(ncid, temp_varid, "_FillValue", NC_FLOAT, 1, &fill_value)))
    ERR(retval);

/* End define mode */
if ((retval = nc_enddef(ncid)))
    ERR(retval);

/* Write coordinate variables */
if ((retval = nc_put_var_float(ncid, lat_varid, lat)))
    ERR(retval);
if ((retval = nc_put_var_float(ncid, lon_varid, lon)))
    ERR(retval);

/* Write temperature data */
if ((retval = nc_put_var_float(ncid, temp_varid, &temperature[0][0])))
    ERR(retval);

/* Close the file */
if ((retval = nc_close(ncid)))
    ERR(retval);

```

```

printf("*** SUCCESS writing file!\n");

/* ===== READ PHASE ===== */
printf("\nReopening file for validation...\n");

/* Open the file for reading */
if ((retval = nc_open(FILE_NAME, NC_NOWRITE, &ncid)))
    ERR(retval);

/* Verify metadata */
int ndims_in, nvars_in;
if ((retval = nc_inq(ncid, &ndims_in, &nvars_in, NULL, NULL)))
    ERR(retval);

if (ndims_in != 2) {
    printf("Error: Expected 2 dimensions, found %d\n", ndims_in);
    exit(ERRCODE);
}
printf("Verified: %d dimensions\n", ndims_in);

if (nvars_in != 3) {
    printf("Error: Expected 3 variables, found %d\n", nvars_in);
    exit(ERRCODE);
}
printf("Verified: %d variables (lat, lon, temperature)\n", nvars_in);

/* Verify latitude attributes */
char att_text[256];
size_t att_len;

if ((retval = nc_inq_attlen(ncid, lat_varid, "units", &att_len)))
    ERR(retval);
if ((retval = nc_get_att_text(ncid, lat_varid, "units", att_text)))
    ERR(retval);
att_text[att_len] = '\0';
if (strcmp(att_text, "degrees_north") != 0) {
    printf("Error: lat units = '%s', expected 'degrees_north'\n", att_text);
    exit(ERRCODE);
}
printf("Verified: lat units = '%s'\n", att_text);

if ((retval = nc_inq_attlen(ncid, lat_varid, "standard_name", &att_len)))
    ERR(retval);
if ((retval = nc_get_att_text(ncid, lat_varid, "standard_name", att_text)))
    ERR(retval);
att_text[att_len] = '\0';
if (strcmp(att_text, "latitude") != 0) {
    printf("Error: lat standard_name = '%s', expected 'latitude'\n", att_text);
    exit(ERRCODE);
}
printf("Verified: lat standard_name = '%s'\n", att_text);

if ((retval = nc_inq_attlen(ncid, lat_varid, "axis", &att_len)))
    ERR(retval);
if ((retval = nc_get_att_text(ncid, lat_varid, "axis", att_text)))
    ERR(retval);
att_text[att_len] = '\0';
if (strcmp(att_text, "Y") != 0) {
    printf("Error: lat axis = '%s', expected 'Y'\n", att_text);
    exit(ERRCODE);
}

```

```

}
printf("Verified: lat axis = '%s'\n", att_text);

/* Verify longitude attributes */
if ((retval = nc_inq_attlen(ncid, lon_varid, "units", &att_len)))
    ERR(retval);
if ((retval = nc_get_att_text(ncid, lon_varid, "units", att_text)))
    ERR(retval);
att_text[att_len] = '\0';
if (strcmp(att_text, "degrees_east") != 0) {
    printf("Error: lon units = '%s', expected 'degrees_east'\n", att_text);
    exit(ERRCODE);
}
printf("Verified: lon units = '%s'\n", att_text);

if ((retval = nc_inq_attlen(ncid, lon_varid, "standard_name", &att_len)))
    ERR(retval);
if ((retval = nc_get_att_text(ncid, lon_varid, "standard_name", att_text)))
    ERR(retval);
att_text[att_len] = '\0';
if (strcmp(att_text, "longitude") != 0) {
    printf("Error: lon standard_name = '%s', expected 'longitude'\n", att_text);
    exit(ERRCODE);
}
printf("Verified: lon standard_name = '%s'\n", att_text);

/* Verify temperature attributes */
if ((retval = nc_inq_attlen(ncid, temp_varid, "units", &att_len)))
    ERR(retval);
if ((retval = nc_get_att_text(ncid, temp_varid, "units", att_text)))
    ERR(retval);
att_text[att_len] = '\0';
if (strcmp(att_text, "K") != 0) {
    printf("Error: temperature units = '%s', expected 'K'\n", att_text);
    exit(ERRCODE);
}
printf("Verified: temperature units = '%s'\n", att_text);

float fill_value_in;
if ((retval = nc_get_att_float(ncid, temp_varid, "_FillValue", &fill_value_in)))
    ERR(retval);
if (fill_value_in != fill_value) {
    printf("Error: temperature _FillValue = %f, expected %f\n", fill_value_in, fill_value);
    exit(ERRCODE);
}
printf("Verified: temperature _FillValue = %f\n", fill_value_in);

/* Read coordinate variables */
if ((retval = nc_get_var_float(ncid, lat_varid, lat_in)))
    ERR(retval);
if ((retval = nc_get_var_float(ncid, lon_varid, lon_in)))
    ERR(retval);

/* Verify coordinate data */
int errors = 0;
for (int i = 0; i < NLAT; i++) {
    if (lat_in[i] != lat[i]) {
        printf("Error: lat[%d] = %f, expected %f\n", i, lat_in[i], lat[i]);
        errors++;
    }
}

```



```

}

for (int j = 0; j < NLON; j++) {
    if (lon_in[j] != lon[j]) {
        printf("Error: lon[%d] = %f, expected %f\n", j, lon_in[j], lon[j]);
        errors++;
    }
}

if (errors == 0) {
    printf("Verified: coordinate arrays correct\n");
    printf("  lat: [%g, %g, %g, %g]\n", lat[0], lat[1], lat[2], lat[3]);
    printf("  lon: [%g, %g, %g, %g, %g]\n", lon[0], lon[1], lon[2], lon[3], lon[4]);
}

/* Read temperature data */
if ((retval = nc_get_var_float(ncid, temp_varid, &temperature_in[0][0])))
    ERR(retval);

/* Verify temperature data */
for (int i = 0; i < NLAT; i++) {
    for (int j = 0; j < NLON; j++) {
        if (temperature_in[i][j] != temperature[i][j]) {
            printf("Error: temperature[%d][%d] = %f, expected %f\n",
                    i, j, temperature_in[i][j], temperature[i][j]);
            errors++;
        }
    }
}

if (errors > 0) {
    printf("**** FAILED: %d data validation errors\n", errors);
    exit(ERRCODE);
}

printf("Verified: all temperature data correct (%d values)\n", NLAT * NLON);

/* Close the file */
if ((retval = nc_close(ncid)))
    ERR(retval);

printf("\n*** SUCCESS: All validation checks passed!\n");
return 0;
}

```

The file produced by this example has the following CDL:

```

netcdf coord_vars {
dimensions:
    lat = 4 ;
    lon = 5 ;
variables:
    float lat(lat) ;
        lat:units = "degrees_north" ;
        lat:standard_name = "latitude" ;
        lat:long_name = "Latitude" ;
        lat:axis = "Y" ;
    float lon(lon) ;
        lon:units = "degrees_east" ;
        lon:standard_name = "longitude" ;
        lon:long_name = "Longitude" ;
}

```

```

        lon:axis = "X" ;
        float temperature(lat, lon) ;
        temperature:units = "K" ;
        temperature:standard_name = "air_temperature" ;
        temperature:long_name = "Air Temperature" ;
        temperature:_FillValue = -999.f ;
data:

    lat = -45, -15, 15, 45 ;

    lon = -120, -60, 0, 60, 120 ;

    temperature =
        273.15, 275.15, 277.15, 279.15, 281.15,
        278.15, 280.15, 282.15, 284.15, 286.15,
        283.15, 285.15, 287.15, 289.15, 291.15,
        288.15, 290.15, 292.15, 294.15, 296.15 ;
}

```

Enhanced Model Examples in C

The examples in this section rely on the enhanced data model, or other advanced features not available in the classic formats.

Different NetCDF Binary Files

This example is `format_variants.c` from the NetCDF Expansion Pack project.

This example creates identical data structures in all five NetCDF binary formats to illustrate their differences in file size limits, compatibility, storage backend, and use cases. Understanding format variants is crucial for choosing the right format for your data requirements.

The program creates five files with the same 3D temperature and pressure data ($\text{time} \times \text{lat} \times \text{lon}$) in different formats, then compares their characteristics including file size, format detection, and size limitations.

Learning Objectives

- Understand all five NetCDF binary format variants
- Learn when to use each format based on size requirements
- Master format detection with `nc_inq_format()` and `nc_inq_format_extended()`
- Compare file sizes and format overhead
- Make informed format choices for different use cases

Key Concepts

- **NC_CLASSIC_MODEL (CDF-1):** Original NetCDF format, 2GB limits
- **NC_64BIT_OFFSET (CDF-2):** Extended format with 4GB variable limit
- **NC_64BIT_DATA (CDF-5):** Modern format with unlimited variable sizes
- **NC_NETCDF4:** NetCDF-4/HDF5 format with groups, compression, user types

- **NC_NETCDF4|NC_CLASSIC_MODEL:** HDF5 storage with classic data model
- **Format Detection:** Runtime identification of file format
- **Backward Compatibility:** Older formats work with all NetCDF versions

Format Comparison

Format	File Limit	Var Limit	NetCDF Version	Backend	Use Case
CDF-1	2 GB	2 GB	2.0.0	CDF-1	Max compatibility
CDF-2	unlimited	4 GB	3.6.0	CDF-2	Don't use for new data, use CDF-5 instead
CDF-5	unlimited	unlimited	4.4.0	CDF-5	Best classic format for new data
NetCDF/HDF5	unlimited	unlimited	4.0.0	HDF5	Best format for new data.
NetCDF/HDF5 Classic Model	unlimited	unlimited	4.0.0	HDF5	Only needed to maintain compatibility with very old software

```
#include <stdio.h>
#include <stdlib.h>
#include <netcdf.h>
#include <sys/stat.h>

#define NTIME 10
#define NLAT 20
#define NLON 30
#define ERRCODE 2
#define ERR(e) {printf("Error: %s\n", nc_strerror(e)); exit(ERRCODE);}

/* Get file size in bytes */
long get_file_size(const char *filename)
{
    struct stat st;
    if (stat(filename, &st) == 0)
        return st.st_size;
    return -1;
}

/* Create a file in the specified format with identical data structure */
void create_format_file(const char *filename, int format_flag, const char *format_name)
{
    int ncid, time_dimid, lat_dimid, lon_dimid;
    int temp_varid, pressure_varid;
    int dimids[3];
    int retval;

    float temperature[NTIME][NLAT][NLON];
    float pressure[NTIME][NLAT][NLON];

    printf("Creating %s format file: %s\n", format_name, filename);
}
```

```

/* Initialize data */
for (int t = 0; t < NTIME; t++)
    for (int i = 0; i < NLAT; i++)
        for (int j = 0; j < NLON; j++) {
            temperature[t][i][j] = 273.15 + t * 1.0 + i * 0.5 + j * 0.2;
            pressure[t][i][j] = 1013.25 + t * 0.1 + i * 0.05 + j * 0.02;
        }

/* Create file */
if ((retval = nc_create(filename, format_flag | NC_CLOBBER, &ncid)))
    ERR(retval);

/* Define dimensions */
if ((retval = nc_def_dim(ncid, "time", NTIME, &time_dimid)))
    ERR(retval);
if ((retval = nc_def_dim(ncid, "lat", NLAT, &lat_dimid)))
    ERR(retval);
if ((retval = nc_def_dim(ncid, "lon", NLON, &lon_dimid)))
    ERR(retval);

/* Define variables */
dimids[0] = time_dimid;
dimids[1] = lat_dimid;
dimids[2] = lon_dimid;

if ((retval = nc_def_var(ncid, "temperature", NC_FLOAT, 3, dimids, &temp_varid)))
    ERR(retval);
if ((retval = nc_def_var(ncid, "pressure", NC_FLOAT, 3, dimids, &pressure_varid)))
    ERR(retval);

/* Add attributes */
if ((retval = nc_put_att_text(ncid, temp_varid, "units", 1, "K")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, pressure_varid, "units", 3, "hPa")))
    ERR(retval);

/* End define mode */
if ((retval = nc_enddef(ncid)))
    ERR(retval);

/* Write data */
if ((retval = nc_put_var_float(ncid, temp_varid, &temperature[0][0][0])))
    ERR(retval);
if ((retval = nc_put_var_float(ncid, pressure_varid, &pressure[0][0][0])))
    ERR(retval);

/* Close file */
if ((retval = nc_close(ncid)))
    ERR(retval);

printf("  File created successfully\n");
}

/* Verify a format file */
void verify_format_file(const char *filename, int expected_format,
                       const char *expected_format_name)
{
    int ncid, retval;
    int format_in;
    int ndims, nvars;

```

```

float temperature[NTIME][NLAT][NLON];
float pressure[NTIME][NLAT][NLON];
int temp_varid, pressure_varid;

printf("\nVerifying file: %s\n", filename);

/* Open file */
if ((retval = nc_open(filename, NC_NOWRITE, &ncid)))
    ERR(retval);

/* Check format */
if ((retval = nc_inq_format(ncid, &format_in)))
    ERR(retval);

/* Determine format name */
const char *detected_format = "UNKNOWN";
if (format_in == NC_FORMAT_CLASSIC)
    detected_format = "NC_FORMAT_CLASSIC (CDF-1)";
else if (format_in == NC_FORMAT_64BIT_OFFSET)
    detected_format = "NC_FORMAT_64BIT_OFFSET (CDF-2)";
else if (format_in == NC_FORMAT_64BIT_DATA)
    detected_format = "NC_FORMAT_64BIT_DATA (CDF-5)";
else if (format_in == NC_FORMAT_NETCDF4)
    detected_format = "NC_FORMAT_NETCDF4 (HDF5)";
else if (format_in == NC_FORMAT_NETCDF4_CLASSIC)
    detected_format = "NC_FORMAT_NETCDF4_CLASSIC (HDF5/Classic)";

printf("  Format detected: %s\n", detected_format);

/* Verify expected format */
if (format_in != expected_format) {
    printf("Error: Expected format %s (%d), got %s (%d)\n",
        expected_format_name, expected_format, detected_format, format_in);
    exit(ERRCODE);
}

/* Get file size */
long file_size = get_file_size(filename);
if (file_size >= 0) {
    printf("  File size: ");
    if (file_size >= 1048576)
        printf("%.2f MB (%ld bytes)\n", file_size / 1048576.0, file_size);
    else if (file_size >= 1024)
        printf("%.2f KB (%ld bytes)\n", file_size / 1024.0, file_size);
    else
        printf("%ld bytes\n", file_size);
}

/* Verify metadata */
if ((retval = nc_inq(ncid, &ndims, &nvars, NULL, NULL)))
    ERR(retval);

if (ndims != 3 || nvars != 2) {
    printf("Error: Expected 3 dimensions and 2 variables, found %d dims, %d vars\n",
        ndims, nvars);
    exit(ERRCODE);
}
printf("  Metadata: %d dimensions, %d variables\n", ndims, nvars);

/* Get variable IDs */

```

```

if ((retval = nc_inq_varid(ncid, "temperature", &temp_varid)))
    ERR(retval);
if ((retval = nc_inq_varid(ncid, "pressure", &pressure_varid)))
    ERR(retval);

/* Read data */
if ((retval = nc_get_var_float(ncid, temp_varid, &temperature[0][0][0])))
    ERR(retval);
if ((retval = nc_get_var_float(ncid, pressure_varid, &pressure[0][0][0])))
    ERR(retval);

/* Verify a few data values */
int errors = 0;
float expected_temp = 273.15;
float expected_pressure = 1013.25;

if (temperature[0][0][0] != expected_temp) {
    printf("Error: temperature[0][0][0] = %f, expected %f\n",
        temperature[0][0][0], expected_temp);
    errors++;
}

if (pressure[0][0][0] != expected_pressure) {
    printf("Error: pressure[0][0][0] = %f, expected %f\n",
        pressure[0][0][0], expected_pressure);
    errors++;
}

if (errors == 0)
    printf(" Data validation: %d values verified\n", NTIME * NLAT * NLON * 2);
else {
    printf("*** FAILED: %d data validation errors\n", errors);
    exit(ERRCODE);
}

/* Close file */
if ((retval = nc_close(ncid)))
    ERR(retval);
}

int main()
{
    printf("NetCDF Format Variants Comparison\n\n");

    printf("This program creates five files with identical data structures\n");
    printf("in all five NetCDF binary formats to demonstrate their differences.\n\n");

    printf("Data structure:\n");
    printf(" Dimensions: time=%d, lat=%d, lon=%d\n", NTIME, NLAT, NLON);
    printf(" Variables: temperature(time,lat,lon), pressure(time,lat,lon)\n");
    printf(" Data type: NC_FLOAT (4 bytes per value)\n");
    printf(" Total data: %d values per variable\n\n", NTIME * NLAT * NLON);

    /* Create files in each format */
    printf("=== Creating Format Files ===\n\n");

    create_format_file("format_classic.nc", NC_CLASSIC_MODEL, "NC_CLASSIC_MODEL");
    create_format_file("format_64bit_offset.nc", NC_64BIT_OFFSET, "NC_64BIT_OFFSET");
    create_format_file("format_64bit_data.nc", NC_64BIT_DATA, "NC_64BIT_DATA");
    create_format_file("format_netcdf4.nc", NC_NETCDF4, "NC_NETCDF4");

```

```

create_format_file("format_netcdf4_classic.nc", NC_NETCDF4 | NC_CLASSIC_MODEL,
                  "NC_NETCDF4|NC_CLASSIC_MODEL");

/* Verify files */
printf("\n=== Verifying Format Files ===\n");

verify_format_file("format_classic.nc", NC_FORMAT_CLASSIC,
                  "NC_FORMAT_CLASSIC");
verify_format_file("format_64bit_offset.nc", NC_FORMAT_64BIT_OFFSET,
                  "NC_FORMAT_64BIT_OFFSET");
verify_format_file("format_64bit_data.nc", NC_FORMAT_64BIT_DATA,
                  "NC_FORMAT_64BIT_DATA");
verify_format_file("format_netcdf4.nc", NC_FORMAT_NETCDF4,
                  "NC_FORMAT_NETCDF4");
verify_format_file("format_netcdf4_classic.nc", NC_FORMAT_NETCDF4_CLASSIC,
                  "NC_FORMAT_NETCDF4_CLASSIC");

/* Summary */
printf("\n=== Format Comparison Summary ===\n\n");

long size_classic = get_file_size("format_classic.nc");
long size_offset = get_file_size("format_64bit_offset.nc");
long size_data = get_file_size("format_64bit_data.nc");
long size_nc4 = get_file_size("format_netcdf4.nc");
long size_nc4_classic = get_file_size("format_netcdf4_classic.nc");

printf("File sizes:\n");
printf("  NC_CLASSIC_MODEL:          %ld bytes\n", size_classic);
printf("  NC_64BIT_OFFSET:           %ld bytes\n", size_offset);
printf("  NC_64BIT_DATA:             %ld bytes\n", size_data);
printf("  NC_NETCDF4:                %ld bytes\n", size_nc4);
printf("  NC_NETCDF4|NC_CLASSIC_MODEL: %ld bytes\n", size_nc4_classic);

printf("\nFormat Characteristics:\n\n");

printf("NC_CLASSIC_MODEL (CDF-1):\n");
printf("  File size limit: 2GB\n");
printf("  Variable size limit: 2GB\n");
printf("  Storage backend: CDF binary\n");
printf("  Compatibility: NetCDF 3.0+, all tools\n");
printf("  Use when: Maximum compatibility needed, files < 2GB\n\n");

printf("NC_64BIT_OFFSET (CDF-2):\n");
printf("  File size limit: effectively unlimited\n");
printf("  Variable size limit: 4GB per variable\n");
printf("  Storage backend: CDF binary\n");
printf("  Compatibility: NetCDF 3.6.0+\n");
printf("  Use when: Large files needed, variables < 4GB each\n\n");

printf("NC_64BIT_DATA (CDF-5):\n");
printf("  File size limit: effectively unlimited\n");
printf("  Variable size limit: effectively unlimited\n");
printf("  Storage backend: CDF binary\n");
printf("  Compatibility: NetCDF 4.4.0+ or PnetCDF\n");
printf("  Use when: Very large variables needed (> 4GB)\n\n");

printf("NC_NETCDF4 (HDF5):\n");
printf("  File size limit: effectively unlimited\n");
printf("  Variable size limit: effectively unlimited\n");
printf("  Storage backend: HDF5\n");

```

```

printf("  Compatibility: NetCDF 4.0+\n");
printf("  Features: groups, compression, chunking, user-defined types\n");
printf("  Use when: Advanced features needed (compression, groups, etc.)\n\n");

printf("NC_NETCDF4|NC_CLASSIC_MODEL (HDF5 Classic Model):\n");
printf("  File size limit: effectively unlimited\n");
printf("  Variable size limit: effectively unlimited\n");
printf("  Storage backend: HDF5\n");
printf("  Compatibility: NetCDF 4.0+\n");
printf("  Features: compression, chunking (no groups, no user-defined types)\n");
printf("  Use when: HDF5 storage benefits needed with classic data model\n\n");

printf("Key Observations:\n");
printf("  - All five formats store identical data correctly\n");
printf("  - Classic formats (CDF-1/2/5) have smaller overhead for small files\n");
printf("  - NetCDF-4 formats (HDF5) have larger overhead but support compression\n");
printf("  - NC4 classic model is a useful middle ground: HDF5 storage, simple model\n");
printf("  - Use nc_inq_format() to detect format type when reading files\n\n");

printf("*** SUCCESS: All format tests passed! ***\n");
return 0;
}

```

This results in a number of output files. Using `ncdump` with the `-s` (secret) and the `-h` (header) options, we can see the format of the files produced. Note the special `_Format` attribute – it is not a real attribute in the file, it is added by the `ncdump` to show the format:

```

ncdump -sh format_classic.nc
netcdf format_classic {
dimensions:
    time = 10 ;
    lat = 20 ;
    lon = 30 ;
variables:
    float temperature(time, lat, lon) ;
        temperature:units = "K" ;
    float pressure(time, lat, lon) ;
        pressure:units = "hPa" ;

// global attributes:
    :_Format = "classic" ;
}
ncdump -sh format_64bit_offset.nc
netcdf format_64bit_offset {
dimensions:
    time = 10 ;
    lat = 20 ;
    lon = 30 ;
variables:
    float temperature(time, lat, lon) ;
        temperature:units = "K" ;
    float pressure(time, lat, lon) ;
        pressure:units = "hPa" ;

// global attributes:
    :_Format = "64-bit offset" ;
}
ncdump -sh format_netcdf4_classic.nc
netcdf format_netcdf4_classic {
dimensions:

```



```

    time = 10 ;
    lat = 20 ;
    lon = 30 ;
variables:
    float temperature(time, lat, lon) ;
        temperature:units = "K" ;
        temperature:_Storage = "contiguous" ;
        temperature:_Endianness = "little" ;
    float pressure(time, lat, lon) ;
        pressure:units = "hPa" ;
        pressure:_Storage = "contiguous" ;
        pressure:_Endianness = "little" ;

// global attributes:
    :_NCProperties = "version=2,netcdf=4.10.0-development,hdf5=1.14.6" ;
    :_SuperblockVersion = 2 ;
    :_IsNetcdf4 = 1 ;
    :_Format = "netCDF-4 classic model" ;
}
ncdump -sh format_netcdf4.nc
netcdf format_netcdf4 {
dimensions:
    time = 10 ;
    lat = 20 ;
    lon = 30 ;
variables:
    float temperature(time, lat, lon) ;
        temperature:units = "K" ;
        temperature:_Storage = "contiguous" ;
        temperature:_Endianness = "little" ;
    float pressure(time, lat, lon) ;
        pressure:units = "hPa" ;
        pressure:_Storage = "contiguous" ;
        pressure:_Endianness = "little" ;

// global attributes:
    :_NCProperties = "version=2,netcdf=4.10.0-development,hdf5=1.14.6" ;
    :_SuperblockVersion = 2 ;
    :_IsNetcdf4 = 1 ;
    :_Format = "netCDF-4" ;
}

```

Compression

This example explores NetCDF-4's built-in compression capabilities by creating multiple files with different compression settings and measuring their performance characteristics. Compression is essential for reducing storage requirements and I/O bandwidth for large scientific datasets.

The program generates realistic 3D temperature data (time×lat×lon) and creates files with various compression configurations: no compression, deflate only, shuffle only, and shuffle+deflate combinations at different compression levels.

It measures write/read times, file sizes, and compression ratios.

Learning Objectives:

- Understand NetCDF-4 compression filters (deflate, shuffle)

- Learn to configure compression with `nc_def_var_deflate()` and `nc_def_var_shuffle()`
- Master compression level selection (1-9 tradeoff between speed and ratio)
- Measure compression performance (time, size, ratio)
- Make informed compression decisions for different data types

Key Concepts:

- Deflate Filter: GZIP compression (levels 1-9, higher = better compression)
- Shuffle Filter: Byte reordering to improve compression of typed data
- Compression Ratio: Original size / compressed size
- Compression Overhead: Extra CPU time for compression/decompression
- Filter Pipeline: Shuffle then deflate for optimal results

Compression Strategies:

- No Compression: Fastest I/O, largest files, use for small datasets
- Deflate Only: Good compression, slower, use for mixed data types
- Shuffle Only: Minimal overhead, modest gains, use for fast I/O
- Shuffle+Deflate: Best compression, moderate overhead, recommended default
- High Deflate Levels (7-9): Maximum compression, slow, use for archival

When to Use Compression:

- Large datasets where storage/bandwidth is limited
- Floating-point data with spatial/temporal correlation
- Archival data where read performance is less critical
- Network transfers where bandwidth is constrained

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <math.h>
#include <netcdf.h>

#define NTIME 50
#define NLAT 90
#define NLON 180
#define NDIMS 3
#define ERRCODE 2
#define ERR(e) {printf("Error: %s\n", nc_strerror(e)); exit(ERRCODE);}

typedef struct {
    char name[64];
    char filename[128];
    int shuffle;
```

```

    int deflate;
    int deflate_level;
    double write_time;
    double read_time;
    long file_size;
    double compression_ratio;
} CompressionTest;

/* Generate realistic temperature data with spatial/temporal patterns */
void generate_temperature_data(float *data) {
    for (int t = 0; t < NTIME; t++) {
        for (int lat = 0; lat < NLAT; lat++) {
            for (int lon = 0; lon < NLON; lon++) {
                int idx = t * NLAT * NLON + lat * NLON + lon;

                /* Base temperature with latitude gradient */
                float base_temp = 15.0 - (lat - NLAT/2) * 0.5;

                /* Seasonal variation */
                float seasonal = 10.0 * sin(2.0 * M_PI * t / NTIME);

                /* Spatial variation */
                float spatial = 5.0 * sin(2.0 * M_PI * lon / NLON) *
                    cos(2.0 * M_PI * lat / NLAT);

                data[idx] = base_temp + seasonal + spatial;
            }
        }
    }
}

/* Get file size */
long get_file_size(const char *filename) {
    FILE *fp = fopen(filename, "rb");
    if (!fp) return -1;
    fseek(fp, 0, SEEK_END);
    long size = ftell(fp);
    fclose(fp);
    return size;
}

/* Create compressed file and measure performance */
void create_compressed_file(CompressionTest *test, float *data) {
    int ncid, varid;
    int time_dimid, lat_dimid, lon_dimid;
    int dimids[NDIMS];
    int retval;
    struct timespec start, end;

    printf("\n=== %s ===\n", test->name);

    /* Start timing */
    clock_gettime(CLOCK_MONOTONIC, &start);

    /* Create file */
    if ((retval = nc_create(test->filename, NC_CLOBBER|NC_NETCDF4, &ncid)))
        ERR(retval);

    /* Define dimensions */
    if ((retval = nc_def_dim(ncid, "time", NTIME, &time_dimid)))

```

```

        ERR(retval);
    if ((retval = nc_def_dim(ncid, "lat", NLAT, &lat_dimid)))
        ERR(retval);
    if ((retval = nc_def_dim(ncid, "lon", NLON, &lon_dimid)))
        ERR(retval);

    /* Define variable */
    dimids[0] = time_dimid;
    dimids[1] = lat_dimid;
    dimids[2] = lon_dimid;
    if ((retval = nc_def_var(ncid, "temperature", NC_FLOAT, NDIMS, dimids, &varid)))
        ERR(retval);

    /* Set compression */
    if (test->deflate || test->shuffle) {
        if ((retval = nc_def_var_deflate(ncid, varid, test->shuffle,
                                         test->deflate, test->deflate_level)))
            ERR(retval);
    }

    /* End define mode */
    if ((retval = nc_enddef(ncid)))
        ERR(retval);

    /* Write data */
    if ((retval = nc_put_var_float(ncid, varid, data)))
        ERR(retval);

    /* Close file */
    if ((retval = nc_close(ncid)))
        ERR(retval);

    /* End timing */
    clock_gettime(CLOCK_MONOTONIC, &end);
    test->write_time = (end.tv_sec - start.tv_sec) +
                      (end.tv_nsec - start.tv_nsec) / 1e9;

    /* Get file size */
    test->file_size = get_file_size(test->filename);

    printf("Write time: %.3f seconds\n", test->write_time);
    printf("File size: %ld bytes (%.2f MB)\n",
           test->file_size, test->file_size / 1048576.0);

    if (test->shuffle) printf("Shuffle: enabled\n");
    if (test->deflate) printf("Deflate: level %d\n", test->deflate_level);
}

/* Read and validate compressed file */
void read_compressed_file(CompressionTest *test, float *original_data) {
    int ncid, varid;
    int retval;
    struct timespec start, end;
    int shuffle, deflate, deflate_level;

    float *data = malloc(NTIME * NLAT * NLON * sizeof(float));
    if (!data) {
        printf("Error: Memory allocation failed\n");
        exit(ERRCODE);
    }
}

```

```

/* Start timing */
clock_gettime(CLOCK_MONOTONIC, &start);

/* Open file */
if ((retval = nc_open(test->filename, NC_NOWRITE, &ncid)))
    ERR(retval);

/* Get variable ID */
if ((retval = nc_inq_varid(ncid, "temperature", &varid)))
    ERR(retval);

/* Verify compression settings */
if ((retval = nc_inq_var_deflate(ncid, varid, &shuffle, &deflate, &deflate_level)))
    ERR(retval);

if (shuffle != test->shuffle || deflate != test->deflate ||
    (deflate && deflate_level != test->deflate_level)) {
    printf("Error: Compression settings mismatch\n");
    exit(ERRCODE);
}

/* Read data */
if ((retval = nc_get_var_float(ncid, varid, data)))
    ERR(retval);

/* Close file */
if ((retval = nc_close(ncid)))
    ERR(retval);

/* End timing */
clock_gettime(CLOCK_MONOTONIC, &end);
test->read_time = (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1e9;

/* Validate data (check first 100 points) */
int errors = 0;
for (int i = 0; i < 100 && i < NTIME * NLAT * NLON; i++) {
    if (fabs(data[i] - original_data[i]) > 0.001) {
        printf("Error: data[%d] = %f, expected %f\n", i, data[i], original_data[i]);
        errors++;
    }
}

if (errors > 0) {
    printf("*** FAILED: %d validation errors\n", errors);
    exit(ERRCODE);
}

printf("Read time: %.3f seconds\n", test->read_time);
printf("Data validated successfully\n");

free(data);
}

int main() {
    printf("Compression Filter Demonstration\n");
    printf("=====\n");
    printf("Dataset dimensions: [time=%d, lat=%d, lon=%d]\n", NTIME, NLAT, NLON);
    printf("Total data points: %d\n", NTIME * NLAT * NLON);
}

```

```

printf("Total data size: %.2f MB\n",
      (NTIME * NLAT * NLON * sizeof(float)) / 1048576.0);

/* Generate realistic temperature data */
float *data = malloc(NTIME * NLAT * NLON * sizeof(float));
if (!data) {
    printf("Error: Memory allocation failed\n");
    return ERRCODE;
}
generate_temperature_data(data);

/* Define compression tests */
CompressionTest tests[] = {
    {"Uncompressed (baseline)", "compress_none.nc", 0, 0, 0, 0, 0, 0, 0},
    {"Shuffle only", "compress_shuffle.nc", 1, 0, 0, 0, 0, 0, 0},
    {"Deflate level 1 (preferred)", "compress_deflate1.nc", 0, 1, 1, 0, 0, 0, 0},
    {"Deflate level 5", "compress_deflate5.nc", 0, 1, 5, 0, 0, 0, 0},
    {"Deflate level 9", "compress_deflate9.nc", 0, 1, 9, 0, 0, 0, 0},
    {"Shuffle + Deflate 1 (recommended)", "compress_shuffle_deflate1.nc", 1, 1, 1, 0, 0,
0, 0},
};
int num_tests = sizeof(tests) / sizeof(tests[0]);

/* Run all tests */
for (int i = 0; i < num_tests; i++) {
    create_compressed_file(&tests[i], data);
    read_compressed_file(&tests[i], data);
}

/* Calculate compression ratios */
long baseline_size = tests[0].file_size;
for (int i = 0; i < num_tests; i++) {
    tests[i].compression_ratio = (double)baseline_size / tests[i].file_size;
}

/* Print summary table */
printf("\n=== Performance Summary ===\n");
printf("%-35s %12s %12s %12s %10s\n",
      "Strategy", "Write (s)", "Read (s)", "Size (MB)", "Ratio");
printf("%-35s %12s %12s %12s %10s\n",
      "-----", "-----", "-----", "-----", "-----");

for (int i = 0; i < num_tests; i++) {
    printf("%-35s %12.3f %12.3f %12.2f %10.2fx\n",
          tests[i].name,
          tests[i].write_time,
          tests[i].read_time,
          tests[i].file_size / 1048576.0,
          tests[i].compression_ratio);
}

/* Print recommendations */
printf("\n=== Recommendations ===\n");
printf("- Uncompressed: Fastest I/O but largest files\n");
printf("- Shuffle only: Reorganizes bytes for better compression (use with deflate)\n");
printf("- Deflate level 1: PREFERRED for almost all real-world data\n");
printf("- Deflate level 5: Marginally better ratio, significantly slower\n");
printf("- Deflate level 9: Maximum compression, much slower, rarely worth it\n");
printf("- Shuffle + Deflate 1: RECOMMENDED default for scientific data\n");
printf("- Level 1 gives nearly the same compression as higher levels\n");

```

```

printf("- Higher levels cost much more CPU time for diminishing returns\n");
printf("- Read performance generally similar across compression levels\n");
printf("- Compression effectiveness depends on data patterns\n");

free(data);
printf("\n*** SUCCESS: All compression strategies tested!\n");
return 0;
}

```

Doing an `ncdump -hs` on one of the the resulting files shows the metadata:

```

ncdump -hs ../netcdf-4/compress_shuffle_deflate1.nc
netcdf compress_shuffle_deflate1 {
dimensions:
    time = 50 ;
    lat = 90 ;
    lon = 180 ;
variables:
    float temperature(time, lat, lon) ;
        temperature:_Storage = "chunked" ;
        temperature:_ChunkSizes = 50, 90, 180 ;
        temperature:_Shuffle = "true" ;
        temperature:_DeflateLevel = 1 ;
        temperature:_Endianness = "little" ;

// global attributes:
    :_NCProperties = "version=2,netcdf=4.10.0-development,hdf5=1.14.6" ;
    :_SuperblockVersion = 2 ;
    :_IsNetcdf4 = 1 ;
    :_Format = "netCDF-4" ;
}

```

User-Defined Types

This example showcases NetCDF-4's powerful type system, which extends beyond the basic atomic types (int, float, etc.) to include user-defined types similar to C structures and enumerations. User-defined types enable more expressive and efficient data models for complex scientific data.

The program demonstrates four types of user-defined types:

Compound types: Structured records like C structs (weather observations)

Enum types: Named integer constants (cloud cover categories)

Variable-length types: Arrays of varying length (daily measurements)

Opaque types: Binary blobs for arbitrary data (calibration data)

Learning Objectives:

- Understand NetCDF-4 user-defined type system
- Learn to create compound types with `nc_def_compound()`
- Master enum type definition with `nc_def_enum()`
- Work with variable-length arrays using `nc_def_vlen()`

- Use opaque types for binary data with `nc_def_opaque()`
- Use `NC_STRING` for variable-length string data
- Write and read complex structured data

Key Concepts:

- **Compound Type:** Structured type with named fields (like C struct)
- **Enum Type:** Integer type with named values (like C enum)
- **Variable-Length Type:** Array whose length varies per element
- **Opaque Type:** Uninterpreted binary data of fixed size
- **String Type:** Variable-length character strings (`NC_STRING`)
- **Type ID:** Unique identifier for each user-defined type

Compound Types:

- Define with `nc_def_compound()`, add fields with `nc_insert_compound()`
- Fields can be atomic types or other user-defined types
- Useful for observations, records, structured metadata
- Example: Weather observation with time, temp, pressure, humidity

Enum Types:

- Define with `nc_def_enum()`, add members with `nc_insert_enum()`
- Based on integer type (`NC_BYTE`, `NC_SHORT`, `NC_INT`, etc.)
- Provides named constants for categorical data
- Example: Cloud cover (`CLEAR`, `PARTLY_CLOUDY`, `CLOUDY`, `OVERCAST`)

Variable-Length Types:

- Define with `nc_def_vlen()` specifying base type
- Each array element can have different length
- Useful for ragged arrays, particle lists, event sequences
- Example: Daily measurements (different counts per day)

Opaque Types:

- Define with `nc_def_opaque()` specifying byte size
- Store arbitrary binary data (images, encrypted data, proprietary formats)
- NetCDF doesn't interpret the content
- Example: Instrument calibration data

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <netcdf.h>

#define FILE_NAME "user_types.nc"
#define NOBS 5
#define NDAYS 3
#define NSTATIONS 4
#define CALIB_SIZE 16
#define ERRCODE 2
#define ERR(e) {printf("Error: %s\n", nc_strerror(e)); exit(ERRCODE);}

/* Compound type: Weather observation */
typedef struct {
    double time;
    float temperature;
    float pressure;
    float humidity;
} WeatherObs;

/* Enum type: Cloud cover categories */
typedef enum {
    CLEAR = 0,
    PARTLY_CLOUDY = 1,
    CLOUDY = 2,
    OVERCAST = 3
} CloudCover;

int main() {
    int ncid, retval;

    printf("User-Defined Types Demonstration\n");
    printf("=====\n");

    /* ===== CREATE FILE AND DEFINE TYPES ===== */
    printf("\n=== Phase 1: Create file and define user types ===\n");

    if ((retval = nc_create(FILE_NAME, NC_CLOBBER|NC_NETCDF4, &ncid)))
        ERR(retval);

    /* 1. Define Compound Type */
    printf("\n--- Compound Type (weather observation) ---\n");
    nc_type compound_typeid;
    if ((retval = nc_def_compound(ncid, sizeof(WeatherObs), "weather_obs_t",
    &compound_typeid)))
        ERR(retval);
```

```

if ((retval = nc_insert_compound(ncid, compound_typeid, "time",
                                offsetof(WeatherObs, time), NC_DOUBLE)))
    ERR(retval);
if ((retval = nc_insert_compound(ncid, compound_typeid, "temperature",
                                offsetof(WeatherObs, temperature), NC_FLOAT)))
    ERR(retval);
if ((retval = nc_insert_compound(ncid, compound_typeid, "pressure",
                                offsetof(WeatherObs, pressure), NC_FLOAT)))
    ERR(retval);
if ((retval = nc_insert_compound(ncid, compound_typeid, "humidity",
                                offsetof(WeatherObs, humidity), NC_FLOAT)))
    ERR(retval);
printf("Defined compound type with 4 fields\n");

/* 2. Define Variable-Length Type */
printf("\n--- Variable-Length Type (ragged arrays) ---\n");
nc_type vlen_typeid;
if ((retval = nc_def_vlen(ncid, "obs_per_day_t", NC_INT, &vlen_typeid)))
    ERR(retval);
printf("Defined vlen type for variable-length integer arrays\n");

/* 3. Define Enumeration Type */
printf("\n--- Enumeration Type (cloud cover) ---\n");
nc_type enum_typeid;
if ((retval = nc_def_enum(ncid, NC_INT, "cloud_cover_t", &enum_typeid)))
    ERR(retval);
CloudCover clear = CLEAR;
CloudCover partly = PARTLY_CLOUDY;
CloudCover cloudy = CLOUDY;
CloudCover overcast = OVERCAST;
if ((retval = nc_insert_enum(ncid, enum_typeid, "CLEAR", &clear)))
    ERR(retval);
if ((retval = nc_insert_enum(ncid, enum_typeid, "PARTLY_CLOUDY", &partly)))
    ERR(retval);
if ((retval = nc_insert_enum(ncid, enum_typeid, "CLOUDY", &cloudy)))
    ERR(retval);
if ((retval = nc_insert_enum(ncid, enum_typeid, "OVERCAST", &overcast)))
    ERR(retval);
printf("Defined enum type with 4 categories\n");

/* 4. Define Opaque Type */
printf("\n--- Opaque Type (binary calibration data) ---\n");
nc_type opaque_typeid;
if ((retval = nc_def_opaque(ncid, CALIB_SIZE, "calibration_t", &opaque_typeid)))
    ERR(retval);
printf("Defined opaque type with %d-byte size\n", CALIB_SIZE);

/* ===== DEFINE DIMENSIONS AND VARIABLES ===== */
printf("\n=== Phase 2: Define dimensions and variables ===\n");

int obs_dimid, day_dimid;
if ((retval = nc_def_dim(ncid, "obs", NOBS, &obs_dimid)))
    ERR(retval);
if ((retval = nc_def_dim(ncid, "day", NDAYS, &day_dimid)))
    ERR(retval);

/* Variables using custom types */
int compound_varid, vlen_varid, enum_varid, opaque_varid, string_varid;

/* 5. Define String Variable */

```

```

printf("\n--- String Type (station names) ---\n");
int station_dimid;
if ((retval = nc_def_dim(ncid, "station", NSTATIONS, &station_dimid)))
    ERR(retval);
if ((retval = nc_def_var(ncid, "station_name", NC_STRING, 1, &station_dimid,
&string_varid)))
    ERR(retval);
printf("Defined NC_STRING variable for %d station names\n", NSTATIONS);

if ((retval = nc_def_var(ncid, "observations", compound_typeid, 1, &obs_dimid,
&compound_varid)))
    ERR(retval);
if ((retval = nc_def_var(ncid, "obs_per_day", vlen_typeid, 1, &day_dimid, &vlen_varid)))
    ERR(retval);
if ((retval = nc_def_var(ncid, "cloud_cover", enum_typeid, 1, &obs_dimid, &enum_varid)))
    ERR(retval);
if ((retval = nc_def_var(ncid, "calibration", opaque_typeid, 0, NULL, &opaque_varid)))
    ERR(retval);

if ((retval = nc_enddef(ncid)))
    ERR(retval);

/* ===== WRITE DATA ===== */
printf("\n=== Phase 3: Write data ===\n");

/* Write compound data */
WeatherObs obs_data[NOBS];
for (int i = 0; i < NOBS; i++) {
    obs_data[i].time = 1000.0 + i * 3600.0;
    obs_data[i].temperature = 20.0 + i * 2.0;
    obs_data[i].pressure = 1013.0 + i * 0.5;
    obs_data[i].humidity = 60.0 - i * 5.0;
}
if ((retval = nc_put_var(ncid, compound_varid, obs_data)))
    ERR(retval);
printf("Wrote %d compound observations\n", NOBS);

/* Write vlen data */
nc_vlen_t vlen_data[NDAYS];
int day1_obs[] = {10, 15, 20};
int day2_obs[] = {12, 18, 22, 25};
int day3_obs[] = {8, 14};

vlen_data[0].len = 3;
vlen_data[0].p = day1_obs;
vlen_data[1].len = 4;
vlen_data[1].p = day2_obs;
vlen_data[2].len = 2;
vlen_data[2].p = day3_obs;

if ((retval = nc_put_var(ncid, vlen_varid, vlen_data)))
    ERR(retval);
printf("Wrote vlen data: day1=%zu obs, day2=%zu obs, day3=%zu obs\n",
    vlen_data[0].len, vlen_data[1].len, vlen_data[2].len);

/* Write enum data */
CloudCover cloud_data[NOBS] = {CLEAR, PARTLY_CLOUDY, CLOUDY, PARTLY_CLOUDY, OVERCAST};
if ((retval = nc_put_var(ncid, enum_varid, cloud_data)))
    ERR(retval);
printf("Wrote %d cloud cover values\n", NOBS);

```

```

/* Write string data */
const char *station_names[NSTATIONS] = {
    "Boulder, CO",
    "Cape Canaveral, FL",
    "Wallops Island, VA",
    "White Sands, NM"
};
if ((retval = nc_put_var_string(ncid, string_varid, station_names)))
    ERR(retval);
printf("Wrote %d station name strings\n", NSTATIONS);

/* Write opaque data */
unsigned char calib_data[CALIB_SIZE];
for (int i = 0; i < CALIB_SIZE; i++) {
    calib_data[i] = (unsigned char)(i * 17);
}
if ((retval = nc_put_var(ncid, opaque_varid, calib_data)))
    ERR(retval);
printf("Wrote %d bytes of opaque calibration data\n", CALIB_SIZE);

if ((retval = nc_close(ncid)))
    ERR(retval);

/* ===== READ AND VALIDATE ===== */
printf("\n=== Phase 4: Read and validate data ===\n");

if ((retval = nc_open(FILE_NAME, NC_NOWRITE, &ncid)))
    ERR(retval);

/* Verify compound type */
printf("\n--- Validating Compound Type ---\n");
if ((retval = nc_inq_varid(ncid, "observations", &compound_varid)))
    ERR(retval);

WeatherObs obs_read[NOBS];
if ((retval = nc_get_var(ncid, compound_varid, obs_read)))
    ERR(retval);

int errors = 0;
for (int i = 0; i < NOBS; i++) {
    if (fabs(obs_read[i].time - obs_data[i].time) > 0.001 ||
        fabs(obs_read[i].temperature - obs_data[i].temperature) > 0.001 ||
        fabs(obs_read[i].pressure - obs_data[i].pressure) > 0.001 ||
        fabs(obs_read[i].humidity - obs_data[i].humidity) > 0.001) {
        printf("Error: compound data mismatch at index %d\n", i);
        errors++;
    }
}
if (errors == 0) {
    printf("Verified: all %d compound observations correct\n", NOBS);
}

/* Verify vlen type */
printf("\n--- Validating Variable-Length Type ---\n");
if ((retval = nc_inq_varid(ncid, "obs_per_day", &vlen_varid)))
    ERR(retval);

nc_vlen_t vlen_read[NDAYS];
if ((retval = nc_get_var(ncid, vlen_varid, vlen_read)))

```

```

    ERR(retval);

for (int d = 0; d < NDAYS; d++) {
    if (vlen_read[d].len != vlen_data[d].len) {
        printf("Error: vlen length mismatch for day %d\n", d);
        errors++;
    } else {
        int *vals = (int *)vlen_read[d].p;
        int *expected = (int *)vlen_data[d].p;
        for (size_t i = 0; i < vlen_read[d].len; i++) {
            if (vals[i] != expected[i]) {
                printf("Error: vlen data mismatch day %d, obs %zu\n", d, i);
                errors++;
            }
        }
    }
}

if (errors == 0) {
    printf("Verified: all vlen data correct (lengths: %zu, %zu, %zu)\n",
        vlen_read[0].len, vlen_read[1].len, vlen_read[2].len);
}

/* Free vlen memory */
if ((retval = nc_free_vlen(vlen_read)))
    ERR(retval);

/* Verify enum type */
printf("\n--- Validating Enumeration Type ---\n");
if ((retval = nc_inq_varid(ncid, "cloud_cover", &enum_varid)))
    ERR(retval);

CloudCover cloud_read[NOBS];
if ((retval = nc_get_var(ncid, enum_varid, cloud_read)))
    ERR(retval);

for (int i = 0; i < NOBS; i++) {
    if (cloud_read[i] != cloud_data[i]) {
        printf("Error: enum data mismatch at index %d\n", i);
        errors++;
    }
}

if (errors == 0) {
    printf("Verified: all %d cloud cover values correct\n", NOBS);
}

/* Verify string type */
printf("\n--- Validating String Type ---\n");
if ((retval = nc_inq_varid(ncid, "station_name", &string_varid)))
    ERR(retval);

char *station_read[NSTATIONS];
if ((retval = nc_get_var_string(ncid, string_varid, station_read)))
    ERR(retval);

for (int i = 0; i < NSTATIONS; i++) {
    if (strcmp(station_read[i], station_names[i]) != 0) {
        printf("Error: string data mismatch at index %d: '%s' != '%s'\n",
            i, station_read[i], station_names[i]);
        errors++;
    }
}

```

```

    }
    if (errors == 0) {
        printf("Verified: all %d station name strings correct\n", NSTATIONS);
    }

    /* Free string memory */
    if ((retval = nc_free_string(NSTATIONS, station_read)))
        ERR(retval);

    /* Verify opaque type */
    printf("\n--- Validating Opaque Type ---\n");
    if ((retval = nc_inq_varid(ncid, "calibration", &opaque_varid)))
        ERR(retval);

    unsigned char calib_read[CALIB_SIZE];
    if ((retval = nc_get_var(ncid, opaque_varid, calib_read)))
        ERR(retval);

    for (int i = 0; i < CALIB_SIZE; i++) {
        if (calib_read[i] != calib_data[i]) {
            printf("Error: opaque data mismatch at byte %d\n", i);
            errors++;
        }
    }
    if (errors == 0) {
        printf("Verified: all %d bytes of opaque data correct\n", CALIB_SIZE);
    }

    if ((retval = nc_close(ncid)))
        ERR(retval);

    if (errors > 0) {
        printf("\n*** FAILED: %d validation errors\n", errors);
        return ERRCODE;
    }

    printf("\n=== Use Cases ===\n");
    printf("- Compound types: Group related fields (like C structs)\n");
    printf("- Variable-length types: Store ragged arrays efficiently\n");
    printf("- Enumeration types: Categorical data with named values\n");
    printf("- Opaque types: Binary metadata or proprietary formats\n");
    printf("- String types: Variable-length text data (station names, labels)\n");

    printf("\n*** SUCCESS: All user-defined types demonstrated!\n");
    return 0;
}

```

Groups and New Atomic Types

This example showcases NetCDF-4's hierarchical group feature, which enables organizing datasets into logical groupings similar to directories in a filesystem. Groups provide namespace isolation for variables while allowing dimensions to be shared across the hierarchy through dimension visibility rules.

The program creates a three-level group hierarchy (root → SubGroup1, root → SubGroup2 → NestedGroup), demonstrates dimension visibility across group boundaries, and showcases all five new NetCDF-4 integer types (NC_UBYTE, NC_USHORT, NC_UINT, NC_INT64, NC_UINT64).

Learning Objectives:

- Understand NetCDF-4 hierarchical group structures
- Learn to create and navigate nested groups
- Master dimension visibility rules across group boundaries
- Work with all five new NetCDF-4 integer types
- Recognize when groups provide organizational benefits

Key Concepts:

- Hierarchical Groups: Organize datasets into logical groupings (like directories)
- Dimension Visibility: Parent dimensions visible in all child groups
- Variable Scoping: Variables only visible in their defining group
- Group Navigation: Use `nc_inq_grp_ncid()` to navigate by name
- New Integer Types: `NC_UBYTE`, `NC_USHORT`, `NC_UINT`, `NC_INT64`, `NC_UINT64`

NetCDF-4 Group Architecture:

- Groups implemented via `NC_GRP_INFO_T` structures (`libsrc4/libhdf5`)
- Dimensions visible in child groups via parent chain lookup
- Variables NOT inherited (scoped to defining group only)
- Requires `NC_NETCDF4` flag (HDF5 backend)
- Not compatible with `NC_CLASSIC_MODEL`

Dimension Visibility Rules:

- Dimensions defined in a group are visible in that group and all descendants
- Root dimensions (x, y) visible in `SubGroup1`, `SubGroup2`, and `NestedGroup`
- Local dimensions (z in `NestedGroup`) only visible in defining group
- Dimension lookup walks parent chain: child → parent → root

Use Cases for Groups:

- Multi-instrument datasets: Group data by instrument or sensor
- Model ensembles: Separate ensemble members into groups
- Quality levels: Organize raw, calibrated, and derived products
- Temporal organization: Group data by year, month, or campaign
- Namespace management: Avoid variable name conflicts

```
/* Create the NetCDF-4 file */
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <netcdf.h>

#define FILE_NAME "groups.nc"
#define NX 3
#define NY 4
#define NZ 2
#define NDIMS_2D 2
#define NDIMS_3D 3
#define ERRCODE 2
#define ERR(e) {printf("Error: %s\n", nc_strerror(e)); exit(ERRCODE);}

int main()
{
    int ncid, grp1_id, grp2_id, nested_id;
    int x_dimid, y_dimid, z_dimid;
    int dimids_2d[NDIMS_2D], dimids_3d[NDIMS_3D];
    int ubyte_varid, ushort_varid, uint_varid, int64_varid, uint64_varid;
    int retval;

    /* Data arrays for all five new integer types */
    unsigned char ubyte_data[NY][NX];
    unsigned short ushort_data[NY][NX];
    unsigned int uint_data[NY][NX];
    long long int64_data[NY][NX];
    unsigned long long uint64_data[NY][NX][NZ];

    printf("NetCDF-4 Groups Example\n");
    printf("=====\n\n");

    /* ===== WRITE PHASE ===== */
    printf("=== Phase 1: Create file with group hierarchy ===\n");

    /* Create the NetCDF-4 file */
    printf("Creating NetCDF-4 file: %s\n", FILE_NAME);
    if ((retval = nc_create(FILE_NAME, NC_CLOBBER|NC_NETCDF4, &ncid)))
        ERR(retval);

    /* Define root dimensions (visible in all groups) */
    printf("Defining root dimensions: x=%d, y=%d\n", NX, NY);
    if ((retval = nc_def_dim(ncid, "x", NX, &x_dimid)))
        ERR(retval);
    if ((retval = nc_def_dim(ncid, "y", NY, &y_dimid)))
        ERR(retval);

    /* Create SubGroup1 */
    printf("Creating SubGroup1\n");
    if ((retval = nc_def_grp(ncid, "SubGroup1", &grp1_id)))
        ERR(retval);

    /* Create SubGroup2 */
    printf("Creating SubGroup2\n");
    if ((retval = nc_def_grp(ncid, "SubGroup2", &grp2_id)))
        ERR(retval);

    /* Create NestedGroup under SubGroup2 */
    printf("Creating NestedGroup under SubGroup2\n");
    if ((retval = nc_def_grp(grp2_id, "NestedGroup", &nested_id)))

```



```

    ERR(retval);

/* Define local dimension z in NestedGroup */
printf("Defining local dimension z=%d in NestedGroup\n", NZ);
if ((retval = nc_def_dim(nested_id, "z", NZ, &z_dimid)))
    ERR(retval);

/* Define variables in each group using all 5 new integer types */
printf("\nDefining variables with new integer types:\n");

/* Root group: NC_UBYTE variable (2D: x, y) */
printf("  Root: ubyte_var (NC_UBYTE, 2D: x, y)\n");
dimids_2d[0] = y_dimid;
dimids_2d[1] = x_dimid;
if ((retval = nc_def_var(ncid, "ubyte_var", NC_UBYTE, NDIMS_2D, dimids_2d, &ubyte_varid)))
    ERR(retval);

/* SubGroup1: NC_USHORT variable (2D: x, y) */
printf("  SubGroup1: ushort_var (NC_USHORT, 2D: x, y)\n");
if ((retval = nc_def_var(grp1_id, "ushort_var", NC_USHORT, NDIMS_2D, dimids_2d,
&ushort_varid)))
    ERR(retval);

/* SubGroup2: NC_UINT variable (2D: x, y) */
printf("  SubGroup2: uint_var (NC_UINT, 2D: x, y)\n");
if ((retval = nc_def_var(grp2_id, "uint_var", NC_UINT, NDIMS_2D, dimids_2d, &uint_varid)))
    ERR(retval);

/* NestedGroup: NC_INT64 variable (2D: x, y) */
printf("  NestedGroup: int64_var (NC_INT64, 2D: x, y)\n");
if ((retval = nc_def_var(nested_id, "int64_var", NC_INT64, NDIMS_2D, dimids_2d,
&int64_varid)))
    ERR(retval);

/* NestedGroup: NC_UINT64 variable (3D: x, y, z) */
printf("  NestedGroup: uint64_var (NC_UINT64, 3D: x, y, z)\n");
dimids_3d[0] = y_dimid;
dimids_3d[1] = x_dimid;
dimids_3d[2] = z_dimid;
if ((retval = nc_def_var(nested_id, "uint64_var", NC_UINT64, NDIMS_3D, dimids_3d,
&uint64_varid)))
    ERR(retval);

/* End define mode */
if ((retval = nc_enddef(ncid)))
    ERR(retval);

/* Initialize data with sequential values starting from 1 */
printf("\nInitializing data with sequential values (1, 2, 3, ...):\n");
int value = 1;

/* NC_UBYTE data (3x4 = 12 values) */
for (int i = 0; i < NY; i++)
    for (int j = 0; j < NX; j++)
        ubyte_data[i][j] = (unsigned char)value++;

/* NC_USHORT data (3x4 = 12 values) */
for (int i = 0; i < NY; i++)
    for (int j = 0; j < NX; j++)
        ushort_data[i][j] = (unsigned short)value++;

```

```

/* NC_UINT data (3x4 = 12 values) */
for (int i = 0; i < NY; i++)
    for (int j = 0; j < NX; j++)
        uint_data[i][j] = (unsigned int)value++;

/* NC_INT64 data (3x4 = 12 values) */
for (int i = 0; i < NY; i++)
    for (int j = 0; j < NX; j++)
        int64_data[i][j] = (long long)value++;

/* NC_UINT64 data (3x4x2 = 24 values) */
for (int i = 0; i < NY; i++)
    for (int j = 0; j < NX; j++)
        for (int k = 0; k < NZ; k++)
            uint64_data[i][j][k] = (unsigned long long)value++;

/* Write data to all variables */
printf("Writing data to all variables...\n");
if ((retval = nc_put_var_uchar(ncid, ubyte_varid, &ubyte_data[0][0])))
    ERR(retval);
if ((retval = nc_put_var_ushort(grp1_id, ushort_varid, &ushort_data[0][0])))
    ERR(retval);
if ((retval = nc_put_var_uint(grp2_id, uint_varid, &uint_data[0][0])))
    ERR(retval);
if ((retval = nc_put_var_longlong(nested_id, int64_varid, &int64_data[0][0])))
    ERR(retval);
if ((retval = nc_put_var_ulonglong(nested_id, uint64_varid, &uint64_data[0][0][0])))
    ERR(retval);

/* Close the file */
if ((retval = nc_close(ncid)))
    ERR(retval);

printf("**** SUCCESS writing file!\n");

/* ===== READ AND VALIDATE PHASE ===== */
printf("\n=== Phase 2: Read and validate file structure ===\n");

/* Open the file for reading */
printf("Reopening file for validation...\n");
if ((retval = nc_open(FILE_NAME, NC_NOWRITE, &ncid)))
    ERR(retval);

/* Query and validate number of groups in root */
int ngrps;
int grpids[NC_MAX_VARS];
if ((retval = nc_inq_grps(ncid, &ngrps, grpids)))
    ERR(retval);

if (ngrps != 2) {
    printf("Error: Expected 2 groups in root, found %d\n", ngrps);
    exit(ERRCODE);
}
printf("Verified: Root has %d child groups\n", ngrps);

/* Navigate to groups by name */
printf("\nNavigating to groups by name:\n");
if ((retval = nc_inq_grp_ncid(ncid, "SubGroup1", &grp1_id)))
    ERR(retval);

```

```

printf(" Found SubGroup1 (ncid=%d)\n", grp1_id);

if ((retval = nc_inq_grp_ncid(ncid, "SubGroup2", &grp2_id)))
    ERR(retval);
printf(" Found SubGroup2 (ncid=%d)\n", grp2_id);

if ((retval = nc_inq_grp_ncid(grp2_id, "NestedGroup", &nested_id)))
    ERR(retval);
printf(" Found NestedGroup (ncid=%d)\n", nested_id);

/* Validate group names */
char grpname[NC_MAX_NAME + 1];
size_t grpname_len;

if ((retval = nc_inq_grpname(grp1_id, grpname)))
    ERR(retval);
if (strcmp(grpname, "SubGroup1") != 0) {
    printf("Error: Expected group name 'SubGroup1', found '%s'\n", grpname);
    exit(ERRCODE);
}

if ((retval = nc_inq_grpname_len(grp2_id, &grpname_len)))
    ERR(retval);
if ((retval = nc_inq_grpname(grp2_id, grpname)))
    ERR(retval);
if (strcmp(grpname, "SubGroup2") != 0) {
    printf("Error: Expected group name 'SubGroup2', found '%s'\n", grpname);
    exit(ERRCODE);
}

if ((retval = nc_inq_grpname(nested_id, grpname)))
    ERR(retval);
if (strcmp(grpname, "NestedGroup") != 0) {
    printf("Error: Expected group name 'NestedGroup', found '%s'\n", grpname);
    exit(ERRCODE);
}
printf("Verified: All group names correct\n");

/* Test dimension visibility across group boundaries */
printf("\n=== Phase 3: Test dimension visibility ===\n");
printf("Testing that root dimensions (x, y) are visible in all groups:\n");

int test_dimid;

/* Test x dimension visibility in SubGroup1 */
if ((retval = nc_inq_dimid(grp1_id, "x", &test_dimid)))
    ERR(retval);
printf(" ✓ SubGroup1 can see dimension 'x' from root\n");

/* Test y dimension visibility in SubGroup1 */
if ((retval = nc_inq_dimid(grp1_id, "y", &test_dimid)))
    ERR(retval);
printf(" ✓ SubGroup1 can see dimension 'y' from root\n");

/* Test x dimension visibility in SubGroup2 */
if ((retval = nc_inq_dimid(grp2_id, "x", &test_dimid)))
    ERR(retval);
printf(" ✓ SubGroup2 can see dimension 'x' from root\n");

/* Test y dimension visibility in SubGroup2 */

```

```

if ((retval = nc_inq_dimid(grp2_id, "y", &test_dimid)))
    ERR(retval);
printf(" ✓ SubGroup2 can see dimension 'y' from root\n");

/* Test x dimension visibility in NestedGroup */
if ((retval = nc_inq_dimid(nested_id, "x", &test_dimid)))
    ERR(retval);
printf(" ✓ NestedGroup can see dimension 'x' from root\n");

/* Test y dimension visibility in NestedGroup */
if ((retval = nc_inq_dimid(nested_id, "y", &test_dimid)))
    ERR(retval);
printf(" ✓ NestedGroup can see dimension 'y' from root\n");

/* Test local dimension z in NestedGroup */
if ((retval = nc_inq_dimid(nested_id, "z", &test_dimid)))
    ERR(retval);
printf(" ✓ NestedGroup can see its local dimension 'z'\n");

printf("Verified: Dimension visibility follows parent chain rules\n");

/* Validate dimension sizes */
printf("\nValidating dimension sizes:\n");
size_t len_x, len_y, len_z;

if ((retval = nc_inq_dimid(ncid, "x", &x_dimid)))
    ERR(retval);
if ((retval = nc_inq_dimlen(ncid, x_dimid, &len_x)))
    ERR(retval);
if (len_x != NX) {
    printf("Error: Expected x dimension = %d, found %zu\n", NX, len_x);
    exit(ERRCODE);
}

if ((retval = nc_inq_dimid(ncid, "y", &y_dimid)))
    ERR(retval);
if ((retval = nc_inq_dimlen(ncid, y_dimid, &len_y)))
    ERR(retval);
if (len_y != NY) {
    printf("Error: Expected y dimension = %d, found %zu\n", NY, len_y);
    exit(ERRCODE);
}

if ((retval = nc_inq_dimid(nested_id, "z", &z_dimid)))
    ERR(retval);
if ((retval = nc_inq_dimlen(nested_id, z_dimid, &len_z)))
    ERR(retval);
if (len_z != NZ) {
    printf("Error: Expected z dimension = %d, found %zu\n", NZ, len_z);
    exit(ERRCODE);
}

printf(" x = %zu, y = %zu, z = %zu\n", len_x, len_y, len_z);
printf("Verified: All dimension sizes correct\n");

/* Query and validate all variable metadata */
printf("\n=== Phase 4: Validate variable metadata ===\n");

char varname[NC_MAX_NAME + 1];
nc_type vartype;

```

```

int varndims;

/* Validate ubyte_var in root */
if ((retval = nc_inq_varid(ncid, "ubyte_var", &ubyte_varid)))
    ERR(retval);
if ((retval = nc_inq_var(ncid, ubyte_varid, varname, &vartype, &varndims, NULL, NULL)))
    ERR(retval);
if (vartype != NC_UBYTE || varndims != NDIMS_2D) {
    printf("Error: ubyte_var has wrong type or dimensions\n");
    exit(ERRCODE);
}
printf("  ✓ Root: ubyte_var (NC_UBYTE, %d)\n", varndims);

/* Validate ushort_var in SubGroup1 */
if ((retval = nc_inq_varid(grp1_id, "ushort_var", &ushort_varid)))
    ERR(retval);
if ((retval = nc_inq_var(grp1_id, ushort_varid, varname, &vartype, &varndims, NULL,
NULL)))
    ERR(retval);
if (vartype != NC_USHORT || varndims != NDIMS_2D) {
    printf("Error: ushort_var has wrong type or dimensions\n");
    exit(ERRCODE);
}
printf("  ✓ SubGroup1: ushort_var (NC_USHORT, %d)\n", varndims);

/* Validate uint_var in SubGroup2 */
if ((retval = nc_inq_varid(grp2_id, "uint_var", &uint_varid)))
    ERR(retval);
if ((retval = nc_inq_var(grp2_id, uint_varid, varname, &vartype, &varndims, NULL, NULL)))
    ERR(retval);
if (vartype != NC_UINT || varndims != NDIMS_2D) {
    printf("Error: uint_var has wrong type or dimensions\n");
    exit(ERRCODE);
}
printf("  ✓ SubGroup2: uint_var (NC_UINT, %d)\n", varndims);

/* Validate int64_var in NestedGroup */
if ((retval = nc_inq_varid(nested_id, "int64_var", &int64_varid)))
    ERR(retval);
if ((retval = nc_inq_var(nested_id, int64_varid, varname, &vartype, &varndims, NULL,
NULL)))
    ERR(retval);
if (vartype != NC_INT64 || varndims != NDIMS_2D) {
    printf("Error: int64_var has wrong type or dimensions\n");
    exit(ERRCODE);
}
printf("  ✓ NestedGroup: int64_var (NC_INT64, %d)\n", varndims);

/* Validate uint64_var in NestedGroup */
if ((retval = nc_inq_varid(nested_id, "uint64_var", &uint64_varid)))
    ERR(retval);
if ((retval = nc_inq_var(nested_id, uint64_varid, varname, &vartype, &varndims, NULL,
NULL)))
    ERR(retval);
if (vartype != NC_UINT64 || varndims != NDIMS_3D) {
    printf("Error: uint64_var has wrong type or dimensions\n");
    exit(ERRCODE);
}
printf("  ✓ NestedGroup: uint64_var (NC_UINT64, %d)\n", varndims);

```

```

printf("Verified: All variable metadata correct\n");

/* Read and validate all data */
printf("\n=== Phase 5: Read and validate data values ===\n");

unsigned char ubyte_in[NY][NX];
unsigned short ushort_in[NY][NX];
unsigned int uint_in[NY][NX];
long long int64_in[NY][NX];
unsigned long long uint64_in[NY][NX][NZ];

if ((retval = nc_get_var_uchar(ncid, ubyte_varid, &ubyte_in[0][0])))
    ERR(retval);
if ((retval = nc_get_var_ushort(grp1_id, ushort_varid, &ushort_in[0][0])))
    ERR(retval);
if ((retval = nc_get_var_uint(grp2_id, uint_varid, &uint_in[0][0])))
    ERR(retval);
if ((retval = nc_get_var_longlong(nested_id, int64_varid, &int64_in[0][0])))
    ERR(retval);
if ((retval = nc_get_var_ulonglong(nested_id, uint64_varid, &uint64_in[0][0][0])))
    ERR(retval);

/* Validate data correctness */
int errors = 0;
value = 1;

/* Validate NC_UBYTE data */
for (int i = 0; i < NY; i++) {
    for (int j = 0; j < NX; j++) {
        if (ubyte_in[i][j] != (unsigned char)value) {
            printf("Error: ubyte_var[%d][%d] = %u, expected %d\n",
                i, j, ubyte_in[i][j], value);
            errors++;
        }
        value++;
    }
}

/* Validate NC_USHORT data */
for (int i = 0; i < NY; i++) {
    for (int j = 0; j < NX; j++) {
        if (ushort_in[i][j] != (unsigned short)value) {
            printf("Error: ushort_var[%d][%d] = %u, expected %d\n",
                i, j, ushort_in[i][j], value);
            errors++;
        }
        value++;
    }
}

/* Validate NC_UINT data */
for (int i = 0; i < NY; i++) {
    for (int j = 0; j < NX; j++) {
        if (uint_in[i][j] != (unsigned int)value) {
            printf("Error: uint_var[%d][%d] = %u, expected %d\n",
                i, j, uint_in[i][j], value);
            errors++;
        }
        value++;
    }
}

```

```

}

/* Validate NC_INT64 data */
for (int i = 0; i < NY; i++) {
    for (int j = 0; j < NX; j++) {
        if (int64_in[i][j] != (long long)value) {
            printf("Error: int64_var[%d][%d] = %lld, expected %d\n",
                i, j, int64_in[i][j], value);
            errors++;
        }
        value++;
    }
}

/* Validate NC_UINT64 data */
for (int i = 0; i < NY; i++) {
    for (int j = 0; j < NX; j++) {
        for (int k = 0; k < NZ; k++) {
            if (uint64_in[i][j][k] != (unsigned long long)value) {
                printf("Error: uint64_var[%d][%d][%d] = %llu, expected %d\n",
                    i, j, k, uint64_in[i][j][k], value);
                errors++;
            }
            value++;
        }
    }
}

if (errors > 0) {
    printf("*** FAILED: %d data validation errors\n", errors);
    exit(ERRCODE);
}

int total_values = NX * NY * 4 + NX * NY * NZ; /* 4 2D arrays + 1 3D array */
printf("Verified: All %d data values correct (sequential 1 to %d)\n",
    total_values, total_values);

/* Close the file */
if ((retval = nc_close(ncid)))
    ERR(retval);

/* Summary */
printf("\n=== Summary ===\n");
printf("Group hierarchy:\n");
printf("  Root\n");
printf("    └─ SubGroup1\n");
printf("    └─ SubGroup2\n");
printf("        └─ NestedGroup\n");
printf("\nDimensions:\n");
printf("  Root: x=%d, y=%d (visible in all groups)\n", NX, NY);
printf("  NestedGroup: z=%d (local only)\n", NZ);
printf("\nVariables (all 5 new integer types):\n");
printf("  Root: ubyte_var (NC_UBYTE)\n");
printf("  SubGroup1: ushort_var (NC_USHORT)\n");
printf("  SubGroup2: uint_var (NC_UINT)\n");
printf("  NestedGroup: int64_var (NC_INT64), uint64_var (NC_UINT64)\n");
printf("\nKey Concepts Demonstrated:\n");
printf("  ✓ Hierarchical group structures (3 levels)\n");
printf("  ✓ Nested groups (NestedGroup under SubGroup2)\n");
printf("  ✓ Dimension visibility across group boundaries\n");

```

```

    printf("  ✓ All 5 new NetCDF-4 integer types\n");
    printf("  ✓ Variable scoping to defining group\n");

    printf("\n*** SUCCESS: All validation checks passed!\n");
    printf("Use 'ncdump groups.nc' to view the file structure.\n");

    return 0;
}

```

This results in the following CDL file:

```

netcdf groups {
dimensions:
    x = 3 ;
    y = 4 ;
variables:
    ubyte ubyte_var(y, x) ;
data:

    ubyte_var =
    1, 2, 3,
    4, 5, 6,
    7, 8, 9,
    10, 11, 12 ;

group: SubGroup1 {
    variables:
        ushort ushort_var(y, x) ;
    data:

        ushort_var =
        13, 14, 15,
        16, 17, 18,
        19, 20, 21,
        22, 23, 24 ;
} // group SubGroup1

group: SubGroup2 {
    variables:
        uint uint_var(y, x) ;
    data:

        uint_var =
        25, 26, 27,
        28, 29, 30,
        31, 32, 33,
        34, 35, 36 ;

    group: NestedGroup {
        dimensions:
            z = 2 ;
        variables:
            int64 int64_var(y, x) ;
            uint64 uint64_var(y, x, z) ;
        data:

            int64_var =
            37, 38, 39,
            40, 41, 42,
            43, 44, 45,
            46, 47, 48 ;
    }
}

```



```
        uint64_var =  
49, 50,  
51, 52,  
53, 54,  
55, 56,  
57, 58,  
59, 60,  
61, 62,  
63, 64,  
65, 66,  
67, 68,  
69, 70,  
71, 72 ;  
    } // group NestedGroup  
} // group SubGroup2  
}
```

Chapter 7 NetCDF Fortran Examples

Fortran Classic Model Examples

The examples in the rest of this chapter are from the NetCDF Expansion Pack. The examples can be built and run as part of that package.

Simple Example with Classic Format

This is the Fortran equivalent of `simple_2D.c`, demonstrating the fundamental workflow for working with NetCDF files using the Fortran 90 NetCDF API. The program creates a 2D integer array, writes it to a NetCDF file with a global attribute ("title") and a variable attribute ("units"), then reopens the file to verify metadata, attributes, and data correctness using `nf90_inquire()`, `nf90_inquire_dimension()`, and `nf90_inquire_variable()`.

Learning Objectives

- Understand Fortran NetCDF API (`nf90_*` functions)
- Learn Fortran column-major vs C row-major array ordering
- Add global and variable attributes
- Query file metadata with `nf90_inquire()`, `nf90_inquire_dimension()`, `nf90_inquire_variable()`
- Master error handling with `nf90_noerr` and `nf90_strerror()`
- Work with Fortran array indexing (1-based vs C's 0-based)
- Verify equivalence with C version (`simple_2D.c`)

Key Concepts

- **Fortran Column-Major:** Arrays stored column-first `[i,j]` vs C row-first `[j][i]`
- **Dimension Ordering:** Fortran reverses dimension order from C
- **1-Based Indexing:** Fortran arrays start at 1, C arrays start at 0
- **nf90 Module:** Fortran 90 NetCDF interface (use `netcdf`)
- **Error Handling:** Check `retval` against `nf90_noerr`

Fortran vs C Differences

Array Declaration: Fortran `data_out(NX, NY)` vs C `data_out[NY][NX]`

Dimension Order: Fortran `dimids(1)=x, dimids(2)=y` vs C `dimids[0]=y, dimids[1]=x`

Indexing: Fortran 1-based (1 to N) vs C 0-based (0 to N-1)

API Prefix: Fortran `nf90_*` vs C `nc_*`

Error Handling: Fortran subroutine call vs C macro

```

program f_simple_2D
  use netcdf
  implicit none

  character(len=*), parameter :: FILE_NAME = "f_simple_2D.nc"
  integer, parameter :: NDIMS = 2
  integer, parameter :: NX = 6, NY = 12

  integer :: ncid, varid
  integer :: x_dimid, y_dimid
  integer :: dimids(NDIMS)
  integer :: retval

  integer :: data_out(NX, NY)
  integer :: data_in(NX, NY)

  integer :: i, j
  integer :: ndims_in, nvars_in, ngatts_in, unlimdimid_in
  integer :: len_x, len_y
  character(len=NF90_MAX_NAME) :: dim_name
  character(len=NF90_MAX_NAME) :: var_name_in
  integer :: var_type, var_ndims
  integer :: var_dimids(NDIMS)
  character(len=100) :: title_in, units_in
  integer :: att_len
  integer :: errors
  integer :: expected

  ! ===== WRITE PHASE =====
  print *, "Creating NetCDF file: ", FILE_NAME

  ! Initialize data with sequential integers (0, 1, 2, 3, ...)
  ! Note: Fortran is column-major, so we fill by column to match C row-major layout
  do j = 1, NY
    do i = 1, NX
      data_out(i, j) = (j-1) * NX + (i-1)
    end do
  end do

  ! Create the NetCDF file (NF90_Clobber overwrites existing file)
  retval = nf90_create(FILE_NAME, NF90_Clobber, ncid)
  if (retval /= nf90_noerr) call handle_err(retval)

  ! Define dimensions
  retval = nf90_def_dim(ncid, "x", NX, x_dimid)
  if (retval /= nf90_noerr) call handle_err(retval)
  retval = nf90_def_dim(ncid, "y", NY, y_dimid)
  if (retval /= nf90_noerr) call handle_err(retval)

  ! Define the variable (dimension order: x, y for Fortran column-major)
  dimids(1) = x_dimid
  dimids(2) = y_dimid
  retval = nf90_def_var(ncid, "data", NF90_INT, dimids, varid)
  if (retval /= nf90_noerr) call handle_err(retval)

  ! Add a global attribute

```

```

retval = nf90_put_att(ncid, NF90_GLOBAL, "title", "Simple 2D Example")
if (retval /= nf90_noerr) call handle_err(retval)

! Add a variable attribute
retval = nf90_put_att(ncid, varid, "units", "m/s")
if (retval /= nf90_noerr) call handle_err(retval)

! End define mode
retval = nf90_enddef(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

! Write the data to the file
retval = nf90_put_var(ncid, varid, data_out)
if (retval /= nf90_noerr) call handle_err(retval)

! Close the file
retval = nf90_close(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

print *, "*** SUCCESS writing file!"

! ===== READ PHASE =====
print *, ""
print *, "Reopening file for validation..."

! Open the file for reading
retval = nf90_open(FILE_NAME, NF90_NOWRITE, ncid)
if (retval /= nf90_noerr) call handle_err(retval)

! Verify metadata: check number of dimensions and variables
retval = nf90_inquire(ncid, ndims_in, nvars_in, ngatts_in, unlimdimid_in)
if (retval /= nf90_noerr) call handle_err(retval)

if (ndims_in /= NDIMS) then
    print *, "Error: Expected ", NDIMS, " dimensions, found ", ndims_in
    stop 2
end if
print *, "Verified: ", ndims_in, " dimensions"

if (nvars_in /= 1) then
    print *, "Error: Expected 1 variable, found ", nvars_in
    stop 2
end if
print *, "Verified: ", nvars_in, " variable"

if (ngatts_in /= 1) then
    print *, "Error: Expected 1 global attribute, found ", ngatts_in
    stop 2
end if
print *, "Verified: ", ngatts_in, " global attribute"

if (unlimdimid_in /= -1) then
    print *, "Error: Expected no unlimited dimension, found dimid ", unlimdimid_in
    stop 2
end if
print *, "Verified: no unlimited dimension"

! Verify dimensions using nf90_inquire_dimension()
retval = nf90_inquire_dimension(ncid, x_dimid, name=dim_name, len=len_x)
if (retval /= nf90_noerr) call handle_err(retval)

```

```

if (trim(dim_name) /= "x") then
    print *, "Error: Expected dimension name 'x', found '", trim(dim_name), "'"
    stop 2
end if
if (len_x /= NX) then
    print *, "Error: Expected x dimension = ", NX, ", found ", len_x
    stop 2
end if
print *, "Verified: dimension '", trim(dim_name), "' = ", len_x

retval = nf90_inquire_dimension(ncid, y_dimid, name=dim_name, len=len_y)
if (retval /= nf90_noerr) call handle_err(retval)

if (trim(dim_name) /= "y") then
    print *, "Error: Expected dimension name 'y', found '", trim(dim_name), "'"
    stop 2
end if
if (len_y /= NY) then
    print *, "Error: Expected y dimension = ", NY, ", found ", len_y
    stop 2
end if
print *, "Verified: dimension '", trim(dim_name), "' = ", len_y

! Verify variable using nf90_inquire_variable()
retval = nf90_inquire_variable(ncid, varid, name=var_name_in, xtype=var_type, &
                             ndims=var_ndims, dimids=var_dimids)
if (retval /= nf90_noerr) call handle_err(retval)

if (trim(var_name_in) /= "data") then
    print *, "Error: Expected variable name 'data', found '", trim(var_name_in), "'"
    stop 2
end if
if (var_type /= NF90_INT) then
    print *, "Error: Expected variable type NF90_INT, found ", var_type
    stop 2
end if
if (var_ndims /= NDIMS) then
    print *, "Error: Expected ", NDIMS, " dimensions, found ", var_ndims
    stop 2
end if
if (var_dimids(1) /= x_dimid .or. var_dimids(2) /= y_dimid) then
    print *, "Error: Unexpected dimension IDs for variable"
    stop 2
end if
print *, "Verified: variable '", trim(var_name_in), "' type NF90_INT, ", var_ndims, " dims"

! Verify global attribute
retval = nf90_inquire_attribute(ncid, NF90_GLOBAL, "title", len=att_len)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_get_att(ncid, NF90_GLOBAL, "title", title_in)
if (retval /= nf90_noerr) call handle_err(retval)
if (title_in(1:att_len) /= "Simple 2D Example") then
    print *, "Error: Expected title 'Simple 2D Example', found '", &
           title_in(1:att_len), "'"
    stop 2
end if
print *, "Verified: global attribute 'title' = '", title_in(1:att_len), "'"

! Verify variable attribute

```

```

retval = nf90_inquire_attribute(ncid, varid, "units", len=att_len)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_get_att(ncid, varid, "units", units_in)
if (retval /= nf90_noerr) call handle_err(retval)
if (units_in(1:att_len) /= "m/s") then
    print *, "Error: Expected units 'm/s', found '", units_in(1:att_len), "'"
    stop 2
end if
print *, "Verified: variable attribute 'units' = '", units_in(1:att_len), "'"

! Read the data back
retval = nf90_get_var(ncid, varid, data_in)
if (retval /= nf90_noerr) call handle_err(retval)

! Verify data correctness
errors = 0
do j = 1, NY
    do i = 1, NX
        expected = (j-1) * NX + (i-1)
        if (data_in(i, j) /= expected) then
            print *, "Error: data(", i, ",", j, ") = ", data_in(i, j), &
                ", expected ", expected
            errors = errors + 1
        end if
    end do
end do

if (errors > 0) then
    print *, "**** FAILED: ", errors, " data validation errors"
    stop 2
end if

print *, "Verified: all ", NX * NY, " data values correct (0, 1, 2, ..., ", &
    NX * NY - 1, ") "

! Close the file
retval = nf90_close(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

print *, ""
print *, "**** SUCCESS: All validation checks passed!"
contains
    subroutine handle_err(status)
        integer, intent(in) :: status
        print *, "Error: ", trim(nf90_strerror(status))
        stop 2
    end subroutine handle_err
end program f_simple_2D

```

This yields the following metadata:

```

netcdf f_simple_2D {
dimensions:
    x = 6 ;
    y = 12 ;
variables:
    int data(y, x) ;
        data :units = "m/s" ;

```

```
// global attributes:
      :title = "Simple 2D Example" ;
data:
  data =
    0, 1, 2, 3, 4, 5,
    6, 7, 8, 9, 10, 11,
    12, 13, 14, 15, 16, 17,
    18, 19, 20, 21, 22, 23,
    24, 25, 26, 27, 28, 29,
    30, 31, 32, 33, 34, 35,
    36, 37, 38, 39, 40, 41,
    42, 43, 44, 45, 46, 47,
    48, 49, 50, 51, 52, 53,
    54, 55, 56, 57, 58, 59,
    60, 61, 62, 63, 64, 65,
    66, 67, 68, 69, 70, 71 ;
}
```

Coordinate Variables

This is the Fortran equivalent of `coord_vars.c`, demonstrating coordinate variables and CF (Climate and Forecast) convention metadata using the Fortran 90 NetCDF API. The program creates a 2D temperature field with latitude and longitude coordinate variables following CF conventions.

Learning Objectives

- Understand coordinate variables in Fortran NetCDF API
- Learn CF convention attributes (`nf90_put_att`)
- Master attribute definition and retrieval in Fortran
- Work with geospatial data in Fortran
- Verify equivalence with C version (`coord_vars.c`)

Key Fortran Concepts

- Character Attributes: Use `nf90_put_att` with character strings
- Attribute Length: Fortran handles string length automatically
- Array Ordering: Temperature(NLON, NLAT) vs C temperature[NLAT][NLON]

```
program f_coord_vars
  use netcdf
  implicit none

  character(len=*), parameter :: FILE_NAME = "f_coord_vars.nc"
  integer, parameter :: NLAT = 4, NLON = 5

  integer :: ncid, lat_varid, lon_varid, temp_varid
  integer :: lat_dimid, lon_dimid
  integer :: dimids(2)
  integer :: retval

  real :: lat(NLAT) = (/ -45.0, -15.0, 15.0, 45.0 /)
  real :: lon(NLON) = (/ -120.0, -60.0, 0.0, 60.0, 120.0 /)
```

```

real :: temperature(NLON, NLAT)

real :: lat_in(NLAT)
real :: lon_in(NLON)
real :: temperature_in(NLON, NLAT)

integer :: i, j
integer :: ndims_in, nvars_in
integer :: errors
character(len=256) :: att_text
integer :: att_len
real :: fill_value, fill_value_in

! ===== WRITE PHASE =====
print *, "Creating NetCDF file: ", FILE_NAME

! Initialize temperature data (synthetic: varies with lat and lon)
do i = 1, NLAT
  do j = 1, NLON
    temperature(j, i) = 273.15 + (i-1) * 5.0 + (j-1) * 2.0
  end do
end do

! Create the NetCDF file
retval = nf90_create(FILE_NAME, NF90_CLOBBER, ncid)
if (retval /= nf90_noerr) call handle_err(retval)

! Define dimensions
retval = nf90_def_dim(ncid, "lat", NLAT, lat_dimid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_def_dim(ncid, "lon", NLON, lon_dimid)
if (retval /= nf90_noerr) call handle_err(retval)

! Define coordinate variables (same name as dimension)
retval = nf90_def_var(ncid, "lat", NF90_FLOAT, lat_dimid, lat_varid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_def_var(ncid, "lon", NF90_FLOAT, lon_dimid, lon_varid)
if (retval /= nf90_noerr) call handle_err(retval)

! Add CF convention attributes to latitude
retval = nf90_put_att(ncid, lat_varid, "units", "degrees_north")
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_put_att(ncid, lat_varid, "standard_name", "latitude")
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_put_att(ncid, lat_varid, "long_name", "Latitude")
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_put_att(ncid, lat_varid, "axis", "Y")
if (retval /= nf90_noerr) call handle_err(retval)

! Add CF convention attributes to longitude
retval = nf90_put_att(ncid, lon_varid, "units", "degrees_east")
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_put_att(ncid, lon_varid, "standard_name", "longitude")
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_put_att(ncid, lon_varid, "long_name", "Longitude")
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_put_att(ncid, lon_varid, "axis", "X")
if (retval /= nf90_noerr) call handle_err(retval)

! Define temperature variable (Fortran order: lon, lat)

```



```

dimids(1) = lon_dimid
dimids(2) = lat_dimid
retval = nf90_def_var(ncid, "temperature", NF90_FLOAT, dimids, temp_varid)
if (retval /= nf90_noerr) call handle_err(retval)

! Add CF convention attributes to temperature
retval = nf90_put_att(ncid, temp_varid, "units", "K")
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_put_att(ncid, temp_varid, "standard_name", "air_temperature")
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_put_att(ncid, temp_varid, "long_name", "Air Temperature")
if (retval /= nf90_noerr) call handle_err(retval)

fill_value = -999.0
retval = nf90_put_att(ncid, temp_varid, "_FillValue", fill_value)
if (retval /= nf90_noerr) call handle_err(retval)

! End define mode
retval = nf90_enddef(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

! Write coordinate variables
retval = nf90_put_var(ncid, lat_varid, lat)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_put_var(ncid, lon_varid, lon)
if (retval /= nf90_noerr) call handle_err(retval)

! Write temperature data
retval = nf90_put_var(ncid, temp_varid, temperature)
if (retval /= nf90_noerr) call handle_err(retval)

! Close the file
retval = nf90_close(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

print *, "*** SUCCESS writing file!"

! ===== READ PHASE =====
print *, ""
print *, "Reopening file for validation..."

! Open the file for reading
retval = nf90_open(FILE_NAME, NF90_NOWRITE, ncid)
if (retval /= nf90_noerr) call handle_err(retval)

! Verify metadata
retval = nf90_inquire(ncid, ndims_in, nvars_in)
if (retval /= nf90_noerr) call handle_err(retval)

if (ndims_in /= 2) then
    print *, "Error: Expected 2 dimensions, found ", ndims_in
    stop 2
end if
print *, "Verified: ", ndims_in, " dimensions"

if (nvars_in /= 3) then
    print *, "Error: Expected 3 variables, found ", nvars_in
    stop 2
end if
print *, "Verified: ", nvars_in, " variables (lat, lon, temperature)"

```

```

! Verify latitude attributes
retval = nf90_inquire_attribute(ncid, lat_varid, "units", len=att_len)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_get_att(ncid, lat_varid, "units", att_text)
if (retval /= nf90_noerr) call handle_err(retval)
if (trim(att_text) /= "degrees_north") then
    print *, "Error: lat units = '", trim(att_text), "', expected 'degrees_north'"
    stop 2
end if
print *, "Verified: lat units = '", trim(att_text), "'"

retval = nf90_get_att(ncid, lat_varid, "standard_name", att_text)
if (retval /= nf90_noerr) call handle_err(retval)
if (trim(att_text) /= "latitude") then
    print *, "Error: lat standard_name = '", trim(att_text), "', expected 'latitude'"
    stop 2
end if
print *, "Verified: lat standard_name = '", trim(att_text), "'"

retval = nf90_get_att(ncid, lat_varid, "axis", att_text)
if (retval /= nf90_noerr) call handle_err(retval)
if (trim(att_text) /= "Y") then
    print *, "Error: lat axis = '", trim(att_text), "', expected 'Y'"
    stop 2
end if
print *, "Verified: lat axis = '", trim(att_text), "'"

! Verify longitude attributes
retval = nf90_get_att(ncid, lon_varid, "units", att_text)
if (retval /= nf90_noerr) call handle_err(retval)
if (trim(att_text) /= "degrees_east") then
    print *, "Error: lon units = '", trim(att_text), "', expected 'degrees_east'"
    stop 2
end if
print *, "Verified: lon units = '", trim(att_text), "'"

retval = nf90_get_att(ncid, lon_varid, "standard_name", att_text)
if (retval /= nf90_noerr) call handle_err(retval)
if (trim(att_text) /= "longitude") then
    print *, "Error: lon standard_name = '", trim(att_text), "', expected 'longitude'"
    stop 2
end if
print *, "Verified: lon standard_name = '", trim(att_text), "'"

! Verify temperature attributes
retval = nf90_get_att(ncid, temp_varid, "units", att_text)
if (retval /= nf90_noerr) call handle_err(retval)
if (trim(att_text) /= "K") then
    print *, "Error: temperature units = '", trim(att_text), "', expected 'K'"
    stop 2
end if
print *, "Verified: temperature units = '", trim(att_text), "'"

retval = nf90_get_att(ncid, temp_varid, "_FillValue", fill_value_in)
if (retval /= nf90_noerr) call handle_err(retval)
if (fill_value_in /= fill_value) then
    print *, "Error: temperature _FillValue = ", fill_value_in, ", expected ", fill_value
    stop 2
end if

```

```

print *, "Verified: temperature _FillValue = ", fill_value_in

! Read coordinate variables
retval = nf90_get_var(ncid, lat_varid, lat_in)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_get_var(ncid, lon_varid, lon_in)
if (retval /= nf90_noerr) call handle_err(retval)

! Verify coordinate data
errors = 0
do i = 1, NLAT
  if (lat_in(i) /= lat(i)) then
    print *, "Error: lat(", i, ") = ", lat_in(i), ", expected ", lat(i)
    errors = errors + 1
  end if
end do

do j = 1, NLON
  if (lon_in(j) /= lon(j)) then
    print *, "Error: lon(", j, ") = ", lon_in(j), ", expected ", lon(j)
    errors = errors + 1
  end if
end do

if (errors == 0) then
  print *, "Verified: coordinate arrays correct"
  print *, "  lat: [", lat(1), ", ", lat(2), ", ", lat(3), ", ", lat(4), "]"
  print *, "  lon: [", lon(1), ", ", lon(2), ", ", lon(3), ", ", lon(4), ", ", lon(5),
"]"
end if

! Read temperature data
retval = nf90_get_var(ncid, temp_varid, temperature_in)
if (retval /= nf90_noerr) call handle_err(retval)

! Verify temperature data
do i = 1, NLAT
  do j = 1, NLON
    if (temperature_in(j, i) /= temperature(j, i)) then
      print *, "Error: temperature(", j, ", ", i, ") = ", temperature_in(j, i), &
        ", expected ", temperature(j, i)
      errors = errors + 1
    end if
  end do
end do

if (errors > 0) then
  print *, "**** FAILED: ", errors, " data validation errors"
  stop 2
end if

print *, "Verified: all temperature data correct (", NLAT * NLON, " values)"

! Close the file
retval = nf90_close(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

print *, ""
print *, "**** SUCCESS: All validation checks passed!"

```

```
contains
  subroutine handle_err(status)
    integer, intent(in) :: status
    print *, "Error: ", trim(nf90_strerror(status))
    stop 2
  end subroutine handle_err
end program f_coord_vars
```

This program produces a file with this ncdump output:

```
netcdf f_coord_vars {
dimensions:
    lat = 4 ;
    lon = 5 ;
variables:
    float lat(lat) ;
        lat:units = "degrees_north" ;
        lat:standard_name = "latitude" ;
        lat:long_name = "Latitude" ;
        lat:axis = "Y" ;
    float lon(lon) ;
        lon:units = "degrees_east" ;
        lon:standard_name = "longitude" ;
        lon:long_name = "Longitude" ;
        lon:axis = "X" ;
    float temperature(lat, lon) ;
        temperature:units = "K" ;
        temperature:standard_name = "air_temperature" ;
        temperature:long_name = "Air Temperature" ;
        temperature:_FillValue = -999.f ;
data:

    lat = -45, -15, 15, 45 ;

    lon = -120, -60, 0, 60, 120 ;

    temperature =
        273.15, 275.15, 277.15, 279.15, 281.15,
        278.15, 280.15, 282.15, 284.15, 286.15,
        283.15, 285.15, 287.15, 289.15, 291.15,
        288.15, 290.15, 292.15, 294.15, 296.15 ;
}
```

Fortran Enhanced Model Examples

These example in Fortan all make use of the enhanced data model, or other features of the HDF5 library.

Different NetCDF Binary Files

This example is f_format_variants.f90 from the examples section of the NetCDF Expansion Pack project.

This is the Fortran equivalent of format_variants.c, demonstrating all five NetCDF binary format variants using the Fortran 90 NetCDF API. The program creates identical data structures in each format and compares their characteristics.

Learning Objectives

- Understand format flags in Fortran (NF90_CLASSIC_MODEL, NF90_64BIT_OFFSET, NF90_64BIT_DATA, NF90_NETCDF4, IOR(NF90_NETCDF4, NF90_CLASSIC_MODEL))
- Learn format detection with nf90_inq_format()
- Compare file sizes and format characteristics
- Make informed format choices in Fortran applications

Fortran Format Constants

- **NF90_CLASSIC_MODEL** CDF-1 format (2GB limits)
- **NF90_64BIT_OFFSET** CDF-2 format (4GB variable limit)
- **NF90_64BIT_DATA** CDF-5 format (unlimited sizes)
- **NF90_NETCDF4** NetCDF-4/HDF5 format (groups, compression, user types)
- **IOR(NF90_NETCDF4, NF90_CLASSIC_MODEL)** HDF5 storage, classic data model

```
program f_format_variants
  use netcdf
  implicit none

  integer, parameter :: NTIME = 10, NLAT = 20, NLON = 30
  integer, parameter :: ERRCODE = 2

  print *, "NetCDF Format Variants Comparison"
  print *, ""
  print *, "This program creates five files with identical data structures"
  print *, "in all five NetCDF binary formats to demonstrate their differences."
  print *, ""
  print *, "Data structure:"
  print *, "  Dimensions: time=", NTIME, ", lat=", NLAT, ", lon=", NLON
  print *, "  Variables: temperature(time,lat,lon), pressure(time,lat,lon)"
  print *, "  Data type: NF90_FLOAT (4 bytes per value)"
  print *, "  Total data: ", NTIME * NLAT * NLON, " values per variable"
  print *, ""

  ! Create files in each format
  print *, "=== Creating Format Files ==="
  print *, ""

  call create_format_file("f_format_classic.nc", NF90_CLASSIC_MODEL, &
    "NF90_CLASSIC_MODEL")
  call create_format_file("f_format_64bit_offset.nc", NF90_64BIT_OFFSET, &
    "NF90_64BIT_OFFSET")
  call create_format_file("f_format_64bit_data.nc", NF90_64BIT_DATA, &
    "NF90_64BIT_DATA")
  call create_format_file("f_format_netcdf4.nc", NF90_NETCDF4, &
    "NF90_NETCDF4")
  call create_format_file("f_format_netcdf4_classic.nc", &
    IOR(NF90_NETCDF4, NF90_CLASSIC_MODEL), &
    "NF90_NETCDF4+NF90_CLASSIC_MODEL")

  ! Verify files
  print *, ""
```

```

print *, "=== Verifying Format Files ==="

call verify_format_file("f_format_classic.nc", NF90_FORMAT_CLASSIC, &
    "NF90_FORMAT_CLASSIC")
call verify_format_file("f_format_64bit_offset.nc", NF90_FORMAT_64BIT_OFFSET, &
    "NF90_FORMAT_64BIT_OFFSET")
call verify_format_file("f_format_64bit_data.nc", NF90_FORMAT_64BIT_DATA, &
    "NF90_FORMAT_64BIT_DATA")
call verify_format_file("f_format_netcdf4.nc", NF90_FORMAT_NETCDF4, &
    "NF90_FORMAT_NETCDF4")
call verify_format_file("f_format_netcdf4_classic.nc", &
    NF90_FORMAT_NETCDF4_CLASSIC, &
    "NF90_FORMAT_NETCDF4_CLASSIC")

! Summary
print *, ""
print *, "=== Format Comparison Summary ==="
print *, ""
print *, "Format Characteristics:"
print *, ""
print *, "NF90_CLASSIC_MODEL (CDF-1):"
print *, "  File size limit: 2GB"
print *, "  Variable size limit: 2GB"
print *, "  Storage backend: CDF binary"
print *, "  Compatibility: NetCDF 3.0+, all tools"
print *, "  Use when: Maximum compatibility needed, files < 2GB"
print *, ""
print *, "NF90_64BIT_OFFSET (CDF-2):"
print *, "  File size limit: effectively unlimited"
print *, "  Variable size limit: 4GB per variable"
print *, "  Storage backend: CDF binary"
print *, "  Compatibility: NetCDF 3.6.0+"
print *, "  Use when: Large files needed, variables < 4GB each"
print *, ""
print *, "NF90_64BIT_DATA (CDF-5):"
print *, "  File size limit: effectively unlimited"
print *, "  Variable size limit: effectively unlimited"
print *, "  Storage backend: CDF binary"
print *, "  Compatibility: NetCDF 4.4.0+ or PnetCDF"
print *, "  Use when: Very large variables needed (> 4GB)"
print *, ""
print *, "NF90_NETCDF4 (HDF5):"
print *, "  File size limit: effectively unlimited"
print *, "  Variable size limit: effectively unlimited"
print *, "  Storage backend: HDF5"
print *, "  Compatibility: NetCDF 4.0+"
print *, "  Features: groups, compression, chunking, user-defined types"
print *, "  Use when: Advanced features needed (compression, groups, etc.)"
print *, ""
print *, "IOR(NF90_NETCDF4, NF90_CLASSIC_MODEL) (HDF5 Classic Model):"
print *, "  File size limit: effectively unlimited"
print *, "  Variable size limit: effectively unlimited"
print *, "  Storage backend: HDF5"
print *, "  Compatibility: NetCDF 4.0+"
print *, "  Features: compression, chunking (no groups, no user-defined types)"
print *, "  Use when: HDF5 storage benefits needed with classic data model"
print *, ""
print *, "Key Observations:"
print *, "  - All five formats store identical data correctly"
print *, "  - Classic formats (CDF-1/2/5) have smaller overhead for small files"

```

```

print *, " - NetCDF-4 formats (HDF5) have larger overhead but support compression"
print *, " - NC4 classic model is a useful middle ground: HDF5 storage, simple model"
print *, " - Use nf90_inq_format() to detect format type when reading files"
print *, ""
print *, "**** SUCCESS: All format tests passed! ****"

```

contains

```

subroutine create_format_file(filename, format_flag, format_name)
  character(len=*), intent(in) :: filename, format_name
  integer, intent(in) :: format_flag

  integer :: ncid, time_dimid, lat_dimid, lon_dimid
  integer :: temp_varid, pressure_varid
  integer :: dimids(3)
  integer :: retval

  real :: temperature(NLON, NLAT, NTIME)
  real :: pressure(NLON, NLAT, NTIME)
  integer :: t, i, j

  print *, "Creating ", trim(format_name), " format file: ", trim(filename)

  ! Initialize data
  do t = 1, NTIME
    do i = 1, NLAT
      do j = 1, NLON
        temperature(j, i, t) = 273.15 + (t-1) * 1.0 + (i-1) * 0.5 + (j-1) * 0.2
        pressure(j, i, t) = 1013.25 + (t-1) * 0.1 + (i-1) * 0.05 + (j-1) * 0.02
      end do
    end do
  end do

  ! Create file
  retval = nf90_create(filename, format_flag, ncid)
  if (retval /= nf90_noerr) call handle_err(retval)

  ! Define dimensions
  retval = nf90_def_dim(ncid, "time", NTIME, time_dimid)
  if (retval /= nf90_noerr) call handle_err(retval)
  retval = nf90_def_dim(ncid, "lat", NLAT, lat_dimid)
  if (retval /= nf90_noerr) call handle_err(retval)
  retval = nf90_def_dim(ncid, "lon", NLON, lon_dimid)
  if (retval /= nf90_noerr) call handle_err(retval)

  ! Define variables (Fortran order: lon, lat, time)
  dimids(1) = lon_dimid
  dimids(2) = lat_dimid
  dimids(3) = time_dimid

  retval = nf90_def_var(ncid, "temperature", NF90_FLOAT, dimids, temp_varid)
  if (retval /= nf90_noerr) call handle_err(retval)
  retval = nf90_def_var(ncid, "pressure", NF90_FLOAT, dimids, pressure_varid)
  if (retval /= nf90_noerr) call handle_err(retval)

  ! Add attributes
  retval = nf90_put_att(ncid, temp_varid, "units", "K")
  if (retval /= nf90_noerr) call handle_err(retval)
  retval = nf90_put_att(ncid, pressure_varid, "units", "hPa")
  if (retval /= nf90_noerr) call handle_err(retval)

```

```

! End define mode
retval = nf90_enddef(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

! Write data
retval = nf90_put_var(ncid, temp_varid, temperature)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_put_var(ncid, pressure_varid, pressure)
if (retval /= nf90_noerr) call handle_err(retval)

! Close file
retval = nf90_close(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

print *, " File created successfully"
end subroutine create_format_file

subroutine verify_format_file(filename, expected_format, expected_format_name)
character(len=*) intent(in) :: filename, expected_format_name
integer, intent(in) :: expected_format

integer :: ncid, retval
integer :: format_in
integer :: ndims, nvars
real :: temperature(NLON, NLAT, NTIME)
real :: pressure(NLON, NLAT, NTIME)
integer :: temp_varid, pressure_varid
character(len=50) :: detected_format
integer :: errors
real :: expected_temp, expected_pressure

print *, ""
print *, "Verifying file: ", trim(filename)
print *, " Expected format: ", trim(expected_format_name)

! Open file
retval = nf90_open(filename, NF90_NOWRITE, ncid)
if (retval /= nf90_noerr) call handle_err(retval)

! Check format
retval = nf90_inq_format(ncid, format_in)
if (retval /= nf90_noerr) call handle_err(retval)

! Determine format name
if (format_in == NF90_FORMAT_CLASSIC) then
    detected_format = "NF90_FORMAT_CLASSIC (CDF-1)"
else if (format_in == NF90_FORMAT_64BIT_OFFSET) then
    detected_format = "NF90_FORMAT_64BIT_OFFSET (CDF-2)"
else if (format_in == NF90_FORMAT_64BIT_DATA) then
    detected_format = "NF90_FORMAT_64BIT_DATA (CDF-5)"
else if (format_in == NF90_FORMAT_NETCDF4) then
    detected_format = "NF90_FORMAT_NETCDF4 (HDF5)"
else if (format_in == NF90_FORMAT_NETCDF4_CLASSIC) then
    detected_format = "NF90_FORMAT_NETCDF4_CLASSIC (HDF5/CM)"
else
    detected_format = "UNKNOWN"
end if

print *, " Format detected: ", trim(detected_format)

```



```

! Verify expected format
if (format_in /= expected_format) then
  print *, "Error: Expected format ", trim(expected_format_name), &
    " (", expected_format, "), got ", trim(detected_format), &
    " (", format_in, ")"
  stop ERRCODE
end if

! Verify metadata
retval = nf90_inquire(ncid, ndims, nvars)
if (retval /= nf90_noerr) call handle_err(retval)

if (ndims /= 3 .or. nvars /= 2) then
  print *, "Error: Expected 3 dimensions and 2 variables, found ", &
    ndims, " dims, ", nvars, " vars"
  stop ERRCODE
end if
print *, " Metadata: ", ndims, " dimensions, ", nvars, " variables"

! Get variable IDs
retval = nf90_inq_varid(ncid, "temperature", temp_varid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_inq_varid(ncid, "pressure", pressure_varid)
if (retval /= nf90_noerr) call handle_err(retval)

! Read data
retval = nf90_get_var(ncid, temp_varid, temperature)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_get_var(ncid, pressure_varid, pressure)
if (retval /= nf90_noerr) call handle_err(retval)

! Verify a few data values
errors = 0
expected_temp = 273.15
expected_pressure = 1013.25

if (temperature(1, 1, 1) /= expected_temp) then
  print *, "Error: temperature(1,1,1) = ", temperature(1, 1, 1), &
    ", expected ", expected_temp
  errors = errors + 1
end if

if (pressure(1, 1, 1) /= expected_pressure) then
  print *, "Error: pressure(1,1,1) = ", pressure(1, 1, 1), &
    ", expected ", expected_pressure
  errors = errors + 1
end if

if (errors == 0) then
  print *, " Data validation: ", NTIME * NLAT * NLON * 2, " values verified"
else
  print *, "*** FAILED: ", errors, " data validation errors"
  stop ERRCODE
end if

! Close file
retval = nf90_close(ncid)
if (retval /= nf90_noerr) call handle_err(retval)
end subroutine verify_format_file

```

```

subroutine handle_err(status)
  integer, intent(in) :: status
  print *, "Error: ", trim(nf90_strerror(status))
  stop ERRCODE
end subroutine handle_err

```

```
end program f_format_variants
```

When we do an `ncdump` of the resulting files, with the `-h` (header) and `-s` (secret) options, we can see the underlying binary format in the artificial `_Format` attribute:

```

ncdump -hs f_format_classic.nc
netcdf f_format_classic {
dimensions:
    time = 10 ;
    lat = 20 ;
    lon = 30 ;
variables:
    float temperature(time, lat, lon) ;
        temperature:units = "K" ;
    float pressure(time, lat, lon) ;
        pressure:units = "hPa" ;

// global attributes:
    :_Format = "classic" ;
}
ncdump -hs f_format_64bit_offset.nc
netcdf f_format_64bit_offset {
dimensions:
    time = 10 ;
    lat = 20 ;
    lon = 30 ;
variables:
    float temperature(time, lat, lon) ;
        temperature:units = "K" ;
    float pressure(time, lat, lon) ;
        pressure:units = "hPa" ;

// global attributes:
    :_Format = "64-bit offset" ;
}
ncdump -hs f_format_netcdf4.nc
netcdf f_format_netcdf4 {
dimensions:
    time = 10 ;
    lat = 20 ;
    lon = 30 ;
variables:
    float temperature(time, lat, lon) ;
        temperature:units = "K" ;
        temperature:_Storage = "contiguous" ;
        temperature:_Endianness = "little" ;
    float pressure(time, lat, lon) ;
        pressure:units = "hPa" ;
        pressure:_Storage = "contiguous" ;
        pressure:_Endianness = "little" ;

// global attributes:
    :_NCProperties = "version=2,netcdf=4.10.0-development,hdf5=1.14.6" ;
    :_SuperblockVersion = 2 ;
}

```

```

        :_IsNetcdf4 = 1 ;
        :_Format = "netCDF-4" ;
}
ncdump -hs f_format_netcdf4_classic.nc
netcdf f_format_netcdf4_classic {
dimensions:
    time = 10 ;
    lat = 20 ;
    lon = 30 ;
variables:
    float temperature(time, lat, lon) ;
        temperature:units = "K" ;
        temperature:_Storage = "contiguous" ;
        temperature:_Endianness = "little" ;
    float pressure(time, lat, lon) ;
        pressure:units = "hPa" ;
        pressure:_Storage = "contiguous" ;
        pressure:_Endianness = "little" ;

// global attributes:
    :_NCProperties = "version=2,netcdf=4.10.0-development,hdf5=1.14.6" ;
    :_SuperblockVersion = 2 ;
    :_IsNetcdf4 = 1 ;
    :_Format = "netCDF-4 classic model" ;
}

```

Simple File with NetCDF/HDF5 Format

This example is the Fortran equivalent of `simple_nc4.c`, demonstrating NetCDF-4 format using the Fortran 90 NetCDF API. It creates a simple 2D array with `NF90_NETCDF4` flag.

Learning Objectives:

- Understand `NF90_NETCDF4` flag in Fortran
- Learn format detection with `nf90_inq_format()`
- Prepare for NetCDF-4 features (compression, chunking, groups)

Fortran NetCDF-4 Constants:

- `NF90_NETCDF4` Create NetCDF-4/HDF5 format file
- `NF90_FORMAT_NETCDF4` Format detection constant

```

! Create the NetCDF-4 file with NF90_NETCDF4 flag
retval = nf90_create(FILE_NAME, NF90_Clobber + NF90_NETCDF4, ncid)
if (retval /= nf90_noerr) call handle_err(retval)

! Define dimensions
retval = nf90_def_dim(ncid, "x", NX, x_dimid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_def_dim(ncid, "y", NY, y_dimid)
if (retval /= nf90_noerr) call handle_err(retval)

! Define the variable (dimension order: x, y for Fortran column-major)
dimids(1) = x_dimid

```

```

dimids(2) = y_dimid
retval = nf90_def_var(ncid, "data", NF90_INT, dimids, varid)
if (retval /= nf90_noerr) call handle_err(retval)

! End define mode
retval = nf90_enddef(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

! Write the data to the file
retval = nf90_put_var(ncid, varid, data_out)
if (retval /= nf90_noerr) call handle_err(retval)

! Close the file
retval = nf90_close(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

print *, "**** SUCCESS writing file!"

```

This results in the following metadata in the output file:

```

ncdump -hs f_simple_nc4.nc
netcdf f_simple_nc4 {
dimensions:
    x = 6 ;
    y = 12 ;
variables:
    int data(y, x) ;
        data:_Storage = "contiguous" ;
        data:_Endianness = "little" ;

// global attributes:
    :_NCProperties = "version=2,netcdf=4.9.3,hdf5=1.14.6" ;
    :_SuperblockVersion = 2 ;
    :_IsNetcdf4 = 1 ;
    :_Format = "netCDF-4" ;
}

```

Compression

This example is the Fortran equivalent of `compression.c`, exploring NetCDF-4 compression using the Fortran 90 NetCDF API. It tests various compression configurations and measures performance.

Learning Objectives:

- Configure compression with `nf90_def_var_deflate()` in Fortran
- Measure compression performance in Fortran applications
- Select appropriate compression levels for different data

Fortran Compression Functions:

- `nf90_def_var_deflate(ncid, varid, shuffle, deflate, deflate_level)`
- `shuffle`: 0=off, 1=on
- `deflate`: 0=off, 1=on
- `deflate_level`: 1-9 (higher=better compression, slower)

```

retval = nf90_create(filename, NF90_CLOBBER + NF90_NETCDF4, ncid)
if (retval /= nf90_noerr) call handle_err(retval)

retval = nf90_def_dim(ncid, "time", NTIME, time_dimid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_def_dim(ncid, "lat", NLAT, lat_dimid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_def_dim(ncid, "lon", NLON, lon_dimid)
if (retval /= nf90_noerr) call handle_err(retval)

dimids(1) = lon_dimid
dimids(2) = lat_dimid
dimids(3) = time_dimid
retval = nf90_def_var(ncid, "temperature", NF90_FLOAT, dimids, varid)
if (retval /= nf90_noerr) call handle_err(retval)

if (deflate == 1 .or. shuffle == 1) then
    retval = nf90_def_var_deflate(ncid, varid, shuffle, deflate, deflate_level)
    if (retval /= nf90_noerr) call handle_err(retval)
end if

retval = nf90_enddef(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

retval = nf90_put_var(ncid, varid, data)
if (retval /= nf90_noerr) call handle_err(retval)

retval = nf90_close(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

```

This results in the following metadata:

```

ncdump -hs f_compress_shuffle_deflate5.nc
netcdf f_compress_shuffle_deflate5 {
dimensions:
    time = 50 ;
    lat = 90 ;
    lon = 180 ;
variables:
    float temperature(time, lat, lon) ;
        temperature:_Storage = "chunked" ;
        temperature:_ChunkSizes = 50, 90, 180 ;
        temperature:_Shuffle = "true" ;
        temperature:_DeflateLevel = 5 ;
        temperature:_Endianness = "little" ;

// global attributes:
    :_NCProperties = "version=2,netcdf=4.9.3,hdf5=1.14.6" ;
    :_SuperblockVersion = 2 ;
    :_IsNetcdf4 = 1 ;
    :_Format = "netCDF-4" ;
}

```

User-Defined Types

This example is the Fortran equivalent of `user_types.c`, demonstrating NetCDF-4 user-defined types using the Fortran 90 NetCDF API. Note: Fortran support for compound types is limited compared to C, so this focuses on enum, vlen, and opaque types.

Learning Objectives:

- Understand Fortran limitations with compound types
- Work with enum types in Fortran (nf90_def_enum)
- Use vlen types for variable-length arrays
- Handle opaque types for binary data

Fortran User Type Limitations:

- Compound types have limited Fortran support
- Enum, vlen, and opaque types fully supported
- Type definitions similar to C API

```
retval = nf90_create(FILE_NAME, NF90_CLOBBER + NF90_NETCDF4, ncid)
if (retval /= nf90_noerr) call handle_err(retval)

call define_and_test_opaque(ncid)

retval = nf90_close(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

print *, ""
print *, "=== Use Cases ==="
print *, "- Opaque types: Binary metadata or proprietary formats"
print *, "- For compound, vlen, and enum types: See user_types.c (C API)"
print *, "- Newer netcdf-fortran versions may have better support"

print *, ""
print *, "**** SUCCESS: User-defined types demonstrated!"
```

contains

```
subroutine define_and_test_opaque(ncid)
  integer, intent(in) :: ncid
  integer :: opaque_typeid
  integer :: retval

  print *, ""
  print *, "--- Opaque Type (binary calibration data) ---"

  retval = nf90_def_opaque(ncid, CALIB_SIZE, "calibration_t", opaque_typeid)
  if (retval /= nf90_noerr) call handle_err(retval)
  print *, "Defined opaque type with ", CALIB_SIZE, "-byte size"
  print *, "Note: Full opaque type I/O requires C API or newer Fortran bindings"
  print *, "(Type definition successful, but data I/O not demonstrated)"

  retval = nf90_enddef(ncid)
  if (retval /= nf90_noerr) call handle_err(retval)

end subroutine define_and_test_opaque
```

This yields the following metadata in the output file:

```
ncdump -hs f_user_types.nc
```

```

netcdf f_user_types {
types:
  opaque(16) calibration_t ;

// global attributes:
  _NCProperties = "version=2,netcdf=4.9.3,hdf5=1.14.6" ;
  _SuperblockVersion = 2 ;
  _IsNetcdf4 = 0 ;
  _Format = "netCDF-4" ;
}

```

Groups and New Atomic Types

Fortran equivalent of groups.c, demonstrating NetCDF-4's hierarchical group feature using the Fortran 90 NetCDF API. Groups enable organizing datasets into logical groupings similar to directories in a filesystem, providing namespace isolation for variables while allowing dimensions to be shared across the hierarchy through dimension visibility rules.

The program creates a three-level group hierarchy (root → SubGroup1, root → SubGroup2 → NestedGroup), demonstrates dimension visibility across group boundaries, and showcases all five new NetCDF-4 integer types (NF90_UBYTE, NF90_USHORT, NF90_UINT, NF90_INT64, NF90_UINT64).

Learning Objectives:

- Understand NetCDF-4 hierarchical group structures in Fortran
- Learn to create and navigate nested groups
- Master dimension visibility rules across group boundaries
- Work with all five new NetCDF-4 integer types
- Recognize when groups provide organizational benefits

Key Concepts:

- Hierarchical Groups: Organize datasets into logical groupings (like directories)
- Dimension Visibility: Parent dimensions visible in all child groups
- Variable Scoping: Variables only visible in their defining group
- Group Navigation: Use `nf90_inq_grp_ncid()` to navigate by name
- New Integer Types: NF90_UBYTE, NF90_USHORT, NF90_UINT, NF90_INT64, NF90_UINT64

NetCDF-4 Group Architecture:

- Groups implemented via NC_GRP_INFO_T structures (libsrc4/libhdf5)
- Dimensions visible in child groups via parent chain lookup
- Variables NOT inherited (scoped to defining group only)
- Requires NF90_NETCDF4 flag (HDF5 backend)

- Not compatible with NF90_CLASSIC_MODEL

Dimension Visibility Rules:

- Dimensions defined in a group are visible in that group and all descendants
- Root dimensions (x, y) visible in SubGroup1, SubGroup2, and NestedGroup
- Local dimensions (z in NestedGroup) only visible in defining group
- Dimension lookup walks parent chain: child → parent → root

Use Cases for Groups:

- Multi-instrument datasets: Group data by instrument or sensor
- Model ensembles: Separate ensemble members into groups
- Quality levels: Organize raw, calibrated, and derived products
- Temporal organization: Group data by year, month, or campaign

Namespace management: Avoid variable name conflicts

```

program f_groups
  use netcdf
  implicit none

  character(len=*), parameter :: FILE_NAME = "f_groups.nc"
  integer, parameter :: NX = 3
  integer, parameter :: NY = 4
  integer, parameter :: NZ = 2
  integer, parameter :: NDIMS_2D = 2
  integer, parameter :: NDIMS_3D = 3
  integer, parameter :: ERRCODE = 2

  integer :: ncid, grp1_id, grp2_id, nested_id
  integer :: x_dimid, y_dimid, z_dimid
  integer :: dimids_2d(NDIMS_2D), dimids_3d(NDIMS_3D)
  integer :: ubyte_varid, ushort_varid, uint_varid, int64_varid, uint64_varid
  integer :: retval

  ! Data arrays for all five new integer types
  ! Note: Fortran uses column-major ordering (dimensions reversed from C)
  integer(kind=1), dimension(NX, NY) :: ubyte_data
  integer(kind=2), dimension(NX, NY) :: ushort_data
  integer(kind=4), dimension(NX, NY) :: uint_data
  integer(kind=8), dimension(NX, NY) :: int64_data
  integer(kind=8), dimension(NX, NY, NZ) :: uint64_data

  integer :: i, j, k, value

  print *, "NetCDF-4 Groups Example (Fortran)"
  print *, "======"
  print *, ""

  ! ===== WRITE PHASE =====
  print *, "=== Phase 1: Create file with group hierarchy ==="

  ! Create the NetCDF-4 file

```



```

print *, "Creating NetCDF-4 file: ", FILE_NAME
retval = nf90_create(FILE_NAME, NF90_CLOBBER + NF90_NETCDF4, ncid)
if (retval /= nf90_noerr) call handle_err(retval)

! Define root dimensions (visible in all groups)
print *, "Defining root dimensions: x=", NX, ", y=", NY
retval = nf90_def_dim(ncid, "x", NX, x_dimid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_def_dim(ncid, "y", NY, y_dimid)
if (retval /= nf90_noerr) call handle_err(retval)

! Create SubGroup1
print *, "Creating SubGroup1"
retval = nf90_def_grp(ncid, "SubGroup1", grp1_id)
if (retval /= nf90_noerr) call handle_err(retval)

! Create SubGroup2
print *, "Creating SubGroup2"
retval = nf90_def_grp(ncid, "SubGroup2", grp2_id)
if (retval /= nf90_noerr) call handle_err(retval)

! Create NestedGroup under SubGroup2
print *, "Creating NestedGroup under SubGroup2"
retval = nf90_def_grp(grp2_id, "NestedGroup", nested_id)
if (retval /= nf90_noerr) call handle_err(retval)

! Define local dimension z in NestedGroup
print *, "Defining local dimension z=", NZ, " in NestedGroup"
retval = nf90_def_dim(nested_id, "z", NZ, z_dimid)
if (retval /= nf90_noerr) call handle_err(retval)

! Define variables in each group using all 5 new integer types
print *, ""
print *, "Defining variables with new integer types:"

! Root group: NF90_UBYTE variable (2D: x, y)
! Note: Fortran dimension order is reversed from C
print *, " Root: ubyte_var (NF90_UBYTE, 2D: x, y)"
dimids_2d(1) = x_dimid
dimids_2d(2) = y_dimid
retval = nf90_def_var(ncid, "ubyte_var", NF90_UBYTE, dimids_2d, ubyte_varid)
if (retval /= nf90_noerr) call handle_err(retval)

! SubGroup1: NF90_USHORT variable (2D: x, y)
print *, " SubGroup1: ushort_var (NF90_USHORT, 2D: x, y)"
retval = nf90_def_var(grp1_id, "ushort_var", NF90_USHORT, dimids_2d, ushort_varid)
if (retval /= nf90_noerr) call handle_err(retval)

! SubGroup2: NF90_UINT variable (2D: x, y)
print *, " SubGroup2: uint_var (NF90_UINT, 2D: x, y)"
retval = nf90_def_var(grp2_id, "uint_var", NF90_UINT, dimids_2d, uint_varid)
if (retval /= nf90_noerr) call handle_err(retval)

! NestedGroup: NF90_INT64 variable (2D: x, y)
print *, " NestedGroup: int64_var (NF90_INT64, 2D: x, y)"
retval = nf90_def_var(nested_id, "int64_var", NF90_INT64, dimids_2d, int64_varid)
if (retval /= nf90_noerr) call handle_err(retval)

! NestedGroup: NF90_UINT64 variable (3D: x, y, z)
print *, " NestedGroup: uint64_var (NF90_UINT64, 3D: x, y, z)"

```

```

dimids_3d(1) = x_dimid
dimids_3d(2) = y_dimid
dimids_3d(3) = z_dimid
retval = nf90_def_var(nested_id, "uint64_var", NF90_UINT64, dimids_3d, uint64_varid)
if (retval /= nf90_noerr) call handle_err(retval)

! End define mode
retval = nf90_enddef(ncid)
if (retval /= nf90_noerr) call handle_err(retval)

! Initialize data with sequential values starting from 1
print *, ""
print *, "Initializing data with sequential values (1, 2, 3, ...):"
value = 1

! NF90_UBYTE data (3x4 = 12 values)
do j = 1, NY
  do i = 1, NX
    ubyte_data(i, j) = int(value, kind=1)
    value = value + 1
  end do
end do

! NF90_USHORT data (3x4 = 12 values)
do j = 1, NY
  do i = 1, NX
    ushort_data(i, j) = int(value, kind=2)
    value = value + 1
  end do
end do

! NF90_UINT data (3x4 = 12 values)
do j = 1, NY
  do i = 1, NX
    uint_data(i, j) = int(value, kind=4)
    value = value + 1
  end do
end do

! NF90_INT64 data (3x4 = 12 values)
do j = 1, NY
  do i = 1, NX
    int64_data(i, j) = int(value, kind=8)
    value = value + 1
  end do
end do

! NF90_UINT64 data (3x4x2 = 24 values)
do k = 1, NZ
  do j = 1, NY
    do i = 1, NX
      uint64_data(i, j, k) = int(value, kind=8)
      value = value + 1
    end do
  end do
end do

! Write data to all variables
print *, "Writing data to all variables..."
retval = nf90_put_var(ncid, ubyte_varid, ubyte_data)

```

```

if (retval != nf90_noerr) call handle_err(retval)
retval = nf90_put_var(grp1_id, ushort_varid, ushort_data)
if (retval != nf90_noerr) call handle_err(retval)
retval = nf90_put_var(grp2_id, uint_varid, uint_data)
if (retval != nf90_noerr) call handle_err(retval)
retval = nf90_put_var(nested_id, int64_varid, int64_data)
if (retval != nf90_noerr) call handle_err(retval)
retval = nf90_put_var(nested_id, uint64_varid, uint64_data)
if (retval != nf90_noerr) call handle_err(retval)

! Close the file
retval = nf90_close(ncid)
if (retval != nf90_noerr) call handle_err(retval)

print *, "*** SUCCESS writing file!"

! ===== READ AND VALIDATE PHASE =====
print *, ""
print *, "=== Phase 2: Read and validate file structure ==="

! Open the file for reading
print *, "Reopening file for validation..."
retval = nf90_open(FILE_NAME, NF90_NOWRITE, ncid)
if (retval != nf90_noerr) call handle_err(retval)

! Query and validate number of groups in root
call validate_groups(ncid, grp1_id, grp2_id, nested_id)

! Test dimension visibility across group boundaries
call test_dimension_visibility(grp1_id, grp2_id, nested_id)

! Validate dimension sizes
call validate_dimensions(ncid, nested_id)

! Query and validate all variable metadata
call validate_variables(ncid, grp1_id, grp2_id, nested_id, &
                      ubyte_varid, ushort_varid, uint_varid, int64_varid, uint64_varid)

! Read and validate all data
call validate_data(ncid, grp1_id, grp2_id, nested_id, &
                  ubyte_varid, ushort_varid, uint_varid, int64_varid, uint64_varid)

! Close the file
retval = nf90_close(ncid)
if (retval != nf90_noerr) call handle_err(retval)

! Summary
print *, ""
print *, "=== Summary ==="
print *, "Group hierarchy:"
print *, "  Root"
print *, "    └─ SubGroup1"
print *, "    └─ SubGroup2"
print *, "        └─ NestedGroup"
print *, ""
print *, "Dimensions:"
print *, "  Root: x=", NX, ", y=", NY, " (visible in all groups)"
print *, "  NestedGroup: z=", NZ, " (local only)"
print *, ""
print *, "Variables (all 5 new integer types):"

```

```

print *, " Root: ubyte_var (NF90_UBYTE)"
print *, " SubGroup1: ushort_var (NF90_USHORT)"
print *, " SubGroup2: uint_var (NF90_UINT)"
print *, " NestedGroup: int64_var (NF90_INT64), uint64_var (NF90_UINT64)"
print *, ""
print *, "Key Concepts Demonstrated:"
print *, "  ✓ Hierarchical group structures (3 levels)"
print *, "  ✓ Nested groups (NestedGroup under SubGroup2)"
print *, "  ✓ Dimension visibility across group boundaries"
print *, "  ✓ All 5 new NetCDF-4 integer types"
print *, "  ✓ Variable scoping to defining group"
print *, ""
print *, "**** SUCCESS: All validation checks passed!"
print *, "Use 'ncdump f_groups.nc' to view the file structure."

```

contains

```

subroutine validate_groups(ncid, grp1_id, grp2_id, nested_id)
  integer, intent(in) :: ncid
  integer, intent(out) :: grp1_id, grp2_id, nested_id
  integer :: ngrps, retval
  integer, dimension(NF90_MAX_VARS) :: grpids
  character(len=NF90_MAX_NAME) :: grpname

  ! Query number of groups in root
  retval = nf90_inq_grps(ncid, ngrps, grpids)
  if (retval /= nf90_noerr) call handle_err(retval)

  if (ngrps /= 2) then
    print *, "Error: Expected 2 groups in root, found ", ngrps
    stop ERRCODE
  end if
  print *, "Verified: Root has ", ngrps, " child groups"

  ! Navigate to groups by name
  print *, ""
  print *, "Navigating to groups by name:"
  retval = nf90_inq_grp_ncid(ncid, "SubGroup1", grp1_id)
  if (retval /= nf90_noerr) call handle_err(retval)
  print *, " Found SubGroup1 (ncid=", grp1_id, ")"

  retval = nf90_inq_grp_ncid(ncid, "SubGroup2", grp2_id)
  if (retval /= nf90_noerr) call handle_err(retval)
  print *, " Found SubGroup2 (ncid=", grp2_id, ")"

  retval = nf90_inq_grp_ncid(grp2_id, "NestedGroup", nested_id)
  if (retval /= nf90_noerr) call handle_err(retval)
  print *, " Found NestedGroup (ncid=", nested_id, ")"

  ! Validate group names
  retval = nf90_inq_grpname(grp1_id, grpname)
  if (retval /= nf90_noerr) call handle_err(retval)
  if (trim(grpname) /= "SubGroup1") then
    print *, "Error: Expected group name 'SubGroup1', found '", trim(grpname), "'"
    stop ERRCODE
  end if

  retval = nf90_inq_grpname(grp2_id, grpname)
  if (retval /= nf90_noerr) call handle_err(retval)
  if (trim(grpname) /= "SubGroup2") then

```

```

        print *, "Error: Expected group name 'SubGroup2', found '", trim(grpname), "'"
        stop ERRCODE
    end if

    retval = nf90_inq_grpname(nested_id, grpname)
    if (retval /= nf90_noerr) call handle_err(retval)
    if (trim(grpname) /= "NestedGroup") then
        print *, "Error: Expected group name 'NestedGroup', found '", trim(grpname), "'"
        stop ERRCODE
    end if
    print *, "Verified: All group names correct"
end subroutine validate_groups

subroutine test_dimension_visibility(grp1_id, grp2_id, nested_id)
    integer, intent(in) :: grp1_id, grp2_id, nested_id
    integer :: test_dimid, retval

    print *, ""
    print *, "=== Phase 3: Test dimension visibility ==="
    print *, "Testing that root dimensions (x, y) are visible in all groups:"

    ! Test x dimension visibility in SubGroup1
    retval = nf90_inq_dimid(grp1_id, "x", test_dimid)
    if (retval /= nf90_noerr) call handle_err(retval)
    print *, "  ✓ SubGroup1 can see dimension 'x' from root"

    ! Test y dimension visibility in SubGroup1
    retval = nf90_inq_dimid(grp1_id, "y", test_dimid)
    if (retval /= nf90_noerr) call handle_err(retval)
    print *, "  ✓ SubGroup1 can see dimension 'y' from root"

    ! Test x dimension visibility in SubGroup2
    retval = nf90_inq_dimid(grp2_id, "x", test_dimid)
    if (retval /= nf90_noerr) call handle_err(retval)
    print *, "  ✓ SubGroup2 can see dimension 'x' from root"

    ! Test y dimension visibility in SubGroup2
    retval = nf90_inq_dimid(grp2_id, "y", test_dimid)
    if (retval /= nf90_noerr) call handle_err(retval)
    print *, "  ✓ SubGroup2 can see dimension 'y' from root"

    ! Test x dimension visibility in NestedGroup
    retval = nf90_inq_dimid(nested_id, "x", test_dimid)
    if (retval /= nf90_noerr) call handle_err(retval)
    print *, "  ✓ NestedGroup can see dimension 'x' from root"

    ! Test y dimension visibility in NestedGroup
    retval = nf90_inq_dimid(nested_id, "y", test_dimid)
    if (retval /= nf90_noerr) call handle_err(retval)
    print *, "  ✓ NestedGroup can see dimension 'y' from root"

    ! Test local dimension z in NestedGroup
    retval = nf90_inq_dimid(nested_id, "z", test_dimid)
    if (retval /= nf90_noerr) call handle_err(retval)
    print *, "  ✓ NestedGroup can see its local dimension 'z'"

    print *, "Verified: Dimension visibility follows parent chain rules"
end subroutine test_dimension_visibility

subroutine validate_dimensions(ncid, nested_id)

```

```

integer, intent(in) :: ncid, nested_id
integer :: x_dimid, y_dimid, z_dimid, retval
integer :: len_x, len_y, len_z

print *, ""
print *, "Validating dimension sizes:"

retval = nf90_inq_dimid(ncid, "x", x_dimid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_inquire_dimension(ncid, x_dimid, len=len_x)
if (retval /= nf90_noerr) call handle_err(retval)
if (len_x /= NX) then
    print *, "Error: Expected x dimension = ", NX, ", found ", len_x
    stop ERRCODE
end if

retval = nf90_inq_dimid(ncid, "y", y_dimid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_inquire_dimension(ncid, y_dimid, len=len_y)
if (retval /= nf90_noerr) call handle_err(retval)
if (len_y /= NY) then
    print *, "Error: Expected y dimension = ", NY, ", found ", len_y
    stop ERRCODE
end if

retval = nf90_inq_dimid(ncid, "z", z_dimid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_inquire_dimension(ncid, z_dimid, len=len_z)
if (retval /= nf90_noerr) call handle_err(retval)
if (len_z /= NZ) then
    print *, "Error: Expected z dimension = ", NZ, ", found ", len_z
    stop ERRCODE
end if

print *, " x = ", len_x, ", y = ", len_y, ", z = ", len_z
print *, "Verified: All dimension sizes correct"
end subroutine validate_dimensions

subroutine validate_variables(ncid, grp1_id, grp2_id, nested_id, &
                             ubyte_varid, ushort_varid, uint_varid, int64_varid,
uint64_varid)
integer, intent(in) :: ncid, grp1_id, grp2_id, nested_id
integer, intent(out) :: ubyte_varid, ushort_varid, uint_varid, int64_varid, uint64_varid
integer :: retval, vartype, varndims

print *, ""
print *, "=== Phase 4: Validate variable metadata ==="

! Validate ubyte_var in root
retval = nf90_inq_varid(ncid, "ubyte_var", ubyte_varid)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_inquire_variable(ncid, ubyte_varid, xtype=vartype, ndims=varndims)
if (retval /= nf90_noerr) call handle_err(retval)
if (vartype /= NF90_UBYTE .or. varndims /= NDIMS_2D) then
    print *, "Error: ubyte_var has wrong type or dimensions"
    stop ERRCODE
end if
print *, "  ✓ Root: ubyte_var (NF90_UBYTE, ", varndims, "D)"

! Validate ushort_var in SubGroup1

```

```

    retval = nf90_inq_varid(grp1_id, "ushort_var", ushort_varid)
    if (retval /= nf90_noerr) call handle_err(retval)
    retval = nf90_inquire_variable(grp1_id, ushort_varid, xtype=vartype, ndims=varndims)
    if (retval /= nf90_noerr) call handle_err(retval)
    if (vartype /= NF90_USHORT .or. varndims /= NDIMS_2D) then
        print *, "Error: ushort_var has wrong type or dimensions"
        stop ERRCODE
    end if
    print *, "  ✓ SubGroup1: ushort_var (NF90_USHORT, ", varndims, "D)"

    ! Validate uint_var in SubGroup2
    retval = nf90_inq_varid(grp2_id, "uint_var", uint_varid)
    if (retval /= nf90_noerr) call handle_err(retval)
    retval = nf90_inquire_variable(grp2_id, uint_varid, xtype=vartype, ndims=varndims)
    if (retval /= nf90_noerr) call handle_err(retval)
    if (vartype /= NF90_UINT .or. varndims /= NDIMS_2D) then
        print *, "Error: uint_var has wrong type or dimensions"
        stop ERRCODE
    end if
    print *, "  ✓ SubGroup2: uint_var (NF90_UINT, ", varndims, "D)"

    ! Validate int64_var in NestedGroup
    retval = nf90_inq_varid(nested_id, "int64_var", int64_varid)
    if (retval /= nf90_noerr) call handle_err(retval)
    retval = nf90_inquire_variable(nested_id, int64_varid, xtype=vartype, ndims=varndims)
    if (retval /= nf90_noerr) call handle_err(retval)
    if (vartype /= NF90_INT64 .or. varndims /= NDIMS_2D) then
        print *, "Error: int64_var has wrong type or dimensions"
        stop ERRCODE
    end if
    print *, "  ✓ NestedGroup: int64_var (NF90_INT64, ", varndims, "D)"

    ! Validate uint64_var in NestedGroup
    retval = nf90_inq_varid(nested_id, "uint64_var", uint64_varid)
    if (retval /= nf90_noerr) call handle_err(retval)
    retval = nf90_inquire_variable(nested_id, uint64_varid, xtype=vartype, ndims=varndims)
    if (retval /= nf90_noerr) call handle_err(retval)
    if (vartype /= NF90_UINT64 .or. varndims /= NDIMS_3D) then
        print *, "Error: uint64_var has wrong type or dimensions"
        stop ERRCODE
    end if
    print *, "  ✓ NestedGroup: uint64_var (NF90_UINT64, ", varndims, "D)"

    print *, "Verified: All variable metadata correct"
end subroutine validate_variables

subroutine validate_data(ncid, grp1_id, grp2_id, nested_id, &
    ubyte_varid, ushort_varid, uint_varid, int64_varid, uint64_varid)
    integer, intent(in) :: ncid, grp1_id, grp2_id, nested_id
    integer, intent(in) :: ubyte_varid, ushort_varid, uint_varid, int64_varid, uint64_varid
    integer :: retval, i, j, k, value, errors, total_values

    integer(kind=1), dimension(NX, NY) :: ubyte_in
    integer(kind=2), dimension(NX, NY) :: ushort_in
    integer(kind=4), dimension(NX, NY) :: uint_in
    integer(kind=8), dimension(NX, NY) :: int64_in
    integer(kind=8), dimension(NX, NY, NZ) :: uint64_in

    print *, ""
    print *, "=== Phase 5: Read and validate data values ==="

```

```

retval = nf90_get_var(ncid, ubyte_varid, ubyte_in)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_get_var(grp1_id, ushort_varid, ushort_in)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_get_var(grp2_id, uint_varid, uint_in)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_get_var(nested_id, int64_varid, int64_in)
if (retval /= nf90_noerr) call handle_err(retval)
retval = nf90_get_var(nested_id, uint64_varid, uint64_in)
if (retval /= nf90_noerr) call handle_err(retval)

! Validate data correctness
errors = 0
value = 1

! Validate NF90_UBYTE data
do j = 1, NY
  do i = 1, NX
    if (ubyte_in(i, j) /= int(value, kind=1)) then
      print *, "Error: ubyte_var(", i, ",", j, ") = ", ubyte_in(i, j), &
        ", expected ", value
      errors = errors + 1
    end if
    value = value + 1
  end do
end do

! Validate NF90_USHORT data
do j = 1, NY
  do i = 1, NX
    if (ushort_in(i, j) /= int(value, kind=2)) then
      print *, "Error: ushort_var(", i, ",", j, ") = ", ushort_in(i, j), &
        ", expected ", value
      errors = errors + 1
    end if
    value = value + 1
  end do
end do

! Validate NF90_UINT data
do j = 1, NY
  do i = 1, NX
    if (uint_in(i, j) /= int(value, kind=4)) then
      print *, "Error: uint_var(", i, ",", j, ") = ", uint_in(i, j), &
        ", expected ", value
      errors = errors + 1
    end if
    value = value + 1
  end do
end do

! Validate NF90_INT64 data
do j = 1, NY
  do i = 1, NX
    if (int64_in(i, j) /= int(value, kind=8)) then
      print *, "Error: int64_var(", i, ",", j, ") = ", int64_in(i, j), &
        ", expected ", value
      errors = errors + 1
    end if
  end do
end do

```



```

        value = value + 1
    end do
end do

! Validate NF90_UINT64 data
do k = 1, NZ
    do j = 1, NY
        do i = 1, NX
            if (uint64_in(i, j, k) /= int(value, kind=8)) then
                print *, "Error: uint64_var(", i, ",", j, ",", k, ") = ", uint64_in(i, j,
k), &
                    ", expected ", value
                errors = errors + 1
            end if
            value = value + 1
        end do
    end do
end do

if (errors > 0) then
    print *, "**** FAILED: ", errors, " data validation errors"
    stop ERRCODE
end if

total_values = NX * NY * 4 + NX * NY * NZ
print *, "Verified: All ", total_values, " data values correct (sequential 1 to ",
total_values, ")")
end subroutine validate_data

subroutine handle_err(status)
    integer, intent(in) :: status
    print *, "Error: ", trim(nf90_strerror(status))
    stop ERRCODE
end subroutine handle_err

end program f_groups

```

This yields a file with this ncdump output:

```

netcdf f_groups {
dimensions:
    x = 3 ;
    y = 4 ;
variables:
    ubyte ubyte_var(y, x) ;
data:

    ubyte_var =
        1, 2, 3,
        4, 5, 6,
        7, 8, 9,
        10, 11, 12 ;

group: SubGroup1 {
    variables:
        ushort ushort_var(y, x) ;
    data:

        ushort_var =
            13, 14, 15,
            16, 17, 18,

```

```

19, 20, 21,
22, 23, 24 ;
} // group SubGroup1

group: SubGroup2 {
  variables:
    uint uint_var(y, x) ;
  data:

    uint_var =
25, 26, 27,
28, 29, 30,
31, 32, 33,
34, 35, 36 ;

  group: NestedGroup {
    dimensions:
      z = 2 ;
    variables:
      int64 int64_var(y, x) ;
      uint64 uint64_var(z, y, x) ;
    data:

      int64_var =
37, 38, 39,
40, 41, 42,
43, 44, 45,
46, 47, 48 ;

      uint64_var =
49, 50, 51,
52, 53, 54,
55, 56, 57,
58, 59, 60,
61, 62, 63,
64, 65, 66,
67, 68, 69,
70, 71, 72 ;
  } // group NestedGroup
} // group SubGroup2
}

```

Most Important Fortran API Functions

This reference lists the most commonly used NetCDF Fortran 90 API functions organized by category. All functions use the NF90_ prefix.

Dataset Operations

Function	Purpose
NF90_CREATE	Create a new NetCDF dataset
NF90_OPEN	Open an existing dataset
NF90_CLOSE	Close an open dataset
NF90_REDEF	Enter define mode

Function	Purpose
NF90_ENDDEF	Exit define mode, enter data mode
NF90_SYNC	Synchronize dataset to disk
NF90_ABORT	Close dataset without saving changes
NF90_INQUIRE	Get information about dataset
NF90_SET_FILL	Set fill mode for variables

Dimension Operations

Function	Purpose
NF90_DEF_DIM	Define a dimension
NF90_INQ_DIMID	Get dimension ID from name
NF90_INQUIRE_DIMENSION	Get dimension information
NF90_RENAME_DIM	Rename a dimension

Variable Operations

Function	Purpose
NF90_DEF_VAR	Define a variable
NF90_INQ_VARID	Get variable ID from name
NF90_INQUIRE_VARIABLE	Get variable information
NF90_RENAME_VAR	Rename a variable
NF90_DEF_VAR_FILL	Define fill parameters
NF90_INQ_VAR_FILL	Get fill parameters
NF90_DEF_VAR_FILTER	Define filter/compression
NF90_INQ_VAR_FILTER	Get filter information

Variable I/O

Function	Purpose
NF90_PUT_VAR	Write data to a variable (entire array or subset)
NF90_GET_VAR	Read data from a variable (entire array or subset)

Note: NF90_PUT_VAR and NF90_GET_VAR are generic interfaces that support:

- Entire array access
- Hyperslab access (with **start** and **count** parameters)
- Strided access (with **stride** parameter)

- Mapped access (with **map** parameter)

Attribute Operations

Function	Purpose
NF90_PUT_ATT	Write an attribute
NF90_GET_ATT	Read an attribute
NF90_INQ_ATTNAME	Get attribute name from number
NF90_INQUIRE_ATTRIBUTE	Get attribute information
NF90_RENAME_ATT	Rename an attribute
NF90_DEL_ATT	Delete an attribute
NF90_COPY_ATT	Copy attribute to another variable

NetCDF-4 Group Operations

Function	Purpose
NF90_DEF_GRP	Create a group
NF90_INQ_NCID	Get group ID from name
NF90_INQ_GRPS	Get child group IDs
NF90_INQ_GRPNAME	Get group name
NF90_RENAME_GRP	Rename a group

NetCDF-4 User-Defined Types

Function	Purpose
NF90_DEF_COMPOUND	Define compound type
NF90_DEF_VLEN	Define variable-length type
NF90_DEF_OPAQUE	Define opaque type
NF90_DEF_ENUM	Define enumeration type

NetCDF-4 Chunking and Compression

Function	Purpose
NF90_DEF_VAR_CHUNKING	Set chunking parameters
NF90_INQ_VAR_CHUNKING	Get chunking parameters
NF90_DEF_VAR_DEFLATE	Set deflate compression
NF90_INQ_VAR_DEFLATE	Get deflate compression settings
NF90_DEF_VAR_FLETCHER32	Set Fletcher32 checksum

Function	Purpose
NF90_INQ_VAR_FLETCHER32	Get Fletcher32 checksum setting

NetCDF-4 Parallel I/O

Function	Purpose
NF90_CREATE_PAR	Create file with parallel access
NF90_OPEN_PAR	Open file with parallel access
NF90_VAR_PAR_ACCESS	Set collective/independent access mode

Error Handling

Function	Purpose
NF90_STRERROR	Get descriptive error message from error code

Data Types

NetCDF Type	Fortran 90 Constant	Bits
byte	NF90_BYTE	8
char	NF90_CHAR	8
short	NF90_SHORT	16
int	NF90_INT	32
float	NF90_FLOAT	32
double	NF90_DOUBLE	64
ubyte	NF90_UBYTE	8
ushort	NF90_USHORT	16
uint	NF90_UINT	32
int64	NF90_INT64	64
uint64	NF90_UINT64	64
string	NF90_STRING	-

Mode Flags

Flag	Purpose
NF90_NOWRITE	Open read-only
NF90_WRITE	Open for writing
NF90_CLOBBER	Overwrite existing file
NF90_NOCLOBBER	Fail if file exists
NF90_SHARE	Disable buffering for immediate writes

Flag	Purpose
NF90_NETCDF4	Create NetCDF-4/HDF5 file
NF90_CLASSIC_MODEL	Use classic data model with NetCDF-4
NF90_64BIT_OFFSET	Use CDF-2 format (large file support)
NF90_64BIT_DATA	Use CDF-5 format (large variable support)
NF90_MPIO	Use MPI I/O for parallel access

Storage Options (NetCDF-4)

Flag	Purpose
NF90_CHUNKED	Use chunked storage
NF90_CONTIGUOUS	Use contiguous storage
NF90_COMPACT	Use compact storage (small variables)

Special Constants

Constant	Purpose
NF90_UNLIMITED	Size for unlimited dimension
NF90_GLOBAL	Global attribute identifier
NF90_FILL	Enable fill values
NF90_NOFILL	Disable fill values
NF90_COLLECTIVE	Collective parallel I/O
NF90_INDEPENDENT	Independent parallel I/O
NF90_NOERR	No error (success)

Error Codes

Code	Meaning
NF90_NOERR	No error
NF90_EBADID	Invalid NetCDF ID
NF90_ENOTVAR	Variable not found
NF90_EINDEFINE	Operation not allowed in define mode
NF90_ENOTINDEFINE	Operation requires define mode
NF90_EINVAL	Invalid argument

Programming with NetCDF in Fortran – Key Takeaways

- APIs: F77 (**nf_***) mirrors C, F90 (**nf90_***) uses optional args.

- **Dim order** reversed versus C.
- Use **nf90_inq_varid** to avoid hard-coding IDs.
- **Compile:** `gfortran prog.f90 -lnetcdff -lnetcdf`.

Chapter 8 Programming with NetCDF in Java

Learning Objectives

- Access data via Common Data Model
- Utilize NetCDF-Java features
- Implement thread-safe access

Re-implementation in Java

The netCDF-Java library is a complete re-implementation of netCDF in pure Java. Unlike the C and Fortran APIs which are wrappers around the netCDF-C library, netCDF-Java is an independent implementation that provides access to netCDF functionality through Java-friendly interfaces. The library implements the Common Data Model (CDM), which unifies netCDF, OPeNDAP, HDF5, and other scientific data formats under a single API.

For C programming examples, see **Chapter 6: Programming with NetCDF in C**. For Fortran programming examples, see **Chapter 7: Programming with NetCDF in Fortran**.

Understanding the NetCDF-Java API

The netCDF-Java library provides a modern, object-oriented interface for working with scientific data. It uses Java's type system, exception handling, and resource management features to create a clean programming experience.

The primary classes for data access are contained in the `ucar.nc2` package. Programs must import the necessary classes to access the API:

```
import ucar.nc2.NetcdfFile;
import ucar.nc2.NetcdfFiles;
import ucar.nc2.Variable;
import ucar.nc2.Dimension;
import ucar.nc2.Attribute;
import ucar.ma2.Array;
import ucar.ma2.Index;
import java.io.IOException;
public class Example {
    public static void main(String[] args) {
        String filename = "example.nc";
        try (NetcdfFile ncfile = NetcdfFiles.open(filename)) {
            // Find a variable
            Variable tempVar = ncfile.findVariable("temperature");
            if (tempVar == null) {
                System.err.println("Variable not found");
                return;
            }
            // Read data
            Array data = tempVar.read();
            // Get attributes
            Attribute unitsAttr = tempVar.findAttribute("units");
```



```

        if (unitsAttr != null) {
            System.out.println("Units: " + unitsAttr.getStringValue());
        }
    } catch (IOException e) {
        System.err.println("Error: " + e.getMessage());
        e.printStackTrace();
    }
}
}

```

The netCDF-Java API uses Java's try-with-resources statement to ensure files are properly closed. The `NetcdfFile` class implements `AutoCloseable`, so it will be automatically closed when the try block exits, even if an exception occurs.

Unlike the C and Fortran APIs, netCDF-Java uses zero-based indexing for arrays, consistent with Java conventions. When reading a subset of data, the first element is at index 0, not 1.

The library provides automatic type conversion between the on-disk data type and Java data types. For example, you can read float data from a file into a Java double array, and the library will handle the conversion.

Error Handling

NetCDF-Java uses Java's exception mechanism for error handling. Most operations can throw `IOException` or its subclasses. Always wrap netCDF operations in try-catch blocks or declare that your method throws `IOException`.

```

try (NetcdfFile ncfile = NetcdfFiles.open(filename)) {
    Variable var = ncfile.findVariable("data");
    if (var == null) {
        throw new IOException("Variable 'data' not found");
    }
    Array data = var.read();
} catch (IOException e) {
    System.err.println("Error reading file: " + e.getMessage());
    e.printStackTrace();
}

```

The library provides detailed error messages that include information about what went wrong. Unlike the C API's numeric error codes, Java exceptions carry descriptive messages and stack traces that help identify the source of problems.

Logging

NetCDF-Java uses the SLF4J (Simple Logging Facade for Java) logging framework. To see logging output, you need to include an SLF4J implementation like Logback or Log4j in your project.

To enable detailed logging for debugging:

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

```

```

public class Example {
    private static final Logger logger = LoggerFactory.getLogger(Example.class);
    public static void main(String[] args) {
        logger.info("Opening netCDF file");
        try (NetcdfFile ncfile = NetcdfFiles.open(filename)) {
            logger.debug("File opened successfully");
            // ... operations ...
        } catch (IOException e) {
            logger.error("Failed to open file", e);
        }
    }
}

```

Configure logging levels in your `logback.xml` or `log4j2.xml` configuration file:

```

<logger name="ucar.nc2" level="DEBUG"/>
<logger name="ucar.ma2" level="INFO"/>

```

Logging Level	Meaning	Notes
ERROR	Critical errors only.	Minimal output.
WARN	Warnings and errors.	Useful for production.
INFO	General information.	Shows file operations.
DEBUG	Detailed debugging info.	High verbosity.
TRACE	Very detailed tracing.	Extremely verbose, use sparingly.

Opening, Creating, and Closing Files

The `NetcdfFiles.open()` method is used to open existing files. It automatically detects the file format (netCDF-3, netCDF-4, HDF5, etc.).

```

try (NetcdfFile ncfile = NetcdfFiles.open(filename)) {
    // File operations here
} // File automatically closed

```

For remote files, netCDF-Java supports various protocols:

```

// OPeNDAP
NetcdfFile ncfile = NetcdfFiles.open("https://server.org/dods/dataset");
// HTTP
NetcdfFile ncfile = NetcdfFiles.open("https://server.org/data/file.nc");
// AWS S3
NetcdfFile ncfile = NetcdfFiles.open("cdms3://bucket-name/path/to/file.nc");

```

Opening Files with Options

For more control over file opening, use `NetcdfFile.Builder`:

```

NetcdfFile ncfile = NetcdfFile.builder()
    .setLocation(filename)
    .setIosp(ioServiceProvider) // Optional: specify format handler
    .setCancelTask(cancelTask) // Optional: allow cancellation
    .build();

```

Creating Files

NetCDF-Java can write netCDF-3 files natively using the `NetcdfFormatWriter` class:

```
import ucar.nc2.write.NetcdfFormatWriter;
import ucar.ma2.DataType;
String outputFile = "output.nc";
try (NetcdfFormatWriter writer = NetcdfFormatWriter.createNewNetcdf3(outputFile)
    .build()) {

    // Define dimensions
    Dimension timeDim = writer.addUnlimitedDimension("time");
    Dimension latDim = writer.addDimension("lat", 180);
    Dimension lonDim = writer.addDimension("lon", 360);

    // Define variable
    Variable.Builder<?> tempVar = writer.addVariable("temperature", DataType.FLOAT,
        "time lat lon");
    tempVar.addAttribute(new Attribute("units", "Celsius"));
    tempVar.addAttribute(new Attribute("long_name", "Air Temperature"));

    // Add global attribute
    writer.addAttribute(new Attribute("title", "Temperature Data"));

    // Create the file (writes metadata)
    writer.create();

    // Write data (after create())
    Array data = Array.factory(DataType.FLOAT, new int[]{1, 180, 360});
    // ... fill data array ...
    writer.write(tempVar.getFullName(), data);

} catch (IOException e) {
    e.printStackTrace();
}
```

Note: For netCDF-4/HDF5 file creation, netCDF-Java requires the netCDF-C library via JNI (Java Native Interface). Pure Java can only write netCDF-3 format files.

Metadata

When working with netCDF files in Java, metadata is accessed through object methods rather than function calls. The netCDF-Java API provides a clean object-oriented interface for dimensions, variables, and attributes.

Reading Dimensions

Dimensions are accessed through the `NetcdfFile` object:

```
try (NetcdfFile ncfile = NetcdfFiles.open(filename)) {
    // List all dimensions
    for (Dimension dim : ncfile.getDimensions()) {
        System.out.println("Dimension: " + dim.getFullName());
        System.out.println("  Length: " + dim.getLength());
        System.out.println("  Unlimited: " + dim.isUnlimited());
    }
}
```

```

    }

    // Find specific dimension
    Dimension timeDim = ncfile.findDimension("time");
    if (timeDim != null) {
        int timeLength = timeDim.getLength();
        System.out.println("Time dimension has " + timeLength + " steps");
    }
}

```

Reading Variables

Variables are found by name and provide access to both metadata and data:

```

try (NetcdfFile ncfile = NetcdfFiles.open(filename)) {
    // Find a variable
    Variable tempVar = ncfile.findVariable("temperature");
    if (tempVar == null) {
        System.err.println("Variable not found");
        return;
    }

    // Get variable metadata
    System.out.println("Variable: " + tempVar.getFullName());
    System.out.println("Data type: " + tempVar.getDataType());
    System.out.println("Shape: " + java.util.Arrays.toString(tempVar.getShape()));
    System.out.println("Rank: " + tempVar.getRank());

    // Get dimensions
    for (Dimension dim : tempVar.getDimensions()) {
        System.out.println("    Dimension: " + dim.getFullName() +
            " (" + dim.getLength() + ")");
    }

    // List all variables
    for (Variable var : ncfile.getVariables()) {
        System.out.println("Variable: " + var.getFullName());
    }
}

```

Reading Attributes

Attributes can be attached to variables or to the file globally:

```

try (NetcdfFile ncfile = NetcdfFiles.open(filename)) {
    Variable tempVar = ncfile.findVariable("temperature");

    // Find specific attribute
    Attribute unitsAttr = tempVar.findAttribute("units");
    if (unitsAttr != null) {
        String units = unitsAttr.getStringValue();
        System.out.println("Units: " + units);
    }

    // List all variable attributes
    for (Attribute attr : tempVar.attributes()) {
        System.out.println("Attribute: " + attr.getFullName());
        System.out.println("    Type: " + attr.getDataType());
        System.out.println("    Value: " + attr.getValue());
    }
}

```

```

// Get global attributes
for (Attribute attr : ncfile.getGlobalAttributes()) {
    System.out.println("Global: " + attr.getFullName() +
        " = " + attr.getValue());
}

// Get numeric attribute
Attribute validMin = tempVar.findAttribute("valid_min");
if (validMin != null) {
    double minValue = validMin.getNumericValue().doubleValue();
    System.out.println("Valid minimum: " + minValue);
}
}

```

Writing Metadata

When creating files, metadata is defined using builder patterns:

```

try (NetcdfFormatWriter writer = NetcdfFormatWriter.createNewNetcdf3(outputFile)
    .build()) {

    // Define dimensions
    Dimension timeDim = writer.addUnlimitedDimension("time");
    Dimension latDim = writer.addDimension("lat", 180);
    Dimension lonDim = writer.addDimension("lon", 360);

    // Define variable with dimensions
    Variable.Builder<?> tempVar = writer.addVariable("temperature",
        DataType.FLOAT, "time lat lon");

    // Add variable attributes
    tempVar.addAttribute(new Attribute("units", "Celsius"));
    tempVar.addAttribute(new Attribute("long_name", "Air Temperature"));
    tempVar.addAttribute(new Attribute("valid_min", -50.0));
    tempVar.addAttribute(new Attribute("valid_max", 50.0));

    // Add global attributes
    writer.addAttribute(new Attribute("title", "Temperature Dataset"));
    writer.addAttribute(new Attribute("institution", "Research Center"));
    writer.addAttribute(new Attribute("source", "Model output"));

    // Create the file
    writer.create();
}

```

Metadata Best Practices

When creating netCDF files in Java, follow the same metadata conventions as in C and Fortran. At a minimum, every variable should have a "units" attribute. Additional attributes like "long_name", "valid_min", "valid_max", and "_FillValue" make data self-describing. Following established conventions, such as the CF Conventions, ensures interoperability. For more information on conventions, see **Chapter 9: Attributes and Conventions**.

Reading and Writing Data

After opening a file and locating variables, the next step is to read or write the actual data. NetCDF-Java uses the `Array` class from the `ucar.ma2` package to represent multi-dimensional arrays.

Reading Entire Variables

The simplest way to read data is to read an entire variable at once:

```
Variable tempVar = ncfile.findVariable("temperature");
Array data = tempVar.read();
// Get array properties
int[] shape = data.getShape();
int rank = data.getRank();
long size = data.getSize();
System.out.println("Shape: " + java.util.Arrays.toString(shape));
System.out.println("Total elements: " + size);
```

This reads all data into memory. For large variables (gigabytes or more), this may not be practical due to memory constraints.

Reading Scalar Variables

Scalar variables (zero dimensions) can be read with convenience methods:

```
Variable scalarVar = ncfile.findVariable("scalar_value");
double value = scalarVar.readScalarDouble();
float floatValue = scalarVar.readScalarFloat();
int intValue = scalarVar.readScalarInt();
```

Reading Subsets with Section Specification

NetCDF-Java uses a string-based section specification syntax for reading subsets. The syntax is based on Fortran 90 array sections but uses zero-based indexing:

```
Variable tempVar = ncfile.findVariable("temperature");
// Read subset: first 10 time steps, all lat/lon
// Format: "dim0_range, dim1_range, dim2_range"
Array subset = tempVar.read("0:9, :, :");
// Read with stride (every 2nd element)
Array strided = tempVar.read("0:10:2, :, :");
// Read single time slice
Array slice = tempVar.read("5, :, :");
// Read single point
Array point = tempVar.read("5, 90, 180");
```

Section specification syntax:

- ":" - all elements in dimension
- "start:end" - elements from start to end (inclusive, zero-based)
- "start:end:stride" - elements with stride
- "index" - single element at index

Reading with Explicit Ranges

For more control, use the `Section` class:

```
import ucar.ma2.Section;
import ucar.ma2.Range;
Variable tempVar = ncfile.findVariable("temperature");
// Create section with explicit ranges
Section section = new Section();
section.appendRange(new Range(0, 9));      // time: 0-9
section.appendRange(new Range(0, 179));    // lat: all
section.appendRange(new Range(0, 359));    // lon: all
Array data = tempVar.read(section);
```

Accessing Array Elements

The `Array` class provides several ways to access individual elements:

```
// 1D array access
Array data1d = var1d.read();
for (int i = 0; i < data1d.getSize(); i++) {
    double value = data1d.getDouble(i);
    System.out.println("data[" + i + "] = " + value);
}

// Multi-dimensional array with Index
Array data3d = var3d.read();
Index index = data3d.getIndex();
int[] shape = data3d.getShape();
for (int t = 0; t < shape[0]; t++) {
    for (int y = 0; y < shape[1]; y++) {
        for (int x = 0; x < shape[2]; x++) {
            double value = data3d.getDouble(index.set(t, y, x));
            // Process value...
        }
    }
}

// Direct access with linear index
long linearIndex = 0;
for (int i = 0; i < data3d.getSize(); i++) {
    double value = data3d.getDouble(i);
}
```

Type-Specific Access

The `Array` class provides type-specific getters that handle conversion:

```
Array data = var.read();
// Get as different types (with automatic conversion)
double doubleVal = data.getDouble(index);
float floatVal = data.getFloat(index);
int intVal = data.getInt(index);
long longVal = data.getLong(index);
short shortVal = data.getShort(index);
byte byteVal = data.getBytes(index);
```

Writing Data

When writing data to netCDF-3 files, create an Array and write it to a variable:

```
try (NetcdfFormatWriter writer = NetcdfFormatWriter.createNewNetcdf3(outputFile)
    .build()) {

    // Define structure
    Dimension timeDim = writer.addUnlimitedDimension("time");
    Dimension latDim = writer.addDimension("lat", 180);
    Dimension lonDim = writer.addDimension("lon", 360);

    Variable.Builder<?> tempVar = writer.addVariable("temperature",
        DataType.FLOAT, "time lat lon");
    tempVar.addAttribute(new Attribute("units", "Celsius"));

    writer.create();

    // Create and fill data array
    int[] shape = new int[]{1, 180, 360};
    Array data = Array.factory(DataType.FLOAT, shape);
    Index index = data.getIndex();

    for (int lat = 0; lat < 180; lat++) {
        for (int lon = 0; lon < 360; lon++) {
            float value = (float) (Math.sin(lat * 0.1) * Math.cos(lon * 0.1));
            data.setFloat(index.set(0, lat, lon), value);
        }
    }

    // Write data
    writer.write(tempVar.getFullName(), data);
} catch (IOException e) {
    e.printStackTrace();
}
```

Writing Subsets

To write data incrementally (e.g., one time step at a time):

```
try (NetcdfFormatWriter writer = NetcdfFormatWriter.createNewNetcdf3(outputFile)
    .build()) {

    // Define structure
    Dimension timeDim = writer.addUnlimitedDimension("time");
    Dimension latDim = writer.addDimension("lat", 180);
    Dimension lonDim = writer.addDimension("lon", 360);

    Variable.Builder<?> tempVar = writer.addVariable("temperature",
        DataType.FLOAT, "time lat lon");

    writer.create();

    // Write one time slice at a time
    int[] origin = new int[]{0, 0, 0};
    int[] shape = new int[]{1, 180, 360};

    for (int t = 0; t < 10; t++) {
```



```

        origin[0] = t; // Update time position

        // Create data for this time step
        Array slice = Array.factory(DataType.FLOAT, shape);
        Index index = slice.getIndex();

        for (int lat = 0; lat < 180; lat++) {
            for (int lon = 0; lon < 360; lon++) {
                float value = (float) (t * Math.sin(lat * 0.1) * Math.cos(lon * 0.1));
                slice.setFloat(index.set(0, lat, lon), value);
            }
        }

        // Write this slice
        writer.write(tempVar.getFullName(), origin, slice);
    }
} catch (IOException e) {
    e.printStackTrace();
}

```

Common Pitfalls

Zero-Based Indexing: Java uses zero-based indexing. The first element is at index 0, not 1 as in Fortran.

```

// Correct: first element is at index 0
Array data = var.read("0:9, :, :"); // First 10 time steps

```

Array Shape Mismatch: The data array shape must match the section being read or written.

```

// WRONG: shape mismatch
int[] shape = new int[]{10, 10}; // 10x10 array
Array data = Array.factory(DataType.FLOAT, shape);
writer.write(varName, data); // ERROR if variable is 180x360

```

Null Checks: Always check if variables or attributes exist before using them.

```

Variable var = ncfile.findVariable("temperature");
if (var == null) {
    System.err.println("Variable not found");
    return;
}

```

Memory Management: Large arrays consume significant memory. For large datasets, read and process data in chunks.

```

// SLOW: Read entire large variable
Array allData = largeVar.read(); // May cause OutOfMemoryError
// BETTER: Read in chunks
for (int t = 0; t < timeSteps; t++) {
    Array slice = largeVar.read(t + ", :, :");
    // Process slice...
}

```

Thread Safety: NetcdfFile objects are not thread-safe. Use one instance per thread or synchronize access.

```
// WRONG: Sharing NetcdfFile across threads
NetcdfFile ncfile = NetcdfFiles.open(filename);
// Multiple threads using ncfile... // NOT SAFE
// CORRECT: One NetcdfFile per thread
ExecutorService executor = Executors.newFixedThreadPool(4);
for (int i = 0; i < 4; i++) {
    executor.submit(() -> {
        try (NetcdfFile ncfile = NetcdfFiles.open(filename)) {
            // Thread-safe: each thread has its own NetcdfFile
        } catch (IOException e) {
            e.printStackTrace();
        }
    });
}
```

Performance Considerations

Lazy Loading: NetCDF-Java loads metadata at open time but data is lazy-loaded. Reading metadata is fast; reading data may be slow for large files.

Chunking Alignment: For netCDF-4/HDF5 files, align reads with chunk boundaries when possible. Reading data that spans many chunks is slower than reading within chunks. For details on chunking, see **Chapter 10: NetCDF-4/HDF5 Performance Fundamentals**.

Caching: NetCDF-Java automatically caches compressed remote files. Configure cache location:

```
ucar.nc2.util.cache.FileCache.setRootDirectory("/path/to/cache");
```

Section Specifications: Use section specifications to read only needed data rather than entire variables.

```
// Efficient: read only what you need
Array subset = var.read("0:9, 45:135, 90:270");
// Inefficient: read everything then subset in memory
Array all = var.read();
// ... manual subsetting ...
```

Advanced Features

NetcdfDataset for Enhanced Functionality

NetcdfDataset extends NetcdfFile with coordinate system support and scale/offset processing:

```
import ucar.nc2.dataset.NetcdfDataset;
import ucar.nc2.dataset.NetcdfDatasets;
import ucar.nc2.dataset.CoordinateSystem;
import ucar.nc2.dataset.CoordinateAxis;
try (NetcdfDataset ncd = NetcdfDatasets.openDataset(filename)) {
    // Access coordinate systems
    for (CoordinateSystem cs : ncd.getCoordinateSystems()) {
        System.out.println("Coordinate System: " + cs.getName());
        for (CoordinateAxis axis : cs.getCoordinateAxes()) {
            System.out.println("    Axis: " + axis.getFullName());
            System.out.println("        Type: " + axis.getAxisType());
        }
    }
}
```

```

    }

    // Variables with scale_factor/add_offset are automatically converted
    Variable var = ncd.findVariable("temperature");
    Array data = var.read(); // Already scaled and offset applied
}

```

NetCDF Markup Language (NcML)

NcML is an XML-based language for modifying netCDF metadata without rewriting files:

```

<?xml version="1.0" encoding="UTF-8"?>
<netcdf xmlns="http://www.unidata.ucar.edu/namespaces/netcdf/ncml-2.2"
        location="original_file.nc">

    <!-- Modify global attribute -->
    <attribute name="title" value="Modified Dataset" />

    <!-- Modify variable attribute -->
    <variable name="temperature">
        <attribute name="units" value="Celsius" />
        <attribute name="long_name" value="Air Temperature" />
    </variable>

    <!-- Add new variable -->
    <variable name="new_var" type="float" shape="time lat lon">
        <attribute name="units" value="m/s" />
    </variable>
</netcdf>

```

Open NcML files like regular netCDF files:

```
NetcdfFile ncfile = NetcdfFiles.open("modified.ncml");
```

Aggregation with NcML

NcML supports aggregating multiple files into a virtual dataset:

```

<netcdf xmlns="http://www.unidata.ucar.edu/namespaces/netcdf/ncml-2.2">
    <aggregation dimName="time" type="joinExisting">
        <scan location="/data/model/" suffix=".nc" />
    </aggregation>
</netcdf>

```

Aggregation types:

- **joinExisting**: Concatenate along existing dimension
- **joinNew**: Create new dimension for aggregation
- **union**: Combine variables from multiple files
- **tiled**: Aggregate multidimensional tiles

Working with Groups (NetCDF-4)

NetCDF-4 files support hierarchical groups:

```

import ucar.nc2.Group;
try (NetcdfFile ncfile = NetcdfFiles.open(netcdf4File)) {
    // Get root group
    Group rootGroup = ncfile.getRootGroup();

    // Find nested group
    Group dataGroup = rootGroup.findGroupLocal("data");
    if (dataGroup != null) {
        // Find variable in group
        Variable var = dataGroup.findVariableLocal("temperature");

        // Or use full path
        Variable var2 = ncfile.findVariable("data/temperature");
    }

    // List all groups recursively
    for (Group group : ncfile.getRootGroup().getGroups()) {
        System.out.println("Group: " + group.getFullName());
    }
}

```

Multiple File Formats

NetCDF-Java can read many formats beyond netCDF:

```

// NetCDF-3
NetcdfFile nc3 = NetcdfFiles.open("data.nc");
// NetCDF-4/HDF5
NetcdfFile nc4 = NetcdfFiles.open("data.nc4");
// HDF5
NetcdfFile hdf5 = NetcdfFiles.open("data.h5");
// GRIB
NetcdfFile grib = NetcdfFiles.open("data.grib2");
// OPeNDAP
NetcdfFile dap = NetcdfFiles.open("https://server.org/dods/dataset");

```

The library automatically detects the format. Some formats require additional Maven dependencies.

Maven and Gradle Integration

Maven Configuration

Add netCDF-Java to your pom.xml:

```

<dependencies>
    <!-- Core CDM library -->
    <dependency>
        <groupId>edu.ucar</groupId>
        <artifactId>cdm-core</artifactId>
        <version>5.5.3</version>
    </dependency>

    <!-- For all formats in one artifact -->
    <dependency>
        <groupId>edu.ucar</groupId>
        <artifactId>netcdfAll</artifactId>

```

```

        <version>5.5.3</version>
    </dependency>

    <!-- SLF4J logging implementation -->
    <dependency>
        <groupId>ch.qos.logback</groupId>
        <artifactId>logback-classic</artifactId>
        <version>1.2.11</version>
    </dependency>
</dependencies>
<!-- Unidata repository -->
<repositories>
    <repository>
        <id>unidata-releases</id>
        <url>https://artifacts.unidata.ucar.edu/repository/unidata-releases/</url>
    </repository>
</repositories>

```

Gradle Configuration

Add netCDF-Java to your `build.gradle`:

```

repositories {
    mavenCentral()
    maven {
        url "https://artifacts.unidata.ucar.edu/repository/unidata-releases/"
    }
}
dependencies {
    implementation 'edu.ucar:cdm-core:5.5.3'
    // or for all formats
    implementation 'edu.ucar:netcdfAll:5.5.3'

    // Logging
    implementation 'ch.qos.logback:logback-classic:1.2.11'
}

```

Modular Dependencies

For smaller deployments, use specific modules instead of `netcdfAll`:

```

<dependencies>
    <dependency>
        <groupId>edu.ucar</groupId>
        <artifactId>cdm-core</artifactId>
        <version>5.5.3</version>
    </dependency>
    <dependency>
        <groupId>edu.ucar</groupId>
        <artifactId>netcdf4</artifactId>
        <version>5.5.3</version>
    </dependency>
    <dependency>
        <groupId>edu.ucar</groupId>
        <artifactId>grib</artifactId>
        <version>5.5.3</version>
    </dependency>
</dependencies>

```

Comparison with C and Fortran APIs

Key Differences

Feature	C/Fortran	Java
Implementation	Shared C library	Pure Java
Error Handling	Return codes	Exceptions
Resource Management	Manual close	Try-with-resources
Array Indexing	C: 0-based, Fortran: 1-based	0-based
Type System	Explicit types	Automatic conversion
Memory Management	Manual	Garbage collected
Thread Safety	Thread-safe with care	Not thread-safe per object
Write Support	All formats	NetCDF-3 native, NetCDF-4 via JNI

API Correspondence

Operation	C API	Fortran F90 API	Java API
Open file	nc_open()	nf90_open()	NetcdfFiles.open()
Create file	nc_create()	nf90_create()	NetcdfFormatWriter.createNewNetcdf3()
Close file	nc_close()	nf90_close()	ncfile.close() or auto-close
Find variable	nc_inq_varid()	nf90_inq_varid()	ncfile.findVariable()
Read data	nc_get_var()	nf90_get_var()	variable.read()
Write data	nc_put_var()	nf90_put_var()	writer.write()
Get attribute	nc_get_att()	nf90_get_att()	variable.findAttribute()

Code Example Comparison

C API:

```
int ncid, varid;
nc_open("data.nc", NC_NOWRITE, &ncid);
nc_inq_varid(ncid, "temperature", &varid);
float data[100];
nc_get_var_float(ncid, varid, data);
nc_close(ncid);
```

Fortran F90 API:

```
integer :: ncid, varid
real :: data(100)
call check(nf90_open("data.nc", NF90_NOWRITE, ncid))
```

```
call check(nf90_inq_varid(ncid, "temperature", varid))
call check(nf90_get_var(ncid, varid, data))
call check(nf90_close(ncid))
```

Java API:

```
try (NetcdfFile ncfile = NetcdfFiles.open("data.nc")) {
    Variable var = ncfile.findVariable("temperature");
    Array data = var.read();
    float[] floatData = (float[]) data.get1DJavaArray(DataType.FLOAT);
} catch (IOException e) {
    e.printStackTrace();
}
```

ToolsUI Application

ToolsUI is a graphical application for browsing and debugging netCDF files. It's invaluable for development and troubleshooting.

Download and Run:

```
# Download from Unidata website
wget https://downloads.unidata.ucar.edu/netcdf-java/5.5.3/toolsUI-5.5.3.jar
# Run with adequate memory
java -Xmx1g -jar toolsUI-5.5.3.jar
```

Features:

- Browse file structure and metadata
- View data values in tables
- Plot data arrays
- Test coordinate systems
- Debug IOServiceProviders
- View NcML output
- Test OPeNDAP connections

Best Practices

1. **Always use try-with-resources** to ensure files are properly closed, even if exceptions occur.
2. **Check for null** when finding variables, attributes, or dimensions. The **find*** methods return null if not found.
3. **Read metadata first** - structural metadata is loaded at open time and is fast to access. Data is lazy-loaded.
4. **Use section specifications** to read subsets of large arrays rather than reading entire variables into memory.
5. **Use NetcdfDataset** when you need coordinate system support or automatic scale/offset processing.

6. **Configure logging** during development to understand what the library is doing.
7. **One NetcdfFile per thread** - don't share instances across threads without synchronization.
8. **Cache remote files** for better performance with repeated access to the same remote datasets.
9. **Use appropriate data types** - the **Array** class provides type-specific getters that handle conversion automatically.
10. **Follow CF Conventions** when creating files to ensure interoperability with other tools and users.

Most Important API Classes and Methods

This reference lists the most commonly used NetCDF-Java API classes and methods organized by category.

Core Classes

Class	Purpose
NetcdfFile	Read-only access to datasets
NetcdfFiles	Static methods for opening files
NetcdfDataset	Enhanced access with coordinate system support
NetcdfDatasets	Static methods for opening datasets with enhanced features
Variable	Represents a data variable
Dimension	Represents a dimension
Attribute	Represents an attribute
Group	Represents a group (NetCDF-4)
Array	Multi-dimensional data array
Index	Index for accessing array elements

Opening and Closing Files

Method	Purpose
NetcdfFiles.open(String location)	Open a NetCDF file
NetcdfDatasets.openDataset(String location)	Open with coordinate system support
NetcdfFile.close()	Close the file

Note: Use try-with-resources for automatic closing:

```
try (NetcdfFile ncfile = NetcdfFiles.open(pathToFile)) {
    // File automatically closed
}
```


File Information

Method	Purpose
NetcdfFile.getVariables()	Get list of all variables
NetcdfFile.getDimensions()	Get list of all dimensions
NetcdfFile.getGlobalAttributes()	Get list of global attributes
NetcdfFile.findVariable(String name)	Find variable by name
NetcdfFile.findDimension(String name)	Find dimension by name
NetcdfFile.findGlobalAttribute(String name)	Find global attribute by name

Variable Operations

Method	Purpose
Variable.read()	Read all data from variable
Variable.read(String section)	Read subset using section specification
Variable.readScalarDouble()	Read scalar double value
Variable.readScalarFloat()	Read scalar float value
Variable.readScalarInt()	Read scalar integer value
Variable.getFullName()	Get variable name
Variable.getDataType()	Get variable data type
Variable.getShape()	Get variable dimensions as array
Variable.getDimensions()	Get list of dimension objects
Variable.attributes()	Get iterator over variable attributes
Variable.findAttribute(String name)	Find attribute by name

Array Operations

Method	Purpose
Array.getShape()	Get array dimensions
Array.getSize()	Get total number of elements
Array.getDataType()	Get data type
Array.getDouble(int index)	Get double value at index
Array.getFloat(int index)	Get float value at index
Array.getInt(int index)	Get integer value at index
Array.getDouble(Index index)	Get double value using Index object
Array.getIndex()	Get Index object for multi-dimensional access

Index Operations

Method	Purpose
Index.set(int... indices)	Set index position for all dimensions
Index.set(int dim, int value)	Set index for specific dimension

Dimension Operations

Method	Purpose
Dimension.getFullName()	Get dimension name
Dimension.getLength()	Get dimension length
Dimension.isUnlimited()	Check if dimension is unlimited

Attribute Operations

Method	Purpose
Attribute.getFullName()	Get attribute name
Attribute.getValue()	Get attribute value
Attribute.getStringValue()	Get attribute as string
Attribute.getNumericValue()	Get attribute as number
Attribute.getDataType()	Get attribute data type
Attribute.getLength()	Get attribute length

Group Operations (NetCDF-4)

Method	Purpose
Group.findGroup(String name)	Find child group by name
Group.getGroups()	Get list of child groups
Group.getVariables()	Get variables in this group
Group.getDimensions()	Get dimensions in this group
Group.getAttributes()	Get attributes in this group

Coordinate System Support (NetcdfDataset)

Method	Purpose
NetcdfDataset.getCoordinateSystems()	Get list of coordinate systems
CoordinateSystem.getName()	Get coordinate system name
CoordinateSystem.getCoordinateAxes()	Get coordinate axes
CoordinateAxis.getFullName()	Get axis name
CoordinateAxis.getAxisType()	Get axis type (X, Y, Z, Time, etc.)

Data Types

NetCDF Type	Java DataType Enum
byte	DataType.BYTE
char	DataType.CHAR
short	DataType.SHORT
int	DataType.INT
long	DataType.LONG
float	DataType.FLOAT
double	DataType.DOUBLE
ubyte	DataType.UBYTE
ushort	DataType.USHORT
uint	DataType.UINT
ulong	DataType.ULONG
string	DataType.STRING

Array Section Syntax

NetCDF-Java uses Fortran 90 array section syntax with zero-based indexing:

Syntax	Meaning
:	All elements in dimension
start:end	Elements from start to end (inclusive)
start:end:stride	Elements with stride
index	Single element

Examples:

- "0:10:2, :, 5" - First dimension 0-10 with stride 2, all of second dimension, element 5 of third dimension
- " :, 10:20" - All of first dimension, elements 10-20 of second dimension

Remote File Access

URL Scheme	Purpose
http:// or https://	HTTP server access
dods:// or OPeNDAP URL	OPeNDAP protocol
cdms3://	AWS S3 access

Programming with NetCDF in Java – Key Takeaways

- **Pure Java:** Complete re-implementation, not a wrapper around C library.
- **Object-oriented:** Uses Java classes, exceptions, and try-with-resources.
- **Zero-based indexing:** Consistent with Java conventions.
- **Multiple formats:** Reads netCDF-3, netCDF-4, HDF5, GRIB, and more.
- **Write limitations:** Native Java writes netCDF-3; netCDF-4 requires JNI.
- **Maven/Gradle:** Add `edu.ucar:cdm-core` or `edu.ucar:netcdfAll` dependency.
- **NcML:** XML-based metadata modification and aggregation.
- **Not thread-safe:** Use one `NetcdfFile` instance per thread.

Chapter 9 Attributes and Conventions

Learning Objectives

- Apply CF conventions
- Design effective metadata schemas
- Validate attribute usage

Storing Earth Science Data

Using dimensions, variables, and attributes, netCDF can store array data in a generalized manner. This is helpful for interoperability. Data consumers will be able to open and understand the file, if attributes are used to fully describe the data.

The Climate and Forecast (CF) Conventions for Earth Science Data

The level of interoperability inherent in netCDF proved insufficient for the needs of climate scientists. In order to compare model output in a rigorous way, an exact specification of each variable must be shared by all users.

This resulted in the "standard names list", a list of named quantities with exact definitions. Data producers using the names from the standard names list could achieve a much higher level of interoperability, particularly for model comparisons.

Essential Attributes for Self-Describing Data

At minimum, every netCDF file should include certain attributes to make the data understandable without external documentation. These essential attributes form the foundation of self-describing data.

Variable Attributes

units: The most important attribute for any variable containing numeric data. Specifies the scientific units of the data using standard unit strings.

long_name: A descriptive name for the variable, more detailed than the variable name itself.

standard_name: For CF-compliant files, the CF standard name from the standard names table.

_FillValue: The value used to represent missing or undefined data. This must be the same type as the variable.

valid_min and valid_max: The minimum and maximum valid values for the variable. Values outside this range should be considered invalid.

scale_factor and add_offset: Used for packed data. The unpacked value is calculated as:

$\text{unpacked_value} = \text{packed_value} * \text{scale_factor} + \text{add_offset}$

Global Attributes

title: A succinct description of the dataset.

institution: Where the data were produced.

source: The method of production of the original data (e.g., "climate model", "satellite observation").

history: Provides an audit trail for modifications to the data. Each modification should append a timestamped line.

references: Published or web-based references that describe the data or methods used to produce it.

Conventions: The conventions followed by the dataset (e.g., "CF-1.8").

Adding Attributes in C

Here is a complete example showing how to create a CF-compliant netCDF file with proper attributes in C:

```
#include <netcdf.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define FILE_NAME "temperature_data.nc"
#define ERRCODE 2
#define ERR(e) {printf("Error: %s\n", nc_strerror(e)); exit(ERRCODE);}
int main() {
    int ncid, time_dimid, lat_dimid, lon_dimid;
    int temp_varid, lat_varid, lon_varid, time_varid;
    int dimids[3];
    int retval;
    /* Create file */
    if ((retval = nc_create(FILE_NAME, NC_NETCDF4, &ncid)))
        ERR(retval);
    /* Define dimensions */
    if ((retval = nc_def_dim(ncid, "time", NC_UNLIMITED, &time_dimid)))
        ERR(retval);
    if ((retval = nc_def_dim(ncid, "lat", 180, &lat_dimid)))
        ERR(retval);
    if ((retval = nc_def_dim(ncid, "lon", 360, &lon_dimid)))
        ERR(retval);
    /* Define coordinate variables */
    if ((retval = nc_def_var(ncid, "time", NC_DOUBLE, 1, &time_dimid, &time_varid)))
        ERR(retval);
    if ((retval = nc_def_var(ncid, "lat", NC_FLOAT, 1, &lat_dimid, &lat_varid)))
        ERR(retval);
    if ((retval = nc_def_var(ncid, "lon", NC_FLOAT, 1, &lon_dimid, &lon_varid)))
```

```

    ERR(retval);
/* Define data variable */
dimids[0] = time_dimid;
dimids[1] = lat_dimid;
dimids[2] = lon_dimid;
if ((retval = nc_def_var(ncid, "temperature", NC_FLOAT, 3, dimids, &temp_varid)))
    ERR(retval);
/* Add global attributes */
if ((retval = nc_put_att_text(ncid, NC_GLOBAL, "title", 28,
                             "Global Temperature Dataset")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, NC_GLOBAL, "institution", 27,
                             "Example Research Institute")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, NC_GLOBAL, "source", 13,
                             "Climate Model")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, NC_GLOBAL, "Conventions", 5, "CF-1.8")))
    ERR(retval);
/* Add history with timestamp */
time_t now = time(NULL);
char history[256];
strftime(history, sizeof(history), "%Y-%m-%d %H:%M:%S: Created file",
         localtime(&now));
if ((retval = nc_put_att_text(ncid, NC_GLOBAL, "history",
                             strlen(history), history)))
    ERR(retval);
/* Add attributes to temperature variable */
if ((retval = nc_put_att_text(ncid, temp_varid, "units", 6, "kelvin")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, temp_varid, "long_name", 19,
                             "Air Temperature")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, temp_varid, "standard_name", 16,
                             "air_temperature")))
    ERR(retval);
float fill_value = -999.0f;
if ((retval = nc_put_att_float(ncid, temp_varid, "_FillValue",
                              NC_FLOAT, 1, &fill_value)))
    ERR(retval);
float valid_range[2] = {180.0f, 330.0f};
if ((retval = nc_put_att_float(ncid, temp_varid, "valid_min",
                              NC_FLOAT, 1, &valid_range[0])))
    ERR(retval);
if ((retval = nc_put_att_float(ncid, temp_varid, "valid_max",
                              NC_FLOAT, 1, &valid_range[1])))
    ERR(retval);
/* Add attributes to coordinate variables */
if ((retval = nc_put_att_text(ncid, time_varid, "units", 19,
                             "days since 1900-01-01")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, time_varid, "calendar", 9, "gregorian")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, time_varid, "long_name", 4, "time")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, lat_varid, "units", 13, "degrees_north")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, lat_varid, "long_name", 8, "latitude")))
    ERR(retval);
if ((retval = nc_put_att_text(ncid, lat_varid, "standard_name", 8, "latitude")))

```

```

        ERR(retval);
    if ((retval = nc_put_att_text(ncid, lon_varid, "units", 12, "degrees_east")))
        ERR(retval);
    if ((retval = nc_put_att_text(ncid, lon_varid, "long_name", 9, "longitude")))
        ERR(retval);
    if ((retval = nc_put_att_text(ncid, lon_varid, "standard_name", 9, "longitude")))
        ERR(retval);
    /* End define mode */
    if ((retval = nc_enddef(ncid)))
        ERR(retval);
    /* Close file */
    if ((retval = nc_close(ncid)))
        ERR(retval);
    printf("*** SUCCESS creating CF-compliant file %s!\n", FILE_NAME);
    return 0;
}

```

Adding Attributes in Fortran

The same file can be created using the Fortran 90 API:

```

program create_cf_file
    use netcdf
    implicit none
    integer :: ncid, time_dimid, lat_dimid, lon_dimid
    integer :: temp_varid, lat_varid, lon_varid, time_varid
    integer :: dimids(3)
    real :: fill_value, valid_min, valid_max
    ! Create file
    call check(nf90_create("temperature_data.nc", NF90_NETCDF4, ncid))
    ! Define dimensions
    call check(nf90_def_dim(ncid, "time", NF90_UNLIMITED, time_dimid))
    call check(nf90_def_dim(ncid, "lat", 180, lat_dimid))
    call check(nf90_def_dim(ncid, "lon", 360, lon_dimid))
    ! Define coordinate variables
    call check(nf90_def_var(ncid, "time", NF90_DOUBLE, time_dimid, time_varid))
    call check(nf90_def_var(ncid, "lat", NF90_FLOAT, lat_dimid, lat_varid))
    call check(nf90_def_var(ncid, "lon", NF90_FLOAT, lon_dimid, lon_varid))
    ! Define data variable
    dimids = (/ lon_dimid, lat_dimid, time_dimid /)
    call check(nf90_def_var(ncid, "temperature", NF90_FLOAT, dimids, temp_varid))
    ! Add global attributes
    call check(nf90_put_att(ncid, NF90_GLOBAL, "title", &
        "Global Temperature Dataset"))
    call check(nf90_put_att(ncid, NF90_GLOBAL, "institution", &
        "Example Research Institute"))
    call check(nf90_put_att(ncid, NF90_GLOBAL, "source", "Climate Model"))
    call check(nf90_put_att(ncid, NF90_GLOBAL, "Conventions", "CF-1.8"))
    call check(nf90_put_att(ncid, NF90_GLOBAL, "history", &
        "2026-01-20: Created file"))
    ! Add attributes to temperature variable
    call check(nf90_put_att(ncid, temp_varid, "units", "kelvin"))
    call check(nf90_put_att(ncid, temp_varid, "long_name", "Air Temperature"))
    call check(nf90_put_att(ncid, temp_varid, "standard_name", "air_temperature"))
    fill_value = -999.0
    call check(nf90_put_att(ncid, temp_varid, "_FillValue", fill_value))
    valid_min = 180.0
    valid_max = 330.0

```



```

call check(nf90_put_att(ncid, temp_varid, "valid_min", valid_min))
call check(nf90_put_att(ncid, temp_varid, "valid_max", valid_max))
! Add attributes to coordinate variables
call check(nf90_put_att(ncid, time_varid, "units", "days since 1900-01-01"))
call check(nf90_put_att(ncid, time_varid, "calendar", "gregorian"))
call check(nf90_put_att(ncid, time_varid, "long_name", "time"))
call check(nf90_put_att(ncid, lat_varid, "units", "degrees_north"))
call check(nf90_put_att(ncid, lat_varid, "long_name", "latitude"))
call check(nf90_put_att(ncid, lat_varid, "standard_name", "latitude"))
call check(nf90_put_att(ncid, lon_varid, "units", "degrees_east"))
call check(nf90_put_att(ncid, lon_varid, "long_name", "longitude"))
call check(nf90_put_att(ncid, lon_varid, "standard_name", "longitude"))
! End define mode
call check(nf90_enddef(ncid))
! Close file
call check(nf90_close(ncid))
print *, "**** SUCCESS creating CF-compliant file!"
contains
subroutine check(status)
integer, intent(in) :: status
if (status /= nf90_noerr) then
print *, trim(nf90_strerror(status))
stop "Stopped"
end if
end subroutine check
end program create_cf_file

```

Coordinate Variables and Coordinate Systems

Coordinate variables are a fundamental concept in netCDF. A coordinate variable is a one-dimensional variable with the same name as its dimension. It provides the values along that dimension.

Defining Coordinate Variables

For a dimension named "lat", the coordinate variable "lat" contains the latitude values:

```

/* Define dimension */
nc_def_dim(ncid, "lat", 180, &lat_dimid);
/* Define coordinate variable with same name */
nc_def_var(ncid, "lat", NC_FLOAT, 1, &lat_dimid, &lat_varid);
/* Add coordinate attributes */
nc_put_att_text(ncid, lat_varid, "units", 13, "degrees_north");
nc_put_att_text(ncid, lat_varid, "long_name", 8, "latitude");

```

Auxiliary Coordinate Variables

Sometimes coordinates cannot be expressed as one-dimensional arrays. For example, a curvilinear grid might have latitude and longitude that vary in two dimensions. In this case, use the `coordinates` attribute to specify auxiliary coordinate variables:

```

/* 2D lat/lon arrays for curvilinear grid */
int dimids[2] = {y_dimid, x_dimid};
nc_def_var(ncid, "lat", NC_FLOAT, 2, dimids, &lat_varid);

```

```
nc_def_var(ncid, "lon", NC_FLOAT, 2, dimids, &lon_varid);
/* Data variable references auxiliary coordinates */
nc_def_var(ncid, "temperature", NC_FLOAT, 2, dimids, &temp_varid);
nc_put_att_text(ncid, temp_varid, "coordinates", 7, "lat lon");
```

Vertical Coordinates

Vertical coordinates require special attention. The CF Conventions define several standard vertical coordinate systems:

Pressure levels: Use units of "Pa" or "hPa"

```
nc_put_att_text(ncid, pressure_varid, "units", 3, "hPa");
nc_put_att_text(ncid, pressure_varid, "positive", 4, "down");
nc_put_att_text(ncid, pressure_varid, "standard_name", 19, "air_pressure");
```

Height above surface: Use units of "m"

```
nc_put_att_text(ncid, height_varid, "units", 1, "m");
nc_put_att_text(ncid, height_varid, "positive", 2, "up");
nc_put_att_text(ncid, height_varid, "standard_name", 6, "height");
```

Sigma coordinates: Dimensionless vertical coordinates

```
nc_put_att_text(ncid, sigma_varid, "standard_name", 38, "atmosphere_sigma_coordinate");
nc_put_att_text(ncid, sigma_varid, "formula_terms", 18, "sigma: sigma ps: ps ptop: ptop");
```

Time Encoding and Calendars

Time is one of the most important coordinates in scientific data, and proper encoding is essential for interoperability.

Time Units

Time should be encoded as numeric values with a **units** attribute that specifies the reference time:

```
nc_put_att_text(ncid, time_varid, "units", 29,
    "days since 1900-01-01 00:00:00");
```

Common time units:

"seconds since YYYY-MM-DD HH:MM:SS"

"hours since YYYY-MM-DD HH:MM:SS"

"days since YYYY-MM-DD"

Calendar Systems

The calendar attribute specifies which calendar system to use:

```
nc_put_att_text(ncid, time_varid, "calendar", 9, "gregorian");
```

CF-compliant calendar values:

"gregorian" or "standard": Mixed Gregorian/Julian calendar

"proleptic_gregorian": Gregorian calendar extended backwards

"noleap" or "365_day": All years have 365 days

"all_leap" or "366_day": All years have 366 days

"360_day": All months have 30 days

"julian": Julian calendar

Example: Complete Time Coordinate

```
/* Define time dimension and variable */
nc_def_dim(ncid, "time", NC_UNLIMITED, &time_dimid);
nc_def_var(ncid, "time", NC_DOUBLE, 1, &time_dimid, &time_varid);
/* Add time attributes */
nc_put_att_text(ncid, time_varid, "units", 29, "days since 1900-01-01 00:00:00");
nc_put_att_text(ncid, time_varid, "calendar", 9, "gregorian");
nc_put_att_text(ncid, time_varid, "long_name", 4, "time");
nc_put_att_text(ncid, time_varid, "standard_name", 4, "time");
nc_put_att_text(ncid, time_varid, "axis", 1, "T");
```

Missing Values and Fill Values

Handling missing or undefined data is critical for scientific datasets.

The `_FillValue` Attribute

The `_FillValue` attribute specifies the value used to pre-fill variables and to represent missing data:

```
float fill_value = -999.0f;
nc_def_var(ncid, "temperature", NC_FLOAT, 3, dimids, &temp_varid);
nc_put_att_float(ncid, temp_varid, "_FillValue", NC_FLOAT, 1, &fill_value);
```

Important: The `_FillValue` must be the same type as the variable. It should be set during define mode, before any data are written.

The missing_value Attribute

Some older datasets use `missing_value` instead of or in addition to `_FillValue`:

```
float missing = -999.0f;
nc_put_att_float(ncid, temp_varid, "missing_value", NC_FLOAT, 1, &missing);
```

Valid Range

Use `valid_min`, `valid_max`, or `valid_range` to specify the range of valid values:

```
/* Using separate min and max */
float valid_min = -50.0f;
float valid_max = 50.0f;
nc_put_att_float(ncid, temp_varid, "valid_min", NC_FLOAT, 1, &valid_min);
nc_put_att_float(ncid, temp_varid, "valid_max", NC_FLOAT, 1, &valid_max);
/* Or using valid_range array */
float valid_range[2] = {-50.0f, 50.0f};
nc_put_att_float(ncid, temp_varid, "valid_range", NC_FLOAT, 2, valid_range);
```

Values outside this range should be considered invalid, even if they are not equal to the fill value.

CF Conventions in Detail

The Climate and Forecast (CF) Conventions provide a comprehensive framework for describing climate and forecast data. Following these conventions ensures maximum interoperability.

Standard Names

The CF standard names table provides precise definitions for thousands of physical quantities. Using standard names eliminates ambiguity about what a variable represents.

Examples of CF standard names:

"**air_temperature**": Air temperature

"**sea_surface_temperature**": Temperature of the sea surface

"**eastward_wind**": Eastward component of wind velocity

"**precipitation_flux**": Precipitation rate

"**relative_humidity**": Relative humidity

The complete standard names table is available at: <http://cfconventions.org/standard-names.html>

```
nc_put_att_text(ncid, temp_varid, "standard_name", 15, "air_temperature");
nc_put_att_text(ncid, wind_u_varid, "standard_name", 13, "eastward_wind");
nc_put_att_text(ncid, precip_varid, "standard_name", 18, "precipitation_flux");
```

Units Conventions

CF requires units to be compatible with UDUNITS, a units library developed by Unidata. Common unit strings include:

Temperature: "K", "degree_C", "degree_F"

Pressure: "Pa", "hPa", "mbar"

Length: "m", "km", "cm"

Time: "s", "hours", "days"

Speed: "m s-1", "km h-1"

Precipitation: "kg m-2 s-1", "mm day-1"

For dimensionless quantities, use "1" as the units.

Cell Methods

The `cell_methods` attribute describes how data values represent the grid cell:

```
/* Temperature is a point value */
nc_put_att_text(ncid, temp_varid, "cell_methods", 10, "time: point");
/* Precipitation is a time mean */
nc_put_att_text(ncid, precip_varid, "cell_methods", 10, "time: mean");
/* Maximum temperature over time */
nc_put_att_text(ncid, tmax_varid, "cell_methods", 13, "time: maximum");
```

Common cell methods: "point", "mean", "sum", "maximum", "minimum", "standard_deviation", "variance"

Axis Attributes

The `axis` attribute identifies coordinate variables as spatial or temporal axes:

```
nc_put_att_text(ncid, lon_varid, "axis", 1, "X");
nc_put_att_text(ncid, lat_varid, "axis", 1, "Y");
nc_put_att_text(ncid, height_varid, "axis", 1, "Z");
nc_put_att_text(ncid, time_varid, "axis", 1, "T");
```

Other Common Conventions

ACDD (Attribute Convention for Data Discovery)

ACDD extends CF with attributes for data discovery and cataloging:

```
/* ACDD global attributes */
```

```
nc_put_att_text(ncid, NC_GLOBAL, "title", 28, "Global Temperature Analysis");
nc_put_att_text(ncid, NC_GLOBAL, "summary", 65,
    "Monthly mean temperature analysis on a global 1-degree grid");
nc_put_att_text(ncid, NC_GLOBAL, "keywords", 38,
    "temperature, climate, global, monthly");
nc_put_att_text(ncid, NC_GLOBAL, "creator_name", 8, "John Doe");
nc_put_att_text(ncid, NC_GLOBAL, "creator_email", 17, "jdoe@example.org");
nc_put_att_text(ncid, NC_GLOBAL, "date_created", 10, "2026-01-20");
nc_put_att_text(ncid, NC_GLOBAL, "geospatial_lat_min", 6, "-90.0");
nc_put_att_text(ncid, NC_GLOBAL, "geospatial_lat_max", 5, "90.0");
nc_put_att_text(ncid, NC_GLOBAL, "time_coverage_start", 10, "2020-01-01");
nc_put_att_text(ncid, NC_GLOBAL, "time_coverage_end", 10, "2025-12-31");
```

COARDS (Cooperative Ocean/Atmosphere Research Data Service)

COARDS is an older convention that preceded CF. CF is backward compatible with COARDS, so CF-compliant files are also COARDS-compliant.

Best Practices for Attributes and Conventions

- **Always specify units:** Every numeric variable should have a **units** attribute.
- **Use standard names when available:** Check the CF standard names table before creating custom names.
- **Document your data:** Use **long_name** and **comment** attributes to provide human-readable descriptions.
- **Follow CF Conventions:** Declare **Conventions = "CF-1.8"** and follow the CF guidelines.
- **Include provenance:** Use **history** and **source** attributes to document data origin and processing.
- **Specify coordinate systems clearly:** Use coordinate variables and the **coordinates** attribute appropriately.
- **Handle missing data consistently:** Use **_FillValue** and **valid_range** to clearly identify invalid data.
- **Use appropriate calendars:** Specify the **calendar** attribute for time coordinates.
- **Add discovery metadata:** Include ACDD attributes for datasets that will be cataloged or published.
- **Validate your files:** Use tools like **cf-checker** to verify CF compliance.

Checking CF Compliance

The CF Checker tool can validate whether your netCDF files comply with the CF Conventions:

```
# Install cf-checker (Python)
pip install cfchecker
```

```
# Check a file
cfchecks temperature_data.nc
```

The checker will report any violations of CF Conventions and suggest corrections.

Summary

Attributes and conventions transform netCDF from a generic array storage format into a self-describing data format. By following established conventions like CF and ACDD, you ensure that your data can be understood and used by others without requiring external documentation. The investment in proper metadata pays dividends in data longevity, interoperability, and scientific reproducibility.

Attributes & Conventions – Key Takeaways

- **Global vs. variable attributes**; keep them small and self-describing.
- **CF + ACDD** are de-facto interoperability standards.
- **Units** must follow UDUNITS syntax.
- **ncdump -h** reveals attribute metadata instantly.

Chapter 10 NetCDF-4/HDF5 Performance Fundamentals

Learning Objectives

- Configure chunking strategies
- Apply compression effectively
- Diagnose I/O bottlenecks

Getting the Best from NetCDF

When a file is created with the NC_NETCDF4 mode flag, the netCDF library uses the HDF5 library to access the file. These files are called netCDF-4/HDF5 files.

NetCDF-4/HDF5 files are fully-functional and correct HDF5 files. NetCDF variables are written as HDF5 datasets; dimensions are stored as HDF5 dimension scales.

Optimizing the performance of netCDF-4/HDF5 files requires understanding several key concepts: chunking, compression, and storage parameters. These features work together to provide efficient storage and access to scientific data. For advanced compression techniques beyond the basics covered in this chapter, see **Chapter 12: Advanced Compression Techniques**. For practical benchmarking methods, see **Chapter 15: Performance Testing and Benchmarking**.

Define Mode

Since HDF5 does not store all metadata at the beginning of the file, there is no performance penalty for re-entering define mode and adding new variables or other metadata. HDF5 appends objects, which may define metadata, to the end of the HDF5 file as it is written. When opening the file, HDF5 reads metadata as needed to locate objects and datasets; the fact that metadata is not centralized at the beginning of the file means there is no classic-format-style rewrite penalty when leaving define mode.

Since entering and leaving define mode causes no penalty, the library will automatically enter define mode when needed, and end define mode when data are written, for netCDF-4/HDF5 files (unless NC_CLASSIC_MODEL is used).

Chunk Sizes

The most important setting for netCDF-4/HDF5 file performance are the chunk sizes.

In HDF5 files, datasets may be chunked or contiguous. Chunked datasets are not stored contiguously in the file, as in the classic netCDF formats. Instead, the data are divided into fixed-size chunks that are scattered throughout the file. HDF5 maintains an index that allows it to locate each chunk.

Why Chunking Matters

Chunked data provides several critical performance advantages:

Efficient Partial I/O: When reading a subset of a large array, only the chunks that contain the requested data need to be read from disk. This can dramatically reduce I/O time when working with large datasets.

Compression Support: HDF5 can only compress chunked datasets. Each chunk is compressed independently, which allows for efficient random access to compressed data.

Flexible Growth: Chunking allows datasets to grow along any dimension. In the classic formats, only the unlimited dimension can grow, and it must be the slowest-varying dimension. With chunking, any dimension can be unlimited, and new chunks are simply added to the file as needed.

Setting Chunk Sizes

The chunk sizes for a variable are set with the `nc_def_var_chunking()` call. This function must be called after the variable is defined, but before `nc_enddef()` is called (or before the first data are written, if the file is not using explicit define mode).

```
size_t chunksizes[3] = {1, 100, 100};  
nc_def_var_chunking(ncid, varid, NC_CHUNKED, chunksizes);
```

Choosing Optimal Chunk Sizes

Choosing the right chunk size is critical for performance. If chunks are too small, the overhead of managing many chunks will slow down I/O. If chunks are too large, partial reads will be inefficient because entire large chunks must be read even when only a small portion is needed.

General Guidelines:

Size chunks to be between 100 KB and 1 MB for optimal performance

Match chunk size to expected access patterns

For time slice access: make the time dimension small (perhaps 1), and spatial dimensions larger

For vertical profiles: make the vertical dimension large and horizontal dimensions small

Consider the interaction between chunk size and compression effectiveness: larger chunks generally compress better but may reduce I/O efficiency for partial reads

The Chunk Cache

The chunk cache is another important performance consideration. HDF5 maintains a cache of recently-accessed chunks in memory. The size of this cache can be controlled with the `nc_set_chunk_cache()` function. A larger cache can improve performance for applications that repeatedly access the same chunks, but it also uses more memory.

Compression with Deflate

Compression is one of the most important features for managing large scientific datasets. The deflate filter uses the zlib library to perform lossless compression on the data stored in a variable. The data are automatically compressed as they are written, and automatically uncompressed when read.

The zlib compression algorithm, also known as deflate, uses the same compression method as gzip. It provides lossless compression with nine levels of compression, from 1 (fastest) to 9 (best compression). Level 1 provides modest compression with minimal CPU overhead, while level 9 provides maximum compression but requires significantly more processing time. For most applications, levels 4-6 provide a good balance between compression ratio and speed.

To enable zlib compression on a variable, use the `nc_def_var_deflate()` function:

```
int shuffle = 1;      /* Use shuffle filter */
int deflate = 1;      /* Turn on compression */
int deflate_level = 5; /* Compression level */
nc_def_var_deflate(ncid, varid, shuffle, deflate, deflate_level);
```

Compression requires chunked storage. The chunk size interacts with compression effectiveness. Larger chunks generally compress better but may reduce I/O efficiency for partial reads.

The szip compression algorithm was developed by NASA and offers different compression characteristics than zlib. However, szip has licensing restrictions that limit its use in some contexts. For this reason, zlib is more commonly used in netCDF applications.

Shuffle Filter

NetCDF-4 allows users to turn on the shuffle filter for a variable.

The shuffle filter rearranges the bytes in a chunk to make compression more effective. It works by separating the bytes of multi-byte values. For example, if a chunk contains four 4-byte integers, the shuffle filter will group all the first bytes together, then all the second bytes, then all the third bytes, and finally all the fourth bytes.

This shuffles the byte order of adjacent integers. For example, if you are storing four integers, 10, 11, 12, and 13, they will look like this in hex:

```
0000 000A 0000 000B 0000 000C 0000 000D
```

If the shuffle filter is used, they will be stored like this instead:

```
0000 0000 0000 0000 0000 0000 0A0B 0C0D
```

The point is that this produces long strings of zeros in many cases. In the unshuffled storage, the longest run of zeros is 7, but in the shuffled data, it is 25. This makes deflation (compression based on the zlib library) more effective.

This also works for slowly-varying floating point data. In IEEE floating point, 10.1, 11.1, 12.1, and 13.1 look like this:

```
4121999a 4131999a 4141999a 4151999a
```

The shuffled result looks like this:

```
41414141 21314151 99999999 9a9a9a9a
```

By putting the same numbers next to each other we create long runs with the same number, which compress much better.

The shuffle filter should be used with integer data and slowly-varying floating-point data, but not with random or already-compressed data. The best approach is to test with some sample data and see if the shuffle filter improves compression.

Fill Values

The use of fill values gets more complex with netCDF-4/HDF5 files.

In HDF5, the fill value and fill mode of a dataset must be specified when the dataset is created. This means that when the user sets a fill value for a variable in a netCDF-4/HDF5 file, that variable must be destroyed and recreated in order to set its fill value in the HDF5 file.

Another difference is that netCDF classic applied the fill mode to all variables in the file. In HDF5, fill mode may be set for each dataset.

In netCDF-4 code this is handled by giving each new dataset the file fill mode setting as its default. The function `nc_def_var_fill()` was added with netCDF-4 to allow control of the fill mode on a per-variable basis for netCDF-4/HDF5 files through the netCDF API.

Another aspect of fill values in HDF5 files concerns when the data are written. In netCDF classic, when a file is created, space for all variables, dimensions, and attributes are mapped out in the file. As a result, even if no data have been written, a classic file will take the same disk space.

In HDF5 this is not the case. Unwritten data are not allocated in the file. Creating a file with large variables, and then closing it without writing data, will result in a very small file size. Only when data are added, does HDF5 allocate disk storage for them.

Finally, in HDF5, turning off fill mode does not actually save any time. Instead of writing the netCDF fill value to the file, HDF5 writes its own fill value (zero). Time is saved in HDF5 because HDF5 does not write fill data except to fill in missing data in a chunk. If no part of a chunk is written by the user, then the chunk is not written to disk. Attempts to read data in that chunk will return fill value, but that is

generated in the library. The fill values are not written to disk in this case. Fill values are only written for a chunk when the user writes any values in that chunk. Once any value in the chunk is written to the file, the HDF5 library generates the entire chunk, using the fill value for any values that have not been specified by the user.

Because of this setting, fill mode for netCDF-4/HDF5 files is not recommended and is supported only for backwards compatibility. For performance reasons it is only useful to turn off fill mode for the classic netCDF formats.

Endianness

HDF5 allows data to be stored in either big or little endian form. It also offers a "native" endianness, which uses the endianness of the machine on which the data are written. Since most or all machines are little-endian, there seems to be little reason to ever use anything for endianness other than the default, which is native endianness, which is the endianness of the machine that is writing the data.

In case there is a desire to control endianness, the netCDF C API offers the `nc_def_var_endianness()` function. It must be called after a variable is defined, but before `nc_enddef()` is called.

NetCDF-4/HDF5 Performance Fundamentals – Key Takeaways

- **Chunking** drives both I/O speed and compression.
- **Deflate + shuffle** typical defaults (level 4–6).
- **Chunk cache** tunable via `nc_set_chunk_cache()`.
- **Entering define mode** is cheap in HDF5 (unlike classic).

Chapter 11 Parallel I/O with NetCDF

Learning Objectives

- Implement MPI parallel access
- Configure collective operations
- Optimize for HPC environments

NetCDF on Supercomputers

Parallel I/O allows multiple processes to read and write the same netCDF file simultaneously, enabling efficient data access in high-performance computing (HPC) environments. This capability is essential for applications running on supercomputers and large clusters where data are generated or processed by hundreds or thousands of processors.

NetCDF supports parallel I/O through two primary mechanisms: parallel HDF5 for netCDF-4/HDF5 files, and PnetCDF for CDF-5 and classic format files. Both approaches use MPI (Message Passing Interface) to coordinate access among multiple processes.

Why Parallel I/O Matters

In traditional serial I/O, only one process can write to a file at a time. For parallel applications, this creates a bottleneck: either all processes must send their data to a single process for writing (requiring expensive communication and memory), or each process must write to separate files (creating thousands of small files that are difficult to manage and analyze).

Parallel I/O solves this problem by allowing all processes to write directly to a single shared file. The parallel I/O library coordinates access to ensure data integrity while maximizing I/O bandwidth. This approach provides:

- **Better performance:** Multiple processes can write simultaneously, utilizing parallel file systems
- **Simpler code:** No need to gather data to a single process or manage multiple files
- **Easier analysis:** Output is a single file that can be opened with standard tools
- **Scalability:** Performance scales with the number of processes and I/O nodes

Building NetCDF with Parallel I/O Support

To use parallel I/O, the netCDF library must be built with parallel support enabled.

For NetCDF-4/HDF5 Parallel I/O

First, build HDF5 with parallel support:

```
# HDF5 must be built with parallel support
CC=mpicc ./configure --enable-parallel --prefix=/usr/local/hdf5-parallel
make
make install
```

Then build netCDF-C with parallel HDF5:

```
# Set environment variables to find parallel HDF5
export CPPFLAGS="-I/usr/local/hdf5-parallel/include"
export LDFLAGS="-L/usr/local/hdf5-parallel/lib"
export CC=mpicc
# Configure netCDF with parallel support
./configure --enable-parallel-tests --prefix=/usr/local/netcdf-parallel
make check
make install
```

For PnetCDF (CDF-5) Parallel I/O

Build PnetCDF separately, then build netCDF with PnetCDF support:

```
# Build PnetCDF
CC=mpicc ./configure --prefix=/usr/local/pnetcdf
make
make install
# Build netCDF with PnetCDF support
export CPPFLAGS="-I/usr/local/pnetcdf/include"
export LDFLAGS="-L/usr/local/pnetcdf/lib"
export CC=mpicc
./configure --enable-pnetcdf --prefix=/usr/local/netcdf-parallel
make
make install
```

Parallel I/O with NetCDF-4/HDF5

NetCDF-4/HDF5 files support parallel I/O through the parallel HDF5 library. This approach works with netCDF-4/HDF5 format files and supports all enhanced model features including compression, groups, and user-defined types.

Creating a Parallel NetCDF-4 File

Use `nc_create_par()` instead of `nc_create()`:

```
#include <netcdf.h>
#include <netcdf_par.h>
#include <mpi.h>
int main(int argc, char **argv) {
    int ncid, varid, dimids[3];
    int mpi_size, mpi_rank;
    /* Initialize MPI */
```

```

MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);
MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
/* Create parallel file */
nc_create_par("parallel_output.nc", NC_NETCDF4 | NC_MPIIO,
              MPI_COMM_WORLD, MPI_INFO_NULL, &ncid);
/* Define dimensions and variables (all processes participate) */
nc_def_dim(ncid, "time", NC_UNLIMITED, &dimids[0]);
nc_def_dim(ncid, "lat", 180, &dimids[1]);
nc_def_dim(ncid, "lon", 360, &dimids[2]);
nc_def_var(ncid, "temperature", NC_FLOAT, 3, dimids, &varid);
/* End define mode */
nc_enddef(ncid);
/* Write data (each process writes its portion) */
/* ... */
/* Close file */
nc_close(ncid);
MPI_Finalize();
return 0;
}

```

Opening an Existing Parallel File

Use `nc_open_par()` to open files for parallel access:

```

int ncid;
nc_open_par("parallel_output.nc", NC_NOWRITE | NC_MPIIO,
            MPI_COMM_WORLD, MPI_INFO_NULL, &ncid);

```

Collective vs. Independent I/O

Parallel I/O operations can be either collective or independent:

Collective I/O: All processes in the communicator must participate in the operation. This allows the library to optimize I/O by combining requests from multiple processes.

Independent I/O: Each process operates independently. This is more flexible but may be less efficient.

Set the access mode with `nc_var_par_access()`:

```

/* Set collective access for a variable */
nc_var_par_access(ncid, varid, NC_COLLECTIVE);
/* Set independent access for a variable */
nc_var_par_access(ncid, varid, NC_INDEPENDENT);

```

Writing Data in Parallel

Each process writes its portion of the data:

```

int mpi_rank, mpi_size;
MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);
/* Calculate this process's portion */
size_t time_per_process = total_time / mpi_size;
size_t start_time = mpi_rank * time_per_process;
size_t count_time = (mpi_rank == mpi_size - 1) ?

```

```

        total_time - start_time : time_per_process;
/* Allocate local data */
float *data = malloc(count_time * 180 * 360 * sizeof(float));
/* Fill data array */
/* ... */
/* Write this process's portion */
size_t start[3] = {start_time, 0, 0};
size_t count[3] = {count_time, 180, 360};
nc_var_par_access(ncid, varid, NC_COLLECTIVE);
nc_put_vara_float(ncid, varid, start, count, data);

```

Example: Complete Parallel Write

```

#include <netcdf.h>
#include <netcdf_par.h>
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#define FILE_NAME "parallel_example.nc"
#define NDIMS 3
#define TIME_DIM 100
#define LAT_DIM 180
#define LON_DIM 360
int main(int argc, char **argv) {
    int ncid, varid, dimids[NDIMS];
    int mpi_size, mpi_rank;
    size_t start[NDIMS], count[NDIMS];
    float *data;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);
    MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
    /* Create file */
    nc_create_par(FILE_NAME, NC_NETCDF4 | NC_MPIIO,
        MPI_COMM_WORLD, MPI_INFO_NULL, &ncid);
    /* Define dimensions */
    nc_def_dim(ncid, "time", TIME_DIM, &dimids[0]);
    nc_def_dim(ncid, "lat", LAT_DIM, &dimids[1]);
    nc_def_dim(ncid, "lon", LON_DIM, &dimids[2]);
    /* Define variable */
    nc_def_var(ncid, "temperature", NC_FLOAT, NDIMS, dimids, &varid);
    /* Add attributes (only rank 0 needs to do this) */
    if (mpi_rank == 0) {
        nc_put_att_text(ncid, varid, "units", 6, "kelvin");
    }
    nc_enddef(ncid);
    /* Calculate domain decomposition */
    size_t time_per_proc = TIME_DIM / mpi_size;
    size_t start_time = mpi_rank * time_per_proc;
    size_t count_time = (mpi_rank == mpi_size - 1) ?
        TIME_DIM - start_time : time_per_proc;
    /* Allocate and fill local data */
    data = malloc(count_time * LAT_DIM * LON_DIM * sizeof(float));
    for (size_t i = 0; i < count_time * LAT_DIM * LON_DIM; i++) {
        data[i] = 273.15 + mpi_rank; /* Simple test data */
    }
    /* Set up hyperslab for this process */
    start[0] = start_time;
    start[1] = 0;
    start[2] = 0;

```



```

count[0] = count_time;
count[1] = LAT_DIM;
count[2] = LON_DIM;
/* Use collective I/O */
nc_var_par_access(ncid, varid, NC_COLLECTIVE);
/* Write data */
nc_put_vara_float(ncid, varid, start, count, data);
/* Clean up */
free(data);
nc_close(ncid);
if (mpi_rank == 0) {
    printf("Successfully wrote parallel file: %s\n", FILE_NAME);
}
MPI_Finalize();
return 0;
}

```

Compile with:

```

mpicc -o parallel_write parallel_write.c -lnetcdf -lhdf5
mpirun -np 4 ./parallel_write

```

Parallel I/O with PnetCDF (CDF-5)

PnetCDF provides parallel I/O for CDF-5 and classic format files. It is particularly well-suited for applications that don't need the enhanced model features of netCDF-4/HDF5.

Using PnetCDF Through NetCDF

When netCDF is built with PnetCDF support, you can use the same `nc_create_par()` and `nc_open_par()` functions with CDF-5 files:

```

/* Create CDF-5 file with parallel access */
nc_create_par("pnetcdf_output.nc", NC_64BIT_DATA | NC_MPIIO,
             MPI_COMM_WORLD, MPI_INFO_NULL, &ncid);

```

The API is identical to netCDF-4/HDF5 parallel I/O, but the file format is CDF-5.

PnetCDF Native API

PnetCDF also provides its own API with additional features. The native PnetCDF API uses `ncmpi_*` function names:

```

#include <pnetcdf.h>
int ncid, varid, dimids[3];
int mpi_rank, mpi_size;
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);

```

```

MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);
/* Create file with PnetCDF API */
ncmpi_create(MPI_COMM_WORLD, "pnetcdf_native.nc",
             NC_64BIT_DATA, MPI_INFO_NULL, &ncid);
/* Define dimensions */
ncmpi_def_dim(ncid, "time", NC_UNLIMITED, &dimids[0]);
ncmpi_def_dim(ncid, "lat", 180, &dimids[1]);
ncmpi_def_dim(ncid, "lon", 360, &dimids[2]);
/* Define variable */
ncmpi_def_var(ncid, "temperature", NC_FLOAT, 3, dimids, &varid);
ncmpi_enddef(ncid);
/* Write data */
MPI_Offset start[3], count[3];
start[0] = mpi_rank * 25;
start[1] = 0;
start[2] = 0;
count[0] = 25;
count[1] = 180;
count[2] = 360;
ncmpi_put_vara_float_all(ncid, varid, start, count, data);
ncmpi_close(ncid);
MPI_Finalize();

```

Domain Decomposition Strategies

How you divide data among processes significantly impacts performance.

1D Decomposition (Time)

Each process handles a range of time steps:

```

size_t times_per_proc = total_times / mpi_size;
size_t my_start_time = mpi_rank * times_per_proc;
size_t my_count_time = (mpi_rank == mpi_size - 1) ?
                       total_times - my_start_time : times_per_proc;
start[0] = my_start_time;
start[1] = 0;
start[2] = 0;
count[0] = my_count_time;
count[1] = lat_dim;
count[2] = lon_dim;

```

Best for: Time series data, model output where each timestep is independent.

2D Decomposition (Spatial)

Divide the spatial domain among processes:

```

/* Decompose in latitude direction */
int lat_per_proc = lat_dim / mpi_size;
int my_start_lat = mpi_rank * lat_per_proc;

```

```

int my_count_lat = (mpi_rank == mpi_size - 1) ?
                    lat_dim - my_start_lat : lat_per_proc;
start[0] = 0;
start[1] = my_start_lat;
start[2] = 0;
count[0] = time_dim;
count[1] = my_count_lat;
count[2] = lon_dim;

```

Best for: Spatial analysis, when all time steps are processed together.

3D Decomposition

For very large process counts, decompose in multiple dimensions:

```

/* 2D process grid */
int proc_rows = sqrt(mpi_size);
int proc_cols = mpi_size / proc_rows;
int my_row = mpi_rank / proc_cols;
int my_col = mpi_rank % proc_cols;
/* Calculate local domain */
int lat_per_row = lat_dim / proc_rows;
int lon_per_col = lon_dim / proc_cols;
start[1] = my_row * lat_per_row;
start[2] = my_col * lon_per_col;
count[1] = lat_per_row;
count[2] = lon_per_col;

```

Best for: Very large process counts (1000+), 3D atmospheric or ocean models.

Performance Optimization for Parallel I/O

Align Chunking with Domain Decomposition

For netCDF-4/HDF5, set chunk sizes to match your decomposition:

```

/* If decomposing in time with 1 time slice per process */
size_t chunks[3] = {1, 180, 360};
nc_def_var_chunking(ncid, varid, NC_CHUNKED, chunks);

```

This ensures each process writes complete chunks, maximizing performance. For foundational principles of chunk sizing and cache optimization, refer to **Chapter 10: NetCDF-4/HDF5 Performance Fundamentals**.

Use Collective I/O

Collective I/O allows the library to optimize access patterns:

```
nc_var_par_access(ncid, varid, NC_COLLECTIVE);
```

All processes must call the I/O function, even if some have no data to write.

Tune MPI-IO Hints

MPI-IO hints can improve performance on parallel file systems:

```
MPI_Info info;
MPI_Info_create(&info);
/* Set stripe count for Lustre filesystem */
MPI_Info_set(info, "striping_factor", "8");
MPI_Info_set(info, "striping_unit", "1048576"); /* 1 MB */
/* Use collective buffering */
MPI_Info_set(info, "romio_cb_write", "enable");
MPI_Info_set(info, "romio_cb_read", "enable");
nc_create_par("output.nc", NC_NETCDF4 | NC_MPIIO,
              MPI_COMM_WORLD, info, &ncid);
MPI_Info_free(&info);
```

Avoid Small, Random Writes

Write large, contiguous blocks when possible:

```
/* BAD: Many small writes */
for (int t = 0; t < my_times; t++) {
    size_t start[3] = {my_start_time + t, 0, 0};
    size_t count[3] = {1, lat_dim, lon_dim};
    nc_put_vara_float(ncid, varid, start, count, &data[t][0][0]);
}
/* GOOD: One large write */
size_t start[3] = {my_start_time, 0, 0};
size_t count[3] = {my_times, lat_dim, lon_dim};
nc_put_vara_float(ncid, varid, start, count, data);
```

Common Pitfalls and Solutions

Pitfall 1: Metadata Operations in Parallel

All processes must participate in metadata operations:

```
/* WRONG: Only rank 0 defines variable */
if (mpi_rank == 0) {
    nc_def_var(ncid, "temp", NC_FLOAT, 3, dimids, &varid); /* DEADLOCK! */
}
/* CORRECT: All processes define variable */
```

```
nc_def_var(ncid, "temp", NC_FLOAT, 3, dimids, &varid);
```

Pitfall 2: Mixing Collective and Independent I/O

Once you set the access mode, all processes must use the same mode:

```
/* WRONG: Inconsistent access modes */
if (mpi_rank == 0) {
    nc_var_par_access(ncid, varid, NC_COLLECTIVE);
} else {
    nc_var_par_access(ncid, varid, NC_INDEPENDENT); /* ERROR! */
}
/* CORRECT: All processes use same mode */
nc_var_par_access(ncid, varid, NC_COLLECTIVE);
```

Pitfall 3: Unbalanced Decomposition

Ensure all processes have similar workloads:

```
/* WRONG: Last process gets all remaining work */
size_t my_count = (mpi_rank == mpi_size - 1) ?
    total_size - (mpi_rank * chunk_size) : chunk_size;
/* If total_size is not divisible by mpi_size, last rank may have 2x work */
/* BETTER: Distribute remainder evenly */
size_t base_count = total_size / mpi_size;
size_t remainder = total_size % mpi_size;
size_t my_count = base_count + (mpi_rank < remainder ? 1 : 0);
size_t my_start = mpi_rank * base_count + (mpi_rank < remainder ? mpi_rank : remainder);
```

Fortran Parallel I/O

The Fortran API provides parallel I/O through similar functions:

```
program parallel_example
    use netcdf
    use mpi
    implicit none
    integer :: ncid, varid, dimids(3)
    integer :: ierr, mpi_rank, mpi_size
    real, allocatable :: data(:,:,:)
    integer :: start(3), count(3)
    call MPI_Init(ierr)
    call MPI_Comm_rank(MPI_COMM_WORLD, mpi_rank, ierr)
    call MPI_Comm_size(MPI_COMM_WORLD, mpi_size, ierr)
    ! Create parallel file
    call check(nf90_create_par("parallel.nc", &
        ior(NF90_NETCDF4, NF90_MPIIO), &
        MPI_COMM_WORLD, MPI_INFO_NULL, ncid))
    ! Define dimensions and variables
    call check(nf90_def_dim(ncid, "time", 100, dimids(1)))
    call check(nf90_def_dim(ncid, "lat", 180, dimids(2)))
    call check(nf90_def_dim(ncid, "lon", 360, dimids(3)))
    call check(nf90_def_var(ncid, "temp", NF90_FLOAT, dimids, varid))
    call check(nf90_enddef(ncid))
    ! Calculate domain decomposition
```

```

! ... (similar to C example)
! Set collective access
call check(nf90_var_par_access(ncid, varid, NF90_COLLECTIVE))
! Write data
call check(nf90_put_var(ncid, varid, data, start=start, count=count))
call check(nf90_close(ncid))
call MPI_Finalize(ierr)
contains
  subroutine check(status)
    integer, intent(in) :: status
    if (status /= nf90_noerr) then
      print *, trim(nf90_strerror(status))
      stop "NetCDF error"
    end if
  end subroutine check
end program parallel_example

```

Benchmarking Parallel I/O Performance

Measure parallel I/O performance to optimize your application:

```

#include <mpi.h>
#include <netcdf.h>
#include <netcdf_par.h>
double start_time, end_time, write_time;
double max_write_time, min_write_time, avg_write_time;
/* Start timing */
MPI_Barrier(MPI_COMM_WORLD);
start_time = MPI_Wtime();
/* Perform I/O */
nc_put_vara_float(ncid, varid, start, count, data);
/* End timing */
MPI_Barrier(MPI_COMM_WORLD);
end_time = MPI_Wtime();
write_time = end_time - start_time;
/* Gather statistics */
MPI_Reduce(&write_time, &max_write_time, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
MPI_Reduce(&write_time, &min_write_time, 1, MPI_DOUBLE, MPI_MIN, 0, MPI_COMM_WORLD);
MPI_Reduce(&write_time, &avg_write_time, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
if (mpi_rank == 0) {
  avg_write_time /= mpi_size;
  double total_mb = (total_size * sizeof(float)) / (1024.0 * 1024.0);
  printf("Write performance:\n");
  printf("  Total data: %.2f MB\n", total_mb);
  printf("  Max time: %.3f s\n", max_write_time);
  printf("  Min time: %.3f s\n", min_write_time);
  printf("  Avg time: %.3f s\n", avg_write_time);
  printf("  Bandwidth: %.2f MB/s\n", total_mb / max_write_time);
}

```

Summary

- Parallel I/O is essential for HPC applications working with large datasets. Key points:
- Use `nc_create_par()` and `nc_open_par()` for parallel file access

- Choose between netCDF-4/HDF5 (with compression, groups) or PnetCDF (simpler, classic model)
- Use collective I/O for best performance
- Align chunking with domain decomposition
- All processes must participate in metadata operations
- Tune MPI-IO hints for your parallel file system
- Benchmark to find optimal configuration

For performance optimization techniques, see **Chapter 10: NetCDF-4/HDF5 Performance Fundamentals**. For benchmarking methods, see **Chapter 15: Performance Testing and Benchmarking**.

Parallel I/O – Key Takeaways

- **Create/open** with `nc_create_par` / `nc_open_par`.
- **Collective vs. independent** set via `nc_var_par_access`.
- **Domain decomposition**: match chunks to process layout.
- **PnetCDF** enables parallel classic/CDF-5; parallel HDF5 for netCDF-4.

Chapter 12 Advanced Compression Techniques

Learning Objectives

- Evaluate modern algorithms (Zstandard/Blosc)
- Apply lossy compression appropriately
- Measure compression/performance tradeoffs

Compression in the Modern Age

NetCDF has evolved its compression capabilities over time, adding new algorithms and techniques to improve both compression ratios and performance. This chapter covers advanced compression features beyond the basic deflate and shuffle filters discussed in **Chapter 10: NetCDF-4/HDF5 Performance Fundamentals**, including modern compression algorithms and lossy compression techniques.

NetCDF-4.9.0 - zstd and quantization

Starting with netCDF-4.9.0, support was added for the Zstandard (zstd) compression algorithm and for quantization.

Zstandard is a modern compression algorithm developed by Facebook (now Meta). It offers compression ratios similar to zlib but with significantly faster compression and decompression speeds. Zstandard supports compression levels from 1 to 22, with higher levels providing better compression at the cost of slower compression speed.

To use Zstandard compression, the HDF5 library must be built with the Zstandard plugin, and the plugin must be available at runtime. Once configured, Zstandard can provide substantial performance improvements over zlib, especially for decompression.

Quantization is a lossy compression technique that reduces the precision of floating-point data. By rounding values to a specified number of significant digits, quantization can dramatically improve compression ratios. This is particularly effective for data where the full precision of floating-point representation is not scientifically meaningful.

Quantization is controlled through the `nc_def_var_quantize()` function:

```
int quantize_mode = NC_QUANTIZE_BITGROOM;
int nsd = 3; /* Number of significant digits */
nc_def_var_quantize(ncid, varid, quantize_mode, nsd);
```

The BitGroom and Granular BitRound quantization algorithms are available. These algorithms preserve a specified number of significant digits while setting the remaining bits to values that compress well. Because quantization is lossy, it should only be used when the loss of precision is acceptable for the scientific application.

The NetCDF Expansion Pack (NEP) - lz4 and More

The NetCDF Expansion Pack (NEP) is a collection of additional compression filters that can be used with netCDF-4/HDF5 files. These filters are implemented as HDF5 plugins and include modern, high-performance compression algorithms.

The LZ4 compression algorithm is designed for extremely fast compression and decompression. While it typically achieves lower compression ratios than zlib or Zstandard, its speed makes it ideal for applications where I/O performance is critical. LZ4 can decompress data at speeds approaching memory bandwidth, making it nearly transparent to applications.

Other algorithms available through NEP include:

Bzip2: Provides higher compression ratios than zlib but with slower compression speeds.

LZ4: Faster compression

Choosing a Compression Algorithm

The choice of compression algorithm depends on the characteristics of the data and the requirements of the application. For basic compression using zlib (deflate) and the shuffle filter, see **Chapter 10: NetCDF-4/HDF5 Performance Fundamentals**. This section focuses on when to use advanced compression algorithms:

- **For maximum compatibility:** Use zlib (deflate) as covered in **Chapter 10: NetCDF-4/HDF5 Performance Fundamentals**. It is universally supported and provides good compression for most data types.
- **For best compression ratio:** Use Zstandard at high levels (15-22). Be prepared for slower compression times but excellent decompression performance.
- **For best performance:** Use LZ4 or Zstandard at low levels (1-3). These provide fast compression and very fast decompression, ideal when I/O speed is critical.
- **For floating-point data:** Consider quantization (BitGroom or Granular BitRound) in combination with any compression algorithm to achieve much higher compression ratios when loss of precision is acceptable for your scientific application.
- **For modern HPC workflows:** Zstandard offers the best balance of compression ratio and speed for most applications, replacing zlib in performance-critical scenarios.
- Testing with representative data is the best way to determine optimal settings for a particular application.

Advanced Compression – Key Takeaways

- **Zstandard (zstd)** ≥ netCDF-4.9: high ratio, fast decode.
- **Quantization** (BitGroom) reduces float precision for size.

- **Filter chain:** shuffle → compress for best effect.
- **Test combos** with `nccopy -d <level> -s`

Chapter 13 NetCDF Architecture and Extensibility

Learning Objectives

- Understand dispatch table design
- Extend with custom formats
- Develop format plugins

Extending NetCDF

The netCDF C library can be extended to support additional formats beyond the built-in classic, 64-bit offset, CDF-5, netCDF-4/HDF5, HDF4 SD, and Zarr formats through its dispatch table architecture. For information on the different binary formats available, see **Chapter 4: Binary Formats**.

The Dispatch Table

The dispatch table architecture is what allows netCDF to support multiple storage formats through a single, unified API. This means you can write code once and use it with any netCDF format—classic, HDF5, Zarr, or others—without changing your application code. The library automatically routes your API calls to the appropriate format-specific implementation.

This architectural pattern is the central design principle of the netCDF C library. It provides format independence by using function pointers to route operations to the correct storage backend. Whether you're working with a classic binary file, an HDF5 file with compression, or a Zarr dataset in cloud storage, you use the same `nc_create()`, `nc_def_var()`, `nc_put_vara()`, and `nc_get_vara()` functions.

How It Works

Every open netCDF file is represented by an NC structure, which contains a pointer to its dispatch table. When you call a netCDF function, the library looks up the appropriate function pointer in the dispatch table and calls the format-specific implementation with the parameters you provided.

The `NC_Dispatch` structure contains approximately 70 function pointers, covering all aspects of netCDF operations:

```
struct NC_Dispatch
{
    int model;                /* Format identifier */
    int (create)(...);        /* Create a new file */
    int (open)(...);          /* Open existing file */
    int (close)(...);         /* Close file */
    int (def_dim)(...);       /* Define dimension */
    int (def_var)(...);       /* Define variable */
    int (put_vara)(...);      /* Write variable data */
    int (get_vara)(...);      /* Read variable data */
    /* ... approximately 60 more function pointers */
}
```

```
};
```

Each format implementation provides its own dispatch table. The classic format uses `NC3_dispatcher`, the HDF5 format uses `HDF5_dispatcher`, and the Zarr format uses `NCZ_dispatcher`. These dispatch tables are defined in the netCDF source code in files like `nc3dispatch.c`, `hdf5dispatch.c`, and `zdispatch.c`.

Format Detection

When you open or create a file, the library uses format detection logic to determine which dispatch table to use. This detection is automatic and transparent:

For existing files being opened, the library examines the file's magic number (the first few bytes) to identify the format. Classic files begin with "CDF", HDF5 files begin with a specific HDF5 signature, and so on.

For new files being created, the mode flags passed to `nc_create()` determine the format. `NC_NETCDF4` selects HDF5, `NC_CDF5` selects CDF-5, `NC_ZARR` selects Zarr, and the default is classic format.

For remote access, the URL scheme determines the protocol. URLs beginning with "http://" or "s3://" trigger the appropriate remote access dispatch layer.

Once the format is detected, the library selects the appropriate dispatch table and uses it for all subsequent operations on that file. This happens automatically. Users should never need to think about which dispatch table is being used.

Extensibility

The dispatch table architecture makes it possible to add new formats to netCDF without modifying the public API. As long as a format can implement the required operations defined in the `NC_Dispatch` structure, it can be integrated into netCDF through a custom dispatch table.

This flexibility has allowed netCDF to evolve from supporting a single binary format in the 1980s to supporting seven or more different storage formats today, all accessible through the same API. User-defined formats can even be added by implementing a custom dispatch table and registering it with the library using the `NC_UDF0` or `NC_UDF1` flags.

For most users, the dispatch table is an implementation detail that can be safely ignored. The important takeaway is that netCDF provides format independence: your code works the same way regardless of which storage format you choose.

User-Defined Formats

The netCDF C library allows users to define their own formats by implementing a custom dispatch table. This advanced feature is useful when working with proprietary data formats or when integrating netCDF with specialized storage systems.

To create a user-defined format, the programmer must implement the `NC_Dispatch` structure, which contains function pointers for all netCDF operations. This includes file operations (create, open, close), metadata operations (define dimensions, variables, attributes), and data I/O operations (read and write variable data).

The `NC_UDF0` and `NC_UDF1` flags are reserved for user-defined formats. When creating a file with one of these flags, the netCDF library will use the corresponding user-defined dispatch table instead of one of the built-in format implementations.

User-defined formats must be registered with the netCDF library before they can be used. This is done by calling `NC_Dispatch_register()` with a pointer to the custom dispatch table. Once registered, the format can be used just like any built-in format.

The V2 API

The netCDF version 2 API is an older interface that was used before netCDF-3 was released. It is still supported for backwards compatibility, but should not be used in new code.

The V2 API uses different function names and calling conventions. For example, `nccreate()` instead of `nc_create()`, and `ncvarput()` instead of `nc_put_vara()`. The V2 API is less flexible and does not support many modern netCDF features.

New code should always use the netCDF-3 or later API (the `nc_*` functions in C, or `nf90_*` functions in Fortran 90). The V2 API is maintained only to support legacy applications.

Architecture & Extensibility – Key Takeaways

- **Dispatch table** routes API calls to format drivers.
- **Add new format** via `NC_Dispatch_register()`.
- **V2 API** kept only for legacy; avoid in new code.
- **Opaque & VLEN types** mainly for HDF5 compatibility.

Chapter 14 Remote Data Access with OPeNDAP

Learning Objectives

- Access remote datasets
- Construct efficient data requests
- Troubleshoot connection issues

Getting Remote Data

OPeNDAP (Open-source Project for a Network Data Access Protocol) provides a way to access scientific data over the Internet using standard HTTP protocols. Instead of downloading entire files, OPeNDAP allows programs to request only the specific data they need, making it efficient for working with large remote datasets.

The netCDF C library includes built-in support for OPeNDAP, allowing programs to open and read remote datasets using OPeNDAP URLs just as they would open local files. This network-transparent data access is one of netCDF's most powerful features for distributed scientific computing.

What is OPeNDAP?

OPeNDAP is both a protocol and a software framework for accessing scientific data over the web. It was originally developed as DODS (Distributed Oceanographic Data System) in the 1990s and has evolved into a widely-used standard for remote data access in the Earth sciences and other fields.

The key innovation of OPeNDAP is that it allows data subsetting on the server side. When you request data from an OPeNDAP server, you can specify exactly which variables you want and which portions of those variables to retrieve. The server reads the data from its local storage, extracts only the requested subset, and sends it to your client. This dramatically reduces network bandwidth compared to downloading entire files.

Client/Server Architecture

OPeNDAP uses a web-based client/server model similar to the World Wide Web:

- **Server (Hyrax):** The OPeNDAP server software translates data from local storage formats (netCDF, HDF5, etc.) into the DAP (Data Access Protocol) format for transmission over HTTP.
- **Client:** The client library (built into netCDF-C) sends HTTP requests to the server and translates the DAP responses back into the format expected by your program.
- **Protocol:** The Data Access Protocol (DAP) defines how metadata and data are structured and transmitted.

The data flow looks like this:

User Program → NetCDF Library → HTTP Request → OPeNDAP Server

User Program ← Translated Data ← DAP Response ← Local Files

From the programmer's perspective, opening a remote OPeNDAP dataset looks almost identical to opening a local file. The netCDF library handles all the HTTP communication transparently. This architecture builds on the dispatch table concepts described in **Chapter 13: NetCDF Architecture and Extensibility**.

OPeNDAP URLs

An OPeNDAP dataset is identified by a URL. The simplest form is just the HTTP address of the dataset:
`http://test.opendap.org/dap/data/nc/sst.mnmean.nc.gz`

This URL points to a netCDF file on the OPeNDAP test server. Even though the file is compressed (.gz), the OPeNDAP server can read it and serve the data.

URL Service Endpoints

OPeNDAP servers provide multiple "views" of the same dataset by appending different suffixes to the URL. These service endpoints return different types of information:

Metadata Endpoints:

- .dds** - Dataset Descriptor Structure (shows the shape and structure of variables)
- .das** - Data Attribute Structure (shows metadata and attributes)
- .dmr.xml** - Dataset Metadata Response (DAP4 - combines structure and attributes in XML)
- .info** - Combined DDS and DAS in human-readable HTML format

Data Endpoints:

- .dods** - Binary data in DAP2 format
- .dap** - Binary data in DAP4 format
- .ascii** - ASCII representation of data (useful for testing)
- .html** - Web form interface for building queries

For example, to see the structure of a dataset, you can visit:

`http://test.opendap.org/dap/data/nc/sst.mnmean.nc.gz.dds`

This returns a text description of all variables and their dimensions, which helps you understand the dataset before writing code to access it.

Constraint Expressions

The real power of OPeNDAP comes from constraint expressions, which allow you to subset data on the server. A constraint expression is added to the URL after a question mark:

```
http://server.org/data.nc?variable[start:stop]
```

For example, to retrieve only the first 10 time steps of sea surface temperature:

```
http://test.opendap.org/dap/data/nc/sst.mnmean.nc.gz?sst[0:9][0:88][0:179]
```

The numbers in brackets specify array indices. OPeNDAP uses 0-based indexing, just like C. The syntax `[start:stop]` retrieves elements from index `start` through index `stop`, inclusive.

You can also use stride to sample every `nth` element:

```
?sst[0:2:100][0:89][0:179]
```

This retrieves every other time step (stride of 2) from index 0 to 100.

Using OPeNDAP with NetCDF Programs

The netCDF C library includes built-in OPeNDAP support (if compiled with the `--enable-dap` option, which is the default in most distributions). To use OPeNDAP, simply pass a URL instead of a file path to `nc_open()`:

```
#include <netcdf.h>
int main() {
    int ncid, varid;
    char *url = "http://test.opendap.org/dap/data/nc/sst.mnmean.nc.gz";
    /* Open remote dataset */
    if (nc_open(url, NC_NOWRITE, &ncid) != NC_NOERR) {
        printf("Error opening URL\n");
        return 1;
    }
    /* Get variable ID */
    nc_inq_varid(ncid, "sst", &varid);
    /* Read a subset of data */
    size_t start[3] = {0, 0, 0};
    size_t count[3] = {1, 10, 10};
    float data[10][10];
    nc_get_vara_float(ncid, varid, start, count, &data[0][0]);
    /* Close the dataset */
}
```



```

    nc_close(ncid);
    return 0;
}

```

This program opens a remote dataset, reads a 10x10 spatial subset from the first time step, and closes the connection. The netCDF library automatically handles all the HTTP communication with the OPeNDAP server.

Constraint Expressions in URLs

You can include constraint expressions directly in the URL passed to `nc_open()`:

```

char *url = "http://test.opendap.org/dap/data/nc/sst.mnmean.nc.gz?sst[0:9][20:30][40:50]";
nc_open(url, NC_NOWRITE, &ncid);

```

When you open a URL with a constraint expression, the netCDF library sees only the subset specified in the constraint. The dimensions of variables will reflect the subset size, not the original dataset size. This can be useful for working with very large datasets where you only need a small portion.

However, for most applications, it's more flexible to open the full dataset URL and use the `start` and `count` parameters in `nc_get_vara_*`() functions to specify which data to retrieve. This allows you to make multiple requests for different subsets without reopening the dataset.

Using OPeNDAP with Fortran Programs

The Fortran netCDF APIs also support OPeNDAP URLs:

```

program read_opendap
  use netcdf
  implicit none
  integer :: ncid, varid, status
  character(len=256) :: url
  real :: data(10, 10)
  integer :: start(3), count(3)
  url = "http://test.opendap.org/dap/data/nc/sst.mnmean.nc.gz"
  ! Open remote dataset
  status = nf90_open(url, NF90_NOWRITE, ncid)
  if (status /= NF90_NOERR) then
    print *, "Error opening URL"
    stop
  end if
  ! Get variable ID
  status = nf90_inq_varid(ncid, "sst", varid)
  ! Read subset
  start = (/ 1, 1, 1 /)
  count = (/ 10, 10, 1 /)
  status = nf90_get_var(ncid, varid, data, start=start, count=count)
  ! Close dataset
  status = nf90_close(ncid)
end program read_opendap

```

Remember that Fortran uses 1-based indexing in the API, but OPeNDAP constraint expressions in URLs use 0-based indexing.

Exploring Remote Datasets

Before writing code to access an OPeNDAP dataset, it's helpful to explore its structure using a web browser or command-line tools.

Using a Web Browser

Visit the dataset URL with the `.html` suffix to get an interactive web form:

```
http://test.opendap.org/dap/data/nc/sst.mnmean.nc.gz.html
```

This form allows you to:

- See all variables and their dimensions
- Build constraint expressions using checkboxes and text fields
- Preview data in ASCII format
- Download subsets

Using `ncdump`

If your netCDF installation includes OPeNDAP support, you can use `ncdump` to examine remote datasets:

```
# View metadata
ncdump -h "http://test.opendap.org/dap/data/nc/sst.mnmean.nc.gz"
# View a subset of data
ncdump -v sst "http://test.opendap.org/dap/data/nc/sst.mnmean.nc.gz?sst[0:2][0:10][0:10]"
```

The `-h` flag shows only the header (metadata), while `-v` shows data for specific variables.

Using `.info` Endpoint

The `.info` endpoint provides a convenient combined view of structure and attributes:

```
http://test.opendap.org/dap/data/nc/sst.mnmean.nc.gz.info
```

This returns an HTML page with all the information you need to understand the dataset.

Performance Considerations

OPeNDAP access involves network communication, which is much slower than reading local files. Here are some strategies for efficient OPeNDAP usage:

Request Only What You Need

Always use constraint expressions or the start/count parameters to request only the data you actually need. Requesting a 1 GB variable when you only need 1 MB wastes bandwidth and time.

```
/* INEFFICIENT: Downloads entire 3D variable */
nc_get_var_float(ncid, varid, all_data);
/* EFFICIENT: Downloads only needed subset */
size_t start[3] = {0, 20, 40};
size_t count[3] = {10, 10, 10};
nc_get_vara_float(ncid, varid, start, count, subset_data);
```

Minimize the Number of Requests

Each HTTP request has overhead. It's more efficient to make one large request than many small requests:

```
/* INEFFICIENT: Many small requests */
for (int t = 0; t < 100; t++) {
    size_t start[3] = {t, 0, 0};
    size_t count[3] = {1, 89, 180};
    nc_get_vara_float(ncid, varid, start, count, slice);
    process_slice(slice);
}
/* EFFICIENT: One large request */
size_t start[3] = {0, 0, 0};
size_t count[3] = {100, 89, 180};
nc_get_vara_float(ncid, varid, start, count, all_slices);
for (int t = 0; t < 100; t++) {
    process_slice(&all_slices[t][0][0]);
}
```

However, balance this against memory constraints. If the full dataset is too large to fit in memory, you may need to make multiple requests.

Request Multiple Variables Together

When you need data from multiple variables, request them in the same `nc_open()` session rather than opening the dataset multiple times:

```
/* Open once */
nc_open(url, NC_NOWRITE, &ncid);
/* Read multiple variables */
nc_inq_varid(ncid, "sst", &sst_varid);
nc_inq_varid(ncid, "anom", &anom_varid);
nc_get_vara_float(ncid, sst_varid, start, count, sst_data);
nc_get_vara_float(ncid, anom_varid, start, count, anom_data);
nc_close(ncid);
```

Use Compression-Aware Servers

Many OPeNDAP servers can read compressed files (like .gz or .bz2) directly. The server decompresses the data and serves it to clients. This is transparent to your program but can improve server-side performance.

DAP2 vs DAP4

OPeNDAP has evolved through several protocol versions. The two main versions in use today are DAP2 and DAP4.

DAP2 (Data Access Protocol 2)

- DAP2 is the older, more widely supported protocol. It uses:
- Separate **.dds** and **.das** responses for structure and attributes
- Binary data in **.dods** format
- A simpler data model that maps well to netCDF-3
- Most OPeNDAP servers support DAP2, and it works reliably with the netCDF-C library.

DAP4 (Data Access Protocol 4)

- DAP4 is the newer protocol with enhanced features:
- Unified **.dmr.xml** response combining structure and attributes in XML
- Enhanced data model with support for groups (like netCDF-4)
- Better support for complex types
- Improved performance through better encoding

DAP4 support in netCDF-C is available in recent versions (4.5.0 and later). To use DAP4, the server must support it, and you may need to specify it in the URL or configuration.

For most applications, DAP2 is sufficient and more widely compatible. Use DAP4 when you need its specific features or when working with servers that provide better DAP4 performance.

Common Issues and Troubleshooting

URL Not Recognized

If you get an error like "NetCDF: Unknown file format" when trying to open an OPeNDAP URL, your netCDF library may not be compiled with DAP support. Check by running:

```
nc-config --has-dap
```

If this returns "no", you need to rebuild netCDF with `--enable-dap` or install a version that includes DAP support.

Network Timeouts

OPeNDAP requests can timeout on slow networks or when requesting large amounts of data. The netCDF library has default timeout settings, but you can adjust them if needed. For very large requests, consider breaking them into smaller chunks.

Authentication

Some OPeNDAP servers require authentication. The netCDF library supports HTTP Basic authentication through `.netrc` files or by embedding credentials in the URL (though this is not recommended for security reasons). Check the server's documentation for authentication requirements.

Constraint Expression Errors

If your constraint expression causes an error, check:

- Variable names match exactly (case-sensitive)
- Array indices are within bounds
- Syntax is correct (no spaces in the constraint)
- The server supports the constraint features you're using

You can test constraint expressions by appending them to the `.ascii` endpoint in a web browser to see the results.

OPeNDAP in Scientific Workflows

OPeNDAP is widely used in operational scientific data systems. Some common use cases include:

Climate Data Analysis

Climate model output and reanalysis datasets are often served via OPeNDAP. Researchers can access specific regions and time periods without downloading terabytes of data:

```
/* Access only North Atlantic region from global dataset */
char *url = "http://server.org/climate/global_sst.nc";
nc_open(url, NC_NOWRITE, &ncid);
/* Latitude 30N-60N, Longitude 280E-350E (80W-10W) */
size_t start[3] = {0, 150, 560}; /* time, lat, lon indices */
size_t count[3] = {120, 60, 140}; /* 10 years, region subset */
nc_get_vara_float(ncid, varid, start, count, north_atlantic_data);
```

Satellite Data Distribution

NASA and other space agencies use OPeNDAP to distribute satellite data. Users can request specific orbits, granules, or geographic regions.

Real-Time Data Access

Weather services and oceanographic institutes use OPeNDAP to provide access to real-time observations and model forecasts. Applications can query the latest data without managing file downloads.

Federated Data Systems

OPeNDAP enables federated data systems where data remain at their source institutions but are accessible through a common interface. This is more efficient than centralizing all data in one location.

Best Practices

When using OPeNDAP in your applications:

- **Test with small requests first:** Before requesting large amounts of data, test your constraint expressions with small subsets using the [.asci](#) endpoint.
- **Handle network errors gracefully:** Network requests can fail. Always check return codes and implement retry logic for transient failures.
- **Cache metadata:** If you're making multiple requests to the same dataset, cache the metadata (dimensions, variable IDs) rather than querying it repeatedly.
- **Document your data sources:** Include the OPeNDAP URLs in your code comments or configuration files so others can access the same data.
- **Consider local caching:** For frequently accessed data, consider downloading it once and caching it locally rather than repeatedly fetching it over the network.

- **Use appropriate tools:** For exploratory analysis, tools like Python's xarray or Matlab's built-in netCDF functions may be more convenient than writing C code.

Summary

OPeNDAP extends netCDF's capabilities from local file access to network-based data access. By using HTTP and allowing server-side subsetting, OPeNDAP makes it practical to work with large remote datasets. The integration with the netCDF library is seamless—programs that work with local files can often work with OPeNDAP URLs with minimal changes.

The key advantages of OPeNDAP are:

- Access data anywhere on the Internet
- Request only the data you need (server-side subsetting)
- No need to download and manage large files
- Transparent integration with netCDF APIs

For scientific applications that need to access distributed data sources, OPeNDAP is an essential tool that has become a standard in Earth sciences and many other fields.

Remote Data Access (OPeNDAP) – Key Takeaways

- **Open URL:** `nc_open("https://server/data.nc?var[0:10]", NC_NOWRITE, &id).`
- **Server-side subsetting** saves bandwidth.
- **DAP2 vs. DAP4:** netCDF-C handles both transparently.
- **Security:** configure `.dodsrc` for credentials.

Chapter 15 Performance Testing and Benchmarking

Learning Objectives

- Design performance tests
- Automate benchmarking
- Visualize/interpret results

Testing Performance

The performance topics covered in **Chapter 10: NetCDF-4/HDF5 Performance Fundamentals** and **Chapter 12: Advanced Compression Techniques** provide the tools for optimizing netCDF-4/HDF5 files. However, determining the optimal settings for a particular application requires testing with representative data. This chapter covers practical techniques for measuring performance, comparing different configurations, and making informed optimization decisions.

Why Benchmark?

Performance characteristics of netCDF files depend heavily on:

- **Access patterns:** Sequential vs. random, time slices vs. spatial subsets
- **Data characteristics:** Compressibility, value ranges, data types
- **Hardware:** Disk speed, memory bandwidth, CPU performance
- **Chunk sizes:** Interaction between chunks and access patterns
- **Compression settings:** Algorithm choice and compression level
-

The only way to determine optimal settings for your specific use case is to test with representative data and realistic access patterns.

Basic Timing Techniques

Using the time Command

The simplest way to measure performance is with the Unix `time` command:

```
time ./write_netcdf_file output.nc
```

This reports three times:

- **real:** Wall-clock time (total elapsed time)
- **user:** CPU time spent in user mode
- **sys:** CPU time spent in system calls (I/O operations)

For I/O-intensive operations, sys time is particularly important. High sys time relative to user time indicates I/O bottlenecks.

Timing in C Code

For more precise measurements, use timing functions within your code:

```
#include <sys/time.h>
double get_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec * 1e-6;
}
int main() {
    double start, end;
    start = get_time();
    /* Write data */
    nc_put_vara_float(ncid, varid, start_idx, count, data);
    end = get_time();
    printf("Write time: %.3f seconds\n", end - start);
    printf("Throughput: %.2f MB/s\n",
        data_size_mb / (end - start));
}
```

Timing in Fortran Code

Fortran provides the `system_clock` intrinsic for timing:

```
integer :: count_start, count_end, count_rate
real :: elapsed_time
call system_clock(count_start, count_rate)
! Write data
status = nf90_put_var(ncid, varid, data)
call system_clock(count_end)
elapsed_time = real(count_end - count_start) / real(count_rate)
print *, "Write time:", elapsed_time, "seconds"
```

For Fortran 95 and later, `cpu_time` is also available:

```
real :: start_time, end_time
call cpu_time(start_time)
! Perform operations
call cpu_time(end_time)
print *, "CPU time:", end_time - start_time, "seconds"
```

Measuring File Size and Compression Ratio

File size is a critical metric, especially when compression is used:

```
# Get file size
ls -lh output.nc
# Compare compressed vs. uncompressed
ls -lh classic_format.nc netcdf4_compressed.nc
# Calculate compression ratio
du -b uncompressed.nc compressed.nc | awk 'NR==1{u=$1} NR==2{c=$1; print "Ratio:", u/c}'
```

In your benchmark code, report both file size and throughput:

```
struct stat st;
stat(filename, &st);
double file_size_mb = st.st_size / (1024.0 * 1024.0);
double write_time = end - start;
printf("File size: %.2f MB\n", file_size_mb);
printf("Write time: %.3f seconds\n", write_time);
printf("Write throughput: %.2f MB/s\n", file_size_mb / write_time);
printf("Compression ratio: %.2f:1\n",
       uncompressed_size_mb / file_size_mb);
```

Benchmarking Chunking Strategies

Chunk size has the largest impact on netCDF-4/HDF5 performance. Test multiple configurations:

Example: Time Series Data

For a 3D variable with dimensions [time, lat, lon] of size [1000, 180, 360]:

```
/* Test different chunk configurations */
size_t chunk_configs[][3] = {
    {1, 180, 360}, /* One time slice */
    {10, 180, 360}, /* Ten time slices */
    {100, 180, 360}, /* Hundred time slices */
    {1, 90, 180}, /* Half spatial resolution */
    {10, 90, 180}, /* Compromise */
};
for (int i = 0; i < 5; i++) {
    /* Create file with this chunk configuration */
    nc_def_var_chunking(ncid, varid, NC_CHUNKED, chunk_configs[i]);
    /* Benchmark write performance */
    start = get_time();
    write_all_data(ncid, varid, data);
    write_time = get_time() - start;
    /* Benchmark read performance - time slices */
    start = get_time();
    read_time_slices(ncid, varid, 100); /* Read 100 time slices */
    read_time = get_time() - start;
    printf("Chunks [%zu,%zu,%zu]: write=%.2fs, read=%.2fs\n",
          chunk_configs[i][0], chunk_configs[i][1], chunk_configs[i][2],
          write_time, read_time);
}
```

```
}
```

Guidelines for Chunk Size Testing

- **Start with access pattern:** If you read time slices, make time dimension small in chunks
- **Test chunk sizes from 100 KB to 1 MB:** This range typically performs well
- **Test your actual access patterns:** Don't just test sequential writes
- **Consider cache size:** Chunks should fit reasonably in the chunk cache

Benchmarking Compression Settings

Test different compression algorithms and levels:

```
struct compression_config {
    int shuffle;
    int deflate;
    int level;
    char *name;
};
struct compression_config configs[] = {
    {0, 0, 0, "No compression"},
    {1, 0, 0, "Shuffle only"},
    {0, 1, 1, "Deflate level 1"},
    {1, 1, 1, "Shuffle + Deflate 1"},
    {1, 1, 5, "Shuffle + Deflate 5"},
    {1, 1, 9, "Shuffle + Deflate 9"},
};
for (int i = 0; i < 6; i++) {
    /* Create file with this compression */
    nc_def_var_deflate(ncid, varid,
                      configs[i].shuffle,
                      configs[i].deflate,
                      configs[i].level);

    /* Benchmark */
    start = get_time();
    write_all_data(ncid, varid, data);
    write_time = get_time() - start;
    start = get_time();
    read_all_data(ncid, varid, data);
    read_time = get_time() - start;
    stat(filename, &st);
    file_size = st.st_size / (1024.0 * 1024.0);
    printf("%s: size=%.2f MB, write=%.2fs, read=%.2fs\n",
          configs[i].name, file_size, write_time, read_time);
}
```

Compression Benchmarking Results Interpretation

When analyzing compression results:

- **Compression ratio vs. speed trade-off:** Higher levels compress better but slower

- **Shuffle filter effectiveness:** Varies by data type and characteristics
- **Read performance:** Decompression is typically faster than compression
- **CPU vs. I/O bound:** On fast storage, compression may slow things down; on slow storage or networks, compression can improve overall performance

Testing Access Patterns

Real applications have specific access patterns. Test the patterns you'll actually use:

Pattern 1: Sequential Time Slice Access

```
/* Read all time slices sequentially */
double start = get_time();
for (size_t t = 0; t < time_dim; t++) {
    size_t start_idx[3] = {t, 0, 0};
    size_t count[3] = {1, lat_dim, lon_dim};
    nc_get_vara_float(ncid, varid, start_idx, count, slice);
}
double elapsed = get_time() - start;
printf("Sequential time slice access: %.3f s\n", elapsed);
```

Pattern 2: Random Spatial Subsets

```
/* Read random spatial subsets */
double start = get_time();
for (int i = 0; i < 100; i++) {
    size_t lat_start = rand() % (lat_dim - 50);
    size_t lon_start = rand() % (lon_dim - 50);
    size_t start_idx[3] = {0, lat_start, lon_start};
    size_t count[3] = {time_dim, 50, 50};
    nc_get_vara_float(ncid, varid, start_idx, count, subset);
}
double elapsed = get_time() - start;
printf("Random spatial subset access: %.3f s\n", elapsed);
```

Pattern 3: Time Series at Points

```
/* Read time series at specific locations */
double start = get_time();
for (int i = 0; i < 1000; i++) {
    size_t lat_idx = rand() % lat_dim;
    size_t lon_idx = rand() % lon_dim;
    size_t start_idx[3] = {0, lat_idx, lon_idx};
    size_t count[3] = {time_dim, 1, 1};
    nc_get_vara_float(ncid, varid, start_idx, count, timeseries);
}
double elapsed = get_time() - start;
printf("Time series point access: %.3f s\n", elapsed);
```

Chunk Cache Tuning

The chunk cache can significantly impact performance. Test different cache sizes:

```
size_t cache_sizes[] = {
    1024 * 1024,      /* 1 MB */
    4 * 1024 * 1024, /* 4 MB */
    16 * 1024 * 1024, /* 16 MB */
    64 * 1024 * 1024, /* 64 MB */
};
for (int i = 0; i < 4; i++) {
    /* Set chunk cache size */
    nc_set_chunk_cache(cache_sizes[i], 1009, 0.75);
    /* Open file and benchmark access */
    nc_open(filename, NC_NOWRITE, &ncid);
    start = get_time();
    /* Perform access pattern */
    read_time_slices(ncid, varid, 100);
    elapsed = get_time() - start;
    printf("Cache size %zu MB: %.3f s\n",
           cache_sizes[i] / (1024*1024), elapsed);
    nc_close(ncid);
}
```

Cache Size Guidelines

- **Match working set:** Cache should hold chunks you access repeatedly
- **Monitor cache statistics:** Use `nc_get_chunk_cache()` to check hit rates
- **Balance memory usage:** Larger cache uses more memory
- **Per-variable cache:** Each variable has its own cache

Creating a Benchmark Suite

A systematic benchmark suite helps compare configurations:

```
/* benchmark.c - NetCDF performance benchmark suite */
#include <netcdf.h>
#include <stdio.h>
#include <sys/time.h>
#include <sys/stat.h>
typedef struct {
    char *name;
    size_t chunks[3];
    int shuffle;
    int deflate_level;
} test_config;
void run_benchmark(test_config *config,
                   size_t dims[3],
                   float *data) {
```

```

int ncid, varid, dimids[3];
double start, write_time, read_time;
struct stat st;
/* Create file */
nc_create(config->name, NC_NETCDF4, &ncid);
nc_def_dim(ncid, "time", dims[0], &dimids[0]);
nc_def_dim(ncid, "lat", dims[1], &dimids[1]);
nc_def_dim(ncid, "lon", dims[2], &dimids[2]);
nc_def_var(ncid, "data", NC_FLOAT, 3, dimids, &varid);
/* Set chunking and compression */
nc_def_var_chunking(ncid, varid, NC_CHUNKED, config->chunks);
if (config->deflate_level > 0) {
    nc_def_var_deflate(ncid, varid, config->shuffle,
        1, config->deflate_level);
}
nc_enddef(ncid);
/* Benchmark write */
start = get_time();
nc_put_var_float(ncid, varid, data);
write_time = get_time() - start;
nc_close(ncid);
/* Get file size */
stat(config->name, &st);
double file_mb = st.st_size / (1024.0 * 1024.0);
/* Benchmark read */
nc_open(config->name, NC_NOWRITE, &ncid);
nc_inq_varid(ncid, "data", &varid);
start = get_time();
nc_get_var_float(ncid, varid, data);
read_time = get_time() - start;
nc_close(ncid);
/* Report results */
printf("%-30s: ", config->name);
printf("size=%6.2f MB, ", file_mb);
printf("write=%5.2fs, ", write_time);
printf("read=%5.2fs\n", read_time);
}

int main() {
    size_t dims[3] = {100, 180, 360}; /* time, lat, lon */
    size_t data_size = dims[0] * dims[1] * dims[2];
    float *data = malloc(data_size * sizeof(float));
    /* Initialize with representative data */
    for (size_t i = 0; i < data_size; i++) {
        data[i] = 273.15 + 20.0 * sin(i * 0.001);
    }
    test_config configs[] = {
        {"classic.nc", {0,0,0}, 0, 0},
        {"nc4_1x180x360.nc", {1,180,360}, 0, 0},
        {"nc4_10x180x360.nc", {10,180,360}, 0, 0},
        {"nc4_compressed_5.nc", {1,180,360}, 1, 5},
        {"nc4_compressed_9.nc", {1,180,360}, 1, 9},
    };
    printf("NetCDF Performance Benchmark\n");
    printf("Data size: %zu x %zu x %zu\n\n", dims[0], dims[1], dims[2]);
    for (int i = 0; i < 5; i++) {
        run_benchmark(&configs[i], dims, data);
    }
    free(data);
    return 0;
}

```

Interpreting Benchmark Results

When analyzing benchmark results, consider:

Write Performance

Compression overhead: Higher compression levels slow writes

Chunk size impact: Smaller chunks may slow writes due to overhead

File format: netCDF-4 may be slower than classic for uncompressed data

Read Performance

Cache effectiveness: Good cache hit rates dramatically improve performance

Chunk alignment: Reads aligned with chunks are faster

Decompression speed: Usually faster than compression

File Size

Compression ratio: Varies greatly by data characteristics

Shuffle filter: Can improve compression 2-3x for appropriate data

Chunk overhead: Very small chunks increase metadata overhead

Overall Efficiency

Total workflow time: Consider both write and read times

Storage costs: Smaller files reduce storage and transfer costs

CPU vs. I/O trade-off: Compression trades CPU time for I/O time

Best Practices for Benchmarking

Use representative data: Test with actual data, not synthetic patterns

Test realistic access patterns: Benchmark how you'll actually use the data

Warm up the cache: Run operations once before timing to warm up caches

Multiple runs: Average results over several runs to reduce variance

Isolate variables: Change one parameter at a time

Document configuration: Record all settings (chunk size, compression, cache)

Test on target hardware: Performance varies by system

Consider the full workflow: Benchmark both write and read operations

Example Benchmark Results

Typical results for a 100x180x360 float array (24.6 MB uncompressed):

Configuration	File Size	Write Time	Read Time	Notes
Classic format	24.6 MB	0.15 s	0.12 s	Baseline
NC4, chunks 1x180x360	24.7 MB	0.18 s	0.13 s	Small overhead
NC4, chunks 10x180x360	24.7 MB	0.17 s	0.13 s	Similar
NC4, deflate 5	8.2 MB	1.2 s	0.4 s	3x compression
NC4, deflate 9	7.8 MB	2.8 s	0.4 s	Diminishing returns
NC4, shuffle + deflate 5	4.1 MB	1.3 s	0.4 s	Best ratio

These results show that shuffle + deflate level 5 provides excellent compression with reasonable performance for this data type.

Automated Testing

For systematic testing, create scripts that run benchmarks and collect results:

```
#!/bin/bash
# benchmark_suite.sh
echo "NetCDF Benchmark Suite"
echo "===== "
for chunk in "1,180,360" "10,180,360" "100,180,360"; do
```



```
for level in 0 1 5 9; do
    echo "Testing chunks=$chunk, deflate=$level"
    ./benchmark --chunks=$chunk --deflate=$level
done
```

This systematic approach helps identify optimal configurations for your specific use case.

Performance Testing & Benchmarking – Key Takeaways

- **Measure:** time I/O with `MPI_Wtime()` or POSIX `clock_gettime()`.
- **Vary:** chunk sizes, compression levels, access patterns.
- **Use HDF5 `h5stat`** to inspect chunk layout.
- **Automate:** shell loops + `nccopy` for combo sweeps.

Chapter 16 The Past, Present, and Future of NetCDF

Learning Objectives

- Identify the current state of the netCDF ecosystem and its active components
- Understand how netCDF is maintained and developed today
- Recognize the roles of the major libraries and tools in current use

How We Got Here

NetCDF has no marketing department, does no advertising, and makes no effort to recruit new users. It is selected by scientists and scientific programmers because it provides the features they need. That pattern has held for over 35 years, and it's how netCDF will continue to evolve: by responding to what working scientists actually require.

NetCDF has grown into a world-spanning data standard. There are communities of netCDF users on every continent, and none of those communities are standing still. The pressures they face today will shape what netCDF becomes tomorrow.

The Past

Ancient Times

NetCDF was developed at Unidata/UCAR in the late 1980s by Russ Rew and Glenn Davis as a self-describing, portable format for scientific data. The first release came in 1989.

Version 2.0 (1991) improved portability across platforms. Version 2.3 (1993) added Steve Emmerson's Fortran77 interface, configure-based builds, and advanced data access features like strides and mapped array sections. Testing infrastructure was established early by Russ Rew, Cathy Cormack, Glenn Davis, and Harvey Davies. Version 2.4 (1996) formalized the file format specification and added supercomputer optimizations.

NetCDF-3 Era

Version 3.0 (1997) standardized the Classic data model, defining dimensions, variables, and attributes. Glenn Davis created NetCDF-Java 1.0 in 1998, a pure Java implementation for web-based and cross-platform use. John Caron released NetCDF-Java 2.0 in 2001 with high-performance arrays, simplified interfaces, and OPeNDAP remote access. This evolved into the Common Data Model (CDM) and became central to Unidata's THREDDS Data Server.

In 2003, Northwestern University and Argonne National Laboratory released PnetCDF 1.0, led by Jianwei Li, Wei-Keng Liao, Alok Choudhary, Robert Ross, and Rajeev Thakur. Built on MPI-IO, it delivered high-performance parallel I/O for CDF-1 and CDF-2 formats on HPC systems.

OPeNDAP grew out of the DODS (Distributed Oceanographic Data System) project, which began in 1995 under the leadership of James Gallagher. Dennis Heimbigner integrated DAP2 client support into netcdf-c 3.6.0 (2004), making remote data access transparent to any program using the netCDF API. DAP2 was approved as a NASA Earth Science Data Systems standard in 2005. Heimbigner added DAP4 client support in netcdf-c 4.4.1 (2016), extending remote access to the enhanced data model. Dennis also built the ncZarr cloud storage backend that began taking shape around 2020-2021.

NetCDF-4 Era

Version 4.0 (2008) was a transformative release, introducing the enhanced data model built on HDF5 with hierarchical groups, user-defined types, multiple unlimited dimensions, and compression. It maintained backward compatibility while becoming the preferred format for complex datasets.

Version 4.1 (2010) added HDF4 SD file reading, enabling access to legacy HDF4 datasets without conversion. That same year, Jim Edwards released ParallelIO (PIO) 1.0 for CESM, featuring asynchronous I/O and multiple backend support. PIO 2.0 (2017) added a C API and improved performance; about this time, I joined as co-developer. Also in 2010, John Caron and Ethan Davis released NetCDF-Java 4.2, introducing the Common Data Model for accessing multiple scientific formats (HDF, GRIB, BUFR), NcML metadata support, and the ToolsUI application.

In 2012, I split netcdf-fortran from netcdf-c, allowing independent evolution. The C++ library was similarly separated, establishing modular language bindings. Version 4.3.0 (2013) added CMake build support by Ward Fisher, enabling Windows/Visual Studio builds.

Version 4.4.0 (2016) introduced CDF-5 format by Wei-Keng Liao, using 64-bit integers for dimensions and variables, supporting datasets over 2 billion elements and adding unsigned types. Version 4.6.2 (2018) added file open optimizations, including lazy attribute reading and fast global attribute access, dramatically reducing open times for metadata-heavy files.

Modern Times

Around 2020-2021, Zarr integration began, adding cloud-native storage support for S3 and Google Cloud Storage. In 2021, Charlie Zender and I developed the Community Codec Repository (CCR), an NSF-supported proof-of-concept demonstrating quantization and advanced compression, including Zender's Bit Grooming algorithm.

Version 4.9.0 (2022) introduced Zstandard compression and quantization functions by myself, Charlie Zender, enabling better compression ratios and controlled lossy compression. Version 4.9.3 (2024) improved ncZarr support, added plugin search path API control, and enhanced performance.

PnetCDF 1.14.0 (2024) by Wei-Keng Liao introduced intra-node aggregation for write requests. NetCDF-Fortran 4.6.2 (2025) by Ward Fisher fixed compatibility with NetCDF-C 4.9.3 and improved CMake support.

In 2025 I released version 1.0 of the NetCDF Expansion Pack (NEP), adding LZ4 and BZIP2 compression plus direct reading of NASA CDF and GeoTIFF files through the NetCDF API, maintaining full backward compatibility.

Key Milestones

Year	Version	Milestone	Significance	Developers
1989	netcdf-c-1.0	First public release	Established self-describing, portable scientific data format	Glenn Davis, Russ Rew
~1990	-	Test infrastructure	nctest test suite for C interface validation	Glenn Davis, Russ Rew
1991	netcdf-c-2.0	C interface improvements	Changed dimension lengths from int to long for better portability (MS-DOS, etc.)	Glenn Davis, Russ Rew
~1993	-	Fortran test code	Test suite for Fortran77 interface validation	Glenn Davis, Russ Rew, Cathy Cormack
1993	netcdf-c-2.3	Fortran interface and optimizations	Added Fortran77 interface, strides, mapped array sections, configure-based build	Glenn Davis, Russ Rew, Steve Emmerson
~1996	-	Exhaustive test suite	nc_test comprehensive validation for NetCDF-3 interface	Glenn Davis, Russ Rew, Harvey Davies
1996	netcdf-c-2.4	Format specification and performance	Formal file format specification, supercomputer optimizations, C++ interface support	Glenn Davis, Russ Rew
1997	netcdf-c-3.0	Classic data model	Defined	Glenn Davis, Russ

		standardized	fundamental concepts of dimensions, variables, and attributes	Rew
1998	netcdf-Java-1.0	NetCDF-Java released	Pure Java implementation enabling NetCDF access in Java applications	Glenn Davis
2001	netcdf-Java-2.0	NetCDF-Java rewrite	High-performance arrays, simplified interface, OpenDAP remote access	John Caron, Ethan Davis, Russ Rew, Dennis Heimburger
2003	PnetCDF-1.0	Parallel NetCDF library	MPI-based parallel I/O for CDF-1 and CDF-2 formats on HPC systems	Jianwei Li, Wei-Keng Liao, Alok Choudhary, Rob Ross, Rajeev Thakur
2004	netcdf-c-3.6.0	OPeNDAP/DAP2 client support	Enabled remote data access over HTTP, network-transparent data access	Dennis Heimburger
2005	-	DAP2 standard approved	Data Access Protocol 2 approved as NASA Earth Science Data Systems standard	James Gallagher, OPeNDAP
2008	netcdf-c-4.0	NetCDF-4/HDF5 enhanced model	Added hierarchical groups, user-defined types, multiple unlimited dimensions, improved compression	Edward Hartnett, Dennis Heimburger

2010	netcdf-c-4.1	HDF4 read-only support	Added capability to read HDF4 SD (Scientific Data) files through NetCDF API	Edward Hartnett
2010	PIO-1.0	ParallelIO library	High-level parallel I/O library for structured grid applications, async I/O	Jim Edwards
2010	netcdf-Java-4.2	NetCDF-Java/CDM stable release	Common Data Model, multi-format support, NcML, ToolsUI, read-write netCDF-3	John Caron, Ethan Davis
2012	netcdf-c-4.2	NetCDF-Fortran separated	Fortran library split into independent distribution with separate versioning	Edward Hartnett, Russ Rew
2013	netcdf-c-4.3.0	CMake build system	CMake support for cross-platform builds, Windows/Visual Studio support	Ward Fisher
2014	-	DAP4 released	Data Access Protocol 4 with enhanced data model support	James Gallagher, OPeNDAP, Unidata
2016	netcdf-c-4.4.0	CDF-5 format support	64-bit data format for large dimensions/variables (>2B elements), unsigned types	Wei-Keng Liao
2016	netcdf-c-4.4.1	DAP4 client support	DAP4 protocol support for	Dennis Heimburger

			enhanced data model over networks	
2017	PIO-2.0	ParallelIO major rewrite	Added C API, improved performance, restructured architecture	Jim Edwards, Edward Hartnett
2018	netcdf-c-4.6.2	File open optimizations	Lazy attribute reading and fast global attribute read for improved open performance	Edward Hartnett
2020-2021	netcdf-c-4.8+	Zarr integration	Cloud-native storage backend for object stores (S3, Google Cloud Storage)	Dennis Heimburger
2021	1.0	Community Codec Repository (CCR)	Proof-of-concept for quantization and advanced compression filters	Charlie Zender, Edward Hartnett
2022	netcdf-c-4.9.0	Zstandard compression and quantization	Added zstd compression and quantization for lossy compression, improved performance	Edward Hartnett, Charlie Zender, Ward Fisher
2024	netcdf-c-4.9.3	Quality of life improvements	Bug fixes, improved ncZarr support, plugin search path API, performance improvements	Ward Fisher, Dennis Heimburger
2024	netcdf-Java-5.x	NetCDF-Java modern releases	Continued CDM development, multi-format	Sean Arms, Hailey Johnson

			support, security updates, bug fixes	
2024	PnetCDF-1.14	PnetCDF stable release	Intra-node aggregation for write requests, continued parallel I/O optimizations	Wei-Keng Liao, Northwestern/Argonne team
2024	PIO-2.6.8	ParallelIO stable release	Continued development, bug fixes, performance improvements	Jim Edwards, NCAR team
2025	netcdf-fortran-4.6.2	NetCDF-Fortran stable release	Bug fixes, improved CMake support, stack overflow fixes, compatibility with NetCDF-C 4.9.3	Ward Fisher
2025	NEP 1.0	NetCDF Expansion Pack	LZ4 and BZIP2 compression, CDF and GeoTIFF file readers via NetCDF API	Edward Hartnett

The Present

NetCDF is a mature, stable technology in active use across the global scientific community. The core C library, `netcdf-c`, is maintained by Ward Fisher at Unidata/UCAR. It supports five binary output formats (classic, 64-bit offset, CDF-5, NetCDF-4/HDF5, and `ncZarr`), runs on every major operating system, and builds with both Autotools and CMake. The current release is version 4.9.3.

The Fortran interface, `netcdf-fortran`, is maintained separately and currently at version 4.6.2. Splitting the Fortran and C libraries in 2012 allowed each to evolve on its own release schedule, and the same separation applies to the C++ bindings. This modular structure means a bug fix or new feature in one language binding does not force a release of the others.

NetCDF-Java, now at version 5.x, is maintained by Sean Arms and Hailey Johnson at Unidata. It provides the Common Data Model (CDM), which reads not only netCDF but also HDF5, GRIB, BUFR, and other formats through a unified API. NetCDF-Java powers the THREDDS Data Server (TDS), which serves netCDF data over OPeNDAP and other protocols to thousands of users worldwide.

PnetCDF, maintained by Wei-Keng Liao at Northwestern University, provides MPI-based parallel I/O for CDF-1, CDF-2, and CDF-5 formats. The current release, version 1.14.0, introduced intra-node aggregation to reduce the number of MPI-IO calls during writes. PnetCDF remains the primary choice for parallel access to classic-format files on HPC systems.

The ParallelIO library (PIO), developed at NCAR, sits above both netcdf-c and PnetCDF and provides a high-level parallel I/O layer for structured-grid earth system models. PIO is used by CESM, E3SM, and other major climate models. The current release is version 2.6.8.

OPeNDAP continues to provide remote data access. The DAP2 and DAP4 protocols allow any netCDF client to open a URL as if it were a local file. DAP2 remains widely deployed; DAP4 adds support for the enhanced data model. Dennis Heimburger integrated both protocols into netcdf-c, and the OPeNDAP project, led by James Gallagher, maintains the server-side infrastructure.

NCO (NetCDF Operators), developed by Charlie Zender at UC Irvine, provides command-line tools for subsetting, concatenation, arithmetic, and attribute editing. NCO is widely used in production data pipelines and eliminates the need to write custom programs for many routine operations.

The CF (Climate and Forecast) Conventions define how to store geophysical data in netCDF files. CF specifies standard names for variables, coordinate reference systems, cell methods, and other metadata. Nearly all major data centers require CF compliance, and CF continues to evolve with new conventions for unstructured grids, satellite swath data, and other data types. CF compliance is what makes netCDF files interoperable across tools and institutions.

For compression, netcdf-c now supports both zlib deflate and Zstandard (zstd) compression, along with quantization functions for controlled lossy compression. The HDF5 filter plugin mechanism allows additional compressors to be loaded at runtime without recompiling the library. The Community Codec Repository (CCR), developed by Charlie Zender and myself, demonstrated this approach with Bit Grooming and other algorithms.

The NetCDF Expansion Pack (NEP), which I released in 2025, extends netcdf-c without adding to its codebase. NEP 1.0 provides LZ4 and BZIP2 compression filters, a read-only layer for NASA CDF (Common Data Format) files, and a read-only layer for GeoTIFF files. All access goes through the standard netCDF API.

Testing remains central to the project. The netcdf-c test suite runs thousands of tests across all supported formats and features. Continuous integration builds and tests every commit on multiple platforms. The Fortran, Java, and PnetCDF libraries each maintain their own test suites as well.

NetCDF today is not a single library but an ecosystem: a C library, Fortran and C++ bindings, a Java implementation, parallel I/O libraries, remote access protocols, command-line operators, an extension pack, and a metadata convention. These components are developed by different teams at different institutions, but they all read and write the same file formats and follow the same data models. That interoperability is the defining feature of netCDF in its current state.

The Future

The Cloud

Cloud storage is replacing local disk for many scientific workflows. Data that once lived on a RAID array down the hall now sits in an S3 bucket on the other side of the country. This changes the performance equation. Local disk access is fast and cheap per operation. Cloud object storage is slow per request but scales without limit.

ncZarr addresses this by storing netCDF data in Zarr format, which is designed for cloud object stores. Each chunk becomes a separate object, so a client can fetch exactly the data it needs with a single HTTP range request instead of downloading an entire file. This matters when a file is 50 GB and you need one time slice.

Cloud-native access is still maturing. Chunk indexing, authentication, caching strategies, and cross-region latency all present ongoing challenges. As more scientific data moves to the cloud, netCDF will need to keep pace with the storage systems that host it.

Compression

The zlib deflate compression that shipped with netCDF-4 in 2008 was a major step forward. But compression research has not stood still. Algorithms like Zstandard offer better ratios at higher speeds. Quantization techniques like Bit Grooming and BitRound reduce the precision of floating-point values to what is scientifically meaningful, then let the compressor work on the simplified bit patterns. The combination of quantization and a fast compressor can cut file sizes by 5x or more with no loss of scientific information.

The HDF5 filter plugin mechanism makes it possible to add new compressors without modifying or recompiling the netCDF library. A compression algorithm can be distributed as a shared library, dropped into a plugin directory, and used immediately. This architecture means netCDF does not need to pick winners among compression algorithms. Users can choose the compressor that fits their data and their performance requirements.

The NetCDF Expansion Pack already provides LZ4 and BZIP2 through this mechanism. More algorithms will follow as the community identifies what works best for different data types: smooth atmospheric fields, noisy radar returns, sparse ocean observations, integer satellite counts.

The NetCDF Expansion Pack

The netcdf-c library must remain stable and conservative. Millions of files depend on it. But scientists need new features, and they need them faster than a core library can safely deliver.

The NetCDF Expansion Pack (NEP) solves this by providing new capabilities as a separate, optional library that links against netcdf-c. NEP 1.0 includes LZ4 and BZIP2 compression filters, a read-only layer for NASA CDF (Common Data Format) files used in heliophysics missions, and a read-only layer

for GeoTIFF satellite imagery. All access goes through the standard netCDF API, so existing tools like ncdump work on these files without modification.

The expansion pack model allows experimentation. A new file format reader or compression filter can ship in NEP, get real-world testing, and later migrate into the core library if it proves valuable. Or it can stay in NEP indefinitely, serving the users who need it without burdening those who do not.

Parallel I/O

The computational science community continues to push toward higher resolution. A global atmosphere model at 1 km grid spacing produces terabytes of output per simulated day. Writing that data to disk without becoming a bottleneck requires parallel I/O that scales to thousands of processes.

PnetCDF and PIO address this today, but the demands keep growing. Burst buffers, non-volatile memory, and tiered storage hierarchies on next-generation supercomputers will require new I/O strategies. NetCDF's parallel I/O stack will need to adapt to hardware that does not yet exist.

Data, Data, and More Data

Everyone working with scientific data faces the same pressure: the instruments keep getting better, the models keep getting finer, and the data volumes keep growing. High-resolution satellites, 1 km weather models, and exascale simulations all produce data at rates that would have been unimaginable when netCDF was first released. The supercomputers that run these models cost hundreds of thousands of dollars per minute of compute time. They are built to solve serious problems, and the data they produce contains answers to important questions.

But managing terabytes of data across a dizzying assortment of structures, types, and storage systems is a problem in itself. NetCDF cannot solve every part of that problem, but it can continue to do what it has always done: provide a reliable, self-describing, portable container that lets scientists focus on their science instead of their file formats.

The future of netCDF lies with those scientists and their data needs.

The Past, Present, and Future of NetCDF – Key Takeaways

- 1989-1997 Classic era: NetCDF 1.0-3.0 established the self-describing portable format, Fortran interface, and classic data model.
- 1998-2004 Expansion: NetCDF-Java for web applications, PnetCDF for parallel HPC I/O, OPeNDAP/DAP2 for remote HTTP access.
- 2008 NetCDF-4 milestone: Enhanced model with HDF5 backend added groups, user-defined types, multiple unlimited dimensions, and compression.
- 2010-2018 Maturation: HDF4 read-only support, ParallelIO library, CDF-5 64-bit format, CMake build system, file open optimizations, modular language bindings.

- 2020-2025 Cloud and compression era: ncZarr cloud-native storage, Zstandard compression, quantization, NetCDF Expansion Pack with LZ4/BZIP2 and CDF/GeoTIFF readers.
- Current ecosystem: netcdf-c 4.9.3, netcdf-fortran 4.6.2, NetCDF-Java 5.x, PnetCDF 1.14.0, PIO 2.6.8, NCO, and NEP 1.0, maintained by teams at Unidata, Northwestern, NCAR, UC Irvine, and OPeNDAP.
- CF Conventions: The standard metadata framework that makes netCDF files interoperable across tools, institutions, and disciplines.
- Compression advances: Zlib, Zstandard, quantization, and runtime filter plugins provide a range of options for balancing speed, ratio, and precision.
- Testing and stability: Thousands of automated tests across all libraries and continuous integration on multiple platforms ensure reliability.
- Community-driven development: Features respond to real scientific data needs across atmospheric science, oceanography, climate modeling, satellite missions, and heliophysics.
- Backward compatibility: Every major addition, from NetCDF-4 to ncZarr to NEP, maintains the ability to read older files and use existing APIs.

Alphabetical Index

Architecture.....	
dispatch table.....	239-241, 244
user-defined format.....	240, 241
V2 API.....	241
Attributes.....	
missing_value.....	212
valid_range.....	207, 212, 214
_FillValue , 43, 45, 88, 90, 91, 96, 97, 106-108, 110, 112, 152-155, 188, 205, 207, 208, 211, 212, 214	
Book structure.....	
learning objectives. .1, -, -16, 33, 75, 87, 101, 106, 112, 119, 125, 133, 145, 150, 156, 162, 163, 165, 166, 183, 205, 217, 223, 235, 239, 243, 253, 263	
Build systems.....	
Autotools.....	13, 18, 19, 22-24, 32, 269
CMake.....	13, 17, 18, 21-24, 32, 264, 267, 269, 272
Conda.....	13, 15, 18, 32
Cygwin.....	17, 18
Gradle.....	13, 15, 25, 26, 195, 196, 203
Maven.....	13, 15, 25, 26, 30-32, 195, 203
MSYS2.....	15, 17, 22, 32
package manager.....	13, 15-19, 23, 25, 27, 32, 100
shared library.....	30, 31, 271
Spack.....	13, 15, 18, 19, 32
vcpkg.....	15, 17, 32
WSL.....	15, 18
C API functions.....	
nc_close..6, 7, 28, 36, 48, 103, 105, 108, 111, 114, 116, 122, 123, 130, 132, 136, 141, 197, 208, 225, 227, 246, 248, 258, 259	
nc_create 5, 28, 30, 36, 37, 48, 51, 52, 61, 78, 79, 81-83, 92, 102, 107, 114, 121, 127, 134, 197, 206, 224-227, 230, 232, 233, 239-241, 259	
nc_create_par.....	28, 30, 79, 224-227, 230, 232, 233
nc_def_compound.....	54, 125-127
nc_def_dim 5, 38, 40, 41, 48, 52, 65, 92, 102, 107, 114, 121, 122, 128, 129, 134, 135, 206, 209, 211, 225, 226, 259	
nc_def_enum.....	58, 125, 126, 128

nc_def_grp.....	52, 134
nc_def_opaque.....	59, 126-128
nc_def_var....	5, 40, 41, 44, 45, 48, 52-56, 59, 65, 81, 83, 92, 102, 107, 108, 114, 120, 122, 129, 135, 206, 207, 209-211, 218-221, 225, 226, 229-231, 235, 239, 255, 256, 259
nc_def_var_fill.....	44, 45, 48, 220
nc_def_var_quantize.....	235
nc_def_var_shuffle.....	120
nc_def_vlen.....	56, 125, 126, 128
nc_enddef.....	6, 37, 38, 44, 45, 48, 103, 108, 114, 122, 129, 135, 208, 218, 221, 225, 226, 259
nc_free_string.....	58, 61, 68, 69, 72, 132
nc_free_vlen.....	57, 67, 68, 72, 131
nc_get_att.....	6, 7, 43, 60, 61, 105, 106, 109, 110, 197
nc_get_chunk_cache.....	258
nc_get_var.....	7, 46-48, 62, 63, 65-72, 105, 110, 111, 116, 123, 130-132, 140, 197, 248, 259
nc_get_vara.....	47, 48, 239, 245, 246, 248, 251, 257
nc_inq.....	6, 7, 28, 32, 36, 38, 43, 45, 47, 48, 61, 64, 75, 76, 101, 103-105, 109, 110, 112, 115, 116, 118, 123, 130-133, 136-139, 197, 245, 248, 259
nc_inq_dim.....	38, 48, 101, 104
nc_inq_dimid.....	137, 138
nc_inq_dimlen.....	6, 138
nc_inq_format.....	75, 76, 112, 115, 118
nc_inq_format_extended.....	112
nc_inq_grp_ncid.....	64, 133, 136, 137
nc_inq_grpname.....	137
nc_inq_grps.....	136
nc_inq_libvers.....	28, 32
nc_inq_var.....	45, 48, 101, 104, 123, 139
nc_inq_var_fill.....	45
nc_inq_varid.....	47, 64, 116, 123, 130-132, 139, 197, 245, 248, 259
nc_insert_compound.....	55, 66, 126, 128
nc_insert_enum.....	59, 126, 128
nc_open.....	6, 36, 37, 48, 52, 76, 79, 82, 83, 85, 103, 109, 115, 123, 130, 136, 197, 225, 227, 232, 233, 245, 246, 248, 251, 252, 258, 259
nc_open_par.....	79, 225, 227, 232, 233
nc_put_att.....	5, 42, 43, 45, 48, 60, 103, 106, 108, 114, 207-214, 226

nc_put_att_text.....5, 42, 43, 103, 108, 114, 207-214, 226
 nc_put_var.....6, 40, 41, 46, 48, 55, 56, 59, 60, 62, 63, 65-72, 92, 103, 108, 114, 122, 129, 130, 136, 197, 259
 nc_put_vara.....46-48, 65, 226, 227, 230, 232, 239, 241, 254
 nc_redef.....37, 38
 nc_set_chunk_cache.....218, 221, 258
 nc_set_log_level.....21
 nc_strerror.....5, 101, 102, 107, 113, 120, 127, 134, 206
 nc_type.....54, 56, 58, 59, 104, 127, 128, 138
 nc_var_par_access.....225-227, 230, 231, 233
 nc_vlen_t.....56, 57, 67, 68, 129, 130
 CF Conventions.....
 axis....., 39, 106-112, 151, 153, 155, 193, 201, 211, 213
 cell methods....., 213, 270
 coordinate system.....11, 27, 193, 198, 199, 201, 209, 210, 214
 standard name.....1, , 107, 205, 212, 214, 270
 Command-line utilities.....
 CDL....., 45, 87, 90-93, 96-98, 100, 111, 142
 h5dump.....80, 82
 nc-config....., 13, 17, 18, 27-32, 250
 nccopy....., 75, 84, 86, 87, 93-96, 98, 100, 237, 262
 ncdump 1, 7, 8, 10, , , 45, 75, 76, 82, 87-93, 95-98, 100, 118, 119, 125, 142, 155, 161-165, 171, 176, 215, 247, 272
 ncgen.....1, , 11, 87, 90-92, 96-98, 100
 nf-config....., 13, 17, 18, 27-30
 Compression.....
 deflate.4, , 81, 83, 84, 94, 119-124, 163, 164, 179, 219, 221, 235, 236, 256, 258, 259, 261, 262, 270, 271
 filter....., 81, 84, 87, 94, 120, 123, 178, 219, 220, 236, 237, 257, 260, 270-273
 lz4....., 83, 236
 quantization....., 235, 236, 264, 268, 270, 271, 273
 shuffle filter.....81, 235
 zstd....., 83, 235, 236, 268, 270
 Conventions.....
 ACDD....., 213-215

CF Conventions.....	3, , 40, 41, 45, 101, 106, 150, 188, 199, 205, 210, 212, 214, 215, 273
COARDS.....	, 214
conventions.....	1, 3, , , 12, 33, 40, 41, 45, 49, 83, 88, 96, 97, 101, 106, 150, 184, 188, 199, 203, 205-208, 210, 212-215, 241, 270, 273
Core concepts.....	
attribute.....	5-12, 35, 37, 41-43, 45, 46, 54, 55, 60, 61, 80, 86, 87, 89, 91, 92, 98, 100, 101, 103-106, 118, 145-150, 153, 161, 179, 181, 183, 186-188, 191, 194, 197, 199-201, 205, 209-215, 244, 264, 268, 270
attributes	1-7, 9-12, 33, 35-37, 40-43, 50-52, 58, 60, 61, 74, 80, 83, 87-92, 95-99, 101-103, 106-110, 114, 118, 119, 125, 145, 150-153, 158, 161-164, 166, 183, 186-188, 192, 198, 200, 201, 205-209, 211, 213-215, 220, 226, 241, 244, 247, 249, 263, 266
coordinate variable.., , ,	33, 40, 41, 43, 74, 80, 88, 89, 95, 97, 100, 101, 106-108, 110, 150-152, 154, 206-209, 213, 214
data mode	1, 2, 4, , , 7, 11, 25, 26, 33-35, 37, 38, 46, 48, 50, 57, 72, 74, 75, 77-79, 82, 84, 85, 93, 102, 112, 113, 118, 125, 155-157, 178, 181, 183, 249, 263-265, 267-270, 272
define mode	6, 9, , , 37, 38, 44, 46, 48, 54, 102, 103, 108, 114, 122, 135, 147, 152, 159, 163, 169, 177, 178, 181, 208, 209, 211, 217, 218, 221, 225
dimension....	2, 6, 8, , , , 21, 37-41, 46-49, 51, 52, 57, 58, 65, 69, 72-74, 80, 81, 85, 86, 88, 90, 94, 95, 99, 101-104, 106, 107, 132, 133, 135, 137, 138, 141, 145-148, 151, 162, 166-168, 170-174, 178, 181, 183, 186-189, 191, 194, 199-202, 209, 211, 217, 218, 239, 256, 265
fill mode.....	, 44, 45, 178, 220, 221
fill value.....	, , 12, 44, 45, 48, 57, 89-91, 181, 211, 212, 220, 221
group....., ,	19, 42, 50-53, 61-64, 74, 79, 80, 84, 91, 132-135, 137, 141-143, 166-168, 170-172, 176, 177, 179, 195, 199, 201, 219
groups....	2, -10, 19, 31, 33, 48-50, 52, 61, 62, 64, 73-75, 77-86, 91-93, 101, 112, 118, 132-134, 136, 137, 141, 142, 156, 157, 162, 166-168, 170-172, 176, 194, 195, 201, 224, 233, 249, 264, 266, 272
metadata...1-7, 9-, ,	27, 33, 36, 38, 40, 42, 43, 48, 55, 57, 61, 79, 80, 82, 83, 87, 92, 93, 98-103, 106, 109, 115, 125, 126, 132, 138, 140, 145, 147, 149, 150, 152, 160, 163-165, 170, 173, 174, 186-188, 193, 194, 198, 203, 205, 214, 215, 217, 230, 233, 241, 243, 244, 247, 251, 260, 264, 270, 273
record variable.....	85
self-describing.....	1, 2, 4, , , 40, 188, 205, 215, 263, 265, 272
subgroup.....	52, 53
unlimited dimension.....	, 33, 39, 48, 51, 52, 65, 73-75, 77, 78, 81-85, 89, 93, 264, 266, 272
variable.....	1, 5, 7-11, 20, 31, 33, 35-40, 42-60, 62-72, 74, 77-82, 84, 85, 88-91, 94, 97-99, 101-106, 108, 112, 115-118, 122, 123, 125, 126, 128-130, 132, 133, 135, 138, 140, 142, 145-149, 151, 156, 157, 160, 162, 165-168, 170, 171, 173, 174, 178, 179, 181, 183, 184, 186-192, 194, 195, 197-200, 205-212, 214, 215, 218-221, 225, 226, 228, 230, 239, 241, 245, 246, 248, 250, 251, 255, 258
Data models.....	
Classic model... , ,	9, 33, 34, 38, 39, 48, 49, 51, 52, 61, 65, 72-74, 76, 77, 79, 81, 82, 84, 92, 93, 101, 113, 118, 119, 145, 157, 158, 162, 233

data model. 1, 2, 4, , 7, 11, 25, 26, 33-35, 48, 50, 57, 72, 74, 75, 77-79, 82, 84, 85, 93, 112, 113, 118, 125, 155-157, 181, 183, 249, 263-265, 267-270, 272

enhanced model. -9, 13, 33, 39, 42, 48, 49, 51, 61, 72-74, 78, 81, 83-86, 112, 155, 224, 227, 266, 272

Data types.....

byte.....1, 2, 35, 39, 40, 43, 44, 46, 55, 56, 66, 78, 89-91, 120, 126-128, 132, 165, 180, 190, 202, 219

char..6, 7, 28, 35, 38, 39, 43, 44, 46, 53, 57, 58, 60, 61, 68-71, 73, 104, 105, 109, 113-115, 120, 121, 130-132, 134, 135, 137, 138, 140, 180, 202, 207, 224, 226, 245, 246, 251, 256, 258

double.....29, 30, 32, 35, 39, 41, 43, 44, 46, 54, 55, 88, 90, 91, 96, 97, 121, 124, 127, 128, 180, 184, 188-190, 200, 202, 206, 208, 211, 232, 254, 255, 257, 259

float...4, 35, 39, 41, 43-47, 52, 54, 55, 65, 88, 90, 91, 96, 97, 107, 108, 110-116, 118, 119, 121-125, 127, 128, 151, 152, 155, 156, 158, 161, 162, 164, 180, 184, 186, 188-192, 194, 197, 198, 200, 202, 206-212, 225-228, 230-232, 236, 245, 248, 251, 254, 257-259, 261

int. 5-8, 11, 27, 28, 35, 36, 39, 40, 43-48, 53, 55, 56, 58-60, 66-69, 71, 76, 90, 91, 102-107, 109-111, 113-116, 120-132, 134-137, 139-141, 146, 148, 149, 163, 169, 175, 176, 180, 186, 187, 189-193, 197, 200-202, 206, 209, 219, 224-230, 235, 239, 245, 248, 254-259, 265

int64.....53, 54, 60, 62, 63, 79, 90, 91, 132-136, 139-142, 166-171, 173-175, 177, 180

schar.....43, 46

short.....35, 39, 43, 44, 46, 53, 54, 60, 90, 91, 126, 134, 135, 140, 180, 190, 202

ubyte.....53, 60, 62, 63, 91, 132-136, 139-142, 166-171, 173-176, 180, 202

uchar.....62, 63, 136, 140

uint.....53, 60, 62, 63, 91, 132-136, 139-142, 166-171, 173-175, 177, 180, 202

uint64.....53, 60, 62, 63, 79, 91, 132-136, 139-143, 166-171, 173-177, 180

ulonglong.....60, 62, 63, 136, 140

unsigned integer.....48, 49, 53, 54, 74

ushort.....52-54, 60, 62, 63, 91, 132-136, 139-142, 166-171, 173-176, 180, 202

dimensions5, 7, 10, 21, 36, 39, 51, 52, 78, 83, 99, 132, 133, 142, 162-164, 166, 167, 176, 177, 186-189, 198, 200, 201, 205, 206, 208, 209, 217, 218, 220, 225, 226, 228, 229, 231, 241, 245-247, 251, 255, 266, 267

Dimensions.....36, 38, 51, 77, 133, 166, 167, 186, 187, 200, 201

Earth science.....

climate.....1-, , , , 26, 49, 73, 97, 99, 106, 150, 205-208, 212, 214, 251, 270, 273

oceanography.....1, 3, , 273

scientific data 1, 4, 33, 49, 74, 82, 85, 94, 100, 106, 119, 124, 125, 183, 210, 211, 217, 219, 243, 250, 263, 265, 267, 271-273

weather.....2, , , 26, 39, 49, 51, 54, 73, 125-127, 251, 272

File formats.....

CDF-1.....24, 76, 78, 112, 113, 115, 117, 118, 156, 157, 159, 263, 266, 270

CDF-2.....	24, 76, 112, 113, 115, 117, 156, 157, 159, 181, 263, 266, 270
CDF-5, , 13, 24, 53, 75-79, 84-86, 92, 93, 96, 112, 113, 115, 117, 156, 157, 159, 181, 223, 224, 227, 233, 239, 240, 264, 267, 269, 270, 272	
Classic format....	2, , 11, 13, 14, 20, 21, 23, 24, 37, 38, 48, 51, 53, 75, 77-79, 82-86, 92-95, 99, 112, 113, 118, 145, 157, 218, 223, 227, 240, 261
HDF4.....	, 13, 14, 20, 79, 84, 85, 239, 264, 267, 272
HDF5. .2, , , , 9, 11-15, 17-24, 26-33, 37, 38, 48, 49, 51, 53, 54, 56, 57, 59, 60, 62, 65, 72, 74-88, 92-96, 98, 99, 102, 112, 113, 115, 117, 118, 133, 155-159, 162, 166, 181, 183, 185, 186, 193, 195, 203, 217, 218, 220, 221, 223, 224, 227, 229, 233, 235, 236, 239-241, 243, 253, 255, 262, 264, 266, 269-272	
ncZarr.....	, 13, 14, 75-77, 82, 83, 85, 264, 268, 269, 271, 273
NetCDF.....	1-41, 43-62, 65, 66, 70, 72, 74-102, 105-107, 111-113, 116-120, 125, 127, 132-134, 142, 145, 146, 149-151, 155-158, 161-168, 171, 176, 177, 179-181, 183-186, 188, 189, 191, 193-199, 201-203, 205, 206, 208, 209, 214, 215, 217, 219-221, 223, 224, 226, 227, 229, 231-233, 235, 236, 239-241, 243-247, 249, 250, 252, 253, 255, 258-261, 263-273
NetCDF-3.....	, 58, 72, 195, 197, 263, 265
NetCDF-4. .2-, -, , 19, 26, 27, 31, 37, 50, 54, 57, 58, 60-62, 72, 75-79, 81-85, 88, 90-95, 98, 99, 101, 102, 112, 118, 119, 125, 132-134, 142, 156, 158, 162-164, 166-168, 171, 179-181, 193-195, 197, 199, 201, 217, 219, 221, 224, 229, 233, 235, 236, 253, 264, 266, 269, 272, 273	
PnetCDF., , 13, 15, 22-24, 27, 75, 77-79, 85, 117, 157, 223, 224, 227, 228, 233, 263, 264, 266, 269, 270, 272, 273	
Zarr.....	, 75, 78, 82-84, 239, 240, 264, 268, 271
Fortran API functions.....	
nf90_close.....	9, 10, 37, 147, 149, 152, 154, 159, 160, 163-165, 170, 197, 198, 209, 232, 246
nf90_create.....	8, 36, 62, 78, 79, 81, 82, 146, 151, 158, 162, 164, 165, 168, 197, 208, 231
nf90_create_par.....	231
nf90_def_dim.....	8, 38, 41, 52, 146, 151, 158, 162, 164, 168, 208, 231
nf90_def_enum.....	59, 165
nf90_def_grp.....	53, 168
nf90_def_opaque.....	60, 165
nf90_def_var.....	8, 40, 41, 44, 52, 54, 70, 146, 151, 152, 158, 163, 164, 168, 169, 208, 231
nf90_def_var_fill.....	44
nf90_enddef.....	9, 37, 147, 152, 159, 163-165, 169, 209, 231
nf90_get_att.....	9, 10, 43, 60, 148, 149, 153, 197
nf90_get_var.....	10, 46, 47, 56, 58, 64, 72, 149, 154, 160, 175, 197, 198, 246
nf90_inq_dimid.....	172, 173
nf90_inq_format.....	156, 158, 159, 162
nf90_inq_grp_ncid.....	64, 166, 171

nf90_inq_grpname.....	171, 172
nf90_inq_grps.....	171
nf90_inq_libvers.....	28, 32
nf90_inq_var_fill.....	45
nf90_inq_varid.....	47, 48, 160, 173, 174, 182, 197, 198, 246
nf90_inquire.....	9, 36, 38, 43, 48, 61, 76, 145, 147-149, 152, 153, 160, 173, 174
nf90_inquire_variable.....	48, 145, 148, 173, 174
nf90_insert_enum.....	59
nf90_noerr..	10, 36-38, 40-42, 44-48, 52-54, 59, 60, 62, 64, 70-72, 76, 79, 81, 82, 145-149, 151-154, 158-160, 162-165, 168-175, 209, 232
nf90_open.....	9, 36, 76, 147, 152, 159, 170, 197, 246
nf90_put_att.....	8, 42, 43, 45, 60, 147, 150-152, 158, 208, 209
nf90_put_var.....	9, 40, 41, 46, 47, 56, 58, 64, 72, 147, 152, 159, 163, 164, 169, 170, 197, 232, 254
nf90_redef.....	37
nf90_strerror.....	10, 145, 149, 155, 161, 176, 209, 232
nf90_var_par_access.....	232
HDF5.....	
dimension scale.....	80, 86, 217
Installation.....	
smoke test.....	, 27, 28, 32
Java API.....	
Array.....	, 11, 126, 145, 150, 183, 184, 186, 189-194, 197-200, 202, 250
CDM.....	183, 195, 263, 267-269
JNI.....	, 26, 30-32, 186, 197, 203
NcML.....	, 194, 198, 203, 264, 267
NetcdfDataset.....	, 11, 193, 198, 199, 201
NetcdfFile.....	183-187, 193-195, 198-200, 203
SLF4J.....	25, 31, 184, 196
Languages.....	
Fortran 1, 3-6, 8-19, 21, 22, 24, 25, 27-30, 32, 33, 36-48, 51-62, 64, 66, 68-72, 74-76, 78, 79, 81, 82, 84, 87, 92, 99, 100, 145, 146, 150, 151, 155, 156, 158, 162-168, 177, 180, 181, 183, 184, 188, 189, 192, 197, 202, 208, 231, 241, 246, 247, 254, 264, 265, 267, 269, 270, 272	
NetCDF-Java....	1, 4, , , 13, 15, 25, 26, 31, 32, 183, 184, 186, 189, 193, 195, 199, 202, 263, 264, 266-269, 272, 273
Libraries.....	
netcdf-c.....	7, 13, 14, 17, 18, 20-22, 26, 51, 80, 264-271, 273

- netcdf-fortran..... 16-19, 21, 22, 29, 56-58, 165, 264, 269, 273
- Mode flags.....
 - NC_CLOBBER..... 5, 28, 36, 52, 61, 78, 79, 81, 82, 102, 107, 114, 121, 127, 134
 - NC_NETCDF428, 37, 52, 61, 62, 81-83, 102, 107, 112, 113, 116-118, 121, 127, 133, 134, 206, 217, 225, 226, 230, 240, 259
 - NC_NOWRITE. 6, 36, 76, 83, 103, 109, 115, 123, 130, 136, 197, 225, 245, 246, 248, 251, 252, 258, 259
- NcML.....
 - aggregation..... , 194, 203, 264, 269, 270
- OPeNDAP.....
 - client/server..... , 243
 - constraint expression....., 245-248, 250, 251
 - DAP2..... , 20, 26, 244, 249, 252, 264, 266, 270, 272
 - DAP4..... , 26, 244, 249, 252, 264, 267, 270
 - DDS..... 244
 - federated..... , 251
 - OPeNDAP. .2, 3, 11, , 15, 20, 22, 26, 27, 183, 185, 195, 198, 202, 243-252, 263, 264, 266, 267, 269, 270, 272, 273
 - remote data.....2, 3, 11, 13-15, 26, 199, 243, 245-247, 252, 264, 266, 270
- Operating systems.....
 - POSIX..... 17, 262
 - Unix....., 15-18, 20, 22, 253
 - Windows....., 6, 13, 15, 17, 18, 22, 26, 32, 264, 267
- Organizations.....
 - CMIP.....3, , 11
 - ECMWF.....3, , 11
 - EOSDIS.....2,
 - ESA....., 11
 - Eumetsat.....
 - NASA..... 2, 4, 5, 11, 85, 219, 251, 264, 266, 270, 271
 - NCAR.....3, , 269, 270, 273
 - NOAA.....2, , , 11
 - UCAR.....263, 269
 - Unidata.....3, 25, 30-32, 196, 198, 213, 263, 267, 269, 273
- Parallel I/O.....

- collective I/O..... , 225, 227, 230, 233
- domain decomposition....., 226, 228, 229, 231, 233
- HPC.....4, , 11, 15, 18, 32, 223, 232, 236, 263, 266, 270, 272
- independent I/O..... , 225, 231
- MPI....., 13-15, 18-24, 28, 29, 32, 77, 79, 181, 223-228, 230-233, 262, 263, 266, 270
- MPI-IO....., 230, 233, 263, 270
- parallel I/O 2, , , 11-16, 20-24, 27, 28, 32, 73, 75, 77-79, 85, 180, 181, 223-225, 227, 229, 231-233, 263, 266, 267, 269, 270, 272
- supercomputer.....79, 263, 265
- Performance.....
 - benchmarking.....3, 11, , , 16, 20, 217, 232, 233, 253, 255, 256, 260, 262
 - chunking.....2, , 11-, , 73, 74, 81, 83, 88, 89, 93-96, 98-100, 118, 157, 162, 179, 193, 217, 218, 221, 229, 233, 255, 259
 - Chunking.....
 - chunk cache....., , 218, 221, 256, 258
 - chunk size....., , 81, 83, 86, 94, 217-219, 229, 253, 255, 256, 260, 262
 - compression...2-, , 9, 11-16, 19, 31, 49, 56, 57, 73-75, 77-81, 83-86, 88, 89, 93-96, 98-101, 112, 118-125, 156-158, 162, 163, 178, 179, 217-221, 224, 233, 235, 236, 239, 249, 253, 255-257, 259-262, 264, 266, 268-273
 - endianness....., 88, 119, 125, 161-164, 221
 - Endianness.....
 - little-endian.....221
- Time encoding.....
 - calendar....., 96, 97, 207, 209, 211, 214
 - time coordinate....., 41, 211, 214
 - time units....., 210
- Tools.....
 - ToolsUI....., , 27, 198, 264, 267
- User-defined types.....
 - compound type.....49, 54-56, 65, 66, 68, 73, 125-128, 130, 132, 164, 165, 179
 - enum type.....50, 56, 58, 59, 70, 125-128, 131, 165
 - opaque type.....59, 60, 125-128, 132, 164, 165, 179
 - string type.....53, 57, 58, 126, 129, 131, 132
 - user-defined type. , 9, 31, 33, 48, 50, 54, 56, 61, 70, 74, 75, 77-81, 83-86, 92, 93, 101, 118, 125-127, 132, 157, 164, 165, 179, 224, 264, 266, 272
 - VLEN....., 33, 56, 57, 67, 68, 72, 74, 179, 241

vlen type.....	56-58, 68, 128, 130, 165, 241
variables. .5, 7, 11, 20, 38, 51, 52, 54, 83, 85, 99, 132, 142, 163, 164, 166, 176-178, 181, 186, 187, 189, 192-194, 198, 200, 201, 205-209, 211, 213, 214, 217, 220, 224, 225, 231, 241, 243-248, 260, 266, 267	
Variables.....	36, 39, 40, 133, 166, 187, 189, 194, 200, 201, 209, 248

-